

XF

CROSS-FRAMEWORK ARCHITECTURAL MODEL

Modelo de Arquitectura Cross-Framework

*XF: A Framework-Agnostic Architectural Reference Model
for Unified Development Process*

Israel Sanjurjo Cuenca

ORCID [0009-0006-9584-7977](https://orcid.org/0009-0006-9584-7977)

June 9, 2026

<https://xfarch.org>

Código de documento	XFA-CFAM-001:2026
Versión del modelo	1.0.0
Estado del documento	Versión 1.0.0 — Edición de referencia
Fecha de emisión	June 9, 2026
Autor	Israel Sanjurjo Cuenca
ORCID	0009-0006-9584-7977
Marco editorial	Cross-Framework Architectural Model (CFAM) — https://xfarch.org
Política de cambios	Las modificaciones del modelo se gestionan mediante un proceso público de propuestas. Las versiones se identifican con numeración semántica de tres niveles (mayor.menor.parche): los cambios <i>sustantivos</i> en la taxonomía o en los axiomas incrementan la versión mayor; las adiciones <i>editoriales</i> al catálogo de reglas o las reorganizaciones incrementan la versión menor; las precisiones <i>aclaratorias</i> de redacción, sin alteración del contenido normativo, incrementan la versión de parche.

Copyright

© 2026 Israel Sanjurjo Cuenca. Todos los derechos reservados.

Este documento es la edición de referencia v1.0 del modelo de Arquitectura XF (Cross-Framework). Su reproducción total o parcial sin autorización expresa del autor queda prohibida.

Declaración sobre derechos de patente

El autor del presente documento no es titular de derecho de patente alguno cuya aplicación sea necesaria para la implementación del modelo XF, ni tiene conocimiento de la existencia de derechos de patente de terceros sobre las prescripciones aquí descritas. El autor llama la atención sobre la posibilidad de que la implementación de este documento pueda requerir el uso de patentes todavía no identificadas. Cualquier parte que considere disponer de derechos de patente esenciales para la aplicación del modelo está invitada a comunicarlo por escrito al autor.

Resumen

La proliferación acelerada de lenguajes de programación, marcos de desarrollo y herramientas de construcción ha producido un ecosistema de desarrollo de software altamente fragmentado, en el que cada tecnología impone su propia nomenclatura, sus propios patrones estructurales y sus propias convenciones de organización. Esta fragmentación genera costes operativos medibles — incremento del coste de cambio de contexto, inconsistencias entre implementaciones y dependencia estructural de herramientas propietarias — a pesar de que todos los artefactos software implementan implícitamente las mismas tres etapas invariantes de cualquier proceso formal: interacción, procesamiento y acceso. El presente documento presenta XF (Cross-Framework), un modelo de referencia arquitectónico agnóstico al marco de desarrollo que organiza la estructura interna de cualquier artefacto software mediante dos dimensiones ortogonales de clasificación: una dimensión de abstracción que estratifica el artefacto en tres capas derivadas de las etapas invariantes del proceso formal, y una dimensión funcional que clasifica los componentes en cinco tipos con responsabilidades precisas y mutuamente excluyentes. El modelo prescribe una taxonomía cerrada y exhaustiva de componentes, una nomenclatura canónica y una estructura de carpetas canónica, todas universales e independientes de la tecnología de implementación. El modelo introduce un marco formal de conformidad — un catálogo de reglas verificables organizadas en nueve grupos temáticos y un algoritmo determinista para calcular el nivel de conformidad de cualquier artefacto — que permite a las herramientas de análisis estático automatizado evaluar la corrección arquitectónica sin interpretación humana. Adicionalmente, la ontología formal del modelo actúa como un lenguaje de especificación funcional de alta precisión que reduce la ambigüedad en la descripción de requisitos y habilita la transformación algorítmica de especificaciones funcionales en implementaciones arquitectónicamente correctas y verificables. XF aborda los tres problemas que los modelos previos — Clean Architecture, Domain-Driven Design, arquitecturas en capas — resuelven solo parcialmente o de forma independiente: estructura de capas derivada de primeros principios, taxonomía cerrada y exhaustiva de componentes, y nomenclatura canónica que hace visible la convergencia estructural que el software ya implementa implícitamente.

Abstract

The rapid proliferation of programming languages, frameworks, and development tools has produced a highly fragmented software development ecosystem in which each technology imposes its own nomenclature, structural patterns, and organizational conventions. This fragmentation generates measurable operational costs — increased context-switching overhead, inconsistent implementations, and structural dependency on proprietary tools — despite the fact that all software artifacts implicitly implement the same three invariant stages of any formal process: interaction, processing, and access. This paper presents XF (Cross-Framework), a framework-agnostic architectural reference model that organizes the internal structure of any software artifact through two orthogonal classification dimensions: an abstraction dimension that stratifies the artifact into three layers derived from the invariant stages of formal processes, and a functional dimension that classifies components into five types with precise and non-overlapping responsibilities. The model

prescribes a closed and exhaustive component taxonomy, a canonical nomenclature, and a canonical folder structure that are universal and independent of the implementation technology. The model introduces a formal conformance framework — a catalog of verifiable rules organized in nine thematic groups and a deterministic algorithm for computing the conformance level of any artifact — that enables automated static analysis tools to evaluate architectural correctness without human interpretation. Additionally, the formal ontology of the model acts as a high-precision functional specification language that reduces ambiguity in requirements description and enables the algorithmic transformation of functional specifications into architecturally correct and verifiable implementations. XF addresses the three problems that prior models — Clean Architecture, Domain-Driven Design, layered architectures — solve only partially or independently: layer structure derived from first principles, closed and exhaustive component taxonomy, and canonical nomenclature that makes visible the structural convergence that software already implements implicitly.

Contents

1	Introducción	9
1.1	Motivación	9
1.2	Fundamentos del modelo	12
1.3	Relación con estándares y modelos de referencia existentes	14
1.4	Estructura del documento	18
2	Referencias normativas	20
3	Términos y abreviaturas	21
3.1	Términos	21
3.2	Abreviaturas	26
4	Convenciones normativas y alcance	27
4.1	Verbos auxiliares prescriptivos	27
4.2	Convenciones de notación	28
4.3	Distinción entre contenido normativo e informativo	29
4.4	Criterios de interpretación	30
4.5	Alcance del modelo	30
5	Fundamentos teóricos	31
5.1	El software como automatización de procesos formales	31
5.2	Etapas invariantes de un proceso formal	32
5.3	La tipificación de componentes como instrumento de comprensión	36
5.4	Limitaciones de la tipificación propietaria	38
5.5	Derivación del modelo de tres capas	40
6	Modelo de Arquitectura XF	44
6.1	Visión general	46
6.2	Principios rectores	48
6.2.1	Agnosticismo tecnológico	48

6.2.2	Aislamiento entre capas	50
6.2.3	Precedencia de la arquitectura sobre la herramienta	53
6.2.4	Tipificación cerrada y exhaustiva	56
7	Taxonomía de componentes	60
7.1	Estructura matricial: capas y tipos de componentes	60
7.2	Dimensión de abstracción: las capas	63
7.2.1	Capa de Acceso	64
7.2.2	Capa de Negocio	73
7.2.3	Capa de Interacción	83
7.3	Dimensión funcional: los tipos de componentes	95
7.3.1	Componentes lógicos	95
7.3.2	Componentes de generalización	99
7.3.3	Componentes de inyección	103
7.3.4	Componentes de utilidad	110
7.3.5	Componentes de transferencia	114
7.4	Estructura de carpetas canónica	120
8	Instanciación de la arquitectura	125
8.1	Punto de inicio de ejecución	126
8.2	Ciclo de vida del artefacto	127
8.3	Elemento de punto de inicio XF	134
9	Flujos de datos	137
9.1	Modelo de transmisión de información	137
9.2	Tipos de estructuras de datos	138
9.3	Unificación de estructuras de datos	139
9.4	Direccionalidad y transformación de los componentes de transferencia	140
9.5	Homogeneidad de las transferencias y vía de comunicación	141
10	Compatibilidad e interoperabilidad	145
10.1	Principio de compatibilidad	146
10.2	Mecanismos prescritos	149
10.2.1	Reformulación de funcionalidad no normalizada en XF	150
10.2.2	Inyección de componentes lógicos XF	152
10.2.3	Herencia de componentes	154
10.2.4	Uso directo	155
10.3	Ecosistema de librerías XF	156
11	Conformidad del modelo XF	158
11.1	Modelo de conformidad	158
11.1.1	Elemento verificable	159
11.1.2	Violación	159
11.1.3	Nivel de conformidad	160
11.1.4	Verificabilidad de las reglas	160
11.2	Niveles de conformidad	161
11.2.1	Nivel 0 — No conforme	161
11.2.2	Nivel 1 — Parcialmente conforme	161
11.2.3	Nivel 2 — Imperfectamente conforme	162

11.2.4	Nivel 3 — Estructuralmente conforme	162
11.2.5	Nivel 4 — Perfectamente conforme	162
11.3	Catálogo de reglas	163
11.3.1	Grupo 1 — Estructura de carpetas	165
11.3.2	Grupo 2 — Aislamiento entre capas	166
11.3.3	Grupo 3 — Componentes lógicos	167
11.3.4	Grupo 4 — Componentes de generalización	168
11.3.5	Grupo 5 — Componentes de inyección	169
11.3.6	Grupo 6 — Componentes de utilidad	170
11.3.7	Grupo 7 — Componentes de transferencia	171
11.3.8	Grupo 8 — Elemento punto de inicio XF	171
11.3.9	Grupo 9 — Exclusividad de la orquestación del ciclo de vida	172
11.3.10	Cierre del catálogo	173
11.4	Determinación del nivel de conformidad	173
11.4.1	Algoritmo de determinación	173
11.4.2	Aplicabilidad de reglas	174
12	Ontología de la arquitectura	175
12.1	Mapa conceptual	175
12.2	Diccionario de términos	175
13	Discusión	196
13.1	Comunicación y vocabulario común	196
13.2	Reducción de costes operativos	197
13.3	Normalización y estandarización	199
13.4	Aplicación en modelos generativos de código	200
13.5	Cierre de brecha conceptual entre humano y código generado	201
14	Conclusiones	204
14.1	Limitaciones del modelo	204
14.2	Validación empírica pendiente	205
14.3	Líneas de trabajo futuro	207
15	Bibliografía	209
A	Anexo A — Ejemplos de implementación: Capa de Acceso	212
A.1	Componentes de transferencia	213
A.2	Componentes de utilidad	213
A.3	Componentes de generalización	214
A.4	Componentes lógicos	214
A.5	Componente de inyección	216
B	Anexo B — Ejemplos de implementación: Capa de Negocio	216
B.1	Componentes de transferencia	216
B.2	Componentes de utilidad	217
B.3	Componentes de generalización	218
B.4	Componentes lógicos	218
B.5	Componente de inyección	220

C	Anexo C — Ejemplos de implementación: Capa de Interacción	221
C.1	Componentes de transferencia	221
C.2	Componentes de utilidad	221
C.3	Componentes de generalización	222
C.4	Componentes lógicos	222
C.5	Componente de inyección	224
C.6	Elemento de punto de inicio XF	224
D	Anexo D — Proyección de vocabularios externos en el modelo XF	225
D.1	Equivalencias con la nomenclatura de marcos de desarrollo	225
D.2	Equivalencias con patrones clásicos de inyección de dependencias	227
D.3	Proyección de arquitecturas de referencia	227
E	Anexo E — Equivalencias terminológicas con otros vocabularios	230
F	Anexo F — Historial de revisiones	232
	Índice alfabético	233

List of Tables

1	Verbos auxiliares prescriptivos	28
2	Estructura matricial $\mathbf{L} \times \mathbf{T}$ del modelo XF	61
3	Matriz $L \times T$ aplicada al artefacto termostato	63
4	Contraste: modelado tradicional frente a unificación de estructuras XF	120
5	Estructura de carpetas canónica \ matriz $L \times T$	123
6	Autómata del ciclo de vida del artefacto	134
7	Direccionalidad permitida de referencias a transferencias entre capas .	140
8	Transferencias recurrentes transmitidas como excepciones en artefactos XF	144
9	Niveles de integración entre artefactos	147
10	Matriz de casos de integración	149
11	Mecanismo prescrito según normalización en XF del integrado	150
12	Niveles de conformidad del modelo XF	163
13	Catálogo maestro de reglas de conformidad del modelo XF	164
14	Vectores de coste: mecanismo tradicional frente a mecanismo XF	198
15	Hipótesis de validación empírica del modelo XF	206
16	Equivalencias XF $\leftrightarrow$ marcos de desarrollo	226
17	Equivalencias XF $\leftrightarrow$ patrones DI clásicos	227
18	Proyección de Clean Architecture en XF	228
19	Proyección de Domain-Driven Design en XF	228
20	Proyección de la Arquitectura Hexagonal en XF	229
21	Proyección de la Arquitectura Onion en XF	229
22	Proyección de las arquitecturas en capas en XF	230
23	Equivalencias terminológicas con otros vocabularios	231
24	Historial de revisiones del modelo XF	232

List of Figures

1	El software como automatización de procesos formales	33
2	Las tres etapas invariantes del proceso formal	35
3	Ortogonalidad OSI–XF: comunicación entre artefactos frente a la estructura interna	43
4	Estratificación de tres capas del artefacto XF	45
5	Comunicación ascendente mediante patrón observador	53
6	Causalidad invertida: tradicional frente a XF	56
7	Contenido del componente lógico de Acceso (Repository)	68
8	Contenido del componente lógico de Negocio (Business)	77
9	Contenido del componente lógico de Interacción (servicio o vista) . .	87
10	Autómata del ciclo de vida del artefacto	133
11	Árbol de decisión para la determinación del nivel de conformidad . .	174
12	Mapa conceptual del modelo XF	176
13	Cierre de brecha humano–código generado mediante estructura canónica XF	203

Índice de códigos

1	Ausencia de subdivisión en la Capa de Acceso	70
2	Organización interna de la Capa de Acceso	71
3	Árbol de herencia de la Capa de Acceso	71
4	Ausencia de subdivisión en la Capa de Negocio	80
5	Organización interna de la Capa de Negocio	80
6	Patrones canónicos de generalización en la Capa de Negocio	81
7	Ausencia de subdivisión en la Capa de Interacción	91
8	Organización interna de la Capa de Interacción	92
9	Patrones canónicos de generalización en la Capa de Interacción	93
10	Estructura interna de un componente lógico	97
11	Organización en carpetas: componente lógico	99
12	Atributos mutables y estado	100
13	Organización en carpetas: componente de generalización	103
14	Ciclo de vida (I): instanciación inicial de los componentes lógicos . . .	105
15	Ciclo de vida (II): composición completa con la operación init() . . .	105
16	Ciclo de vida (III): terminación en orden inverso	106
17	Patrón de acceso canónico: forma general	106
18	Patrón de acceso canónico: ejemplo de uso	106
19	Organización en carpetas: componente de inyección	110
20	Alcance local y la excepción de los tipos primitivos	112
21	Organización en carpetas: componente de utilidad	114
22	Componente de transferencia con operaciones autocontenidas	115
23	Transferencia definida en la Capa de Acceso	117
24	Transformación estructural de la transferencia en la Capa de Interacción	117
25	Organización en carpetas: componente de transferencia	119
26	Estructura prescrita	121
27	Raíces de artefacto múltiples	124
28	Fase 0 — Instanciación	128

29	Fase 1 — Inicialización: orden ascendente entre capas	129
30	Fase 1 — Inicialización: cuerpo típico de la operación <code>init()</code>	129
31	Fase 3 — Terminación: orden descendente entre capas	131
32	Fase 3 — Terminación: cuerpo típico de la operación <code>terminate()</code> . . .	131
33	Ubicación canónica	135
34	Operaciones obligatorias	135
35	Contacto por delegación en atributo	151
36	Contacto por decoración con anotación del marco de desarrollo	151
37	Contacto por herencia de una clase base del marco de desarrollo . . .	152
38	Inyección por referencia de componentes lógicos ajenos (estado com- partido)	153
39	Inyección por instanciación de un componente lógico ajeno (estado aislado)	154
40	Herencia de una generalización XF publicada por una librería	154
41	Herencia de una utilidad XF publicada por una librería	154
42	Uso directo de una transferencia y una utilidad XF publicadas por una librería	155
43	Componentes de transferencia de la Capa de Acceso	213
44	Componente de utilidad de Acceso sobre tipos primitivos	213
45	Componente de generalización: <code>RemoteRepository</code>	214
46	Componentes lógicos de Acceso: <code>IdentityRepository</code> y <code>ServerRepository</code>	215
47	Componente de inyección de la Capa de Acceso: <code>R</code>	216
48	Componentes de transferencia de la Capa de Negocio	217
49	Componente de utilidad de Negocio: <code>TemperatureUtils</code>	217
50	Componente de generalización: <code>StatefulBusiness<T></code>	218
51	Componente lógico de Negocio: <code>SessionBusiness</code>	218
52	Componente lógico de Negocio: <code>TemperatureBusiness</code>	219
53	Componente lógico de Negocio: <code>UserBusiness</code>	220
54	Componente de inyección de la Capa de Negocio: <code>B</code>	220
55	Componentes de transferencia de la Capa de Interacción	221
56	Componente de utilidad de Interacción: <code>FormatUtils</code>	222
57	Componente de generalización: <code>RestService</code>	222
58	Componente lógico de Interacción: <code>TemperatureService</code>	222
59	Componente lógico de Interacción: <code>MainView</code>	223
60	Componente de inyección de la Capa de Interacción: <code>A</code>	224
61	Elemento de punto de inicio: <code>XF</code>	224

1 Introducción

Las cláusulas internas tienen carácter informativo. Su propósito es exponer la motivación, los fundamentos conceptuales y la relación del modelo XF con los estándares y modelos de referencia existentes. El contenido normativo del modelo se establece en las cláusulas 2, 3, 4 y 6 a 12.

El presente documento establece las directrices normativas del modelo de Arquitectura *Cross-Framework* (en adelante, **XF**), un modelo de referencia para la estructuración de artefactos software con independencia del lenguaje de programación, el marco de desarrollo o el entorno de ejecución empleados.

El modelo XF define una taxonomía de componentes, una estratificación jerárquica en capas de abstracción y un conjunto de reglas de conformidad que permiten organizar de manera uniforme la lógica de cualquier artefacto software, desde aplicaciones móviles y servicios web hasta sistemas embebidos, dispositivos de computación periférica y soluciones de automatización industrial. El modelo no prescribe implementaciones concretas ni restringe el uso de marcos de desarrollo existentes; en cambio, provee una capa de abstracción conceptual que permite expresar la estructura de cualquier artefacto en términos universalmente comprensibles, con independencia de la tecnología subyacente.

El alcance de este documento comprende:

- La definición formal de las capas de abstracción que componen la Arquitectura XF y las responsabilidades de cada una.
- La tipificación exhaustiva de los componentes software admitidos por el modelo y los criterios de clasificación aplicables.
- Las reglas de conformidad que determinan el grado de ajuste de un artefacto al modelo.
- Los mecanismos de instanciación, inicialización y terminación de la arquitectura.
- Las reglas de compatibilidad e interoperabilidad con artefactos que no implementen el modelo XF.
- La ontología de términos y conceptos del dominio de la Arquitectura XF.

Quedan fuera del alcance de este documento la definición de metodologías de gestión de proyectos software, los criterios de calidad del código fuente más allá de la conformidad arquitectónica, las especificaciones de implementación propias de cada marco de desarrollo, y los procedimientos de certificación de personas o artefactos, que serán objeto de documentos complementarios.

1.1 Motivación

La disciplina de la ingeniería del software se caracteriza por una diversidad tecnológica en constante expansión. La aparición sostenida de nuevos marcos de desarrollo, lenguajes de programación y herramientas de construcción ha dado lugar a un ecosistema altamente heterogéneo en el que cada tecnología impone su propia nomenclatura, sus propios patrones estructurales y sus propias convenciones de organización del código.

Antes de describir las consecuencias de esta heterogeneidad, conviene enunciar una

observación central: **la fragmentación de la industria es, antes que nada, un problema terminológico y estructural.**

En la mayoría de los artefactos software que resuelven un problema real con valor de uso, la tripartición que el modelo XF formaliza emerge de forma implícita: el artefacto ejecuta una lógica de procesamiento como núcleo necesario y, cuando se comunica con su entorno — el caso típico de los sistemas de gestión, transacción y automatización —, lo hace por dos únicas vías estructurales: accede a datos externos cuando los necesita y expone los resultados a sus consumidores cuando recibe invocación.

Fowler [12] documentó que los mismos patrones arquitectónicos emergen de forma recurrente en sistemas *enterprise* heterogéneos, independientemente del lenguaje o marco de desarrollo empleado. Esta observación constituye evidencia de que la estructura subyacente del software converge hacia soluciones comunes que la diversidad terminológica de la industria ha impedido reconocer como equivalentes. A título ilustrativo e informativo: Spring llama `@Service` a lo que Angular llama `Injectable`, a lo que Flutter llama `BLoC`, a lo que Android llama `ViewModel` — cuatro nombres distintos que designan en cada marco el mismo rol arquitectónico. Del mismo modo, Spring llama `@Repository` a lo que Django llama `QuerySet`, a lo que Node.js llama `Model`, a lo que J2EE llama `DAO` — cuatro nombres distintos que cumplen el mismo rol arquitectónico en cada marco.

La práctica industrial documentada sugiere que el software, en su naturaleza profunda, se ajusta al modelado formal de procesos. Lo que no es uniforme es la forma en que esa conformidad se expresa — cada marco de desarrollo, cada equipo y cada proyecto lo nombra, lo estructura y lo organiza de forma distinta. La consecuencia no es que el software esté mal construido — es que no puede ser leído, comprendido ni mantenido con la misma facilidad por desarrolladores que provienen de contextos tecnológicos distintos. La fragmentación no es un problema del fondo del software sino de su superficie: la nomenclatura, la estratificación y el vocabulario con el que se describe y se comunica.

Esta observación tiene una implicación directa sobre el alcance del problema que el modelo XF aborda. No se trata de corregir cómo el software funciona — se trata de hacer explícito y uniforme lo que el software ya hace implícitamente. XF no impone al desarrollador una forma nueva de construir software — le provee un vocabulario común y una estructura prescrita para describir lo que ya construye, de forma que cualquier otro desarrollador formado en el modelo pueda reconocer los mismos conceptos con los mismos nombres, independientemente de la tecnología empleada.

Esta heterogeneidad genera consecuencias operativas medibles en las organizaciones que desarrollan y mantienen software:

Fragmentación del conocimiento. Los desarrolladores especializados en un marco de desarrollo encuentran dificultades para operar con eficacia en otro, aunque los procesos de negocio subyacentes sean equivalentes. Conceptos arquitectónicamente idénticos reciben denominaciones distintas según la tecnología empleada, mientras que un mismo término puede designar conceptos radicalmente diferentes en marcos distintos: el término `Service` hace referencia a la lógica de negocio en Spring, a

una tarea asíncrona en Android y a un punto de interacción sistémica en el modelo XF. Evans [11] introdujo el concepto de *lenguaje ubicuo* como condición necesaria para la comunicación precisa entre los miembros de un equipo de desarrollo, e identificó su ausencia como una de las causas principales de la ambigüedad semántica en proyectos software — un diagnóstico que el modelo XF extiende al nivel de la industria: la fragmentación terminológica entre marcos de desarrollo impide que los desarrolladores reconozcan como equivalentes los conceptos que ya implementan de forma convergente. Esta inconsistencia semántica constituye una barrera estructural para la transferencia de conocimiento entre proyectos, equipos y organizaciones, y confina el conocimiento en silos tecnológicos que dificultan la movilidad interna y encarecen la incorporación de nuevos miembros al equipo.

Incremento del coste de cambio de contexto. Cada transición entre proyectos con tecnologías distintas impone una curva de aprendizaje que no responde a la complejidad del dominio de negocio, sino a las particularidades de la herramienta. Un desarrollador con sólidos conocimientos de arquitectura de software debe invertir tiempo y recursos en aprender las convenciones propietarias de cada marco antes de poder contribuir de manera efectiva, independientemente de que los problemas que debe resolver sean funcionalmente análogos a los que ya ha resuelto en otras tecnologías. Este coste no aporta valor al producto y constituye una ineficiencia estructural del proceso de desarrollo, replicada de forma sistemática en toda la industria.

Inconsistencia en la implementación. La ausencia de un modelo de referencia común conduce a que distintos equipos resuelvan problemas equivalentes con enfoques divergentes dentro de una misma organización, e incluso dentro de un mismo proyecto cuando los equipos han sido formados en tecnologías distintas. Jones [25] identificó la inconsistencia en las convenciones de diseño como uno de los factores con mayor impacto negativo en la velocidad de desarrollo y en la densidad de defectos del producto — consecuencia directa de la fragmentación terminológica que impide a los equipos reconocer y reutilizar soluciones arquitectónicas equivalentes. Esta divergencia dificulta la revisión del código, la auditoría de la solución, la estimación del esfuerzo de mantenimiento y la incorporación de nuevos desarrolladores, que deben comprender no solo el dominio de negocio, sino también las decisiones arquitectónicas adoptadas en cada proyecto.

Dependencia estructural de la herramienta. En el modelo de desarrollo predominante, la arquitectura del artefacto queda supeditada al marco de desarrollo elegido. Las decisiones arquitectónicas no derivan del análisis del proceso que se automatiza, sino de las convenciones impuestas por la herramienta, generando tantas formas de trabajo como herramientas existen en el mercado. Booch [6] estableció que la arquitectura de un sistema debe ser comprensible para todos sus participantes, no solo para sus creadores — una propiedad que la fragmentación terminológica de la industria compromete sistemáticamente al hacer que la misma estructura arquitectónica sea irreconocible para desarrolladores formados en tecnologías distintas. Esta dependencia tiene implicaciones a largo plazo: cuando una tecnología queda obsoleta o es reemplazada, el conocimiento arquitectónico acumulado en torno a ella no es directamente transferible a la nueva herramienta, obligando a las organizaciones a asumir un coste de migración que va más allá del código fuente y afecta a los procesos, la formación y la documentación.

Dificultad de escalabilidad y mantenimiento. Los proyectos software deben evolucionar con el tiempo para adaptarse a nuevos requisitos de negocio, a cambios en las integraciones con terceros y al crecimiento del equipo de desarrollo. La diversidad arquitectónica dificulta esta evolución, ya que las diferencias en la gestión de dependencias, la configuración y la estructura del proyecto generan inconsistencias que se amplifican con la escala.

Estas consecuencias no son independientes entre sí — se refuerzan mutuamente y producen un efecto acumulativo que impacta negativamente en la velocidad de desarrollo, la calidad del software producido y la capacidad de las organizaciones para mantener y evolucionar sus sistemas a lo largo del tiempo. El presente documento parte de la hipótesis de que comparten una causa raíz común: no que el software esté mal construido, sino que el mismo software bien construido se describe, se nombra y se estructura de formas incompatibles entre sí.

En disciplinas de ingeniería consolidadas — civil, eléctrica, mecánica — la existencia de estándares normativos que regulan la terminología, los procesos y las interfaces entre componentes ha sido factor del avance sistemático de la disciplina y de la reducción sostenida de sus costes de producción [7]. Un ingeniero civil no concibe la construcción de una edificación al margen de la normativa aplicable; un ingeniero eléctrico no diseña un sistema de distribución sin atenerse a los esquemas y protocolos de seguridad establecidos. La ingeniería del software, pese a su madurez relativa como disciplina, carece aún de un equivalente en el ámbito de la arquitectura de aplicaciones. El modelo XF constituye un paso en esa dirección — no para cambiar cómo se construye el software, sino para hacer uniforme y explícito lo que el software ya hace.

1.2 Fundamentos del modelo

El modelo XF se fundamenta en dos premisas de carácter universal, derivadas de la teoría del modelado formal de procesos y de los principios de abstracción por capas del modelo OSI [17]. Ambas premisas son observaciones sobre propiedades intrínsecas del software — no son convenciones que el modelo impone sino realidades que el modelo formaliza y hace explícitas.

Primera premisa: todo artefacto software ya automatiza un proceso formal

Todo proceso formal automatizable por software contiene necesariamente una etapa de *procesamiento* — la ejecución ordenada de los procedimientos que producen el resultado esperado — y, cuando se comunica con su entorno, lo hace exclusivamente por dos vías estructuralmente distintas: *interacción*, cuando el proceso recibe la orden de ejecución, verifica sus precondiciones o consume datos del exterior, y *acceso*, cuando el proceso delega parte de su ejecución en sistemas externos a través de un protocolo de comunicación definido. Estas dos vías son opcionales según el proceso — existen procesos puramente computacionales sin comunicación externa, procesos disparados internamente sin interacción y procesos autocontenidos sin acceso — pero son **exhaustivas**: no existe una tercera vía estructural de comunicación con el entorno. Esta propiedad — procesamiento necesario, comunicación exhaustiva en sus dos formas — es la que el modelo XF formaliza como las tres capas de abstracción del artefacto. El estándar BPMN 2.0 [35] estructura los procesos de negocio con la misma

tripartición — *Activities, catch Events, throw Events* — y el cálculo de procesos comunicantes CSP [16] la formaliza matemáticamente. La derivación rigurosa y las tradiciones formales que la sostienen se desarrollan en §5.2.

Todo artefacto software que resuelve un problema con valor de uso implementa el procesamiento como núcleo necesario y, en la mayoría de los casos de interés práctico — aplicaciones de gestión, servicios de negocio, sistemas de automatización —, también las dos vías de comunicación con el entorno. Una aplicación de gestión de temperatura accede a datos del termostato, los procesa según las reglas del dominio y los expone al usuario — acceso, procesamiento, interacción —. Un microservicio de autenticación consulta credenciales en una base de datos, verifica su validez y devuelve un token — acceso, procesamiento, interacción —. Un script de automatización lee ficheros de configuración, transforma los datos y escribe los resultados — acceso, procesamiento, interacción —. El proceso formal subyacente sigue el mismo patrón estructural en los tres casos. Lo que varía es el nombre que cada marco de desarrollo le da a cada etapa y la forma en que la organiza en su estructura.

Parnas [39] estableció que la estructura de un sistema debe reflejar las decisiones de diseño que tienen mayor probabilidad de cambiar, y que estas decisiones están determinadas por la naturaleza del proceso que el sistema implementa. Esta observación anticipa la primera premisa del modelo XF: si la naturaleza del proceso determina la estructura del sistema, y si la tripartición procesamiento–interacción–acceso captura las únicas categorías estructurales posibles de un proceso formal, entonces la estructura de todo artefacto software debería converger hacia una organización común — independientemente de la tecnología. La fragmentación que documenta §1.1 no contradice esta convergencia estructural — la confirma: los marcos de desarrollo convergen hacia las mismas soluciones porque los procesos que modelan tienen la misma estructura subyacente, pero lo hacen con nomenclaturas distintas que ocultan esa convergencia.

Brooks [7] identificó la complejidad accidental — aquella introducida por las herramientas y los procesos de desarrollo, no por el problema en sí — como una de las causas principales de los fallos en proyectos software. La fragmentación terminológica de la industria es precisamente ese tipo de complejidad: no deriva del problema que el software resuelve sino de la forma en que las herramientas han elegido describirlo. El modelo XF elimina esa complejidad accidental al proveer una terminología única para describir lo que el software ya hace — haciendo visible la convergencia estructural que los marcos de desarrollo implementan implícitamente.

Segunda premisa: la arquitectura debe preceder a la herramienta Si todo artefacto software implementa el procesamiento como núcleo necesario y, cuando comunica con el entorno, lo hace por las dos vías exhaustivas del proceso formal, entonces la estructura del artefacto debe derivar de esa tripartición — no de las convenciones del marco de desarrollo. Esta premisa invierte el paradigma predominante en la industria: en lugar de que la herramienta imponga la arquitectura, el desarrollador modela la arquitectura en función de las propiedades del proceso que automatiza y utiliza la herramienta para implementarla. El marco de desarrollo deja de ser el origen de la arquitectura y se convierte en su instrumento.

Martin [30] identifica el acoplamiento de la arquitectura a la herramienta como uno de

los antipatrones más perjudiciales en el diseño de sistemas software, argumentando que una arquitectura que no puede ser descrita independientemente de sus herramientas de implementación ha perdido su función de modelo conceptual. El modelo XF aborda este acoplamiento de forma directa: al derivar su estructura de las propiedades invariantes del proceso formal — universales y tecnológicamente neutras — provee una arquitectura que puede ser descrita, comprendida y comunicada con independencia de cualquier tecnología de implementación.

Esta inversión es posible precisamente porque la tripartición procesamiento–interacción–acceso es estructuralmente universal: el procesamiento como núcleo necesario y las dos vías de comunicación con el entorno como categorías exhaustivas existen en todo artefacto software que las requiera, independientemente de la tecnología. Un desarrollador que conoce el modelo XF puede trabajar en Spring, Angular, Flutter o cualquier otro marco de desarrollo porque reconoce en cada uno de ellos la misma estructura subyacente, expresada con nomenclatura distinta. La curva de aprendizaje de una nueva tecnología se reduce a aprender la nomenclatura propietaria del marco de desarrollo — no a aprender una nueva forma de estructurar el software. Esta propiedad — la transferibilidad del conocimiento arquitectónico entre tecnologías — es un beneficio operativo central del modelo y una consecuencia directa de que la arquitectura preceda a la herramienta.

Derivación del modelo El modelo XF parte de estas dos premisas para establecer que existe una estructura arquitectónica universal — compuesta por tres capas de abstracción que corresponden biunívocamente a las tres etapas invariantes de todo proceso formal — y cuya taxonomía de componentes se define con independencia de cualquier tecnología de implementación. La Capa de Acceso modela la etapa de acceso, la Capa de Negocio modela la etapa de procesamiento y la Capa de Interacción modela la etapa de interacción. Esta estructura no es una invención del modelo — es la formalización explícita de lo que el software ya hace implícitamente. El modelo la nombra, la prescribe y la hace uniforme.

La derivación formal de estas capas a partir de las propiedades estructurales invariantes de los procesos formales se desarrolla en §5. Su definición normativa completa — principios rectores, taxonomía de componentes y estructura de carpetas canónica — se desarrolla en §6.

1.3 Relación con estándares y modelos de referencia existentes

El modelo XF no emerge de forma aislada — se posiciona en continuidad con una tradición consolidada de modelos de referencia y patrones arquitectónicos que han abordado, desde distintas perspectivas y con distintos alcances, el problema de la organización estructural del software. Esta cláusula establece con precisión la relación del modelo XF con los estándares y modelos más relevantes, distinguiendo en cada caso qué aporta XF que esos modelos no aportan y en qué aspectos XF los extiende, generaliza o complementa.

Modelo OSI — [17] El modelo de interconexión de sistemas abiertos es un antecedente formal directo del modelo XF y la referencia normativa sobre la que se

fundamenta su derivación. El modelo OSI organiza los protocolos de comunicación en red mediante siete capas de abstracción con responsabilidades claramente delimitadas, donde cada capa provee servicios a la capa inmediatamente superior y consume los servicios de la capa inmediatamente inferior. Su contribución fundamental a la industria no fue técnica sino semántica: al definir un vocabulario común para describir la comunicación entre sistemas heterogéneos, permitió la interoperabilidad entre implementaciones tecnológicamente incompatibles sin necesidad de que cada implementación conociera los detalles internos de las demás.

El modelo XF extiende el principio de estratificación del modelo OSI a una dimensión ortogonal: mientras OSI estandariza la comunicación *entre* artefactos, XF estandariza la estructura *interna* de cada artefacto. Las dos estratificaciones son técnicamente independientes y conceptualmente complementarias. La extensión preserva las cuatro propiedades formales del modelo OSI — encapsulación de servicios mediante primitivas, dependencia estrictamente unidireccional entre capas, aislamiento de implementación y composición de servicios — aplicadas a la nueva dimensión. El interior de las aplicaciones, espacio que el modelo OSI deja explícitamente fuera de su alcance y que ha permanecido históricamente sin un marco de referencia equivalente, es el dominio que el modelo XF formaliza. La derivación técnica detallada se desarrolla en §5.5.

La analogía entre XF y OSI no es solo estructural sino también metodológica. El modelo OSI demostró que la definición de capas con responsabilidades claramente delimitadas e interfaces bien definidas entre ellas permite la interoperabilidad entre implementaciones heterogéneas. El modelo XF aplica ese mismo principio al interior de las aplicaciones: al definir capas con responsabilidades claramente delimitadas, permite que distintos equipos, tecnologías y marcos de desarrollo colaboren sobre una estructura conceptual común.

Clean Architecture — [30] La Clean Architecture de Robert C. Martin establece el principio de inversión de dependencias como fundamento de una arquitectura que separa la lógica de negocio de los detalles de implementación — marcos de desarrollo, bases de datos, interfaces de usuario. Su contribución central es la regla de dependencia: las dependencias del código fuente deben apuntar siempre hacia el interior — hacia las políticas de negocio — y nunca hacia el exterior — hacia los detalles de implementación.

El modelo XF comparte este principio y lo formaliza en el principio de aislamiento entre capas (§6.2.2) y en el principio de precedencia de la arquitectura sobre la herramienta (§6.2.3). XF extiende el alcance de Clean Architecture en dos aspectos. El primero es la **universalidad de aplicación**: Clean Architecture define una estructura de capas concéntricas — entidades, casos de uso, adaptadores, marcos de desarrollo — que, aunque agnóstica en teoría, ha sido adoptada principalmente en el contexto del desarrollo backend orientado a objetos. El modelo XF prescribe una estructura que aplica con la misma fuerza a artefactos frontend, backend, móviles, embebidos y de cualquier otro tipo. El segundo es la **taxonomía de componentes**: Clean Architecture no prescribe una nomenclatura de componentes — deja a cada equipo definir la suya dentro de la estructura de capas. El modelo XF prescribe una taxonomía cerrada y exhaustiva que cierra el espacio de decisión nominal que Clean Architecture deja al equipo.

Arquitecturas estratificadas — [3] Las arquitecturas estratificadas establecen el principio de organización del software en capas horizontales con responsabilidades diferenciadas, donde cada capa provee servicios a la capa superior y consume los de la inferior. Su contribución es la formalización del concepto de abstracción por capas como instrumento de separación de responsabilidades en sistemas software.

El modelo XF es una especialización de las arquitecturas estratificadas — adopta su principio de organización en capas y lo instancia en tres capas concretas derivadas de las etapas invariantes de los procesos formales. La diferencia fundamental es que las arquitecturas estratificadas son un patrón estructural genérico — no prescriben cuántas capas debe tener un sistema ni qué responsabilidades debe tener cada una. El modelo XF prescribe exactamente eso: tres capas con responsabilidades normativas, derivadas de primeros principios del modelado formal de procesos.

Arquitectura Hexagonal — [8] La Arquitectura Hexagonal de Alistair Cockburn — también conocida como *Ports and Adapters* — introduce el principio de aislamiento del núcleo de la aplicación respecto a sus interfaces externas mediante puertos abstractos que el núcleo expone y adaptadores concretos que los implementan. Su contribución es la formalización explícita de la dirección de las dependencias entre dominio e infraestructura: el dominio define el contrato, los adaptadores lo satisfacen.

El modelo XF comparte con la Arquitectura Hexagonal la separación entre la lógica de negocio y los mecanismos de comunicación con el exterior. La Capa de Negocio cumple el papel del núcleo de aplicación, y la Capa de Acceso instancia el rol de los adaptadores *driven*. La diferencia fundamental es que el modelo Hexagonal no prescribe la dirección de entrada al sistema — *driving adapters* — como una capa diferenciada con identidad propia: la concentra junto al resto de adaptadores bajo una distinción binaria entrada/salida. XF separa explícitamente la dirección de entrada como Capa de Interacción, derivándola de la propiedad de los procesos formales según la cual la invocación y la presentación de resultados constituyen una etapa estructuralmente distinta del acceso a sistemas externos.

Onion Architecture — [38] La Arquitectura Onion de Jeffrey Palermo organiza el sistema en anillos concéntricos donde las dependencias apuntan siempre hacia el centro — el dominio — y nunca hacia los anillos externos. Su contribución es la visualización topológica de la regla de dependencias que también articula la Clean Architecture y, en su estratificación lineal, el modelo XF.

El modelo XF y la Arquitectura Onion comparten la dirección de las dependencias y la centralidad del dominio, pero difieren en la forma de la estructura y en la prescripción de tipos. La Onion no prescribe taxonomía cerrada de componentes ni nomenclatura canónica, ni define cómo se materializan los anillos en una estructura de carpetas verificable. El modelo XF prescribe ambas, manteniendo la dirección de dependencias que la Onion captura visualmente.

Domain-Driven Design — [11] El Domain-Driven Design introduce una rica terminología para modelar dominios de negocio complejos — Aggregates, Repositories, Domain Services, Application Services, Bounded Contexts — y establece el concepto de Ubiquitous Language como condición necesaria para la comunicación precisa entre

desarrolladores y expertos de dominio. Su contribución semántica al modelado del dominio de negocio es considerable y ha influido en la forma en que la industria piensa sobre la separación entre lógica de dominio e infraestructura.

La relación entre DDD y XF es de complementariedad, no de competencia. DDD opera principalmente en el espacio de la Capa de Negocio — sus conceptos de **Repository**, **Domain Service** y **Application Service** tienen correspondencia directa con los componentes lógicos de acceso y negocio del modelo. La compatibilidad se materializa específicamente dentro de `/business/logic/instance`, región del artefacto donde el modelo reconoce el dominio funcional como criterio legítimo de subdivisión (§7.2.2). La organización por subdominios o por contextos delimitados que DDD prescribe encuentra ahí su lugar natural; las capas de Acceso e Interacción se organizan en cambio por criterios técnicos ortogonales al dominio — protocolo y tipo de entrada respectivamente —, de forma que se evita la replicación de resolución técnica que una subdivisión por dominio en todas las capas introduciría. Adicionalmente, DDD no prescribe cómo organizar las capas de acceso e interacción ni provee una taxonomía de componentes aplicable a artefactos frontend, móviles o embebidos. XF extiende el alcance de la estandarización semántica más allá del dominio de negocio — al conjunto completo del artefacto — y lo hace de forma tecnológicamente agnóstica, sin los supuestos de complejidad de dominio que limitan la adopción de DDD a sistemas enterprise de cierta escala.

Patrones de diseño — [14] Los patrones de diseño del Gang of Four describen soluciones recurrentes a problemas de diseño local dentro de componentes o conjuntos de componentes relacionados. Su contribución es un vocabulario compartido para describir soluciones de diseño a nivel de componente — **Factory**, **Observer**, **Strategy**, **Repository**.

El modelo XF no es un patrón de diseño en el sentido de Gamma et al. — no describe una solución a un problema de diseño local. Es un modelo de referencia que opera a un nivel de abstracción superior: prescribe cómo se organiza el conjunto del artefacto, no cómo se resuelve un problema de diseño específico dentro de él. Los patrones de diseño son compatibles con XF — pueden aplicarse dentro de los componentes lógicos, de generalización o de utilidad de cualquier capa — pero son ortogonales a él: su aplicación no afecta a la conformidad con el modelo.

La contribución diferencial del modelo XF Ninguno de los modelos y estándares descritos en esta cláusula aborda simultáneamente los tres problemas que el modelo XF resuelve. El modelo OSI estandarizó la comunicación entre sistemas pero no el interior de las aplicaciones. Las arquitecturas estratificadas, la Hexagonal, la Onion y Clean Architecture estandarizaron la organización en capas y la dirección de las dependencias, pero no la taxonomía de componentes ni la nomenclatura. DDD estandarizó la semántica del dominio de negocio, pero no la organización del conjunto del artefacto ni su aplicabilidad a tecnologías heterogéneas. Los patrones de diseño estandarizaron soluciones locales pero no la estructura global del artefacto.

El modelo XF aborda los tres problemas simultáneamente y de forma integrada: prescribe una estructura de capas derivada de primeros principios, una taxonomía de componentes cerrada y exhaustiva, y una nomenclatura canónica tecnológicamente

agnóstica. Esta integración es lo que permite al modelo resolver el problema que ninguno de sus predecesores resolvía por separado: hacer visible y uniforme la convergencia estructural que el software ya implementa implícitamente, con independencia de la tecnología empleada.

1.4 Estructura del documento

El presente documento se organiza del siguiente modo.

La **cláusula 2** (Referencias normativas) establece las referencias normativas externas sobre las que se fundamenta el modelo — estándares y documentos cuya aplicación es indispensable para la correcta interpretación de las prescripciones del presente documento.

La **cláusula 3** (Términos y abreviaturas) define los términos y abreviaturas empleados en el documento con una acepción específica en el contexto del modelo XF. Los términos aquí definidos constituyen el vocabulario normativo del modelo y deben interpretarse con la precisión que se les asigna en esta cláusula.

La **cláusula 4** (Convenciones normativas) establece las convenciones normativas empleadas en la redacción del documento — el uso de los verbos auxiliares prescriptivos, la distinción entre requisitos obligatorios y recomendaciones, y los criterios de interpretación aplicables en caso de ambigüedad.

La **cláusula 5** (Fundamentos teóricos) expone los fundamentos teóricos del modelo con carácter informativo. Establece el razonamiento que justifica las decisiones normativas de las cláusulas siguientes — el principio de isomorfismo entre proceso y artefacto, la derivación de las tres etapas invariantes de todo proceso formal, la tipificación de componentes como instrumento de comprensión y las limitaciones de la tipificación propietaria — y deriva formalmente el modelo de tres capas a partir de las propiedades estructurales invariantes de los procesos formales.

La **cláusula 6** (Modelo de Arquitectura XF) define el modelo en sus términos normativos: presenta su visión general y establece los cuatro principios rectores — agnosticismo tecnológico, aislamiento entre capas, precedencia de la arquitectura sobre la herramienta y tipificación cerrada y exhaustiva.

La **cláusula 7** (Taxonomía de componentes) organiza todo artefacto en la matriz de clasificación de dos dimensiones. Describe la estructura matricial de capas y tipos; la *dimensión de abstracción*, que define las tres capas — Acceso, Negocio e Interacción — con su responsabilidad, sus límites y el modelo de proceso de su lógica interna; la *dimensión funcional*, que define exhaustivamente los cinco tipos de componentes reconocidos por el modelo; y la estructura de carpetas canónica que materializa la matriz en el sistema de ficheros.

La **cláusula 8** (Instanciación de la arquitectura) especifica el mecanismo de instanciación de la arquitectura. Describe el punto de inicio de ejecución del artefacto, el ciclo de vida completo con sus cuatro fases — instanciación, inicialización, ejecución y terminación —, la instanciación inicial de los componentes lógicos por parte del componente de inyección (que establece el estado inicial σ_0 de cada lógico por construcción del singleton, no por invocación) y las dos operaciones invocables — inicialización y terminación —, el orden de inicialización derivado del grafo de

dependencias entre componentes y el elemento **XF** como punto de control centralizado de la inicialización.

La **cláusula 9** (Flujos de datos) describe el tratamiento de los datos en la arquitectura XF. Establece que las Transferencias constituyen un dominio único y homogéneo cuya transmisión se realiza por la vía de comunicación de las operaciones, materializada en los constructos sintácticos nativos del lenguaje de implementación, sin que el modelo defina mecanismos de transmisión propios. Formaliza los tipos de estructuras de datos que el modelo reconoce, establece la diferencia con el modelo OSI en cuanto al tratamiento de los datos entre capas, desarrolla el principio de unificación de estructuras de datos y la regla de transformación estructural, y prescribe las restricciones de direccionalidad de los componentes de transferencia entre capas.

La **cláusula 10** (Compatibilidad e interoperabilidad) establece las reglas de compatibilidad e interoperabilidad del modelo. Define el principio de retrocompatibilidad bidireccional, formaliza los dos niveles de integración entre artefactos — a nivel de aplicación y a nivel de artefacto —, desarrolla los patrones de integración para las cuatro combinaciones posibles entre artefactos XF y tradicionales y describe el ecosistema de librerías XF como consecuencia natural de las formas de integración.

La **cláusula 11** (Conformidad del modelo XF) define el modelo de conformidad del modelo XF. Introduce los conceptos específicos de la conformidad — elemento verificable, violación y nivel de conformidad —, establece los cinco niveles de conformidad discretos, presenta el catálogo exhaustivo de reglas verificables organizadas en nueve grupos temáticos y define el algoritmo de determinación del nivel de conformidad con su expresión formal.

La **cláusula 12** (Ontología de la arquitectura) presenta la ontología de la arquitectura XF. Incluye el mapa conceptual del modelo y el diccionario de términos con la definición precisa de cada concepto del modelo — tanto los términos propios de XF como aquellos términos de uso general en ingeniería del software a los que el modelo da una acepción normativa exacta.

La **cláusula 13** (Discusión) contrasta el modelo con el estado del arte, analiza sus limitaciones y desarrolla sus implicaciones para la generación automática de código y el *Specification-Driven Development*. La **cláusula 14** (Conclusiones) sintetiza la contribución del modelo, enuncia las hipótesis pendientes de validación empírica y establece las líneas de trabajo futuro.

El documento se completa con seis anexos, todos informativos.

El **Anexo A** (Ejemplos de implementación: Capa de Acceso) ilustra mediante código real cómo se materializan en una tecnología concreta los componentes prescritos para la Capa de Acceso.

El **Anexo B** (Ejemplos de implementación: Capa de Negocio) ilustra la materialización equivalente para la Capa de Negocio.

El **Anexo C** (Ejemplos de implementación: Capa de Interacción) ilustra la materialización equivalente para la Capa de Interacción, distinguiendo entre componentes de servicio y componentes de vista.

El **Anexo D** (Proyección de vocabularios externos en el modelo XF) formaliza la

correspondencia entre el vocabulario del modelo y el de las arquitecturas de referencia previas (Clean Architecture, Domain-Driven Design, Hexagonal y Onion) y de los marcos de desarrollo más adoptados.

El [Anexo E](#) (Equivalencias terminológicas con otros vocabularios) presenta la tabla de equivalencias entre términos canónicos del modelo XF y los términos análogos del estado del arte de la ingeniería del software.

El [Anexo F](#) (Historial de revisiones) registra las modificaciones aplicadas a sucesivas ediciones del documento.

2 Referencias normativas

Los documentos siguientes son indispensables para la aplicación del presente documento. Para las referencias con fecha, se aplica únicamente la edición citada. Para las referencias sin fecha, se aplica la edición más reciente del documento referenciado. Las referencias se enumeran por organización emisora (ISO/IEC primero, ISO/IEC/IEEE a continuación, otras al final) y, dentro de cada bloque, por número ascendente, conforme a la convención ISO/IEC Directives Part 2 §10.2.

ISO/IEC 7498-1:1994 [17] — *Information technology — Open Systems Interconnection — Basic Reference Model: The Basic Model*. International Organization for Standardization / International Electrotechnical Commission.

Proporciona el modelo de referencia de interconexión de sistemas abiertos sobre el que el modelo XF se fundamenta conceptualmente. XF extiende el principio de estratificación de OSI a una dimensión ortogonal — la arquitectura interna del artefacto, no la comunicación entre artefactos — preservando sus propiedades formales: encapsulación de servicios, dependencia unidireccional, aislamiento de implementación y composición. Los conceptos de capa de abstracción, responsabilidad de capa y dependencias entre capas son adoptados en el sentido normativo que les asigna este estándar.

ISO/IEC Directives, Part 2:2021 [21] — *Principles and rules for the structure and drafting of ISO and IEC documents* (9.^a ed.). International Organization for Standardization / International Electrotechnical Commission.

Establece las convenciones de redacción normativa aplicadas en el presente documento — el uso de los verbos auxiliares prescriptivos **debe / no debe, debería / no debería, puede / no puede y se recomienda / no se recomienda** (§4.1), la distinción entre requisitos obligatorios y recomendaciones, y los criterios de estructura y presentación de documentos normativos.

ISO/IEC/IEEE 12207:2017 [19] — *Systems and software engineering — Software life cycle processes*. International Organization for Standardization / International Electrotechnical Commission / Institute of Electrical and Electronics Engineers.

Proporciona el marco normativo de los procesos del ciclo de vida del software sobre el que el modelo XF se posiciona como instrumento de organización arquitectónica del artefacto, sin reemplazar ni modificar las prescripciones de proceso del ciclo de vida. La instanciación, inicialización, ejecución y terminación del artefacto desarrolladas

en §8 se entienden como manifestación en tiempo de ejecución de las actividades del ciclo de vida que esta norma describe.

ISO/IEC/IEEE 24765:2017 [20] — *Systems and software engineering — Vocabulary*. International Organization for Standardization / International Electrotechnical Commission / Institute of Electrical and Electronics Engineers.

Establece el vocabulario normalizado de la ingeniería del software sobre el que se construye la terminología propia del modelo XF. Los términos básicos — *software artifact, component, module, interface* — se interpretan en el sentido asignado por esta norma salvo restricción explícita por parte del modelo XF, según se desarrolla en §3.1.

ISO/IEC/IEEE 42010:2022 [22] — *Software, systems and enterprise — Architecture description*. International Organization for Standardization / International Electrotechnical Commission / Institute of Electrical and Electronics Engineers.

Proporciona el marco conceptual para la descripción de arquitecturas software — incluyendo los conceptos de modelo de referencia arquitectónico, puntos de vista (*viewpoints*), vistas (*views*), intereses (*concerns*) y partes interesadas (*stakeholders*) — que el modelo XF adopta en su posicionamiento como modelo de referencia en el sentido de este estándar, según se desarrolla en §1.3 y §13.3.

OMG BPMN 2.0.2:2014 [35] — *Business Process Model and Notation (BPMN), Version 2.0.2*. Object Management Group. La especificación BPMN 2.0 fue adoptada por ISO/IEC como **ISO/IEC 19510:2013** [18] (*Information technology — Object Management Group Business Process Model and Notation*); la versión 2.0.2 (2014) del OMG incorpora correcciones editoriales posteriores.

Proporciona la definición formal de proceso formal y sus propiedades estructurales — actividades, entradas, salidas y reglas de transición — que el modelo XF adopta en §5.1 y §5.2 como base para la derivación de las tres etapas invariantes de todo proceso formal y la correspondiente estratificación en tres capas de abstracción.

3 Términos y abreviaturas

3.1 Términos

A efectos del presente documento, se aplican los términos y definiciones siguientes. Para las definiciones completas y las relaciones conceptuales entre términos, véase §12.2.

3.1.1 Artefacto — pieza de código autocontenida que formaliza un proceso, define un área de responsabilidad y está sujeta a control de versiones. En el modelo XF, todo artefacto se estructura en exactamente tres capas de abstracción — Acceso, Negocio e Interacción — y se ejecuta siempre en un único espacio de ejecución. Véase §6.1.

Nota 1 a la entrada: El término *Artefacto* es una especialización del concepto homónimo definido en OMG UML 2.5.1 [36] y normalizado en ISO/IEC/IEEE 24765:2017 [20], restringido al subconjunto de elementos **ejecutables y autocontenidos** que automatizan un proceso formal completo. La diferencia operativa del Artefacto XF

respecto a los vocabularios consolidados de la disciplina reside en la prescripción simultánea de tres condiciones que ninguno de ellos exige conjuntamente: autocontención, exactamente tres capas de abstracción derivadas de las etapas invariantes del proceso formal y un único espacio de ejecución. La comparativa detallada con los términos análogos de la literatura se desarrolla en el [Anexo E](#).

3.1.2 Aplicación — unidad ejecutable resultante de cargar uno o más artefactos en un mismo espacio de ejecución, típicamente un único proceso del sistema operativo. El término *aplicación* designa la unidad funcional ejecutable; el término *artefacto* designa la unidad estructural autocontenida que la aplicación carga y compone. Véase [§6.1](#) y [§10.2](#).

3.1.3 Capa de abstracción — división horizontal del artefacto que agrupa los componentes que implementan la misma etapa del proceso formal que el artefacto automatiza. El modelo XF define exactamente tres capas — Acceso, Negocio e Interacción. Véase [§6.1](#).

3.1.4 Componente — unidad atómica de clasificación del modelo XF con identidad propia, nombre único dentro del artefacto y clasificación definida en la matriz de capas y tipos. Véase [§7.1](#).

3.1.5 Tipo funcional — clasificación de un componente según la función que desempeña dentro de su capa. El modelo XF define exactamente cinco tipos funcionales — Lógico, Generalización, Inyección, Utilidad y Transferencia. Véase [§7.3](#).

3.1.6 Lógica efectiva — conjunto mínimo de operaciones que justifican la existencia de un componente lógico en el artefacto, que no pueden ser abstraídas en ningún otro tipo de componente sin perder la especificidad del concepto que el componente modela. Véase [§7.3.1](#).

3.1.7 Lógica contextual — lógica auxiliar que sirve de soporte a la lógica efectiva de un componente lógico. Se subdivide formalmente en tres especies según el tipo de componente que la aloja: **lógica estructural** en los componentes de generalización, **lógica de utilidad** en los componentes de utilidad y **lógica de transferencia** en los componentes de transferencia — la representación estructural del concepto y, opcionalmente, operaciones autocontenidas de transformación entre representaciones equivalentes, sin lógica de comportamiento sobre el dominio. La subdivisión es exhaustiva: toda lógica contextual cabe en exactamente una de estas tres especies. Véase [§7.3.1](#).

3.1.8 Estado — conjunto de datos de un componente lógico que tienen correspondencia directa con información del dominio de negocio y que determinan el comportamiento observable del componente ante las operaciones que lo invocan. Solo los componentes lógicos pueden mantener estado. Véase [§7.3.1](#).

3.1.9 Operación — subconjunto de la lógica efectiva de un componente lógico que se ejecuta de forma atómica ante un evento o invocación, produciendo una transformación del estado del componente, un resultado para el invocante, o ambos. Véase [§7.3.1](#).

3.1.10 Condición — sentencia de una operación que evalúa el estado del componente o los parámetros de entrada antes de ejecutar la lógica efectiva. Constituye la precondition de ejecución de la operación. Véase [§7.3.1](#).

3.1.11 *Singleton Gathering* — patrón de diseño implementado por los componentes de inyección que agrega en un único punto las instancias únicas de los componentes lógicos de una capa, garantizando su unicidad y proveyendo un punto de acceso nombrado y estable. El término se mantiene en inglés por carecer de equivalente consolidado en la literatura hispanohablante; designa específicamente la concurrencia de las propiedades de unicidad (*singleton*) y agregación (*gathering*) que caracterizan al componente de inyección. Véase §7.3.3.

3.1.12 *Instanciación* — creación de un objeto en memoria mediante el constructor de un componente. En el modelo XF, la instanciación de los **componentes lógicos** es una operación exclusiva del componente de inyección de la capa correspondiente; la instanciación de componentes de otros tipos — en particular las transferencias — no está sujeta a esa restricción. Se distingue de la inicialización. Véase §7.3.3.

3.1.13 *Inicialización* — ejecución de la lógica de arranque de un componente lógico que lo prepara para procesar las operaciones para las que fue diseñado. Se distingue de la instanciación. Véase §7.3.3.

3.1.14 *Terminación* — ejecución de la lógica de cierre de un componente lógico que libera los recursos adquiridos durante la inicialización. Véase §7.3.3.

3.1.15 *Tipo primitivo* — tipo de dato primitivo en el sentido de ISO/IEC/IEEE 24765:2017 [20]: tipo del que se construyen otros tipos, presente como tipo base en la mayoría de lenguajes de programación. En el modelo XF, los tipos primitivos entran al artefacto a través de la Capa de Acceso — frontera con la Capa de Presentación del modelo OSI (Nivel 6) —, son anteriores a cualquier transformación semántica y constituyen la materia prima sobre la que operan todas las capas. Los formatos de intercambio del sistema operativo o del entorno de ejecución cuya estructura está determinada por la plataforma se equiparan a tipos primitivos a efectos del modelo. Véase §7.3.4.

3.1.16 *Estructura* — agrupación de tipos primitivos u otras estructuras que modela un concepto del dominio del artefacto. Sinónimo de componente de transferencia en su acepción estructural. Unifica los términos DTO, **Entity**, **Model**, **POJO**, **Record**, **VO** y **Schema** bajo un único concepto. Véase §9.2.

3.1.17 *Protocolo* — conjunto de reglas y formatos que rigen la comunicación entre dos artefactos software a través de un canal de comunicación. La gestión de los protocolos es responsabilidad estructural de las dos capas frontera del artefacto, en roles complementarios: la **Capa de Interacción** *define y expone* el protocolo por el que los consumidores externos invocan al artefacto; la **Capa de Acceso** *consume* los protocolos publicados por los sistemas externos a los que el artefacto delega parte de su ejecución. Incluye tanto protocolos de red como protocolos de acceso local. Véase §7.2.1 y §7.2.3.

3.1.18 *Punto de entrada* — mecanismo declarado por la Capa de Interacción a través del cual un consumidor externo — humano o sistémico — invoca las operaciones que el artefacto expone. Cada componente lógico de la Capa de Interacción declara exactamente un punto de entrada. No debe confundirse con el punto de inicio de ejecución del artefacto, que es interno al artefacto y no es invocable por consumidores. Véase §7.2.3.

3.1.19 Punto de inicio de ejecución — fragmento de código que el entorno de ejecución invoca para arrancar el proceso del artefacto — `main`, `App.main()`, `index.ts` o equivalente según el lenguaje. Es el primer código que se ejecuta del artefacto y recibe el control del entorno; su única responsabilidad es delegar la inicialización XF y, al final, la terminación. No es un componente clasificable en la matriz de capas y tipos y reside fuera de la raíz del artefacto. No debe confundirse con el punto de entrada (§3.1.18), que es invocable por consumidores externos a través de la Capa de Interacción. Véase §8.1.

3.1.20 Dependencia — toda relación entre dos componentes A y B en la que A requiere conocer la definición o el comportamiento de B para poder ser compilado, instanciado o ejecutado. Toda dependencia está sujeta al principio de aislamiento entre capas. Véase §6.2.2.

3.1.21 Espacio de ejecución — contexto computacional en el que los componentes de un artefacto operan. Criterio determinante para clasificar una integración entre artefactos como integración a nivel de aplicación o a nivel de artefacto. Véase §10.2.

3.1.22 Elemento verificable — todo elemento del artefacto sobre el cual una regla de conformidad puede ser evaluada. El modelo XF reconoce dos tipos disjuntos de elemento verificable: el **elemento de código** (clase, módulo o fichero de funciones cohesionadas; comprende el componente — clasificado en la matriz de capas y tipos — y el elemento de punto de inicio XF) y el **elemento estructural** (carpeta de la estructura canónica). Véase §11.1.2.

3.1.23 Regla de conformidad — condición verificable que operacionaliza una prescripción normativa del modelo XF en forma evaluable sobre un elemento verificable del artefacto (§3.1.22). Tiene un identificador canónico en formato kebab-case y se caracteriza por su **verificabilidad** — estructural o semántica, según si su evaluación puede realizarse de forma determinista a partir de propiedades sintácticas observables del código fuente o requiere interpretación semántica de la responsabilidad funcional. El incumplimiento de cualquier regla del catálogo produce una violación que afecta al nivel de conformidad del artefacto. El catálogo cerrado y exhaustivo se desarrolla en §11.3. Véase §11.1 y §11.3.

3.1.24 Violación — constatación de que un elemento verificable (§3.1.22) del artefacto no satisface la condición prescrita por una regla de conformidad (§3.1.23) del catálogo. Toda violación está asociada a exactamente una regla y exactamente un elemento verificable. Véase §11.1.2.

3.1.25 Totalidad del código — condición que un artefacto satisface cuando todo componente que contiene tiene una posición definida en la matriz de capas y tipos. El elemento XF, cuando se declara, no es componente y queda fuera de la condición de totalidad por construcción. Condición necesaria para alcanzar los niveles de conformidad 2, 3 o 4. Véase §11.1.3.

3.1.26 Nivel de conformidad — propiedad discreta de un artefacto que determina en qué medida satisface las prescripciones normativas del modelo XF. Toma valores en el conjunto {0, 1, 2, 3, 4}. Véase §11.2.

3.1.27 Compatibilidad — propiedad del modelo XF que prescribe cómo un

artefacto XF integra componentes de otros artefactos: reconociendo la responsabilidad funcional del componente integrado, asignándole una celda de su propia matriz de capas y tipos y aplicando el mecanismo prescrito para esa celda y ese tipo de componente. Es bidireccional y no invasiva: ningún artefacto integrado debe modificarse para hacer posible su integración. Véase §10.1.

3.1.28 Lógica estructural — especie de lógica contextual (§3.1.7) encapsulada en un componente de generalización, que abstrae un patrón de comportamiento estructural común entre dos o más componentes lógicos de la **misma capa** y se transmite a estos por herencia. Es **paramétrica respecto al dominio**: su lógica aplica con independencia del concepto concreto que cada componente lógico que la hereda modela. La presencia en un componente de generalización de lógica ligada a un concepto concreto del dominio del artefacto indica que ha dejado de ser lógica estructural y debe trasladarse al componente lógico al que pertenezca. Véase §7.3.2.

3.1.29 Verificabilidad estructural — propiedad de una regla del catálogo de conformidad (§3.1.23) cuya evaluación puede realizarse de forma determinista a partir de propiedades sintácticas observables del código fuente del artefacto — nombres de ficheros, ubicación en la estructura de carpetas, declaraciones de importación entre módulos, sufijos y prefijos de identificadores, grafo de dependencias entre componentes. Las reglas con verificabilidad estructural son evaluables algorítmicamente por una herramienta de análisis estático. Véase §11.1.4.

3.1.30 Verificabilidad semántica — propiedad de una regla del catálogo de conformidad (§3.1.23) cuya evaluación requiere comprender la responsabilidad funcional del código y no es decidible exclusivamente por análisis estructural. Su evaluación exige análisis adicional al de la herramienta de análisis estático — revisión arquitectónica humana, análisis de flujo de datos o análisis de intención del código. La existencia de reglas con verificabilidad semántica determina que la aplicación de la función de conformidad no sea completamente automatizable: parte de las violaciones no puede detectarse solo por análisis estático y requiere evaluación humana. Véase §11.1.4.

3.1.31 Componente lógico — tipo funcional que implementa la lógica efectiva de la capa a la que pertenece. Es la unidad funcional principal de cada capa — el único tipo que puede mantener estado en el sentido formal del modelo y el único cuyas instancias son gestionadas por el componente de inyección. Nomenclatura canónica: sufijo **Repository** en Acceso, **Business** en Negocio, **Service** o **View** en Interacción. Véase §7.3.1.

3.1.32 Componente de generalización — tipo funcional que abstrae comportamiento estructural común a dos o más componentes lógicos de la **misma capa** y lo transmite por herencia. No mantiene estado de dominio y no es accesible directamente a través del componente de inyección. Véase §7.3.2.

3.1.33 Componente de inyección — tipo funcional singleton que centraliza el ciclo de vida y el acceso a los componentes lógicos de su capa. Exactamente uno por capa: R en Acceso, B en Negocio, A en Interacción. Véase §7.3.3.

3.1.34 Componente de utilidad — tipo funcional que provee operaciones auxiliares puras, sin estado, de alcance local a la capa que lo aloja. Nomenclatura canónica: sufijo **Utils**. Véase §7.3.4.

3.1.35 Componente de transferencia — tipo funcional que modela las estructuras de datos que circulan entre componentes del artefacto. No modela procesos de negocio; puede declarar operaciones autocontenidas sobre sus propios datos. Las excepciones son un subtipo de componente de transferencia con nomenclatura propia: sufijo *Exception*. Véase §7.3.5.

3.1.36 Sentencia — instrucción elemental del cuerpo de una operación: consulta o transforma el estado del componente, evalúa una condición o delega en otro componente a través del componente de inyección de la capa correspondiente. El cuerpo de toda operación es una secuencia ordenada de sentencias. Véase §7.3.1.

3.2 Abreviaturas

A efectos del presente documento, se aplican las abreviaturas siguientes. Las abreviaturas son acrónimos textuales sustituibles por su forma completa cuando se invoca el término en prosa; los símbolos matemáticos del aparato formal del modelo (conjuntos, funciones, relaciones) se introducen en las cláusulas donde se emplean. Cuando la sola expansión literal del acrónimo no basta para identificar inequívocamente el referente, la forma completa incluye entre paréntesis un contexto desambiguador — norma de origen, autor del modelo o ámbito de aplicación.

Acrónimos del modelo XF

XF	Cross-Framework (modelo de Arquitectura XF)
CFAM	Cross-Framework Architectural Model (denominación pública internacional del modelo XF)

Estándares, normas y vocabularios de ingeniería del software

OMG	Object Management Group
UML	Unified Modeling Language
BPMN	Business Process Model and Notation
OSI	Open Systems Interconnection (modelo ISO/IEC 7498-1)

Paradigmas, patrones y metodologías arquitectónicas

IPO	Input-Process-Output (modelo de descomposición de la ingeniería estructurada)
CSP	Communicating Sequential Processes (cálculo de procesos comunicantes de Hoare)
DDD	Domain-Driven Design

Patrones de implementación y tipos de objeto

DTO	Data Transfer Object
DAO	Data Access Object
BLoC	Business Logic Component (patrón de gestión de estado de Flutter)

Tecnologías y protocolos del ecosistema

API	Application Programming Interface
------------	-----------------------------------

SDK	Software Development Kit
IDE	Integrated Development Environment
JPA	Java Persistence API
REST	Representational State Transfer
gRPC	Google Remote Procedure Call
HTTP	HyperText Transfer Protocol
URL	Uniform Resource Locator
SQL	Structured Query Language
JSON	JavaScript Object Notation
IoT	Internet of Things

4 Convenciones normativas y alcance

4.1 Verbos auxiliares prescriptivos

El presente documento emplea los verbos auxiliares prescriptivos definidos en la norma ISO/IEC Directives, Part 2:2021 [21] con el significado preciso que se establece a continuación. La interpretación correcta de estos verbos es condición necesaria para la correcta aplicación de las prescripciones del modelo.

debe / no debe El verbo **debe** indica un requisito — una prescripción cuyo cumplimiento es obligatorio para la conformidad con el modelo. Su incumplimiento constituye una violación que afecta al nivel de conformidad del artefacto según las condiciones definidas en §11.2.

El verbo **no debe** indica una prohibición — una prescripción cuyo cumplimiento es igualmente obligatorio para la conformidad. Su incumplimiento constituye igualmente una violación.

Las reglas del catálogo de §11.3 corresponden a prescripciones expresadas con **debe** o **no debe** en el cuerpo normativo del documento.

debería / no debería El verbo **debería** indica una recomendación normativa — una prescripción cuyo cumplimiento es preferido pero no obligatorio para la conformidad con el modelo. Su incumplimiento no constituye violación y no afecta al nivel de conformidad del artefacto, pero indica que el artefacto se aparta de la práctica preferida que el modelo prescribe. El verbo **no debería** indica una recomendación negativa con el mismo estatus.

se recomienda / no se recomienda El presente documento utiliza adicionalmente las formas **se recomienda** y **no se recomienda** como equivalentes normativos de **debería** y **no debería**, respectivamente. Esta elección no amplía el catálogo de seis categorías canónicas de ISO/IEC Directives Part 2:2021 [21]: corresponde a las formas equivalentes para *should / should not* explícitamente admitidas en el Anexo E de la propia norma (“*it is recommended that*” / “*it is not recommended that*”). Su uso responde a la voz natural del español, donde la pasiva refleja resulta más idiomática que el modal en numerosos giros, sin alterar el estatus normativo ni el impacto en la conformidad de la categoría *recomendación* a la que pertenecen.

puede / no puede El verbo **puede** indica una acción o decisión que está explícitamente autorizada por el modelo pero que no es obligatoria ni recomendada. Su no aplicación no constituye violación de ningún tipo.

El verbo **no puede** indica una **imposibilidad estructural**: una propiedad que el modelo no puede expresar o realizar por construcción de su propio aparato formal. No es prescripción normativa — no genera violación — sino constatación de un límite del modelo. Para las prohibiciones que sí constituyen violación se emplea siempre **no debe**.

La [tabla siguiente](#) resume el estatus normativo de cada verbo auxiliar:

Table 1. Verbos auxiliares prescriptivos y su impacto en la conformidad.

Verbo auxiliar	Estatus normativo	Impacto en conformidad
debe	Requisito obligatorio	Violación <i>afecta al nivel de conformidad</i>
no debe	Prohibición obligatoria	Violación <i>afecta al nivel de conformidad</i>
debería	Recomendación normativa	Sin impacto
no debería	Recomendación normativa negativa	Sin impacto
se recomienda	Forma equivalente de <i>debería</i> (Anexo E)	Sin impacto
no se recomienda	Forma equivalente de <i>no debería</i> (Anexo E)	Sin impacto
puede	Acción autorizada, no obligatoria	Sin impacto
no puede	Imposibilidad estructural	Sin impacto

4.2 Convenciones de notación

El presente documento emplea las siguientes convenciones de notación en el cuerpo normativo y en los anexos.

Notación de código Los nombres de componentes, operaciones, carpetas y patrones de acceso se representan en tipografía monoespaciada — `UserBusiness`, `R.init()`, `/src/repository/logic`, `B.<negocio>.<operación>()`. Esta notación indica que el término debe interpretarse como un identificador de código, no como un término del lenguaje natural.

Notación matemática Las expresiones formales del modelo emplean la notación matemática estándar de teoría de conjuntos y funciones, con las siguientes convenciones tipográficas por familia:

- **Conjuntos de componentes del artefacto** (blackboard bold): $\mathbb{C}$ para el conjunto finito de componentes; sus subconjuntos se denotan con subíndice, como $\mathbb{C}_T$ para los componentes de transferencia.
- **Componentes de inyección** (latinas mayúsculas): R , B , A para las tres capas según el contexto.
- **Artefacto completo** (fraktur): $\mathfrak{A}$.

- **Funciones estructurales y de conformidad** (letra griega): ϕ para la función de clasificación, que asigna a cada componente su celda en la matriz $L \times T$ ($\phi: \mathbb{C} \rightarrow L \times T$); Λ para la función de nivel de conformidad, cuya aplicación a un artefacto produce $\Lambda(\mathfrak{A}) \in \{0, 1, 2, 3, 4\}$; y λ para su resultado, el nivel de conformidad alcanzado por el artefacto.
- **Estado de un componente lógico** (letra griega con subíndice): σ_0 para el estado inicial, posterior a la construcción del componente y anterior a la invocación de `init()`.
- **Notación conjuntista y de funciones**: pertenencia $\in$, unión $\cup$, producto cartesiano $\times$ (matriz $L \times T$) y la flecha de función $\rightarrow$. Las cuantificaciones y conectivas lógicas se expresan en prosa.

Notación de referencias cruzadas El documento emplea el término *cláusula* de forma unificada para designar cualquier subdivisión jerárquica del cuerpo normativo y los anexos, con independencia de su nivel de profundidad. Las denominaciones alternativas *sección*, *subapartado*, *subsección* y *sub-cláusula* no se usan; el contexto y el número de subdivisión expresan la profundidad cuando es relevante. Las referencias a cláusulas del documento se expresan mediante el símbolo § seguido del número de cláusula — §6.2.2, §11.3.1. Las referencias a anexos se expresan mediante la letra del anexo seguida del número de cláusula interna.

Figuras del documento Las figuras del documento se incluyen como diagramas vectoriales con caption descriptivo y etiqueta numérica única. Toda figura es referenciable desde el cuerpo del texto mediante el mecanismo de referencias cruzadas descrito en el párrafo anterior. El catálogo completo aparece en el *Índice de figuras* al inicio del documento.

Tablas del documento Las tablas del documento se insertan no flotantes en el lugar argumental donde aparecen, con partición de página cuando exceden el espacio disponible, de forma que se preserva el hilo argumental del texto. Cada tabla lleva caption descriptivo, opcionalmente acompañado de un *caption corto* entre corchetes para la entrada del *Índice de tablas*, y una etiqueta única que la hace referenciable mediante el mecanismo de referencias cruzadas. El catálogo completo aparece en el *Índice de tablas* al inicio del documento.

Listings de código Los fragmentos de código del cuerpo normativo se presentan en bloques delimitados con caption descriptivo y etiqueta única referenciable. No se expresan en ningún lenguaje de programación concreto: emplean un pseudocódigo orientado a objetos neutro — con construcciones comunes a los lenguajes mayoritarios — elegido por claridad de lectura y en coherencia con el agnosticismo tecnológico del modelo; ilustran la estructura y las relaciones entre componentes, no prescriben una sintaxis ni un lenguaje de implementación. El catálogo completo aparece en el *Índice de códigos* al inicio del documento.

4.3 Distinción entre contenido normativo e informativo

El presente documento distingue explícitamente entre contenido normativo y contenido informativo.

El **contenido normativo** establece requisitos, prohibiciones, recomendaciones y permisiones que determinan la conformidad de un artefacto con el modelo XF. Constituye el cuerpo prescriptivo del documento. Las cláusulas §2, §3, §4 y §6 a §12 tienen carácter normativo.

El **contenido informativo** provee contexto, justificación, ejemplos y orientación de implementación que facilitan la comprensión y la aplicación del contenido normativo, pero que no establecen requisitos adicionales. §1, §5, los Anexos A a F, §13 (Discusión) y §14 (Conclusiones) tienen carácter informativo.

Cuando el contenido informativo entre en aparente contradicción con el contenido normativo, prevalece el contenido normativo.

4.4 Criterios de interpretación

En caso de ambigüedad en la interpretación de cualquier prescripción normativa del presente documento, se aplican los siguientes criterios en orden de prioridad.

El primer criterio es la **consistencia con los principios rectores**: entre dos interpretaciones posibles, prevalece la que mejor satisfaga los cuatro principios rectores del modelo establecidos en §6.2 — agnosticismo tecnológico, aislamiento entre capas, precedencia de la arquitectura sobre la herramienta y tipificación cerrada y exhaustiva.

El segundo criterio es la **verificabilidad**: entre dos interpretaciones igualmente consistentes con los principios, prevalece la que produce una condición verificable de forma determinista. El modelo XF aspira a que todas sus prescripciones sean verificables algorítmicamente — una interpretación que produce una condición no verificable es indicativa de ambigüedad en la prescripción que debe ser resuelta en una versión futura del documento.

El tercer criterio es la **referencia al diccionario de términos**: cuando la ambigüedad derive del significado de un término técnico, la definición de §12 es la referencia normativa. Si el término no aparece en §12, se aplica su acepción en la norma ISO/IEC/IEEE 24765:2017 [20]; en defecto de esta, su acepción en ISO/IEC/IEEE 42010:2022 [22].

4.5 Alcance del modelo

El modelo XF normaliza la **arquitectura del código** del artefacto. Su dominio de aplicación comienza en el código fuente del artefacto y termina en los límites externos del mismo. Los elementos del proyecto que rodean al código fuente — y que son necesarios para construir, empaquetar y desplegar el artefacto — quedan fuera del alcance directo del modelo.

Quedan explícitamente fuera del alcance del modelo:

- La estructura del proyecto fuera del código fuente del artefacto: la gestión de dependencias externas, los descriptores de empaquetado, los recursos asociados al despliegue, los parámetros de configuración del entorno de ejecución y los ficheros propios de las herramientas de construcción son responsabilidad del marco de desarrollo, no del modelo XF.

- La gestión del ciclo de desarrollo: el control de versiones, la integración continua, la automatización del despliegue y los procesos de aseguramiento de calidad externos al código son instrumentos distintos del modelo y su normalización corresponde a otros marcos normativos y metodológicos.
- El código generado automáticamente por herramientas externas que no conocen el modelo XF: la organización interna de ese código depende de la herramienta que lo produce. A efectos de integración y conformidad XF, ese código es funcionalidad no normalizada en XF y su tratamiento se rige por las prescripciones generales de §10.2.1 — se reformula en un componente XF propio que lo encapsula, o se mantiene fuera del alcance de la conformidad ubicándolo fuera de la raíz del artefacto según la prescripción de §7.4.

Estos elementos pueden coexistir con un artefacto XF y su correcta organización es relevante para la calidad del proyecto en su conjunto, pero su normalización es responsabilidad de instrumentos distintos del modelo. El modelo XF describe la organización del código que se desarrolla, no el proceso por el que se desarrolla ni el entorno en el que se ejecuta.

5 Fundamentos teóricos

Las cláusulas internas tienen carácter informativo. Su propósito es establecer el razonamiento teórico del que se deriva el modelo de Arquitectura XF definido en las cláusulas siguientes. El contenido de esta cláusula no constituye requisito normativo.

La presente cláusula expone los fundamentos teóricos de los que se deriva el modelo de Arquitectura XF. Su propósito es establecer el razonamiento que justifica las decisiones normativas adoptadas en las cláusulas siguientes, de forma que el lector pueda comprender no solo qué prescribe el modelo, sino por qué lo prescribe.

El argumento se desarrolla en cinco cláusulas internas con estructura encadenada. Las cláusulas §5.1 y §5.2 establecen las propiedades estructurales invariantes de todo proceso formal, que constituyen el fundamento del que se deriva la arquitectura de tres capas. Las cláusulas §5.3 y §5.4 analizan la tipificación de componentes como instrumento de gestión de la complejidad y documentan las limitaciones del modelo de tipificación propietaria predominante en la industria. La cláusula §5.5 sintetiza ambas líneas argumentales en la derivación formal del modelo de tres capas y establece las propiedades formales que debe satisfacer todo artefacto XF.

5.1 El software como automatización de procesos formales

La ingeniería del software tiene como objeto la construcción de sistemas que ejecuten, de forma autónoma y reproducible, procesos que preexisten en el dominio que modelan. Esta caracterización no es nueva: la conferencia de la OTAN sobre ingeniería del software de 1968 [33] marca el reconocimiento institucional de la disciplina y la formulación de la denominada *crisis del software*, cuya superación pasa por la producción sistemática y reproducible de sistemas ejecutables fiables a partir de especificaciones de proceso. Lo que ha cambiado desde entonces es la escala y la heterogeneidad de los procesos que el software modela, no su naturaleza fundamental.

Un sistema software no crea procesos; los formaliza. Esta distinción tiene una implicación arquitectónica de primer orden: si la función del software es representar un proceso preexistente, la estructura interna del artefacto es isomórfica a la estructura del proceso que modela. La arquitectura del sistema no es, por tanto, una decisión discrecional del desarrollador ni una convención impuesta por el marco de desarrollo — es una propiedad derivada de la naturaleza del proceso automatizado.

Este principio de isomorfismo entre proceso y artefacto conecta directamente con la teoría del modelado formal de procesos. El estándar BPMN 2.0 [35, 18] caracteriza un proceso — en su dominio de negocio — como una secuencia de actividades con semántica precisa, entradas y salidas bien definidas, y reglas de transición verificables, caracterización que el modelo XF generaliza al concepto de proceso formal. Desde esta perspectiva, cualquier proceso susceptible de ser descrito con precisión suficiente es, en principio, automatizable mediante software — desde la gestión de pedidos en una organización hasta el control de actuadores en un sistema embebido, pasando por la sincronización de datos entre servicios distribuidos.

La implicación práctica es directa: el análisis de la estructura del proceso debe preceder al diseño de la arquitectura del artefacto. Brooks [7] identificó la complejidad accidental — aquella introducida por las herramientas y los procesos de desarrollo, no por el problema en sí — como una de las causas principales de los fallos en proyectos software. Una arquitectura que no derive de la estructura del proceso que modela introduce precisamente ese tipo de complejidad: inconsistencias estructurales que no tienen origen en el dominio de negocio, sino en decisiones de implementación arbitrarias que se acumulan como deuda técnica y se manifiestan durante el mantenimiento y la evolución del sistema.

El modelo XF parte de esta premisa para establecer que existe un conjunto de propiedades estructurales comunes a todo proceso formal, con independencia de su dominio, complejidad o tecnología de implementación. La identificación y formalización de estas propiedades es el objeto de las cláusulas siguientes.

5.2 Etapas invariantes de un proceso formal

Si todo artefacto software modela un proceso formal, y si la arquitectura del artefacto es isomórfica a la estructura de ese proceso, entonces la identificación de las propiedades estructurales comunes a todo proceso formal constituye, en la propuesta del modelo XF, el paso previo a la derivación de cualquier modelo de arquitectura de software con vocación universal.

El estudio de los procesos formales en el ámbito de la teoría de la computación y la ingeniería de sistemas revela una propiedad estructural notable. Con independencia del dominio, la complejidad o la naturaleza del proceso considerado, todo proceso formal admite una descomposición canónica en tres etapas con estatus formal distinto — **procesamiento**, **interacción** y **acceso** — cuyo desarrollo individual y formalización del estatus se desarrollan en los siguientes apartados. El modelo XF postula esta descomposición no como convención de diseño ni elección metodológica, sino como propiedad estructural derivable del análisis de las condiciones mínimas que cualquier proceso formal debe satisfacer para ser ejecutable.

Parnas [39], en su trabajo seminal sobre la descomposición modular de sistemas

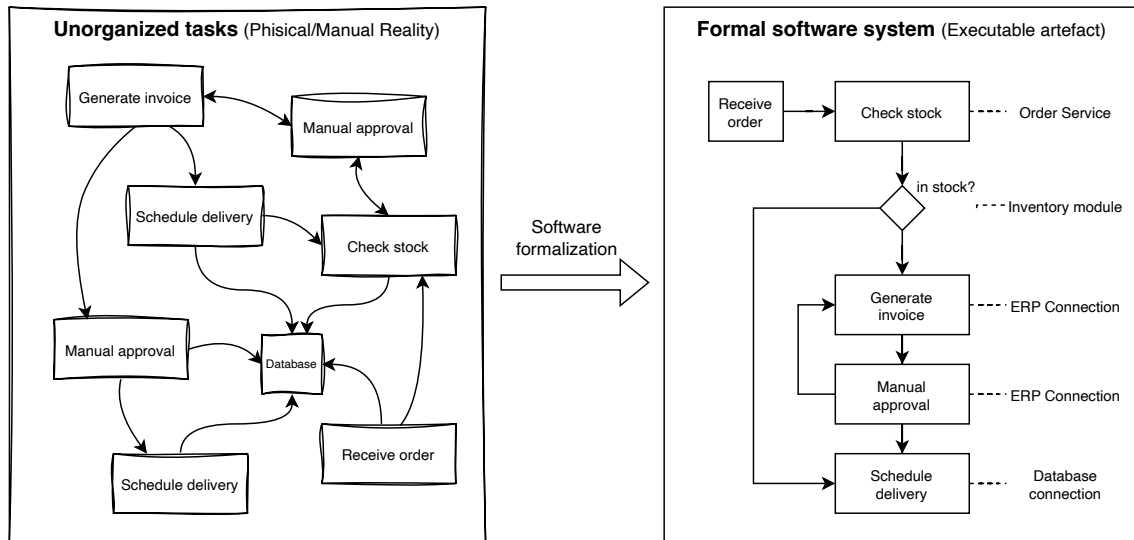


Figure 1. El software como automatización de procesos formales. Todo artefacto software ejecuta una secuencia ordenada de operaciones que transforma un estado inicial en un estado final mediante la aplicación de reglas deterministas; en esa correspondencia descansa la posibilidad de derivar una arquitectura universal a partir de propiedades estructurales del proceso modelado.

software, observó que la estructura de un sistema debe reflejar las decisiones de diseño que tienen mayor probabilidad de cambiar, y que estas decisiones están determinadas por la naturaleza del proceso que el sistema implementa. Esta observación anticipa el principio de isomorfismo descrito en la cláusula anterior y apunta hacia la existencia de una estructura de proceso subyacente que todo sistema debe reconocer y representar.

Las tres etapas que componen todo proceso formal son las siguientes:

- **Interacción.** Todo proceso formal requiere un mecanismo por el cual recibe la orden de ser ejecutado y verifica que se satisfacen las precondiciones necesarias para su inicio. Esta etapa no forma parte de la lógica interna del proceso — es la interfaz entre el proceso y su entorno, el punto en el que el proceso se hace visible y accesible para los agentes externos que lo invocan. Sin esta etapa, el proceso no puede ser iniciado de forma controlada ni puede garantizarse que las condiciones de ejecución sean las adecuadas. En términos de la teoría de contratos de Meyer [31], esta etapa corresponde a la verificación de las precondiciones del proceso — aquellas condiciones que deben ser verdaderas en el momento de la invocación para que el proceso pueda ejecutarse correctamente.
- **Procesamiento.** Una vez satisfechas las precondiciones de ejecución, el proceso aplica de forma ordenada y determinista el conjunto de operaciones que lo definen para transformar sus entradas en el resultado esperado. Esta es la etapa en la que reside la lógica propia del proceso — los algoritmos, las reglas de negocio y las transformaciones de datos que constituyen el valor que el proceso aporta. Turing [43] demostró que cualquier función computable puede ser descompuesta en una secuencia finita de operaciones elementales; el procesamiento es precisamente esa secuencia, instanciada para el dominio específico del proceso considerado.
- **Acceso.** Todo proceso formal que opera en un entorno no trivial requiere

información o capacidades que residen fuera de sus propios límites. La etapa de acceso es el mecanismo por el cual el proceso delega parte de su ejecución en otros procesos o sistemas externos, con los que se comunica a través de un protocolo de interacción definido. Esta etapa es el reflejo exacto de la etapa de interacción vista desde la perspectiva del proceso delegado: lo que para el proceso invocante es acceso, para el proceso invocado es interacción. Esta simetría no es casual — es una consecuencia directa de la composicionalidad de los procesos formales, propiedad que permite construir procesos complejos mediante la composición de procesos más simples [16].

Estas tres etapas tienen un estatus estructural que conviene precisar. El **procesamiento** es la etapa estructuralmente necesaria de todo proceso formal: sin lógica que transforme entradas en resultados no hay proceso. Las etapas de **interacción** y **acceso** son estructuralmente opcionales según el contexto del proceso — existen procesos disparados internamente que no reciben invocación externa, y existen procesos puramente computacionales que no requieren colaboración con sistemas externos — pero son **exhaustivas y mutuamente excluyentes** como categorías de comunicación con el entorno. Todo proceso que se comunica con su entorno lo hace por exactamente una de dos vías — recibir invocación o invocar al exterior — sin que exista un tercer tipo estructural de comunicación. Esta propiedad — procesamiento necesario, comunicación exhaustiva en sus dos formas — es la base formal de la tripartición y, cuando una etapa de comunicación está presente, su semántica respecto al procesamiento es invariante: la interacción precede causalmente al procesamiento y el acceso ocurre como delegación durante o después de él.

La propiedad de exhaustividad estructural de las dos vías de comunicación tiene apoyo en al menos cuatro tradiciones formales independientes que llegan a la misma conclusión. El estándar BPMN 2.0 [35] estructura cada proceso en *Activities* — procesamiento — *catch Events* — interacción entrante — y *throw Events* — acceso saliente —, siendo *Activities* el único elemento estructuralmente necesario. El cálculo de procesos comunicantes CSP [16], junto con su extensión móvil — el cálculo π de Milner [32] — demuestra que toda computación concurrente puede expresarse con tres primitivas: acción interna (τ), recepción por canal ($c?x$) y emisión por canal ($c!v$). El *Actor Model* [1] define todo actor por el procesamiento del mensaje en curso, el envío opcional de mensajes a otros actores — equivalente al acceso — y la recepción opcional en su buzón — equivalente a la interacción —. El modelo *Input-Process-Output* (IPO) de la ingeniería estructurada [47] formaliza la descomposición clásica con *Process* necesario e *Input/Output* opcionales. Las cuatro tradiciones convergen en la misma estructura: una etapa de transformación interna como núcleo necesario y dos — y solo dos — direcciones de comunicación con el entorno.

La identificación de estas tres etapas — procesamiento necesario, interacción y acceso como vías exhaustivas de comunicación con el entorno — constituye el punto de partida para la derivación del modelo de tres capas que define la Arquitectura XF. Dicha derivación, incluyendo la correspondencia formal entre cada etapa y su capa de abstracción correspondiente, se desarrolla en §5.5 una vez establecidos los fundamentos complementarios de los que también depende.

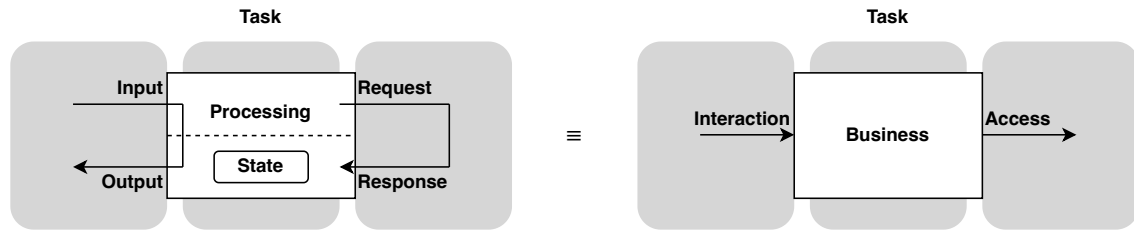


Figure 2. Las tres etapas invariantes del proceso formal: Interacción, Negocio (procesamiento) y Acceso, representadas como secuencia de ejecución con sus flechas de entrada y salida.

El artefacto como escala canónica de uniformización terminológica El patrón estructural IPO no es exclusivo del artefacto como unidad: aparece de forma recurrente a múltiples escalas en todo sistema software bien construido — a nivel de sentencia individual, de función, de método, de componente y de subsistema. Esta autosimilaridad escalar ha sido reconocida en al menos cuatro tradiciones formales independientes. La cibernética [45] estableció el bucle entrada–proceso–salida–retroalimentación como primitiva universal de los sistemas adaptativos. La teoría general de sistemas [5] postuló que todos los sistemas — biológicos, sociales y técnicos — comparten esta estructura con independencia de la escala en que se observen. La ingeniería estructurada [47] la formalizó como técnica recursiva de descomposición funcional del software. El cálculo de procesos comunicantes [16] demostró su composicionalidad algebraica mediante tres primitivas: acción interna, recepción por canal y emisión por canal. La autosimilaridad escalar es, por tanto, una propiedad emergente del sistema software bien construido, reconocida transversalmente en la literatura y derivada de la naturaleza misma del proceso formal — noción que Mandelbrot [28] generalizó al concepto de autosimilaridad fractal como propiedad estructural de los sistemas naturales.

El alcance del modelo XF no es esa autosimilaridad ni su formalización recursiva. Dentro de la jerarquía de escalas, el modelo identifica una específica — la del **artefacto** — y la adopta como nivel canónico de uniformización terminológica. La razón de esta elección es estructural y no convencional: el artefacto es la escala mínima en la que un proceso formal completo se realiza biunívocamente como estructura de código. Las tres etapas invariantes del proceso formal — interacción, procesamiento y acceso — se corresponden con tres regiones del código que constituyen capas con identidad estructural propia. Por debajo del artefacto, las unidades de descomposición — componentes, clases, funciones — son encapsulaciones de responsabilidad que sirven al proceso pero no son procesos formales completos en sí mismas: una clase no tiene “etapa de interacción” propia, un módulo no tiene “etapa de acceso” propia. Por encima del artefacto, las unidades de composición — sistemas distribuidos, ecosistemas, organizaciones — coordinan procesos múltiples y no se reducen a un proceso único.

Esta delimitación de la escala canónica es lo que distingue al modelo XF como modelo de referencia — en el sentido de la norma ISO/IEC/IEEE 42010 [22] — y no como una arquitectura más entre las que coexisten en la industria. Lo que el modelo aporta no es estructura — que ya está universalmente presente en cualquier artefacto software por la naturaleza del software como automatización de procesos formales

— sino **vocabulario**. El modelo XF acopla la nomenclatura al modelo formal del proceso para que la estructura, ya existente, sea reconocible y comunicable a través de cualquier tecnología. La fragmentación documentada en §5.4 no es estructural sino terminológica: el mismo concepto arquitectónico recibe nombres incompatibles según el marco de desarrollo empleado, y esa incompatibilidad — no la estructura subyacente — es lo que produce la curva de aprendizaje inter-marco de desarrollo y los costes operativos de la fragmentación. El modelo XF resuelve esa fragmentación normalizando la nomenclatura a la escala donde la estructura del proceso formal admite traducción biunívoca al código, y solo a esa escala.

5.3 La tipificación de componentes como instrumento de comprensión

El desarrollo de un artefacto software implica la construcción y coordinación de un conjunto potencialmente extenso de unidades de código — clases, funciones, módulos — cada una de las cuales resuelve un problema específico dentro del sistema. A medida que la complejidad del artefacto crece, la capacidad de los desarrolladores para comprender la función de cada unidad y su relación con el resto del sistema se convierte en un factor determinante de la productividad del equipo y de la calidad del producto resultante.

La gestión de esta complejidad ha sido reconocida como uno de los problemas centrales de la ingeniería del software desde sus orígenes. Dijkstra [10] argumentó que la calidad intelectual de un programa es inversamente proporcional a la distancia conceptual entre el problema que resuelve y la estructura del código que lo implementa. Esta observación apunta hacia un principio fundamental: la estructura del código debe ser legible, y su legibilidad depende en gran medida de que cada unidad de código comunique de forma inmediata su propósito y su responsabilidad.

La **tipificación de componentes** es el mecanismo mediante el cual los equipos de desarrollo asignan a cada unidad de código una categoría que comunica su función dentro del conjunto del artefacto. Esta categorización se materializa habitualmente en la nomenclatura del componente — mediante sufijos, prefijos o convenciones de nombrado — de forma que el nombre del componente sea, por sí solo, suficientemente informativo sobre su rol en el sistema. Así, un componente denominado **UserRepository** comunica sin ambigüedad que su responsabilidad es el acceso a datos de usuarios; un componente denominado **AuthenticationService** indica que su función es proveer lógica de autenticación; un componente denominado **TemperatureUtils** señala que contiene funciones de utilidad relacionadas con el tratamiento de datos de temperatura.

Martin [29] identifica la claridad de los nombres como uno de los principios fundamentales del código limpio, argumentando que un nombre que requiere comentario adicional para ser comprendido es un nombre que ha fallado en su propósito. La tipificación extiende este principio más allá del nombre individual: no solo comunica qué hace el componente, sino qué tipo de lógica implementa y qué lugar ocupa en la arquitectura del sistema.

Las ventajas de una tipificación consistente son documentadas y medibles. En primer lugar, reduce la carga cognitiva de los desarrolladores al permitirles establecer

expectativas precisas sobre el comportamiento de un componente antes de leer su implementación. En segundo lugar, facilita la incorporación de nuevos miembros al equipo, que pueden orientarse en la estructura del artefacto con mayor rapidez cuando los componentes están categorizados de forma coherente. En tercer lugar, mejora la revisión del código, ya que los revisores pueden verificar no solo la corrección de la implementación sino también su adecuación al tipo asignado. En cuarto lugar, favorece la separación de responsabilidades [47], al hacer explícita la responsabilidad de cada componente y facilitar la detección de violaciones.

La convergencia implícita de la industria Estas ventajas no han pasado inadvertidas para la industria del software. La emergencia recurrente de taxonomías de componentes en arquitecturas heterogéneas [41, 3] es evidencia de que la tipificación de componentes es un instrumento valioso: los marcos de desarrollo más adoptados la implementan de forma autónoma — cada uno con su propia taxonomía, pero todos con el mismo objetivo: hacer comprensible la estructura del artefacto mediante la categorización de sus componentes.

Fowler [12] documentó que los mismos patrones de tipificación emergen de forma recurrente en sistemas enterprise heterogéneos — **Repository** para el acceso a datos, **Service** para la lógica de negocio, **Controller** para los puntos de entrada. Esta recurrencia no es casual — refleja que los desarrolladores convergen hacia las mismas categorías de componentes porque los procesos que automatizan tienen la misma estructura subyacente. Spring tipifica sus componentes con **@Repository**, **@Service** y **@Controller**. Angular tipifica los suyos con **Injectable**, **Component** y **Pipe**. Flutter tipifica los suyos con **BLoC**, **Widget** y **Repository**. Cada marco de desarrollo ha llegado de forma independiente a la misma conclusión: la tipificación de componentes es necesaria para hacer comprensible la estructura del artefacto.

Esta convergencia implícita de la industria, observable en el cuerpo de patrones recurrentes documentados por Bass et al. [3] y Fowler [12], constituye confirmación empírica de que la tipificación de componentes no es una convención arbitraria sino una necesidad estructural derivada de la naturaleza del software como automatización de procesos formales. Si los marcos de desarrollo convergen hacia las mismas categorías de componentes, es porque los procesos que modelan tienen las mismas etapas invariantes. La tipificación no crea esa estructura — la hace visible.

Sin embargo, esta convergencia implícita tiene una limitación estructural que ningún marco de desarrollo por sí solo puede resolver: cada marco de desarrollo tipifica con su propia taxonomía propietaria, incompatible con las taxonomías de los demás. Los marcos de desarrollo convergen hacia las mismas soluciones pero las describen con vocabularios distintos — vocabularios que, como se documentó en §5.2 y se desarrollará en §5.4, producen sinonimia heterotécnica y homonimia interframework que fragmentan el conocimiento y elevan el coste de cambio de contexto entre tecnologías.

La tipificación de componentes es, en definitiva, una forma de documentación estructural embebida en el propio código. A diferencia de la documentación externa — comentarios, wikis, diagramas — la tipificación es inseparable del artefacto y evoluciona con él de forma natural [4]. Esta propiedad la convierte en un instrumento especialmente valioso en entornos de desarrollo ágil, donde la documentación tradi-

cional tiende a quedar desactualizada con rapidez. Sin embargo, su efectividad como instrumento de comunicación — no solo dentro de un equipo sino entre equipos y entre tecnologías — depende de una condición que los modelos de desarrollo actuales no satisfacen: que la taxonomía de tipos sea consistente, exhaustiva y compartida universalmente. Cuando esta condición no se cumple, la tipificación pierde su valor como instrumento de comunicación inter-tecnológica y se convierte en una fuente adicional de fragmentación. Este es precisamente el problema que aborda la cláusula siguiente.

5.4 Limitaciones de la tipificación propietaria

La cláusula anterior estableció que la tipificación de componentes es un instrumento necesario para la gestión de la complejidad en el desarrollo de software, y que los marcos de desarrollo más adoptados la implementan de forma convergente — llegando de forma independiente a las mismas categorías de componentes porque los procesos que modelan tienen la misma estructura subyacente. Esta convergencia implícita, en línea con la recurrencia de patrones arquitectónicos documentada en sistemas *enterprise* heterogéneos [12], sustenta que la tipificación no sea una convención arbitraria sino una necesidad estructural derivada de la naturaleza del software.

Sin embargo, esta misma convergencia pone de manifiesto la limitación fundamental del estado actual de la industria: los marcos de desarrollo convergen hacia las mismas soluciones estructurales, pero las describen con taxonomías propietarias incompatibles entre sí. La convergencia existe — pero es invisible para los desarrolladores porque está oculta detrás de vocabularios distintos. Un desarrollador que trabaja con Spring y uno que trabaja con Angular están tipificando sus componentes con las mismas categorías conceptuales subyacentes, pero con nombres tan distintos que no pueden reconocer esa equivalencia sin un proceso explícito de traducción. El resultado es que la industria paga repetidamente el coste de aprender lo que ya sabe — expresado en otra nomenclatura.

La causa estructural de esta situación reside en que la tipificación de componentes no ha sido históricamente objeto de estandarización independiente, sino que ha quedado subordinada a los marcos de desarrollo. Cada marco de desarrollo define su propia taxonomía de tipos como parte de su modelo de programación, con la consecuencia de que la tipificación es consistente dentro de un marco concreto pero incompatible con las taxonomías de otros marcos. El resultado es un ecosistema en el que la convergencia estructural real del software queda fragmentada en la superficie por taxonomías propietarias que impiden reconocerla.

Este fenómeno puede caracterizarse con precisión mediante el concepto de **acoplamiento semántico al marco de desarrollo**: la arquitectura del artefacto no deriva de la estructura del proceso que modela — que es universal — sino de las convenciones del marco de desarrollo empleado — que son propietarias. Martin [30] identifica este acoplamiento como uno de los antipatrones más perjudiciales en el diseño de sistemas software, argumentando que una arquitectura que no puede ser descrita independientemente de sus herramientas de implementación es una arquitectura que ha perdido su función de modelo conceptual y se ha convertido en un mero artefacto tecnológico.

Los dos patrones de inconsistencia La magnitud del problema se hace evidente al examinar la fragmentación semántica resultante en marcos de desarrollo ampliamente adoptados. Un análisis sistemático de las taxonomías propietarias revela dos patrones de inconsistencia que se producen de forma recurrente y simultánea.

El primero es la **sinonimia heterotécnica**: conceptos arquitectónicamente equivalentes reciben denominaciones distintas en marcos diferentes. La lógica de acceso a datos persistentes se denomina **Repository** en Spring, **QuerySet** en Django, **DAO** en J2EE y **Model** en Node.js. La lógica de interacción HTTP se denomina **Controller** en Spring, **View** en Django, **Router** en Node.js y **Handler** en marcos de desarrollo como Gin o Echo. Un desarrollador que transite entre estas tecnologías debe aprender no solo la sintaxis del nuevo lenguaje, sino una nueva taxonomía conceptual para describir los mismos problemas arquitectónicos que ya resolvía en su tecnología anterior — aunque la estructura subyacente sea idéntica.

El segundo es la **homonimia interframework**: un mismo término designa conceptos arquitectónicamente distintos en marcos diferentes, generando ambigüedad que puede inducir a errores de diseño. El término **Service** es el ejemplo más ilustrativo: en Spring designa un componente de lógica de negocio; en Android designa una tarea asíncrona ejecutada en segundo plano; en Angular designa un objeto singleton de propósito general que puede contener tanto lógica de negocio como lógica de acceso; en la arquitectura de microservicios designa una unidad de despliegue independiente. Cuatro conceptos arquitectónicamente distintos, un único término, cuatro marcos diferentes. La consecuencia práctica es que, en equipos con desarrolladores formados en tecnologías distintas, el uso del término **Service** en una discusión técnica no comunica nada preciso sin un contexto tecnológico explícito — exactamente lo contrario de lo que la tipificación debe conseguir.

Las consecuencias operativas Estas inconsistencias tienen consecuencias operativas que van más allá de la mera incomodidad terminológica. Jones [25], en su análisis cuantitativo de los factores que determinan la productividad en proyectos software, identifica la ambigüedad en los requisitos y las inconsistencias en las convenciones de diseño entre los factores con mayor impacto negativo en la velocidad de desarrollo y en la densidad de defectos del producto. La fragmentación semántica de las taxonomías propietarias contribuye directamente a ambos factores: genera ambigüedad en las discusiones de diseño y produce inconsistencias estructurales que se traducen en decisiones de implementación divergentes dentro de un mismo equipo.

Un segundo efecto de largo alcance es el coste acumulado de formación específica en tecnología que las organizaciones deben asumir cuando incorporan nuevos desarrolladores o cuando migran sistemas a nuevas plataformas. Este coste — análogo en su naturaleza a la deuda técnica descrita por Cunningham [9] pero de naturaleza organizacional y no estructural — se acumula de forma invisible en el producto y se manifiesta en los costes de incorporación, en la dependencia de perfiles especializados escasos y en la dificultad para escalar equipos con rapidez. En un contexto de alta rotación profesional y taxonomías propietarias incompatibles, este tipo de deuda se convierte en una carga estructural que afecta a la totalidad de la industria.

La causa raíz La raíz del problema no reside en los marcos de desarrollo en sí — que cumplen una función legítima y valiosa al proveer abstracciones de implementación especializadas — ni en la calidad del software que producen. El modelo XF identifica una causa raíz estructural: la convergencia que el software ya implementa implícitamente no tiene expresión formal y universal. Los marcos de desarrollo han llegado de forma independiente a las mismas categorías conceptuales subyacentes — porque los procesos que modelan son los mismos — pero cada uno las ha nombrado y organizado de forma propietaria, sin ningún mecanismo que permita reconocer y comunicar esa equivalencia entre tecnologías.

En términos de la arquitectura en capas de Dijkstra [10], el problema es que la capa conceptual y la capa tecnológica no están separadas: el modelo mental del desarrollador está acoplado a la herramienta, cuando debería estar acoplado al proceso que la herramienta implementa. La tipificación propietaria confunde el instrumento con el concepto — hace que el desarrollador aprenda `@Repository` de Spring en lugar de aprender el concepto de componente de acceso a datos que `@Repository` implementa, y que tenga que reaprender ese mismo concepto como `QuerySet` cuando cambia a Django, como `DAO` cuando trabaja en J2EE, y como `Model` cuando trabaja en Node.js.

El modelo XF aborda este problema mediante la definición de una capa de abstracción conceptual independiente cuya taxonomía de componentes deriva de las propiedades estructurales invariantes de los procesos formales — establecidas en §5.1 y §5.2 — y no de las convenciones de ningún marco de desarrollo en particular. Esta taxonomía no reemplaza las taxonomías propietarias de los marcos de desarrollo — los marcos de desarrollo seguirán llamando `@Repository` a sus componentes de acceso. Lo que provee es el nivel de abstracción que permite a los desarrolladores reconocer que `@Repository`, `QuerySet`, `DAO` y `Model` son el mismo concepto expresado con nombres distintos — y comunicarse sobre él con un vocabulario común. La derivación formal de esta taxonomía es el objeto de §5.5.

5.5 Derivación del modelo de tres capas

Recapitulación de proposiciones Las cláusulas precedentes han establecido tres proposiciones que, tomadas conjuntamente, determinan de forma necesaria la estructura del modelo de Arquitectura XF:

1. La **primera proposición** establece que todo artefacto software modela un proceso formal, y que la arquitectura del artefacto es isomórfica a la estructura del proceso que modela (§5.1).
2. La **segunda proposición** establece que todo proceso formal admite una descomposición canónica en tres etapas con estatus formal distinto: el *procesamiento* como etapa estructuralmente necesaria, y la *interacción* y el *acceso* como categorías estructuralmente opcionales pero exhaustivas y mutuamente excluyentes de comunicación con el entorno (§5.2).
3. La **tercera proposición** establece que la tipificación de componentes es el mecanismo mediante el cual la arquitectura de un artefacto hace explícita su estructura, y que su efectividad requiere una taxonomía consistente, exhaustiva e independiente de cualquier marco de desarrollo propietario (§5.3 y §5.4).

De la conjunción de estas tres proposiciones se deriva una conclusión directa: existe una estructura arquitectónica universal, compuesta por tres capas de abstracción que corresponden biunívocamente a las tres etapas invariantes de todo proceso formal, y cuya taxonomía de componentes se define con independencia de cualquier tecnología de implementación. Esta estructura es el modelo de Arquitectura XF.

Derivación formal de las capas La derivación de las tres capas del modelo XF a partir de las etapas invariantes de los procesos formales sigue el principio de correspondencia estructural enunciado en §5.1: cada etapa del proceso se corresponde con exactamente una capa de abstracción del artefacto, y cada capa de abstracción del artefacto da cobertura a exactamente una etapa del proceso.

La **etapa de interacción** del proceso formal — la recepción de la orden de ejecución y la verificación de las precondiciones — se corresponde con la **Capa de Interacción** del artefacto. Esta capa contiene la lógica que hace el artefacto accesible a sus consumidores, ya sean usuarios humanos a través de interfaces gráficas o sistemas externos a través de protocolos de comunicación definidos. Su responsabilidad es exclusivamente la de establecer las condiciones de interacción — validar entradas, definir protocolos de acceso, representar resultados — sin incorporar lógica de procesamiento ni lógica de acceso a datos.

La **etapa de procesamiento** del proceso formal — la ejecución ordenada de las operaciones que transforman las entradas en el resultado esperado — se corresponde con la **Capa de Negocio** del artefacto. Esta capa contiene la lógica que constituye el valor del artefacto: los algoritmos, las reglas de negocio y las transformaciones de datos que definen qué hace el sistema y cómo lo hace. Su responsabilidad es exclusivamente la de procesar — no interactuar con consumidores externos ni acceder directamente a fuentes de datos remotas.

La **etapa de acceso** del proceso formal — la delegación de parte del procesamiento en sistemas o procesos externos — se corresponde con la **Capa de Acceso** del artefacto. Esta capa contiene la lógica que capacita al artefacto para comunicarse con otros sistemas a través de protocolos definidos, abstrayendo los detalles de cada protocolo del resto del artefacto. Su responsabilidad es exclusivamente la de establecer y gestionar las comunicaciones con sistemas externos — sin procesar los datos obtenidos más allá de lo necesario para su normalización, ni exponer interfaces de interacción a consumidores externos.

Posicionamiento en el modelo OSI La estructura de tres capas del modelo XF no emerge de forma aislada, sino en continuidad con un marco de referencia establecido y ampliamente validado: el modelo de interconexión de sistemas abiertos ISO/IEC 7498-1 [17]. El modelo OSI organiza los protocolos de comunicación en red mediante siete capas de abstracción con responsabilidades claramente delimitadas, donde cada capa provee servicios a la capa inmediatamente superior y consume los servicios de la capa inmediatamente inferior.

El nivel más alto del modelo OSI — la Capa de Aplicación — se define como la frontera entre las capas de comunicación y los procesos de aplicación: es el medio de acceso al entorno OSI por parte de los procesos de aplicación, no un contenedor de su lógica interna. Por diseño, ISO/IEC 7498-1 termina en esa frontera. La arquitectura

interna de los procesos de aplicación — cómo se organiza la lógica *dentro* de cada artefacto — queda explícitamente fuera del alcance del modelo OSI y ha permanecido históricamente sin un marco de referencia equivalente.

El modelo XF ocupa ese espacio. No subdivide la Capa de Aplicación de OSI — ISO/IEC 7498-1 no especifica el interior de las aplicaciones y, por tanto, no hay nada que subdividir — sino que extiende el principio de estratificación de OSI a una dimensión ortogonal: mientras OSI estratifica la comunicación *entre* artefactos, XF estratifica la estructura *interna* de cada artefacto. Las dos estratificaciones son técnicamente independientes y conceptualmente complementarias. Un artefacto XF se ejecuta sobre el entorno OSI — típicamente comunicándose con otros artefactos a través de las primitivas de servicio de la Capa de Aplicación — pero su estructura interna se organiza siguiendo la estratificación XF, ortogonal a las siete capas de OSI.

La extensión preserva las cuatro propiedades formales del modelo OSI, aplicadas a la nueva dimensión:

- **Encapsulación de servicios mediante primitivas:** cada capa expone a la capa inmediatamente superior un conjunto definido de servicios a través del componente de inyección de la capa, que actúa como punto de acceso único y nombrado. La capa superior consume esos servicios sin conocer los detalles internos de su implementación.
- **Dependencia estrictamente unidireccional:** cada capa depende exclusivamente de la capa inmediatamente inferior, nunca de la superior ni de capas no adyacentes. Esta propiedad se materializa en el principio de aislamiento entre capas establecido en §6.2.2.
- **Aislamiento de implementación:** una capa puede cambiar internamente sin afectar a la capa superior, siempre que preserve las primitivas de servicio que expone. Esta propiedad habilita la independencia tecnológica que el modelo prescribe en §6.2.1.
- **Composición de servicios:** el comportamiento total del artefacto se obtiene por composición de los servicios que cada capa provee a la inmediatamente superior, partiendo de los servicios externos consumidos en la capa inferior.

La continuidad con el modelo OSI no es meramente formal: es también pragmática. La aportación duradera de OSI a la industria no fue su pila de protocolos — fue TCP/IP el que se adoptó como pila efectiva — sino su contribución estandarizadora a tres niveles. El primero es el **vocabulario común**: OSI dio nombres genéricos — *transport layer*, *session layer* — que permitieron a ingenieros de empresas competidoras describir los mismos problemas con la misma terminología, eliminando la fragmentación semántica entre nomenclaturas propietarias. El segundo es la **independencia respecto a la implementación**: las primitivas de servicio se definieron sin asumir ningún protocolo concreto, permitiendo comparar implementaciones distintas como realizaciones de una misma abstracción. El tercero es la **validación cruzada y la pedagogía**: la estratificación OSI permitió identificar qué protocolos operan en qué capa, facilitando la enseñanza sistemática, la documentación normalizada y la auditoría arquitectónica.

El modelo XF persigue exactamente la misma contribución estandarizadora aplicada

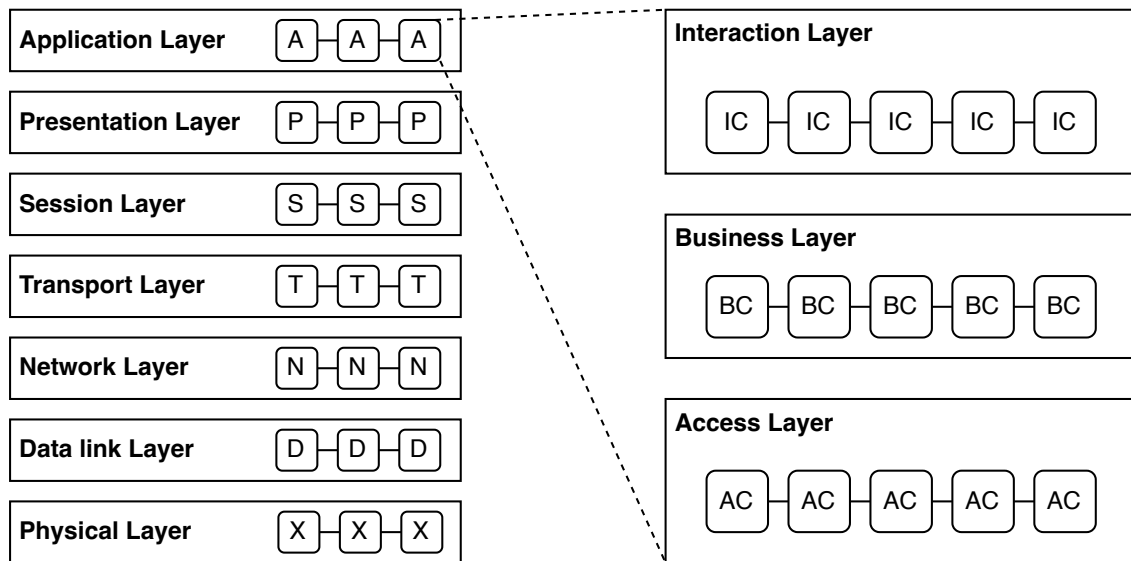


Figure 3. Ortogonalidad entre las dos estratificaciones: las siete capas del modelo OSI organizadas verticalmente como pila de protocolos de comunicación entre artefactos, y las tres capas del modelo XF organizadas en una dimensión ortogonal como estratificación de la estructura interna del artefacto. La Capa de Aplicación OSI (Nivel 7) es el punto de contacto entre ambas dimensiones.

a la arquitectura interna del artefacto. Vocabulario común que sustituya las nomenclaturas propietarias de los marcos de desarrollo. Interfaces independientes de la tecnología que permitan describir un componente de acceso sin asumir un protocolo concreto, una librería de persistencia o un marco específico. Validación cruzada que permita identificar qué lógica de un artefacto opera en cada capa de la estratificación XF, habilitando análisis estático automatizado y enseñanza sistemática. La analogía entre lo que OSI aportó a la comunicación entre sistemas y lo que XF pretende aportar a la arquitectura interna del artefacto es el fundamento del posicionamiento del modelo en continuidad con la tradición de estandarización del software.

Propiedades formales del modelo de tres capas La derivación anterior determina tres propiedades formales que el modelo de tres capas satisface y que constituyen, en última instancia, los criterios de corrección de cualquier artefacto XF.

La primera es la **completitud**: las tres capas cubren de forma exhaustiva y sin solapamiento las tres categorías estructurales del proceso formal — procesamiento necesario, interacción como vía de comunicación entrante, acceso como vía de comunicación saliente. Todo componente de un artefacto software puede ser clasificado en exactamente una de las tres capas según la categoría del proceso que implementa — no existe lógica de programa que no pertenezca a alguna de las tres categorías, ni lógica que pertenezca simultáneamente a más de una. En artefactos que no requieran comunicación con el entorno (procesamiento puro), las capas de Acceso e Interacción pueden quedar vacías sin que ello afecte a la conformidad del artefacto con el modelo.

La segunda es la **separación estricta**: cada capa implementa exclusivamente la lógica correspondiente a su etapa del proceso. La presencia de lógica de una etapa en la capa correspondiente a otra etapa constituye una violación de la correspondencia estructural entre proceso y artefacto, y por tanto una violación del modelo. Esta

separación es la base del principio de aislamiento entre capas que se define formalmente en §6.2.2.

La tercera es la **direccionalidad**: la dependencia entre capas es estrictamente unidireccional en sentido descendente — la Capa de Interacción depende de la Capa de Negocio, y la Capa de Negocio depende de la Capa de Acceso, pero nunca en sentido inverso. Esta propiedad deriva directamente del orden de ejecución de las etapas del proceso formal: la interacción precede al procesamiento, y el procesamiento precede al acceso; una dependencia en sentido inverso implicaría que una etapa anterior del proceso depende de una etapa posterior, lo que contradice la estructura causal del proceso.

Estas tres propiedades — completitud, separación estricta y direccionalidad — constituyen los fundamentos formales sobre los que se construye el modelo normativo desarrollado en las cláusulas siguientes. Su derivación a partir de las propiedades estructurales invariantes de los procesos formales garantiza que el modelo XF no es una convención arbitraria de diseño, sino una consecuencia necesaria de la naturaleza del software como instrumento de automatización de procesos.

6 Modelo de Arquitectura XF

La presente cláusula define el modelo de Arquitectura XF en sus términos normativos. A diferencia de §5, cuyo propósito era exponer el razonamiento teórico que justifica el modelo, el contenido de esta cláusula tiene carácter prescriptivo: establece los principios que todo artefacto XF debe satisfacer, la estructura que debe adoptar y la taxonomía de componentes que debe emplear. Las disposiciones de esta cláusula constituyen los requisitos fundamentales del modelo, de los que derivan todas sus reglas de conformidad.

El modelo de Arquitectura XF se define como un **modelo de referencia arquitectónico** en el sentido establecido por Bass et al. [3]: un conjunto de decisiones de diseño aplicables a un dominio de problemas recurrentes que, al ser adoptadas, establecen restricciones sobre la arquitectura de los sistemas que las implementan y favorecen determinadas cualidades del sistema resultante. En el caso del modelo XF, el dominio de problemas recurrentes es la automatización de procesos formales mediante software, y las cualidades favorecidas son la comprensibilidad, la mantenibilidad, la interoperabilidad entre equipos y la independencia tecnológica.

Es importante distinguir el modelo XF de otros instrumentos arquitectónicos con los que podría confundirse. El modelo XF no es un patrón de diseño en el sentido de Gamma et al. [14] — no describe una solución a un problema de diseño local dentro de un componente o un conjunto de componentes relacionados. Tampoco es un estilo arquitectónico en el sentido de Shaw y Garlan [41] — no describe una familia de sistemas caracterizados por una organización estructural particular. El modelo XF es un modelo de referencia: una abstracción conceptual que establece un vocabulario común, una estructura de organización universal y un conjunto de principios rectores que son aplicables a cualquier artefacto software independientemente de su dominio, tecnología o escala.

Esta distinción tiene implicaciones prácticas directas. Un patrón de diseño se aplica o

tienen rango de axiomas: ninguna disposición normativa de este documento puede interpretarse de forma contraria a ellos.

6.1 Visión general

El modelo de Arquitectura XF organiza la estructura interna de cualquier artefacto software mediante dos dimensiones ortogonales de clasificación: la **dimensión de abstracción**, que determina en qué capa del artefacto reside un componente según la etapa del proceso formal que implementa, y la **dimensión funcional**, que determina qué tipo de rol desempeña ese componente dentro de su capa. La combinación de estas dos dimensiones produce una estructura matricial que categoriza de forma unívoca cada componente del artefacto, eliminando la ambigüedad estructural que caracteriza a las arquitecturas propietarias descritas en §5.4.

La **dimensión de abstracción** divide el artefacto en tres capas jerárquicas ordenadas de menor a mayor nivel de abstracción. La Capa de Acceso constituye el nivel de abstracción más bajo y es responsable de toda comunicación con sistemas externos al artefacto. La Capa de Negocio constituye el nivel de abstracción intermedio y es responsable de la lógica de dominio y las transformaciones de datos que definen el comportamiento del artefacto. La Capa de Interacción constituye el nivel de abstracción más alto y es responsable de los puntos de entrada al artefacto, tanto para usuarios humanos como para sistemas externos que lo consumen. La correspondencia entre estas tres capas y las tres etapas invariantes de los procesos formales — acceso, procesamiento e interacción — ha sido establecida formalmente en §5.5.

La **dimensión funcional** divide los componentes de cada capa en cinco tipos según la función que desempeñan dentro de ella. Los componentes lógicos implementan la lógica efectiva de la capa. Los componentes de generalización abstraen comportamientos comunes entre componentes lógicos de la misma capa. Los componentes de inyección gestionan el ciclo de vida y el acceso centralizado a las instancias de los componentes lógicos. Los componentes de utilidad proveen operaciones auxiliares de alcance local a la capa. Los componentes de transferencia definen las estructuras de datos que circulan entre componentes. Cada uno de estos tipos se define exhaustivamente en §7.3.

Estas dos dimensiones son **independientes entre sí** en el sentido de que la clasificación de un componente en una capa no determina su tipo funcional, y su clasificación en un tipo funcional no determina su capa. Un componente lógico puede pertenecer a cualquiera de las tres capas; un componente de utilidad puede igualmente pertenecer a cualquiera de las tres capas. Lo que el modelo prescribe es que todo componente debe ser clasificado simultáneamente en exactamente una capa y exactamente un tipo — ni más ni menos. Esta propiedad de clasificación unívoca es la que confiere al modelo su capacidad de eliminar la ambigüedad estructural y proveer una base común para la comunicación entre equipos.

El modelo XF reconoce adicionalmente dos elementos que operan sobre el conjunto del artefacto y no quedan inscritos en ninguna celda de la matriz $L \times T$. El primero es el **elemento XF**, denominado XF, que encapsula la inicialización y terminación del conjunto del artefacto en un único punto de control. El segundo es la **estructura de carpetas canónica**, que materializa físicamente en el sistema de ficheros del

proyecto la organización matricial del modelo, haciendo que la arquitectura sea visible e inspeccionable sin necesidad de herramientas adicionales. Los **componentes de inyección** — denominados R, B y A en las capas de Acceso, Negocio e Interacción respectivamente — no son transversales: cada uno pertenece a la celda *Inyección* de su capa y gestiona el ciclo de vida de los componentes lógicos de esa misma capa.

Una propiedad fundamental del modelo es su **universalidad de aplicación**. El modelo XF no establece restricciones sobre el dominio de negocio del artefacto, el lenguaje de programación empleado, el marco de desarrollo utilizado, el paradigma de programación adoptado — orientado a objetos, funcional, reactivo — ni el entorno de ejecución del artefacto — servidor, dispositivo móvil, navegador, sistema embebido, contenedor. La universalidad del modelo deriva directamente de que sus fundamentos — las propiedades estructurales invariantes de los procesos formales — son igualmente universales. Todo artefacto que automatice un proceso formal, independientemente de cualquier otra consideración tecnológica, puede y debe ser organizado según el modelo XF.

Esta universalidad no implica uniformidad de implementación. Dos artefactos desarrollados en tecnologías de naturaleza distinta y que implementen dominios de negocio independientes serán, si ambos siguen el modelo XF, estructuralmente equivalentes en términos de organización de capas, tipología de componentes y flujos de dependencia. Esta equivalencia estructural es precisamente la que permite a un desarrollador formado en el modelo XF orientarse con rapidez en cualquier artefacto XF con independencia de la tecnología en la que esté implementado, reduciendo el coste de cambio de contexto documentado en §5.4.

El modelo XF generaliza la estructura subyacente a todo artefacto software; no se construye como reacción a las arquitecturas vigentes en un momento dado, sino como abstracción de las propiedades estructurales que toda arquitectura comparte. Esta generalidad implica que su aplicación no se reduce a paradigmas con clases nominales ni a artefactos con estado mutable: cualquier unidad de software que automatice un proceso formal admite organización XF, con independencia del paradigma — orientado a objetos, funcional, reactivo, procedural — y de las primitivas sintácticas que el lenguaje ofrezca para expresar componentes. La forma concreta que adoptan los componentes varía con las restricciones del sustrato técnico: un componente lógico en un paradigma funcional puro no mantiene estado interno y sus operaciones se reducen a funciones puras; un componente lógico en un lenguaje sin estructura nominal puede materializarse como un módulo o como una agrupación cohesionada de funciones de alcance global en lugar de como una clase; un componente de inyección puede tomar la forma de objeto con referencias inmutables, de contenedor de estado observable o de ligadura léxica gestionada por la plataforma de ejecución. Las variaciones de forma no debilitan la aplicación del modelo: son su proyección natural sobre el sustrato técnico del artefacto. Lo que el modelo prescribe es la clasificación y las relaciones entre componentes, no las primitivas sintácticas con las que esos componentes se expresan.

La proyección del modelo sobre paradigmas no orientados a objetos clásicos — programación funcional con flujo unidireccional de estado y vistas declarativas, sistemas reactivos basados en streams, arquitecturas con despliegue distribuido — queda fuera del alcance del presente estándar. En todos esos paradigmas la

clasificación $L \times T$ mantiene su carácter total y exhaustivo; cambia únicamente la forma sintáctica con la que cada componente se materializa, no su rol arquitectónico.

6.2 Principios rectores

Los principios rectores del modelo XF tienen rango de axiomas normativos. Ninguna disposición de este documento puede interpretarse de forma contraria a ellos, y en caso de ambigüedad en la aplicación de cualquier regla de conformidad, la interpretación que prevalece es aquella que mejor satisface el conjunto de principios definidos en esta cláusula.

Los principios son independientes entre sí en su enunciado, pero interdependientes en su aplicación: la correcta implementación de cualquiera de ellos presupone la correcta implementación de los demás. Un artefacto que satisfaga el principio de aislamiento entre capas, pero no el de tipificación cerrada no es un artefacto XF conforme — los principios se satisfacen en conjunto o no se satisfacen.

6.2.1 Agnosticismo tecnológico

Enunciado: el modelo XF no establece restricciones sobre el lenguaje de programación, el marco de desarrollo, el paradigma de programación — orientado a objetos, funcional, reactivo, procedural —, la plataforma de ejecución, la **infraestructura de despliegue** — monolítica, distribuida o de cualquier topología intermedia —, el **modelo de gestión de estado** del artefacto — con estado mutable, inmutable o sin estado — ni el dominio de negocio. Todo artefacto software que automatice un proceso formal puede ser organizado según el modelo XF con independencia de cualquier consideración tecnológica.

Las restricciones que un lenguaje o paradigma concreto imponga sobre la forma sintáctica de los componentes — ausencia de estructuras nominales, inmutabilidad obligatoria, ciclo de vida cedido al marco de desarrollo, ausencia de mecanismos de encapsulación — no acotan el alcance del modelo XF: determinan la forma que adoptan sus componentes en ese contexto, sin alterar su clasificación ni sus relaciones.

Motivación El agnosticismo tecnológico es la propiedad que hace posible la función principal del modelo XF: proveer una capa de abstracción conceptual común sobre un ecosistema de desarrollo heterogéneo. Un modelo de referencia que estableciera restricciones sobre la tecnología de implementación sería, en el mejor de los casos, un estándar propietario — útil dentro de su ecosistema, irrelevante fuera de él. La historia de la industria del software está poblada de estándares con esta característica, cuya adopción quedó limitada a los entornos tecnológicos para los que fueron concebidos y cuya obsolescencia llegó con la obsolescencia de esos entornos.

El modelo XF evita esta limitación estructural derivando sus principios de las propiedades invariantes de los procesos formales — establecidas en §5.2 — y no de las convenciones de ninguna tecnología en particular. Dado que las propiedades de los procesos formales son independientes de la tecnología empleada para automatizarlos, el modelo que las formaliza hereda esa misma independencia.

Esta decisión de diseño conecta con el principio de separación de intereses enunciado por Dijkstra [10]: las decisiones de diseño conceptual — qué hace el sistema y cómo

se organiza — deben estar separadas de las decisiones de implementación — con qué herramientas y en qué lenguaje se construye. El agnosticismo tecnológico del modelo XF es la materialización de este principio a nivel arquitectónico: el modelo define la estructura conceptual del artefacto, y la tecnología elegida es el instrumento con el que esa estructura se implementa.

Implicaciones normativas La primera implicación es que el modelo XF debe poder ser implementado en cualquier lenguaje o marco de desarrollo sin requerir extensiones, adaptaciones o excepciones al modelo. Si la implementación del modelo en una tecnología concreta requiere violar alguno de sus principios, la causa de esa violación es una limitación de la tecnología, no una limitación del modelo. El tratamiento de estas situaciones es objeto de §10.

La segunda implicación es que la evaluación de la conformidad de un artefacto con el modelo XF debe poder realizarse con independencia de la tecnología en la que está implementado. Las reglas de conformidad del modelo son necesariamente tecnológicamente neutras: describen propiedades estructurales del artefacto — presencia de capas, clasificación de componentes, direccionalidad de dependencias — que pueden ser verificadas independientemente del lenguaje o el marco empleado.

La tercera implicación afecta a la nomenclatura. El modelo XF prescribe nombres canónicos para los componentes de inyección — R, B, A —, para las carpetas de cada capa — /repository, /business, /api — y para los tipos de componentes — sufijos Repository, Business, Service, View, Utils. Esta nomenclatura es parte del modelo y debe respetarse con independencia de las convenciones de nombrado propias del lenguaje o marco de desarrollo empleado. Cuando las convenciones del marco entren en conflicto con la nomenclatura prescrita por el modelo, prevalece la nomenclatura del modelo.

Alcance y límites del principio El agnosticismo tecnológico tiene un alcance preciso que conviene delimitar para evitar interpretaciones excesivamente amplias. El modelo XF es agnóstico respecto a la tecnología de implementación — lenguaje, marco, plataforma — pero no es agnóstico respecto a la estructura conceptual del artefacto. El modelo prescribe con precisión cómo debe organizarse un artefacto, qué tipos de componentes debe contener y cómo deben relacionarse entre sí. Esta prescripción estructural no es negociable en función de las preferencias tecnológicas del equipo de desarrollo.

Dicho de otro modo: el agnosticismo tecnológico significa que el modelo no dice con qué herramienta se construye el artefacto, pero sí dice cómo debe estar organizado independientemente de la herramienta elegida. La libertad que el principio otorga es libertad de implementación, no libertad de estructura.

Nota. La proyección canónica del modelo sobre cada lenguaje y paradigma concreto — la forma sintáctica que adoptan los componentes en cada caso — es objeto de un documento complementario de compatibilidad por lenguaje, externo al presente estándar. La convergencia que el modelo formaliza se introduce en §5.3 y su proyección detallada por marco de desarrollo, en §D.1.

Relación con otros principios El agnosticismo tecnológico es el principio que da sentido a los tres principios restantes. El aislamiento entre capas (§6.2.2), la precedencia de la arquitectura sobre la herramienta (§6.2.3) y la tipificación cerrada y exhaustiva (§6.2.4) son propiedades que el modelo prescribe precisamente porque son independientes de la tecnología — si dependieran de ella, no podrían ser principios de un modelo agnóstico. En este sentido, el agnosticismo tecnológico es el principio fundacional del que los demás derivan su universalidad.

6.2.2 Aislamiento entre capas

Enunciado: todo componente de un artefacto XF depende exclusivamente de componentes clasificados en su misma capa o en capas de menor nivel de abstracción. Ningún componente puede depender de un componente clasificado en una capa de mayor nivel de abstracción. Las dependencias entre capas son estrictamente unidireccionales en sentido descendente: la Capa de Interacción puede depender de la Capa de Negocio; la Capa de Negocio puede depender de la Capa de Acceso; ninguna dependencia en sentido inverso es admisible.

Motivación El aislamiento entre capas es la propiedad estructural que garantiza que la arquitectura del artefacto sea isomórfica al proceso formal que modela. Como se estableció en §5.2, cuando las etapas de comunicación están presentes en un proceso formal, su orden causal respecto al procesamiento es invariante: la interacción precede causalmente al procesamiento — el proceso recibe la invocación antes de ejecutar su lógica — y el acceso se produce como delegación durante o después del procesamiento — el proceso accede al exterior cuando necesita información o capacidades que residen fuera de sus propios límites. Ese orden causal es unidireccional. Una dependencia de la Capa de Acceso sobre la Capa de Negocio implicaría que la etapa de acceso del proceso depende de la etapa de procesamiento, lo que contradice la estructura causal del proceso y, por tanto, la premisa sobre la que se fundamenta el modelo.

Parnas [39] estableció que la modularidad efectiva requiere que cada módulo oculte una decisión de diseño de los demás, de forma que los cambios en esa decisión no se propaguen más allá del módulo que la encapsula. El aislamiento entre capas es la materialización de este principio a nivel arquitectónico: cada capa encapsula un tipo de lógica — de acceso, de negocio, de interacción — de forma que los cambios en esa lógica no se propaguen a las capas que no deben conocerla. Un cambio en el protocolo de comunicación con un sistema externo — por ejemplo, la migración de REST a GraphQL — debe quedar confinado en la Capa de Acceso y no requerir modificaciones en la Capa de Negocio ni en la Capa de Interacción. Esta propiedad, conocida en la literatura como resistencia al cambio local [47], es una de las cualidades de mantenibilidad más valiosas en sistemas de larga vida.

Implicaciones normativas La implicación normativa del aislamiento entre capas es que toda dependencia entre componentes de capas distintas debe ser estrictamente descendente: un componente puede depender de los de capas de menor nivel de abstracción, nunca de los de mayor nivel. Para fijar con precisión el alcance de esta prescripción, el modelo define a continuación qué constituye una dependencia a sus efectos.

Definición formal de dependencia A efectos del modelo XF, se define **dependencia** entre dos componentes A y B como cualquier relación en la que A requiere conocer la definición o el comportamiento de B para poder ser compilado, instanciado o ejecutado. Esta definición abarca las siguientes relaciones concretas:

- A invoca una operación de B a través del componente de inyección de la capa a la que pertenece B.
- A hereda de B o implementa una interfaz definida por B.
- A declara un atributo, parámetro o valor de retorno cuyo tipo es B o deriva de B.
- A importa o referencia el módulo, paquete o espacio de nombres en el que B está definido.

Esta definición es intencionalmente amplia. El modelo XF no distingue entre tipos de dependencia a efectos del principio de aislamiento — toda relación que implique que A conoce a B es una dependencia, y toda dependencia está sujeta a las restricciones de direccionalidad del modelo.

Componentes de transferencia y aislamiento Los componentes de transferencia se rigen por la misma direccionalidad descendente que el resto de las dependencias del modelo: un componente clasificado en una capa puede referenciar transferencias definidas en su propia capa o en capas de menor nivel de abstracción, nunca en capas de mayor abstracción. Esta direccionalidad no constituye excepción al principio de aislamiento: lo satisface íntegramente.

Aislamiento y generalización El principio de aislamiento entre capas tiene implicaciones directas sobre el uso de componentes de generalización. La herencia entre componentes está restringida a componentes de la misma capa: ningún componente lógico puede heredar de un componente de generalización definido en una capa distinta, y ningún componente de generalización puede abstraer comportamiento de componentes pertenecientes a capas distintas.

Esta restricción tiene una consecuencia que puede resultar contraintuitiva: cuando dos o más capas requieren implementar el mismo patrón estructural — por ejemplo, el patrón de observación de estado mutable o la ejecución periódica de tareas — cada capa **debe** implementar su propio componente de generalización de forma independiente, aunque la implementación sea estructuralmente idéntica.

El modelo XF prescribe esta duplicación de forma explícita porque la alternativa — un componente de generalización transversal a varias capas — violaría el principio de aislamiento e introduciría una dependencia estructural entre capas que comprometería la resistencia al cambio local del artefacto. La duplicación controlada entre capas es un coste deliberado y asumido por el modelo en beneficio de la independencia estructural de cada capa.

Aislamiento y comportamiento compartido no trivial El principio de aislamiento no impide que componentes de distintas capas utilicen lógica común — impide que esa lógica común se implemente como generalización entre capas o que cree dependencias en sentido ascendente. Cuando varios componentes de distintas

capas necesitan acceder a un comportamiento compartido no trivial — registro de trazas de depuración, métricas, cifrado de datos, lectura de configuración persistida — el modelo XF prescribe que ese comportamiento se encapsule en un componente lógico clasificado en la **capa más baja** que lo requiera, de forma que pueda ser invocado por las capas superiores respetando la direccionalidad descendente.

Alcance y límites del principio El principio de aislamiento tiene un alcance preciso que conviene delimitar. Restringe la dirección de las dependencias *entre capas* — qué componente puede conocer, referenciar o heredar de cuál —, admisible únicamente en sentido descendente. No restringe las dependencias entre componentes de una misma capa, que pueden relacionarse entre sí sin restricción de dirección, ni el sentido en que la información circula en tiempo de ejecución.

Dicho de otro modo: el principio determina quién puede depender de quién, no cómo viajan los datos durante la ejecución. Esta distinción — entre la dirección de la dependencia y la del flujo de datos — es la que permite que la información ascienda entre capas sin invertir las dependencias, como se desarrolla a continuación.

Comunicación ascendente — programación orientada a eventos El principio de aislamiento prescribe que la dirección de las dependencias es descendente, pero no impide que la información fluya en sentido ascendente. La distinción es fundamental: la **dirección de la dependencia** determina quién conoce a quién; la **dirección del flujo de datos** determina en qué sentido viaja la información en tiempo de ejecución.

Cuando un componente de una capa inferior necesita comunicar un cambio de estado a componentes de capas superiores — por ejemplo, una pérdida de conectividad detectada por un componente de negocio, o una actualización de datos en tiempo real recibida a través de un canal WebSocket — el modelo XF prescribe que esa comunicación ascienda **sin introducir dependencia estructural ascendente**: el acoplamiento permanece descendente aunque la información fluya hacia las capas superiores. Esta propiedad se realiza típicamente mediante programación orientada a eventos y el patrón de observación — los componentes de capas superiores se registran como observadores del estado y reaccionan a sus mutaciones sin que el componente de capa inferior los invoque directamente —, pero la elección del mecanismo concreto corresponde al implementador.

Este mecanismo preserva el principio de aislamiento porque la dependencia sigue siendo descendente mientras que la notificación fluye en sentido ascendente sin que la capa inferior conozca a sus observadores. El componente de negocio muta su estado y notifica; los observadores registrados reaccionan. La Capa de Negocio nunca invoca directamente a ningún componente de la Capa de Interacción. La [figura siguiente](#) ilustra este mecanismo de comunicación ascendente.

Relación con otros principios El aislamiento entre capas es el principio que da contenido operativo al agnosticismo tecnológico (§6.2.1): sin aislamiento, los cambios tecnológicos en una capa se propagarían a las demás, creando acoplamiento tecnológico entre capas que comprometería la independencia del artefacto respecto a su implementación. Del mismo modo, el aislamiento entre capas es la condición

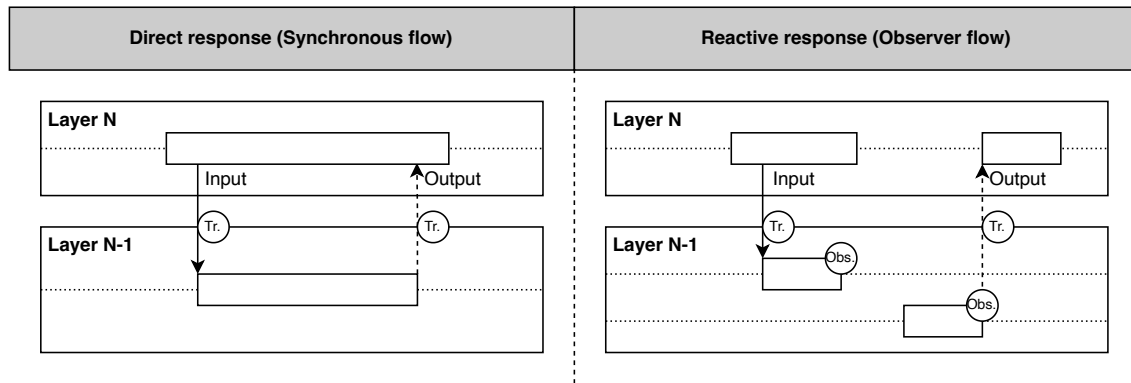


Figure 5. Mecanismo de comunicación ascendente mediante el patrón observador: un componente de negocio muta su estado y notifica a los observadores registrados; un componente de interacción reacciona al cambio sin establecer dependencia directa ascendente sobre el negocio. Esta inversión preserva el principio de aislamiento entre capas.

necesaria para que la tipificación cerrada y exhaustiva (§6.2.4) sea significativa: clasificar un componente en una capa solo tiene valor normativo si esa clasificación implica restricciones reales sobre las dependencias que el componente puede establecer.

6.2.3 Precedencia de la arquitectura sobre la herramienta

Enunciado: la estructura arquitectónica de un artefacto XF deriva de las propiedades del proceso formal que automatiza, y no de las convenciones, patrones o restricciones impuestas por el marco de desarrollo, el lenguaje de programación o la plataforma de ejecución empleados. El marco de desarrollo es el instrumento mediante el cual se implementa la arquitectura; no es su origen ni su determinante.

Motivación El modelo de desarrollo predominante en la industria del software establece implícitamente una relación de subordinación entre la arquitectura y la herramienta: el desarrollador elige un marco de desarrollo y adopta la arquitectura que ese marco prescribe o favorece. Spring prescribe una arquitectura basada en controladores, servicios y repositorios gestionados por un contenedor de inversión de control. Angular prescribe una arquitectura basada en módulos, componentes y servicios con inyección de dependencias. Flutter prescribe una arquitectura basada en widgets, providers y blocs. En todos estos casos, la herramienta precede a la arquitectura — el desarrollador no diseña la arquitectura del artefacto, sino que la hereda del marco.

Esta subordinación tiene consecuencias que van más allá de la mera preferencia estilística. Cuando la arquitectura del artefacto está determinada por el marco de desarrollo, las decisiones de diseño quedan acopladas a las decisiones tecnológicas. Un cambio de marco — motivado por obsolescencia tecnológica, por cambios en los requisitos de rendimiento o por evolución del ecosistema — implica no solo una migración de código, sino una reestructuración arquitectónica completa. El conocimiento arquitectónico acumulado por el equipo durante el desarrollo del artefacto no es transferible a la nueva tecnología porque ese conocimiento está expresado en términos propietarios del marco anterior.

Brooks [7] identificó la **complejidad accidental** — aquella introducida por las

herramientas y los procesos de desarrollo, no por el problema en sí — como una de las causas principales de los fallos en proyectos software. La subordinación de la arquitectura a la herramienta es una fuente sistémica de complejidad accidental: introduce en el artefacto decisiones estructurales que no derivan del dominio de negocio sino de las convenciones del marco, incrementando la distancia conceptual entre el proceso que el artefacto modela y el código que lo implementa.

El modelo XF invierte esta relación de forma explícita y normativa. La arquitectura del artefacto — su organización en capas, la clasificación de sus componentes, las reglas de dependencia entre ellos — deriva de las propiedades del proceso formal que automatiza, propiedades que son independientes de cualquier tecnología. El marco de desarrollo es el instrumento con el que esa arquitectura se implementa, no su origen. Esta inversión no es una preferencia metodológica — es una consecuencia directa del principio de isomorfismo entre proceso y artefacto establecido en §5.1.

Implicaciones normativas La primera implicación es que la adopción del modelo XF en un marco de desarrollo concreto requiere un trabajo de **mapeo tecnológico**: identificar los mecanismos que el marco provee y determinar cómo pueden utilizarse para implementar cada elemento del modelo — capas, tipos de componentes, componentes de inyección, estructura de carpetas — sin adoptar la arquitectura propietaria del marco como sustituto de la arquitectura XF. Este mapeo es específico de cada marco y es objeto del documento complementario de compatibilidad por lenguaje, externo al presente estándar.

La segunda implicación afecta al tratamiento de los constructos propietarios del marco. Todo marco de desarrollo provee constructos propietarios — anotaciones, decoradores, clases base, interfaces — que implementan mecanismos específicos del marco. El modelo XF no prohíbe el uso de estos constructos, pero prescribe que su uso quede **encapsulado** dentro del componente XF correspondiente, de forma que el constructo propietario no sea visible fuera del componente que lo utiliza. Un componente de la Capa de Acceso puede usar la anotación `@Repository` de Spring para beneficiarse de la gestión automática de transacciones que Spring provee, pero esa anotación debe quedar confinada en el componente de acceso correspondiente y no debe propagarse a componentes de otras capas. El artefacto usa el marco; el marco no determina la arquitectura del artefacto.

La tercera implicación es sobre el **orden de las decisiones de diseño**. En un artefacto XF, el análisis del proceso formal que se va a automatizar — su descomposición en etapas, la identificación de los conceptos del dominio, la definición de las operaciones de negocio — precede a cualquier decisión sobre la tecnología de implementación. La elección del marco de desarrollo es una decisión posterior, subordinada a los requisitos arquitectónicos del artefacto y no al revés. Este orden de decisiones es el que garantiza que la arquitectura del artefacto derive del proceso y no de la herramienta.

El marco como implementador de la arquitectura La relación que el modelo XF establece entre arquitectura y herramienta puede caracterizarse con precisión mediante una analogía con disciplinas de ingeniería consolidadas. Un arquitecto de edificios diseña la estructura, la distribución y las propiedades del edificio con

independencia de los materiales con los que vaya a construirse — el diseño arquitectónico precede a la elección de los materiales y los determina, no al revés. Del mismo modo, en el modelo XF el diseño arquitectónico del artefacto precede a la elección del marco de desarrollo y la determina: el marco es el material con el que se construye la arquitectura diseñada, no el origen de esa arquitectura.

Esta analogía no es meramente ilustrativa. Parnas [39] estableció que las decisiones de diseño con mayor impacto sobre la mantenibilidad de un sistema son aquellas que más probable es que cambien durante el ciclo de vida del sistema. En el desarrollo de software, los marcos de desarrollo cambian con mayor frecuencia que los dominios de negocio que los artefactos modelan — los procesos de negocio de una organización evolucionan gradualmente, mientras que el ecosistema tecnológico puede cambiar de forma radical en periodos cortos. Un artefacto cuya arquitectura deriva del proceso de negocio y no del marco es, por tanto, estructuralmente más resistente a los cambios tecnológicos que uno cuya arquitectura está determinada por el marco.

Alcance y límites del principio La precedencia de la arquitectura sobre la herramienta no implica ignorancia de las capacidades y restricciones del marco. Un buen diseño arquitectónico en el modelo XF aprovecha las capacidades del marco elegido para implementar con eficiencia los elementos que el modelo prescribe — los mecanismos de gestión de conexiones del marco para los componentes de acceso, los mecanismos de gestión de estado para los componentes de negocio, los mecanismos de routing para los componentes de interacción. Lo que el principio prescribe es que estas capacidades se usen al servicio de la arquitectura XF, y no que determinen su estructura.

Del mismo modo, cuando un marco impone restricciones que dificultan la implementación de algún elemento del modelo — por ejemplo, la gestión automática del ciclo de vida en marcos de desarrollo con inyección de dependencias automática, que puede entrar en tensión con el mecanismo de inicialización prescrito por los componentes de inyección — el principio de precedencia de la arquitectura establece que la solución debe buscarse en la forma de implementar el espíritu del modelo dentro de las restricciones del marco, no en la relajación de los requisitos del modelo. El tratamiento específico de estas situaciones queda fuera del alcance del presente estándar. La [figura siguiente](#) contrasta el paradigma tradicional con el del modelo XF.

Relación con otros principios La precedencia de la arquitectura sobre la herramienta es la condición que hace posible el agnosticismo tecnológico (§6.2.1): si la arquitectura dependiera de la herramienta, el modelo no podría ser agnóstico respecto a ella. Del mismo modo, este principio refuerza el aislamiento entre capas (§6.2.2): cuando la arquitectura precede a la herramienta, las capas se definen en términos del proceso — acceso, negocio, interacción — y no en términos de los constructos del marco, lo que elimina el riesgo de que las convenciones propietarias del marco distorsionen la estructura de capas del artefacto.

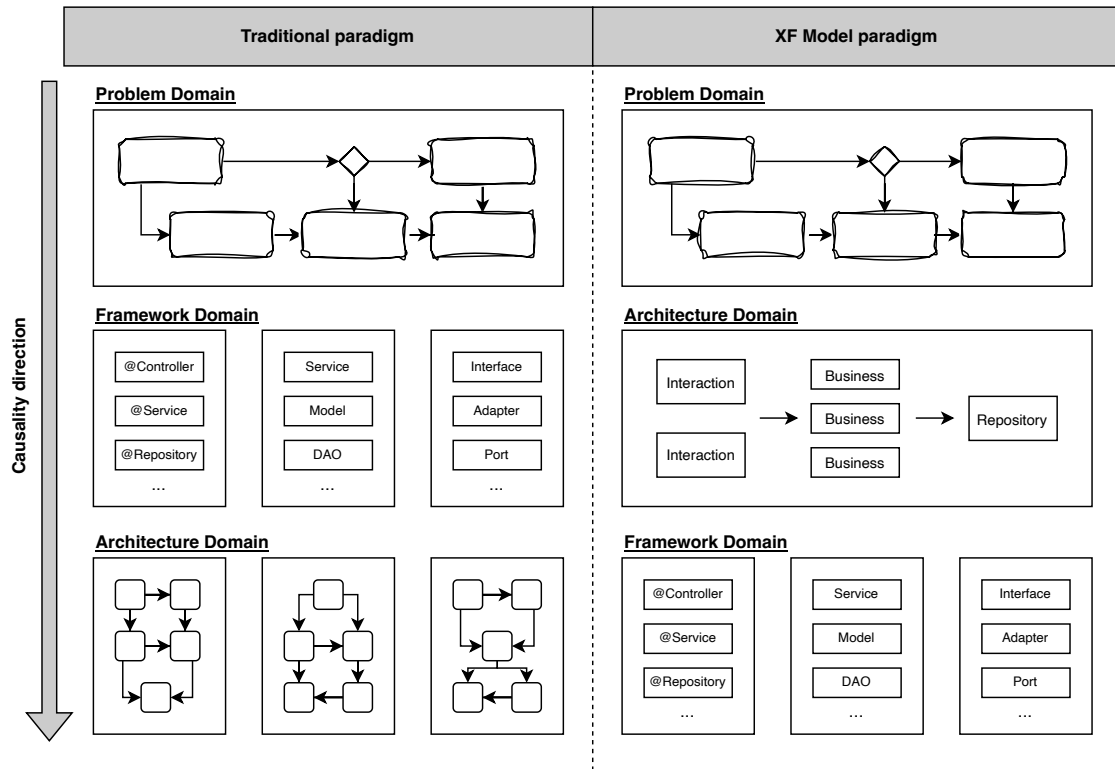


Figure 6. Inversión de la causalidad entre arquitectura y marco de desarrollo. En el paradigma tradicional, el marco precede y determina la estructura del artefacto (Problema → Marco → Arquitectura); en el modelo XF, la arquitectura precede y determina el uso del marco (Problema → Arquitectura → Marco).

6.2.4 Tipificación cerrada y exhaustiva

Enunciado: el modelo XF define un sistema de clasificación de componentes que es simultáneamente **cerrado** y **exhaustivo**. Es cerrado porque el conjunto de tipos de componentes reconocidos por el modelo es finito y no extensible: cinco tipos funcionales distribuidos en tres capas de abstracción producen un espacio de clasificación de quince categorías posibles, fuera del cual el modelo no reconoce ninguna categoría adicional. Es exhaustivo porque todo componente de un artefacto software admite clasificación en exactamente una de esas quince categorías: no existe componente que no tenga clasificación válida en el modelo, y no existe componente que requiera pertenecer simultáneamente a más de una categoría. La exhaustividad es una propiedad estructural del sistema de clasificación — no una afirmación de trivialidad. La clasificación de algunos componentes no es inmediata y requiere análisis contextual de su responsabilidad principal y de su modo de operación; el modelo provee patrones canónicos para los casos no triviales más adelante en esta misma cláusula.

Motivación La tipificación cerrada y exhaustiva es la propiedad que confiere al modelo XF su capacidad de eliminar la ambigüedad estructural documentada en §5.4. Un sistema de clasificación abierto — en el que los equipos de desarrollo pueden definir tipos adicionales según sus necesidades — reproduce exactamente el problema que el modelo pretende resolver: la proliferación de taxonomías propietarias incompatibles entre proyectos y equipos. Un sistema de clasificación no exhaustivo

— en el que existen componentes que no pueden ser clasificados — introduce zonas grises estructurales que son fuente de inconsistencias y de deuda técnica.

La propiedad de cierre y exhaustividad de un sistema de tipos es un fundamento clásico de la teoría de tipos en ingeniería del software. Por analogía con el principio de subtipado conductual — formalizado por Liskov y Wing [27] en el dominio de los sistemas de tipos — la corrección de un sistema con tipos depende de que toda instancia de un tipo preserve las propiedades estructurales y conductuales declaradas por su tipo. La extensión análoga a un sistema de clasificación arquitectónica es directa: una taxonomía cerrada y exhaustiva permite verificar determinísticamente que todo componente respete las propiedades de su categoría, mientras que una taxonomía abierta o no exhaustiva introduce zonas grises donde el contrato del tipo no es vinculante. El modelo XF aplica este principio al sistema de clasificación de componentes: un sistema de clasificación que no puede clasificar todos los elementos de su dominio es un sistema incompleto, y un sistema incompleto no puede ser la base de un estándar normativo. El modelo XF establece que su dominio — el conjunto de todos los componentes de cualquier artefacto software — está completamente cubierto por su sistema de clasificación, y que esta cobertura es una propiedad del modelo que debe preservarse en cualquier evolución futura.

Desde una perspectiva práctica, la tipificación cerrada y exhaustiva tiene un corolario de gran valor operativo: **obliga al desarrollador a tomar una decisión de diseño explícita antes de implementar cualquier componente**. En el modelo de desarrollo tradicional, un desarrollador puede comenzar a implementar un componente sin haber decidido con precisión qué tipo de lógica va a contener — esa decisión se toma implícitamente durante la implementación, con el riesgo de que el resultado sea un componente con responsabilidades mixtas que viole el principio de separación de intereses. En el modelo XF, la clasificación del componente en una capa y un tipo debe preceder a su implementación, porque esa clasificación determina qué lógica puede contener y de qué otros componentes pueden depender. La tipificación es, en este sentido, un instrumento de disciplina de diseño tanto como un instrumento de organización estructural.

Implicaciones normativas La implicación normativa de la tipificación cerrada y exhaustiva es que todo componente del artefacto debe quedar clasificado en exactamente una de las categorías previstas, sin residuos ni solapamientos: ningún componente válido cae fuera de la taxonomía. Esta propiedad de cierre distingue al modelo de las taxonomías previas de la disciplina, como se contrasta a continuación.

Diferenciación respecto a taxonomías cerradas previas La propiedad de cierre del sistema de clasificación tiene precedentes en la disciplina de los Lenguajes de Descripción de Arquitectura. Sistemas formales como ACME [2] proveen sintaxis para definir conjuntos cerrados de tipos de componente y de conector por proyecto, dejando la elección de los tipos concretos al criterio del arquitecto. La norma [22] establece un metamodelo cerrado de conceptos abstractos — vista, punto de vista, preocupación — sobre el que toda descripción arquitectónica debe construirse, sin prescribir tipos concretos de componente.

El modelo XF se distingue de estos precedentes en una dimensión decisiva: prescribe

un conjunto cerrado, exhaustivo y agnóstico al lenguaje de *tipos concretos* de componente — no una sintaxis para definirlos. La consecuencia operativa es que un artefacto XF tiene una clasificación válida y verificable sin dependencia del proyecto, del equipo o del marco de descripción adoptado. La taxonomía cerrada de tipos concretos universales es la novedad estructural del modelo respecto al estado del arte.

El sistema de clasificación El espacio de clasificación del modelo XF está definido por el producto cartesiano de dos conjuntos finitos:

- El conjunto de capas: $\mathbf{L} = \{\text{Acceso, Negocio, Interacción}\}$, ordenado por nivel de abstracción creciente.
- El conjunto de tipos funcionales: $\mathbf{T} = \{\text{Lógico, Generalización, Inyección, Utilidad, Transferencia}\}$.

El producto cartesiano $L \times T$ produce quince categorías de clasificación posibles. Todo componente de un artefacto XF pertenece a exactamente un elemento de $L \times T$. La función de clasificación $\phi : \mathbb{C} \rightarrow L \times T$, donde $\mathbb{C}$ es el conjunto de componentes del artefacto, es total — definida para todo elemento de $\mathbb{C}$, de modo que ningún componente pertenece a más de una categoría.

La clasificación como decisión de diseño previa a la implementación El modelo XF prescribe que la clasificación de un componente en una categoría de $L \times T$ es una decisión que **debe** tomarse antes de comenzar su implementación. Esta prescripción no es meramente organizativa — tiene consecuencias normativas directas sobre lo que el componente puede contener.

La clasificación en una capa determina qué tipos de lógica puede implementar el componente: un componente clasificado en la Capa de Acceso implementa exclusivamente lógica de acceso a sistemas externos; un componente clasificado en la Capa de Negocio implementa exclusivamente lógica de dominio; un componente clasificado en la Capa de Interacción implementa exclusivamente lógica de interacción. La presencia en un componente de lógica correspondiente a una capa distinta a la que está clasificado constituye una violación del modelo, independientemente de que esa lógica sea funcionalmente correcta.

La clasificación en un tipo funcional determina qué función desempeña el componente dentro de su capa: un componente de tipo lógico implementa la lógica efectiva de la capa; un componente de tipo generalización abstrae comportamiento común entre lógicos de la misma capa; un componente de tipo inyección gestiona el ciclo de vida de los lógicos de la capa; un componente de tipo utilidad provee operaciones auxiliares; un componente de tipo transferencia define estructuras de datos. Un componente que no satisfaga la función correspondiente a su tipo clasificado constituye igualmente una violación del modelo.

Alcance y límites del principio

Tratamiento de componentes no clasificables El modelo XF establece que todo componente de un artefacto software admite clasificación en alguna categoría

de $L \times T$. Sin embargo, en la práctica del desarrollo los equipos pueden encontrarse con componentes cuya clasificación no resulte inmediata. El modelo prescribe que en estos casos el equipo **debe** aplicar uno de los siguientes tres análisis antes de proceder a la implementación.

El primero es el **análisis de responsabilidad única**: verificar si el componente que no puede clasificarse está asumiendo responsabilidades que corresponden a más de una categoría de $L \times T$. Si es así, el componente debe descomponerse en tantos componentes clasificables como responsabilidades distintas contiene. Esta situación es la más frecuente y su resolución mediante descomposición es la práctica que el modelo favorece.

El segundo es el **análisis de completitud de la lógica**: verificar si el componente que no puede clasificarse está implementando una lógica incompleta que debería complementarse para ajustarse a alguna de las categorías del modelo. Un componente que realiza transformaciones de datos pero no implementa ninguna regla de negocio puede no encajar en el tipo lógico, pero podría completarse para convertirse en un componente de utilidad o en un componente de transferencia, según la naturaleza de las transformaciones que realiza.

El tercero es el **análisis del contexto operativo**: cuando la responsabilidad del componente sea única y su lógica sea completa pero su clasificación siga sin ser inmediata, el modelo prescribe identificar el patrón canónico aplicable según el contexto en que el componente opera. Esta cláusula documenta los patrones canónicos para los casos que con mayor frecuencia generan duda en la práctica.

Disparadores de ejecución Un disparador de ejecución es la lógica que inicia la ejecución de una operación sin que ésta haya sido invocada explícitamente por otro componente del artefacto en ese instante. El modelo distingue dos subtipos según el origen de la condición de disparo:

- Disparadores **autónomos**: la condición de disparo es interna al componente — transcurso de un intervalo temporal medido por el propio componente, alcance de un estado interno, finalización de un proceso interno previo. La lógica de disparo pertenece a la misma capa del componente lógico que la requiere. El disparo autónomo **no** convierte al componente en un punto de entrada de la Capa de Interacción, porque no media interacción con un consumidor externo: la Capa de Interacción formaliza el protocolo por el que un consumidor externo invoca operaciones del artefacto, no cualquier mecanismo interno que las arranque.
- Disparadores basados en **señales externas**: la condición de disparo es la recepción de una señal de un sistema externo a través de un protocolo de comunicación. La suscripción al sistema externo y la recepción de la señal son comunicación con el exterior; estos componentes pertenecen a la **Capa de Acceso** y se clasifican como componentes lógicos de acceso.

La distinción es relevante porque el origen de la condición de disparo determina la clasificación: si la condición es interna al artefacto, la lógica del disparo vive en la capa del componente disparado; si la condición proviene de un sistema externo, la lógica del disparo es lógica de acceso.

Lógica contextual a múltiples componentes lógicos La lógica contextual — aquella que da soporte a la lógica efectiva sin formar parte del propósito principal de ningún componente lógico — no requiere ningún tipo de componente adicional: se clasifica dentro de los cinco tipos canónicos de la matriz $L \times T$ como cualquier otra lógica del artefacto, según su naturaleza y su capa. La taxonomía permanece cerrada y exhaustiva, sin tipos adicionales.

Si tras la aplicación de los tres análisis — responsabilidad única, completitud de la lógica y contexto operativo — el componente sigue sin poder clasificarse en ninguna categoría de $L \times T$, el modelo prescribe que su existencia debe ser sometida a revisión arquitectónica, ya que la imposibilidad de clasificación indica que el componente no se ajusta a los principios del modelo y constituye una violación de la tipificación exhaustiva.

Relación con otros principios La tipificación cerrada y exhaustiva es el principio que hace verificable el cumplimiento de los demás. El agnosticismo tecnológico (§6.2.1) es verificable porque la clasificación en $L \times T$ es independiente de la tecnología. El aislamiento entre capas (§6.2.2) es verificable porque las reglas de dependencia se aplican sobre categorías de clasificación precisas. La precedencia de la arquitectura sobre la herramienta (§6.2.3) es verificable porque la clasificación de todos los componentes en categorías derivadas del proceso — y no del marco — es una condición comprobable. Sin tipificación cerrada y exhaustiva, los tres principios anteriores serían principios declarativos sin mecanismo de verificación.

Esta propiedad de verificabilidad es la que hace posible la definición de herramientas de análisis estático de conformidad — analizadores de código, plugins de IDE, generadores de informes de conformidad — que pueden evaluar de forma automática el grado de ajuste de un artefacto al modelo XF. La naturaleza formal y cerrada del sistema de clasificación es condición necesaria para que estas herramientas puedan operar con precisión y sin ambigüedad. Las reglas de conformidad del modelo se fundamentan directamente en las propiedades del sistema de clasificación establecidas en este principio.

7 Taxonomía de componentes

La taxonomía de componentes del modelo XF organiza todo artefacto en una matriz de dos dimensiones: la *dimensión de abstracción* — las capas — y la *dimensión funcional* — los tipos de componente —, cuya intersección clasifica cada componente y que se materializa físicamente en la estructura de carpetas del proyecto. Esta cláusula define primero la estructura matricial que articula ambas dimensiones, después cada dimensión por separado y, por último, su materialización canónica en carpetas.

7.1 Estructura matricial: capas y tipos de componentes

La aplicación simultánea de las dos dimensiones de clasificación definidas en §6.1 — la dimensión de abstracción y la dimensión funcional — produce la estructura organizativa fundamental del modelo XF: una matriz de tres filas y cinco columnas en

la que cada fila corresponde a una capa de abstracción y cada columna corresponde a un tipo funcional de componente.

Cada celda de esta matriz representa una categoría de componente válida en el modelo. Todo componente de un artefacto XF ocupa exactamente una celda de la matriz — ni más ni menos. La matriz es, por tanto, la representación canónica del espacio de clasificación $L \times T$ definido en §6.2.4, y constituye la referencia estructural sobre la que se fundamentan todas las reglas de conformidad del modelo.

La estructura matricial se presenta a continuación de forma tabular (Tabla 2).

Representación tabular La tabla siguiente presenta la estructura matricial completa del modelo XF, con la nomenclatura canónica prescrita para cada categoría de componente:

Table 2. Patrón canónico de nomenclatura por categoría de componente. La marca $\langle N \rangle$ denota el nombre del concepto modelado por un componente lógico, de utilidad o de transferencia; $\langle P \rangle$ denota el nombre del patrón estructural abstraído por una generalización. Los componentes de inyección llevan nomenclatura de una sola letra — A, B, R — común a todos los artefactos XF, independiente del concepto que albergan.

Capa \ Tipo	Lógico	Generalización	Inyección	Utilidad	Transferencia
Interacción	$\langle N \rangle$ Service	$\langle P \rangle$ Service	A	$\langle N \rangle$ Utils	$\langle N \rangle$
	$\langle N \rangle$ View	$\langle P \rangle$ View			
Negocio	$\langle N \rangle$ Business	$\langle P \rangle$ Business	B	$\langle N \rangle$ Utils	$\langle N \rangle$
Acceso	$\langle N \rangle$ Repository	$\langle P \rangle$ Repository	R	$\langle N \rangle$ Utils	$\langle N \rangle$

Notación: $\langle N \rangle$ representa el nombre del concepto que el componente modela — típicamente un nombre de dominio para componentes lógicos, de utilidad y de transferencia. $\langle P \rangle$ representa el nombre del patrón estructural que el componente de generalización abstrae. Los componentes de inyección son los únicos cuya nomenclatura es una constante de una sola letra — R, B, A — común a todos los artefactos XF.

Propiedades de la estructura matricial La estructura matricial tiene tres propiedades formales que derivan directamente de los principios rectores establecidos en §6.2 y que deben ser satisfechas por todo artefacto XF.

La primera propiedad es la **completitud**: toda celda de la matriz puede estar poblada en cualquier artefacto XF. No existe celda vacía por definición — la ausencia de componentes en una celda concreta en un artefacto particular es una decisión de diseño válida, no una restricción del modelo. Un artefacto puramente backend puede no tener componentes de tipo **View** en la Capa de Interacción; un artefacto simple puede no requerir componentes de generalización en ninguna capa. La matriz define el espacio de lo posible, no el conjunto de lo obligatorio.

La segunda propiedad es la **unicidad de clasificación**: ningún componente puede pertenecer simultáneamente a más de una celda. La clasificación de un componente es una función total del conjunto de componentes del artefacto en el espacio de clasificación $L \times T$ — cada componente tiene exactamente una imagen en la matriz.

La tercera propiedad es la **restricción de dependencias por fila**: las dependencias entre componentes lógicos — tipo Lógico, Generalización e Inyección — están restringidas por la fila a la que pertenecen, siguiendo el principio de aislamiento entre capas establecido en §6.2.2. Un componente de la fila Interacción puede depender de componentes de las filas Negocio y Acceso, pero nunca al revés. Dentro de una misma fila, todo acceso entre componentes lógicos se realiza exclusivamente a través del componente de inyección de esa fila.

Relaciones entre columnas Además de las restricciones por filas, la estructura matricial establece relaciones funcionales entre las columnas que determinan cómo los distintos tipos de componentes interactúan dentro de cada capa.

Los componentes de tipo **Lógico** son el núcleo funcional de cada capa — implementan la lógica efectiva que justifica la existencia de la capa. Son los únicos componentes que pueden mantener estado mutable en el artefacto, y son los únicos cuyas instancias son gestionadas por el componente de **Inyección** de su capa. Todo acceso a un componente lógico desde fuera de su capa se realiza exclusivamente a través del componente de inyección correspondiente.

Los componentes de tipo **Generalización** existen en función de los componentes lógicos de su misma capa — abstraen comportamiento común entre ellos y no tienen utilidad independiente de los lógicos que los heredan. Su alcance está estrictamente limitado a la capa en la que están definidos: un componente de generalización de la Capa de Negocio no debe ser heredado por un componente lógico de la Capa de Acceso ni de la Capa de Interacción.

Los componentes de tipo **Inyección** tienen una posición singular en la estructura matricial: son el único punto de acceso a los componentes lógicos de su capa desde cualquier otro componente del artefacto, incluyendo componentes de la misma capa. Existe exactamente un componente de inyección por capa — **R** para la Capa de Acceso, **B** para la Capa de Negocio, **A** para la Capa de Interacción — y su definición determina qué componentes lógicos están disponibles en el artefacto y en qué orden se inicializan.

Los componentes de tipo **Utilidad** proveen operaciones auxiliares sin estado y sin efectos laterales sobre las estructuras de datos de su capa. No son instanciables y no mantienen estado. Su alcance es local a la capa en la que están definidos, con la excepción de los componentes de utilidad de la Capa de Acceso que operan sobre tipos primitivos — cadenas, fechas, números, decimales — heredados formalmente de la Capa de Presentación del modelo OSI (Nivel 6). Estos componentes de utilidad sobre tipos primitivos pueden ser referenciados por componentes de cualquier capa sin que ello constituya una violación del principio de aislamiento, dado que los tipos primitivos son anteriores a cualquier transformación semántica del artefacto.

Los componentes de tipo **Transferencia** definen las estructuras de datos que circulan entre componentes. No contienen lógica de comportamiento — su función es exclusivamente la de modelar la forma de los datos en un punto concreto del flujo de procesamiento.

Nomenclatura canónica El modelo XF prescribe una nomenclatura canónica para cada categoría de la matriz que debe ser respetada con independencia de las convenciones de nombrado del lenguaje o marco de desarrollo empleado. Esta nomenclatura es parte del modelo y su consistencia es condición necesaria para la interoperabilidad semántica entre equipos y proyectos que es uno de los objetivos centrales del modelo.

Los componentes lógicos de la Capa de Acceso deben llevar el sufijo **Repository**. Los componentes lógicos de la Capa de Negocio deben llevar el sufijo **Business**. Los componentes lógicos de la Capa de Interacción deben llevar el sufijo **Service** o **View** según sean interacciones sistémicas o gráficas respectivamente. Los componentes de utilidad deben llevar el sufijo **Utils**. Los componentes de transferencia no llevan sufijo adicional — su nombre describe el concepto del dominio del artefacto que modelan (de negocio, de acceso o de interacción). Los componentes de inyección reciben los nombres canónicos **R**, **B** y **A** respectivamente. Los componentes de generalización llevan el prefijo o el nombre del patrón que abstraen seguido del nombre de la capa — por ejemplo, **StatefulBusiness**, **RestRepository**, **FormView**.

La [tabla siguiente](#) ilustra la clasificación matricial $L \times T$ sobre un artefacto concreto (un termostato).

Table 3. Ejemplo concreto de la estructura matricial $L \times T$ aplicada al artefacto termostato. Cada celda contiene los componentes reales del artefacto clasificados en su categoría canónica.

Capa \ Tipo	Lógico	Generalización	Inyección	Utilidad	Transferencia
Interacción	TemperatureService	RestService	A	HttpUtils	Response User
	ThermometerView	FormView			
	LoginView				
Negocio	SessionBusiness	StatefulBusiness	B	TemperatureUtils	Session Temperature
	TemperatureBusiness				
Acceso	IdentityRepository	RestRepository	R	StringUtils	User
	DatabaseRepository	SqlRepository		DateUtils	NetworkException
					AuthResponse

7.2 Dimensión de abstracción: las capas

La presente cláusula define las tres capas de abstracción que componen el modelo XF — Acceso, Negocio e Interacción — en sus términos normativos. Generaliza el proceso de cada capa individual, definiendo su responsabilidad específica, los límites que la separan de las capas adyacentes, el modelo de proceso que prescribe para su lógica interna y la organización estructural que debe adoptar en el sistema de ficheros del proyecto.

La estratificación del modelo XF no es una convención organizativa — es una consecuencia directa del principio de isomorfismo entre proceso y artefacto establecido en §5.1 y formalizado en §5.5. Cada capa corresponde biunívocamente a una etapa invariante de todo proceso formal: la Capa de Acceso modela la etapa de acceso, la Capa de Negocio modela la etapa de procesamiento y la Capa de Interacción modela la etapa de interacción. Esta correspondencia determina con precisión qué lógica pertenece a cada capa y, por exclusión, qué lógica no pertenece a ella — un criterio

de clasificación que es independiente de la tecnología, el dominio de negocio y la escala del artefacto.

Las tres capas comparten la misma estructura interna de tipos de componentes — que se define en §7.3 — y las mismas reglas de dependencia — definidas en §6.2.2. Lo que las diferencia es la naturaleza de la lógica que implementan y el modelo de proceso que esa lógica debe satisfacer. La definición de ese modelo de proceso para cada capa es el objeto central de esta cláusula.

La cláusula se organiza en tres cláusulas internas paralelas y simétricas, una por capa, cada una con la misma estructura interna: responsabilidad y límites, modelo de proceso de la capa, subdivisiones de componentes lógicos y organización interna. Esta simetría es intencional — refleja la simetría estructural del modelo y facilita la navegación del documento por parte de desarrolladores que buscan información sobre una capa concreta sin necesidad de leer la cláusula completa.

7.2.1 Capa de Acceso

La Capa de Acceso constituye el nivel de abstracción más bajo de la Arquitectura XF y es la única capa del modelo que tiene contacto directo con la Capa de Presentación del modelo OSI (Nivel 6). Su posición en la estratificación no es arbitraria — es consecuencia directa de la derivación formal establecida en §5.5: la etapa de acceso de todo proceso formal es aquella en la que el proceso delega parte de su ejecución en sistemas externos, y esa delegación requiere necesariamente resolver los protocolos de comunicación antes de que cualquier lógica de negocio pueda operar sobre los datos obtenidos.

Todo dato que el artefacto necesite obtener de un sistema externo — independientemente de si ese sistema es una base de datos relacional, una API REST, un fichero del sistema operativo, un dispositivo conectado por puerto serie o un servicio del sistema operativo — entra al artefacto a través de la Capa de Acceso. Del mismo modo, todo dato que el artefacto necesite enviar a un sistema externo sale del artefacto a través de esta capa. La Capa de Acceso es, en este sentido, la frontera entre el artefacto y su entorno — el punto en el que el artefacto deja de ser autónomo y necesita comunicarse con otros sistemas para cumplir su función.

Los componentes lógicos de la Capa de Acceso — denominados Repositorios — encapsulan los protocolos de comunicación de forma que las capas superiores nunca conozcan los detalles de cómo se establece esa comunicación. Esta propiedad de abstracción de protocolo es la que hace posible el principio de agnosticismo tecnológico del modelo XF a nivel de datos: un cambio en el protocolo de comunicación con un sistema externo — por ejemplo, la migración de una API REST a GraphQL — queda confinado en el Repositorio correspondiente y no requiere modificaciones en la Capa de Negocio ni en la Capa de Interacción.

Responsabilidad y límites Esta cláusula delimita la responsabilidad de la Capa de Acceso y sus fronteras: el límite inferior con el entorno de datos del artefacto y el límite superior con la Capa de Negocio.

Responsabilidad La responsabilidad de la Capa de Acceso es resolver de forma completa y autónoma los protocolos de comunicación con los sistemas externos al artefacto, abstraer los detalles de esos protocolos de las capas superiores, y proveer a la Capa de Negocio datos normalizados en componentes de transferencia que son independientes del protocolo utilizado para obtenerlos.

Esta responsabilidad tiene tres dimensiones que el modelo XF prescribe:

La primera dimensión es la **abstracción de protocolo**: todo detalle relativo al protocolo de comunicación — cadenas de conexión, cabeceras HTTP, sentencias SQL, formatos de serialización, mecanismos de autenticación con sistemas externos — debe quedar confinado en los componentes lógicos de la Capa de Acceso. Ningún componente de la Capa de Negocio ni de la Capa de Interacción debe conocer el protocolo utilizado para obtener un dato. Si un componente de negocio conoce que los datos de usuario se obtienen mediante una llamada REST a un endpoint concreto, la abstracción de protocolo ha sido violada.

La segunda dimensión es la **normalización de datos**: los datos obtenidos de sistemas externos llegan al artefacto en el formato que cada sistema externo define — streams de bytes, cadenas JSON, cursores de base de datos, tramas de protocolo propietario. La Capa de Acceso es responsable de transformar esos datos en componentes de transferencia estructurados e interpretables por la Capa de Negocio, sin aplicar ninguna transformación semántica sobre ellos. La normalización es una transformación sintáctica — de formato de protocolo a estructura de datos — no una transformación semántica. Aplicar reglas de negocio sobre los datos durante la normalización constituye una violación de la responsabilidad de la capa.

La tercera dimensión es la **gestión de errores de comunicación**: la Capa de Acceso es responsable de detectar y propagar hacia las capas superiores los errores que indiquen condiciones de ejecución inesperadas en la comunicación con sistemas externos. La gestión concreta de cada tipo de error es decisión de implementación del desarrollador — el modelo XF prescribe la responsabilidad, no la implementación.

Límite inferior — Capa de Presentación OSI (Nivel 6) El límite inferior de la Capa de Acceso es la Capa de Presentación del modelo OSI (Nivel 6), que provee al artefacto los tipos primitivos (§3.1.15) que constituyen la materia prima sobre la que opera la Capa de Acceso. Estos tipos primitivos son anteriores a cualquier transformación semántica del artefacto y, como se establece en §7.3.4, los componentes de utilidad que operan sobre ellos se definen en la Capa de Acceso y pueden ser referenciados por cualquier capa del artefacto sin restricción.

Límite superior — Capa de Negocio El límite superior de la Capa de Acceso es la Capa de Negocio, a la que provee datos normalizados en componentes de transferencia. La Capa de Acceso no conoce qué hace la Capa de Negocio con esos datos — su responsabilidad termina en el momento en que entrega un componente de transferencia correctamente normalizado o propaga una excepción que indica que la comunicación no pudo completarse satisfactoriamente. Toda lógica que decida qué hacer con los datos obtenidos pertenece a la Capa de Negocio, no a la Capa de Acceso.

Modelo de proceso El modelo de proceso de acceso define qué caracteriza funcionalmente a la lógica que pertenece a la Capa de Acceso, de forma que todo desarrollador pueda determinar sin ambigüedad si una pieza de lógica concreta pertenece a esta capa o a otra. Las prescripciones de esta cláusula son independientes del protocolo de comunicación empleado, del tipo de sistema externo al que se accede y del lenguaje o marco de desarrollo utilizado para la implementación.

El modelo de proceso de acceso no prescribe cómo debe implementarse la comunicación con sistemas externos — esa decisión corresponde al desarrollador en función del protocolo concreto, las librerías disponibles y las características del sistema externo. El modelo prescribe qué responsabilidades debe satisfacer todo componente de acceso y qué responsabilidades le están explícitamente vedadas. La orientación de implementación para los protocolos más comunes se recoge en el Anexo A de este documento.

Responsabilidades prescritas Todo componente lógico de la Capa de Acceso **debe** satisfacer las siguientes responsabilidades con independencia del protocolo que implemente:

La primera responsabilidad es la **encapsulación completa del protocolo**. Todo detalle relativo al protocolo de comunicación — parámetros de conexión, formatos de serialización, mecanismos de autenticación con sistemas externos, gestión del canal de comunicación — debe quedar confinado en el componente de acceso. Ningún componente de la Capa de Negocio ni de la Capa de Interacción puede conocer el protocolo utilizado para obtener o enviar un dato. La violación de esta responsabilidad es verificable de forma directa: si un componente de negocio importa una librería de protocolo, referencia una URL, construye una sentencia SQL o manipula cabeceras HTTP, la encapsulación ha sido violada.

La segunda responsabilidad es la **normalización de los datos recibidos**. Los datos obtenidos de sistemas externos llegan al artefacto en el formato que cada sistema externo define. El componente de acceso es responsable de transformar esos datos en componentes de transferencia estructurados e interpretables por la Capa de Negocio. Esta transformación es estrictamente sintáctica — convierte el formato de protocolo en una estructura de datos — y no debe incorporar ninguna transformación semántica sobre los datos. Un componente de acceso que filtra, agrega, enriquece o transforma semánticamente los datos durante la normalización está asumiendo responsabilidades que pertenecen a la Capa de Negocio.

La tercera responsabilidad es la **propagación de errores de contexto de uso**. Los errores que indiquen condiciones de ejecución inesperadas en la comunicación con el sistema externo — sistema no disponible, respuesta con formato inesperado, datos sintácticamente incoherentes respecto al contrato del protocolo — deben propagarse hacia las capas superiores mediante componentes de transferencia de excepción. El componente de acceso no debe tomar decisiones de negocio basadas en ellos. La Capa de Negocio es la responsable de decidir cómo reaccionar ante un error de comunicación — el componente de acceso debe limitarse a comunicarlo de forma precisa.

Responsabilidades excluidas El modelo XF prescribe explícitamente que los siguientes tipos de lógica no pertenecen a la Capa de Acceso. Su presencia en un componente de acceso constituye una violación del principio de aislamiento establecido en §6.2.2:

Un componente de acceso **no debe** aplicar reglas de negocio sobre los datos obtenidos. Ninguna transformación semántica, validación de dominio, cálculo derivado o decisión basada en el significado de un dato pertenece a esta capa. Si un componente de acceso toma una decisión basada en el valor de un dato de negocio — y no en la respuesta del protocolo de comunicación — esa lógica pertenece a la Capa de Negocio.

Un componente de acceso **no debe** gestionar el estado del dominio. Mantener en memoria el estado actual de un concepto de negocio, coordinar datos entre múltiples fuentes o notificar a observadores de cambios de estado son responsabilidades de la Capa de Negocio.

Un componente de acceso **no debe** definir interfaces de interacción. Ningún componente de acceso puede definir puntos de entrada al artefacto, contratos de servicio o lógica de presentación de ningún tipo.

Un componente de acceso **no debe** invocar componentes de capas superiores. La Capa de Acceso no debe invocar operaciones de la Capa de Negocio ni de la Capa de Interacción a través de sus componentes de inyección. Toda comunicación ascendente — notificación de cambios de estado, eventos de sistema — se resuelve sin invocación directa ascendente, típicamente mediante el patrón de observación (§6.2.2).

Criterio de clasificación El modelo XF prescribe el siguiente criterio operativo para determinar si una pieza de lógica concreta pertenece a la Capa de Acceso. Una pieza de lógica pertenece a esta capa si y solo si satisface simultáneamente las tres condiciones siguientes:

La primera condición es que **conoce un protocolo de comunicación con un sistema externo** — sabe cómo establecer la comunicación con ese sistema, independientemente de si es remoto o local al dispositivo.

La segunda condición es que **no conoce ninguna regla de negocio del dominio** — no toma decisiones basadas en el significado de los datos, solo en la respuesta del protocolo de comunicación.

La tercera condición es que **su resultado es un componente de transferencia normalizado o una excepción de comunicación** — no produce resultados que impliquen interpretación semántica de los datos ni decisiones de dominio.

Si una pieza de lógica no satisface la primera condición, no pertenece a la Capa de Acceso. Si satisface la primera, pero no satisface la segunda o la tercera, debe ser descompuesta: la parte que resuelve el protocolo pertenece a la Capa de Acceso, y la parte que aplica semántica o toma decisiones de dominio pertenece a la Capa de Negocio.

Terminología de referencia El modelo XF define los siguientes términos para describir con precisión los elementos que componen la lógica de acceso de un artefacto. Esta terminología constituye un vocabulario común que facilita la comunicación entre

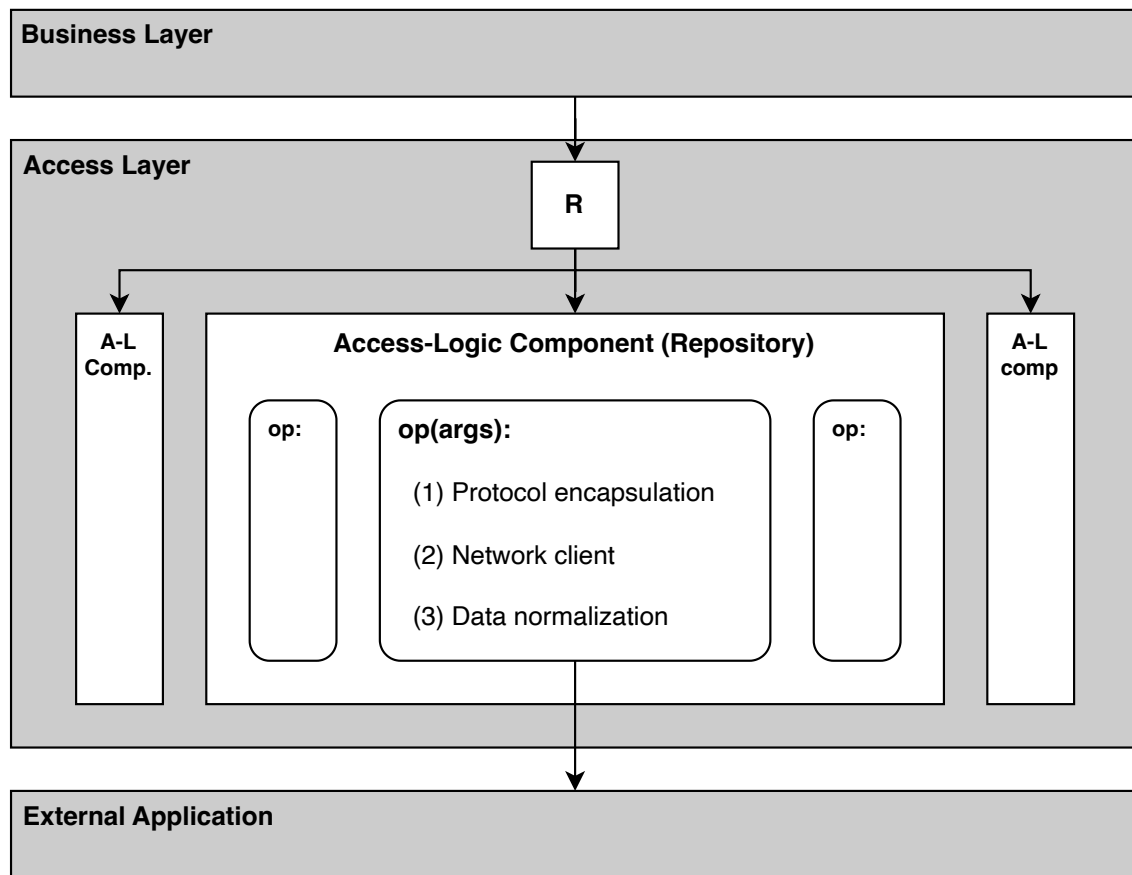


Figure 7. Contenido prescrito de un componente lógico de la Capa de Acceso. El **Repository** encapsula el protocolo de comunicación con un sistema externo, recibe invocaciones a través del componente de inyección **R**, normaliza sintácticamente los datos del protocolo y entrega componentes de transferencia o excepciones de comunicación a la Capa de Negocio. Ninguna lógica de dominio reside en este componente; ningún componente de capa superior se accede desde él.

desarrolladores, analistas y revisores sobre la lógica de acceso de cualquier artefacto XF con independencia de la tecnología empleada.

Se denomina **abstracción de protocolo** a la propiedad de los componentes lógicos de la Capa de Acceso por la cual los detalles del protocolo de comunicación con un sistema externo quedan confinados en el componente y son invisibles para las capas superiores. La abstracción de protocolo es la propiedad estructural que hace posible el principio de agnosticismo tecnológico a nivel de datos: un cambio de protocolo queda confinado en el repositorio correspondiente sin requerir modificaciones en la Capa de Negocio ni en la Capa de Interacción.

Se denomina **normalización de datos** a la transformación que un componente de acceso aplica sobre los datos recibidos de un sistema externo para convertirlos en componentes de transferencia interpretables por la Capa de Negocio. La normalización es una transformación estrictamente sintáctica — de formato de protocolo a estructura de datos — sin transformación semántica ni aplicación de reglas de dominio.

Se denomina **propagación de errores de comunicación** a la transmisión hacia

las capas superiores, mediante componentes de transferencia de excepción, de las condiciones de ejecución inesperadas detectadas durante la comunicación con un sistema externo. La Capa de Acceso propaga el error sin tomar decisiones de negocio sobre él; la interpretación y respuesta corresponden a la Capa de Negocio.

Se denomina **excepción de comunicación** al componente de transferencia originado en la Capa de Acceso que modela una condición de error en la comunicación con un sistema externo — fallo de red, error de protocolo, indisponibilidad del sistema, respuesta inválida. Es un componente de transferencia $C_T \in \mathbb{C}_T$ como cualquier otro, transmitido típicamente como excepción del lenguaje (§9.5).

Subdivisiones de componentes lógicos La carpeta `/logic` de la Capa de Acceso contiene todos los componentes lógicos — Repositorios — del artefacto. En artefactos de pequeña escala, esta carpeta puede contener un número reducido de componentes que no requieren organización adicional. Sin embargo, a medida que el artefacto crece en funcionalidad y el número de sistemas externos con los que se comunica aumenta, la carpeta `/logic` puede llegar a contener un número de componentes suficientemente elevado como para dificultar su navegación y comprensión.

Para estos casos, el modelo XF permite — y recomienda — la creación de subdivisiones internas dentro de la carpeta `/logic` que agrupen los componentes lógicos según el criterio que el equipo de desarrollo considere más apropiado para su artefacto concreto. Estas subdivisiones son una decisión de diseño del equipo — no una prescripción normativa del modelo — y su implementación no afecta a la conformidad del artefacto.

Subdivisión recomendada El modelo XF propone como subdivisión de referencia la división entre accesos locales y accesos remotos, materializada en dos subcarpetas dentro de `/logic`:

- `/repository/logic/remote` — componentes lógicos que acceden a sistemas ubicados fuera del dispositivo que ejecuta el artefacto, cuya comunicación requiere un periférico de comunicación.
- `/repository/logic/local` — componentes lógicos que acceden a sistemas o recursos ubicados en el mismo dispositivo que ejecuta el artefacto, cuya comunicación no requiere periférico de comunicación adicional.

Esta subdivisión es intuitiva para la mayoría de los artefactos porque refleja una diferencia funcional relevante entre los componentes: los accesos remotos y los accesos locales tienen características de comunicación distintas, tipos de errores distintos y patrones de inicialización potencialmente distintos. Agruparlos por separado facilita la localización de componentes y orienta al desarrollador sobre las implicaciones de cada tipo antes de comenzar la implementación.

Sin embargo, el criterio concreto de clasificación de un componente como local o remoto en casos ambiguos — un servicio del sistema operativo con interfaz de red, una base de datos en un contenedor local — es una decisión del implementador. El modelo XF no prescribe cómo resolver estos casos.

Criterio de subdivisión El criterio de subdivisión legítimo en la Capa de Acceso es el que se deriva de su responsabilidad técnica: la resolución de protocolos de co-

municación con sistemas externos. Toda subdivisión interna de `/repository/logic` debe agrupar los componentes lógicos según propiedades de la comunicación que resuelven — ubicación (local o remota), tipo de protocolo, características del canal — y no según el dominio funcional del artefacto. Un artefacto con múltiples protocolos de comunicación puede subdividir por protocolo — `/repository/logic/rest`, `/repository/logic/database`, `/repository/logic/filesystem`. Un artefacto con accesos heterogéneos según la naturaleza del canal puede subdividir por características de la comunicación — síncrona/asíncrona, conectiva/no conectiva.

Subdivisión por dominio: no conforme La subdivisión de `/repository/logic` por dominio funcional del artefacto — por ejemplo, agrupar componentes lógicos en subcarpetas cuyo nombre coincide con conceptos del dominio modelado por la Capa de Negocio — no es conforme al modelo XF. La razón es estructural: la lógica de acceso resuelve protocolo de comunicación, no dominio. Varios dominios del artefacto pueden compartir el mismo protocolo, y subdividir por dominio implicaría replicar la misma resolución de protocolo en varias subcarpetas con el riesgo de redundancia y de mezcla de responsabilidades. El dominio es criterio de subdivisión legítimo exclusivamente en la Capa de Negocio (§7.2.2).

Condiciones formales Toda subdivisión es válida siempre que se respeten tres condiciones. La primera es que todos los componentes lógicos de la Capa de Acceso estén ubicados dentro de la carpeta `/repository/logic`, directamente o en alguna de sus subcarpetas. La segunda es que ninguna subcarpeta contenga componentes de un tipo distinto al lógico — los componentes de generalización, utilidad, transferencia e inyección deben permanecer en sus carpetas canónicas correspondientes independientemente de la subdivisión adoptada para `/logic`. La tercera es que el criterio de subdivisión adoptado sea del vocabulario técnico de la capa — propiedades de la comunicación — y no del vocabulario del dominio modelado por la Capa de Negocio.

Ausencia de subdivisión Un artefacto que no implemente ninguna subdivisión dentro de `/logic` es totalmente conforme al modelo XF. La ausencia de subdivisión es la organización más simple y es apropiada para artefactos con un número reducido de componentes lógicos donde la subdivisión no aporta valor organizativo.

```

1 // Sin subdivisión - válido para artefactos de pequeña escala
2 /repository/logic
3 |--- DatabaseRepository
4 |--- IdentityRepository
5 |--- FileRepository
6
7 // Con subdivisión recomendada - válido para artefactos de mayor escala
8 /repository/logic
9 |--- /remote
10 |   |--- DatabaseRepository
11 |   |--- IdentityRepository
12 |--- /local
13 |   |--- FileRepository
14
15 // Con subdivisión por protocolo - igualmente válido
16 /repository/logic
17 |--- /rest
18 |   |--- IdentityRepository
19 |--- /database
20 |   |--- DatabaseRepository
21 |--- /filesystem
22 |   |--- FileRepository

```

Código 1. Ausencia de subdivisión en la Capa de Acceso

Organización interna La Capa de Acceso se materializa en el sistema de ficheros del proyecto bajo la carpeta canónica `/repository`, cuya estructura interna sigue la organización que se prescribe en §7.4. Esta cláusula consolida esa organización aplicada al contexto específico de la Capa de Acceso, describiendo la función de cada subcarpeta y su relación con las prescripciones del modelo de proceso definido en §7.2.1.

```

1 /repository
2 |--- /general (Componentes de generalizacion)
3 |--- /logic (Componentes logicos)
4 |   |--- /local (Accesos locales -- recomendado)
5 |   |--- /remote (Accesos remotos -- recomendado)
6 |--- /transfers (Componentes de transferencia)
7 |--- /utils (Componentes de utilidad)
8 |--- R (Componente de inyeccion)

```

Código 2. Organización interna de la Capa de Acceso

/general — Componentes de generalización La carpeta `/general` contiene los componentes de generalización de la Capa de Acceso — las abstracciones de comportamiento estructural común entre repositorios. Como se establece en §7.3.2, estos componentes no implementan lógica efectiva de ningún acceso concreto — abstraen lógica contextual de protocolo que es compartida por varios repositorios de la misma capa, liberando a los componentes lógicos concretos para que se concentren exclusivamente en su lógica efectiva: el establecimiento de la conexión con el sistema externo concreto y la obtención o actualización de sus datos específicos.

La generalización de protocolo forma un **árbol de herencia**: bajo la generalización base `Repository` cuelgan, como hermanos, las generalizaciones de los distintos protocolos que el artefacto utiliza — por ejemplo `RestRepository` y `FileRepository` —; bajo cada generalización de protocolo cuelgan, también como hermanos, las generalizaciones de los distintos clientes que comparten ese protocolo; y bajo cada cliente cuelgan los componentes lógicos concretos. Una generalización solo se justifica si abstrae comportamiento *compartido por al menos dos descendientes*: añadir un nivel intermedio con un único hijo introduce cadena lineal sin reutilización y no constituye generalización legítima. Un árbol típico podría ser el siguiente:

```

1 Repository (comportamiento base de cualquier acceso)
2 |--- RestRepository (protocolo HTTP/REST generico)
3 |   |--- MyRestRepository (cliente del servidor MyApi:
4 |       |   autentificacion, estructura de request,
5 |       |   estructura de response { code, description, data })
6 |       |--- UserRepository (logico -- endpoints y estructuras de User)
7 |       |--- PetRepository  (logico -- endpoints y estructuras de Pet)
8 |   |--- AuthRepository (logico -- cliente del servidor de autentificacion)
9 |--- FileRepository (protocolo de acceso a ficheros locales)
10 |   |--- SessionRepository (logico -- persiste la sesion en disco)

```

Código 3. Árbol de herencia de la Capa de Acceso

En este árbol, `UserRepository`, `PetRepository`, `AuthRepository` y `SessionRepository` son los componentes lógicos concretos — heredan toda

la lógica contextual del protocolo de sus generalizaciones e implementan únicamente su lógica efectiva: los endpoints específicos, los esquemas concretos y las estructuras de datos propias de cada recurso. Cuanto más generalización compartida exista en los niveles superiores, más pura es la lógica efectiva de los componentes lógicos concretos.

La forma del árbol es decisión del implementador y depende de la diversidad de protocolos y de clientes que el artefacto utilice. Un artefacto que accede a un único sistema externo con un protocolo simple puede prescindir por completo de generalizaciones de protocolo; un artefacto que combina varios protocolos y varios clientes por protocolo construye un árbol más rico. Ambas opciones son conformes al modelo XF siempre que se respete el principio de aislamiento — todos los componentes del árbol pertenecen a la Capa de Acceso y ningún nodo puede contener lógica que pertenezca a otra capa — y el principio de generalización legítima — ningún nodo intermedio con un único hijo.

/logic — Componentes lógicos La carpeta `/logic` contiene los Repositorios del artefacto — los componentes lógicos de la Capa de Acceso. Cada repositorio encapsula la comunicación con un sistema externo concreto o con un conjunto lógico de datos de un sistema externo, satisfaciendo las responsabilidades prescritas en §7.2.1: encapsulación completa del protocolo, normalización de los datos recibidos y propagación de errores de contexto de uso.

La lógica efectiva de cada repositorio — el establecimiento de la conexión y la obtención o actualización de los datos del sistema externo concreto — es lo que distingue a un componente lógico de un componente de generalización en esta capa. Un componente que solo define comportamiento de protocolo sin implementar ningún acceso concreto a ningún recurso específico es un componente de generalización, no un componente lógico.

La organización interna de `/logic` — con o sin subdivisiones — sigue las prescripciones de §7.2.1. El contenido concreto de esta carpeta es específico de cada artefacto y depende de los sistemas externos con los que el artefacto se comunica.

/transfers — Componentes de transferencia La carpeta `/transfers` contiene los componentes de transferencia definidos en la Capa de Acceso — las estructuras de datos que modelan la información en su forma más próxima al sistema externo que la origina, antes de cualquier transformación semántica por parte de la Capa de Negocio.

En el contexto de la Capa de Acceso, los componentes de transferencia típicos modelan la información tal como la define el sistema externo: las entidades que una base de datos persiste, las estructuras que una API externa publica en sus respuestas, los formatos de datos que un dispositivo conectado transmite. Es igualmente habitual encontrar en esta carpeta estructuras auxiliares de protocolo — envolturas de respuesta, cabeceras de mensaje, estructuras de error específicas del sistema externo — que son necesarias para la implementación del protocolo pero no tienen correspondencia directa con conceptos del dominio de negocio.

Como se prescribe en §7.3.5, estas estructuras pueden ser referenciadas directamente

por componentes de la Capa de Negocio siempre que no sean modificadas. Si la Capa de Negocio necesita transformar una estructura definida en esta carpeta, debe definir su propio componente de transferencia en `/business/transfers`.

/utils — Componentes de utilidad La carpeta `/utils` contiene los componentes de utilidad de la Capa de Acceso — operaciones auxiliares sin estado que sirven de soporte a los repositorios para la consecución de su lógica de acceso.

En el contexto de la Capa de Acceso, los componentes de utilidad típicos proveen operaciones de serialización y deserialización de datos — `JsonUtils`, `XmlUtils` —, operaciones de construcción de parámetros de consulta — `QueryUtils` — y operaciones sobre tipos primitivos — `StringUtils`, `DateUtils`, `NumberUtils`. Como se establece en §7.3.4, los componentes de utilidad sobre tipos primitivos definidos en esta carpeta pueden ser referenciados por componentes de cualquier capa del artefacto sin restricción, dado que los tipos primitivos son anteriores a cualquier transformación semántica del artefacto.

R — Componente de inyección El componente de inyección `R` se ubica en la raíz de la carpeta `/repository` y constituye el único punto de acceso a los repositorios del artefacto desde cualquier otro componente. Su definición, propiedades y ciclo de vida se desarrollan en §7.3.3. En el contexto de la Capa de Acceso, `R` es el punto a través del cual la Capa de Negocio accede a todos los datos externos que el artefacto necesita — toda invocación de un repositorio desde un componente de negocio sigue el patrón `R.<repositorio>.<operación>()`.

La ubicación de `R` en la raíz de `/repository` — al mismo nivel que las subcarpetas de tipos de componentes — refleja su naturaleza transversal dentro de la capa: agrega y da acceso a todos los componentes lógicos de la misma sin pertenecer a ninguna subcarpeta de tipo concreto.

7.2.2 Capa de Negocio

La Capa de Negocio constituye el nivel de abstracción intermedio de la Arquitectura XF y es la capa que define la identidad funcional del artefacto. Si la Capa de Acceso resuelve cómo el artefacto se comunica con el exterior, y la Capa de Interacción resuelve cómo el artefacto es consumido por sus usuarios y sistemas externos, la Capa de Negocio resuelve qué hace el artefacto — cuál es su propósito, qué conceptos del dominio gestiona y qué operaciones sobre esos conceptos son posibles.

Es en esta capa donde reside la ontología de la solución: el conjunto estructurado de conceptos, relaciones y operaciones que describen el dominio que el artefacto automatiza. Un cambio en los requisitos funcionales del artefacto — una nueva regla de negocio, un nuevo caso de uso, una nueva restricción de dominio — se manifiesta siempre como un cambio en la Capa de Negocio. Las otras capas pueden cambiar por razones tecnológicas — un nuevo protocolo de acceso, un nuevo canal de interacción — pero la Capa de Negocio cambia por razones de dominio.

Los componentes lógicos de la Capa de Negocio — denominados Negocios — son el núcleo funcional del artefacto. Aunque todo componente lógico, con independencia de su capa, puede mantener estado en el sentido formal que se define en §7.3.1, el

estado de los lógicos de Acceso modela condiciones de la comunicación con sistemas externos y el de los lógicos de Interacción modela condiciones del protocolo o de la interfaz; solo el estado de los Negocios establece correspondencia con los conceptos, relaciones y operaciones del dominio que el artefacto automatiza. Esta exclusividad ontológica del estado de la Capa de Negocio es lo que la constituye en la memoria semántica del artefacto durante su ejecución.

Responsabilidad y límites Esta cláusula delimita la responsabilidad de la Capa de Negocio y sus fronteras: el límite inferior con la Capa de Acceso y el límite superior con la Capa de Interacción.

Responsabilidad La responsabilidad de la Capa de Negocio es implementar la lógica efectiva del dominio del artefacto — las operaciones, transformaciones y reglas que constituyen el valor funcional del sistema — abstrayendo tanto los detalles de comunicación con sistemas externos como los detalles de presentación e interacción. La Capa de Negocio recibe datos normalizados de la Capa de Acceso, les aplica la lógica del dominio y provee resultados a la Capa de Interacción a través de operaciones bien definidas.

Esta responsabilidad tiene tres dimensiones que el modelo XF prescribe:

La primera dimensión es la **implementación de la ontología del dominio**: la Capa de Negocio define el conjunto de conceptos que el artefacto gestiona y las operaciones que sobre ellos son posibles. Cada componente lógico de negocio modela exactamente un concepto del dominio — un usuario, una sesión, una temperatura, un pedido — y define las operaciones que transforman, validan o consultan ese concepto. El conjunto de todos los componentes lógicos de negocio constituye la ontología completa de la solución.

La segunda dimensión es la **gestión del estado del dominio**: aunque todo componente lógico puede mantener estado en el sentido formal del modelo, la Capa de Negocio es la única en la que ese estado establece correspondencia con la ontología del dominio del artefacto. El estado de los lógicos de Acceso modela condiciones de la comunicación con sistemas externos y el de los lógicos de Interacción modela condiciones del protocolo o de la interfaz; ninguno de ambos modela conceptos del dominio. El estado de un Negocio representa el contexto dinámico del concepto del dominio que modela durante la ejecución del artefacto y es accesible exclusivamente a través del componente de inyección B.

La tercera dimensión es la **abstracción de la fuente de los datos**: la Capa de Negocio no conoce el protocolo utilizado para obtener los datos que procesa. Toda comunicación con sistemas externos está abstraída en la Capa de Acceso y es accesible a través de R. Esta propiedad garantiza que un cambio en el protocolo de acceso no requiera modificaciones en la Capa de Negocio — el componente de negocio invoca `R.<repositorio>.<operación>()` con independencia de si ese repositorio accede a una base de datos relacional, una API REST o un fichero del sistema.

Límite inferior — Capa de Acceso El límite inferior de la Capa de Negocio es la Capa de Acceso, de la que consume datos normalizados en componentes de transferencia a través del componente de inyección R. La Capa de Negocio no conoce

los detalles de cómo esos datos son obtenidos — su responsabilidad comienza en el momento en que recibe un componente de transferencia normalizado o una excepción de comunicación procedente de la Capa de Acceso.

Límite superior — Capa de Interacción El límite superior de la Capa de Negocio es la Capa de Interacción, a la que provee operaciones de dominio bien definidas a través del componente de inyección B. La Capa de Negocio no conoce cómo sus operaciones son invocadas ni cómo sus resultados son presentados — su responsabilidad termina en el momento en que entrega un resultado o propaga una excepción de negocio.

Modelo de proceso El modelo de proceso de negocio define qué caracteriza funcionalmente a la lógica que pertenece a la Capa de Negocio, de forma que todo desarrollador pueda determinar sin ambigüedad si una pieza de lógica concreta pertenece a esta capa o a otra. Al igual que en §7.2.1, el modelo prescribe responsabilidades y restricciones — no implementaciones concretas. La orientación de implementación se recoge en el Anexo B.

El modelo XF introduce en esta cláusula una terminología de referencia para describir los conceptos de la Capa de Negocio con precisión. Esta terminología no es una prescripción normativa sobre la nomenclatura interna de los equipos — es el vocabulario que el modelo utiliza para describir los elementos de la capa y que permite a los equipos comunicarse sobre la lógica de negocio de un artefacto con precisión y sin ambigüedad, independientemente de la tecnología empleada.

Responsabilidades prescritas Todo componente lógico de la Capa de Negocio **debe** satisfacer las siguientes responsabilidades:

La primera responsabilidad es la **implementación de la lógica efectiva del dominio**. La lógica efectiva de un componente de negocio es el conjunto mínimo de operaciones que justifican su existencia en el artefacto — aquellas que resuelven directamente los casos de uso del concepto de dominio que el componente modela y que no pueden ser abstraídas en ningún otro componente sin perder la especificidad de ese concepto. Un componente de negocio que gestiona la autenticación de usuarios define las operaciones “cotejar credenciales”, “refrescar sesión” y “caducar sesión” — estas son operaciones efectivas porque resuelven directamente el problema de autenticación. La gestión del protocolo de acceso al servidor de identidades o el almacenamiento de la sesión en disco son lógica contextual que pertenece a otros tipos de componentes.

La segunda responsabilidad es la **gestión de las precondiciones de ejecución**. Antes de ejecutar la parte de la lógica efectiva que depende de ellas, todo componente de negocio verifica las condiciones necesarias para que la operación pueda completarse correctamente. Estas condiciones — denominadas **condiciones de negocio** en la terminología del modelo — comprueban el estado del componente y los parámetros de entrada de la operación. Cuando una condición no se satisface, la operación suele propagar una excepción al invocante; otras formas de gestión — valor por defecto, reintento, lógica de fallback definida — son admisibles cuando la semántica del proceso lo justifica. El modelo XF recomienda que las condiciones se evalúen tan

pronto como sea posible en la operación, idealmente al inicio cuando la naturaleza del proceso lo permita, de forma que la condición anómala se transmita sin ejecutar lógica innecesaria.

La tercera responsabilidad es la **delegación hacia la Capa de Acceso**. Cuando la lógica efectiva de un componente de negocio requiere datos de sistemas externos, esa obtención debe ser delegada a los componentes lógicos de la Capa de Acceso a través del componente de inyección R. Un componente de negocio nunca accede directamente a sistemas externos — invoca `R.<repositorio>.<operación>()` y recibe el resultado como un componente de transferencia normalizado o una excepción de comunicación.

La cuarta responsabilidad es la **gestión de excepciones de negocio**. Cuando una operación de negocio no puede completarse porque una condición de negocio no se satisface, o porque una dependencia de la Capa de Acceso ha propagado una excepción de comunicación, el componente de negocio decide la respuesta apropiada según la semántica del proceso: típicamente propaga hacia la Capa de Interacción una excepción de negocio que describa con precisión la causa del fallo, pero otras respuestas — valor por defecto, reintento, gestión local del fallo — son admisibles cuando el proceso lo requiere. El modelo XF recomienda la propagación explícita como práctica preferida para preservar la trazabilidad del fallo, pero no la impone como obligación estructural.

Responsabilidades excluidas El modelo XF prescribe explícitamente que los siguientes tipos de lógica no pertenecen a la Capa de Negocio:

Un componente de negocio **no debe** contener lógica de protocolo de comunicación con sistemas externos. Toda comunicación con sistemas externos pertenece a la Capa de Acceso y debe ser accedida a través de R. Si un componente de negocio importa una librería de protocolo, construye una sentencia SQL o manipula cabeceras HTTP, la responsabilidad de la capa ha sido violada.

Un componente de negocio **no debe** contener lógica de presentación ni definición de interfaces de interacción. La composición de elementos gráficos, la definición de contratos de servicio o la validación sintáctica de parámetros de entrada pertenecen a la Capa de Interacción.

Criterio de clasificación El modelo XF prescribe el siguiente criterio operativo para determinar si una pieza de lógica concreta pertenece a la Capa de Negocio. Una pieza de lógica pertenece a esta capa si y solo si satisface simultáneamente las tres condiciones siguientes:

La primera condición es que **conoce al menos un concepto del dominio del artefacto** — opera sobre datos que tienen correspondencia directa con la ontología de la solución.

La segunda condición es que **no conoce ningún protocolo de comunicación con sistemas externos** — no sabe cómo se obtienen los datos sobre los que opera, solo opera sobre ellos.

La tercera condición es que **no conoce cómo sus resultados son presentados o expuestos** — no sabe si sus resultados serán mostrados en una interfaz gráfica,

devueltos en una respuesta HTTP o persistidos en un fichero.

Si una pieza de lógica no satisface la primera condición, no pertenece a la Capa de Negocio. Si satisface la primera pero no satisface la segunda o la tercera, debe ser descompuesta: la parte que conoce el protocolo pertenece a la Capa de Acceso, y la parte que conoce la presentación pertenece a la Capa de Interacción.

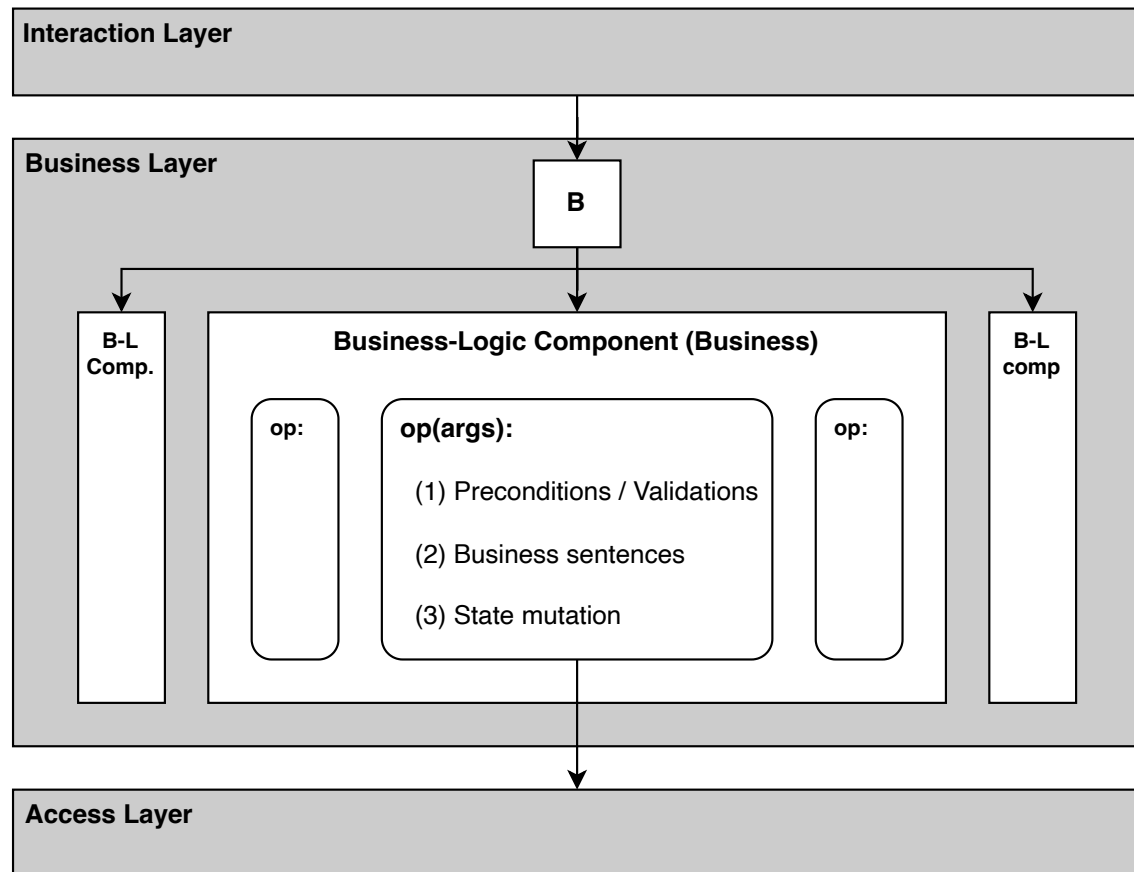


Figure 8. Contenido prescrito de un componente lógico de la Capa de Negocio. El **Business** implementa la lógica efectiva del concepto de dominio que modela: recibe invocaciones a través del componente de inyección **B**, evalúa condiciones de negocio, ejecuta operaciones de dominio, mantiene el estado del concepto y propaga resultados o excepciones de negocio. Delega operaciones de acceso a sistemas externos en componentes lógicos de la Capa de Acceso a través de **R**. Ningún protocolo de comunicación ni lógica de presentación reside en este componente.

Terminología de referencia El modelo XF define los siguientes términos para describir con precisión los elementos que componen la lógica de negocio de un artefacto. Esta terminología constituye un vocabulario común que facilita la comunicación entre desarrolladores, analistas y revisores sobre la lógica de negocio de cualquier artefacto XF con independencia de la tecnología empleada.

Se denomina **ontología del negocio** al modelo conceptual que define el conjunto estructurado de conceptos clave tratados por el artefacto, sus relaciones, las operaciones permitidas sobre ellos y las reglas que los gobiernan. La ontología del negocio es la descripción formal del dominio que el artefacto automatiza.

Se denomina **componente de negocio** a cada uno de los conceptos clave de la ontología que tiene representación como componente lógico en el artefacto. Un componente de negocio modela exactamente un concepto del dominio y define las operaciones que sobre él son posibles.

Se denomina **estructura de negocio** al conjunto de datos estructurados que representa la información gestionada por un componente de negocio y que tiene correspondencia directa con las características, hechos o valores del concepto que modela. Las estructuras de negocio se implementan como componentes de transferencia de la Capa de Negocio.

Se denomina **operación de negocio** a cada uno de los subconjuntos de la lógica efectiva de un componente de negocio que se ejecuta de forma atómica ante un evento o invocación, produciendo una transformación del estado del componente, un resultado para el invocante, o ambos.

Se denomina **sentencia de negocio** a cada uno de los pasos lógicos que componen una operación de negocio. Las sentencias se ejecutan de forma ordenada y su conjunto constituye la implementación completa de la operación.

Se denomina **condición de negocio** a las sentencias específicas de una operación que evalúan el estado del componente o los parámetros de entrada antes de ejecutar la lógica efectiva. Las condiciones constituyen las precondiciones de ejecución de la operación.

Se denomina **excepción de negocio** al componente de transferencia que la Capa de Negocio transmite típicamente como excepción del lenguaje cuando una condición de negocio no se satisface o una dependencia de la Capa de Acceso no puede ser resuelta. La distinción es del constructo sintáctico de transmisión (§9.5), no de dominio: la excepción de negocio es un componente de transferencia $C_T \in \mathbb{C}_T$ como cualquier otro.

Subdivisiones de componentes lógicos La carpeta `/logic` de la Capa de Negocio contiene todos los componentes lógicos de negocio — los Negocios — del artefacto. En artefactos de pequeña escala, esta carpeta puede contener un número reducido de componentes que no requieren organización adicional. Sin embargo, a medida que la ontología del artefacto crece en complejidad y el número de conceptos de dominio gestionados aumenta, la carpeta `/logic` puede llegar a contener un número de componentes suficientemente elevado como para dificultar su navegación y comprensión.

Para estos casos, el modelo XF permite — y recomienda — la creación de subdivisiones internas dentro de la carpeta `/logic` que agrupen los componentes lógicos según el criterio que el equipo de desarrollo considere más apropiado para su artefacto concreto. Estas subdivisiones son una decisión de diseño del equipo — no una prescripción normativa del modelo — y su implementación no afecta a la conformidad del artefacto.

Subdivisión recomendada El modelo XF propone como subdivisión de referencia la división entre negocios de instancia y negocios de dispositivo, materializada en dos subcarpetas dentro de `/logic`:

- `/business/logic/instance` — componentes lógicos que modelan la ontología específica de la solución que el artefacto automatiza. Son los negocios que definen el dominio propio del artefacto — los conceptos, las reglas y las operaciones que constituyen el valor funcional del sistema. Ejemplos típicos: `TemperatureBusiness`, `UserBusiness`, `SessionBusiness`, `ScheduleBusiness`.
- `/business/logic/device` — componentes lógicos que modelan procesos relativos al dispositivo que ejecuta el artefacto o a los periféricos conectados a él. Son los negocios que gestionan las capacidades del entorno de ejecución — conectividad, localización, idioma, cifrado, permisos del sistema operativo. Ejemplos típicos: `NetworkBusiness`, `LocationBusiness`, `KeychainBusiness`, `LanguageBusiness`.

La utilidad de esta subdivisión es doble. Por un lado, facilita la orientación en artefactos con un número elevado de componentes lógicos, separando los negocios que modelan el dominio de la solución de los que gestionan el entorno de ejecución. Por otro lado, favorece la reutilización: los negocios de instancia pueden ser replicados en otros artefactos que modelen la misma ontología de dominio, mientras que los negocios de dispositivo pueden ser replicados en artefactos ejecutados en dispositivos con las mismas características, independientemente del dominio de negocio.

El criterio concreto de clasificación de un componente como de instancia o de dispositivo en casos ambiguos es una decisión del implementador. Un componente que gestiona la configuración del artefacto puede considerarse de instancia — si la configuración es propia del dominio — o de dispositivo — si depende de las capacidades del entorno de ejecución. El modelo XF no prescribe cómo resolver estos casos.

Criterio de subdivisión La Capa de Negocio es la **única capa del modelo XF en la que el dominio funcional es un criterio de subdivisión legítimo**. La razón es estructural: la lógica de negocio es la única lógica del artefacto que tiene como objeto el dominio modelado — las otras dos capas resuelven problemas técnicos (protocolo y tipo de entrada) que son ortogonales al dominio. Un artefacto con una ontología de dominio extensa puede subdividir `/logic/instance` por subdominios — `/business/logic/instance/user`, `/business/logic/instance/temperature`, `/business/logic/instance/schedule`. Un artefacto con múltiples integraciones de dispositivo puede subdividir `/logic/device` por tipo de capacidad — `/business/logic/device/connectivity`, `/business/logic/device/security`. La compatibilidad del modelo XF con metodologías de modelado del dominio — en particular con la organización por subdominios o contextos delimitados — se materializa precisamente en este nivel: dentro de `/business/logic/instance`, no en la estructura top-level del artefacto ni en la subdivisión interna de las otras capas.

Condiciones formales Toda subdivisión es válida siempre que se respeten tres condiciones. La primera es que todos los componentes lógicos de la Capa de Negocio estén ubicados dentro de la carpeta `/business/logic`, directamente o en alguna de sus subcarpetas. La segunda es que ninguna subcarpeta contenga componentes de un tipo distinto al lógico — los componentes de generalización, utilidad, transferencia e inyección deben permanecer en sus carpetas canónicas correspondientes. La tercera

es que el criterio de subdivisión adoptado pertenezca al vocabulario del dominio modelado por la Capa de Negocio — la única capa del modelo XF donde el dominio funcional es criterio de subdivisión legítimo.

Ausencia de subdivisión Un artefacto que no implemente ninguna subdivisión dentro de `/logic` es totalmente conforme al modelo XF. La ausencia de subdivisión es la organización más simple y es apropiada para artefactos con un número reducido de componentes lógicos donde la subdivisión no aporta valor organizativo.

```

1 // Sin subdivisión - válido para artefactos de pequeña escala
2 /business/logic
3 |--- TemperatureBusiness
4 |--- UserBusiness
5 |--- SessionBusiness
6 |--- NetworkBusiness
7
8 // Con subdivisión recomendada - válido para artefactos de mayor escala
9 /business/logic
10 |--- /instance
11 |   |--- TemperatureBusiness
12 |   |--- UserBusiness
13 |   |--- SessionBusiness
14 |--- /device
15 |   |--- NetworkBusiness
16
17 // Con subdivisión por subdominio - igualmente válido
18 /business/logic
19 |--- /instance
20 |   |--- /user
21 |       |--- UserBusiness
22 |       |--- SessionBusiness
23 |   |--- /temperature
24 |       |--- TemperatureBusiness
25 |--- /device
26 |   |--- NetworkBusiness

```

Código 4. Ausencia de subdivisión en la Capa de Negocio

Organización interna La Capa de Negocio se materializa en el sistema de ficheros del proyecto bajo la carpeta canónica `/business`, cuya estructura interna sigue la organización que se prescribe en §7.4. Esta cláusula consolida esa organización aplicada al contexto específico de la Capa de Negocio, describiendo la función de cada subcarpeta y su relación con las prescripciones del modelo de proceso definido en §7.2.2.

```

1 /business
2 |--- /general (Componentes de generalizacion)
3 |--- /logic (Componentes logicos)
4 |   |--- /instance (Negocios de instancia -- recomendado)
5 |   |--- /device (Negocios de dispositivo -- recomendado)
6 |--- /transfers (Componentes de transferencia)
7 |--- /utils (Componentes de utilidad)
8 |--- B (Componente de inyeccion)

```

Código 5. Organización interna de la Capa de Negocio

/general — Componentes de generalización La carpeta `/general` contiene los componentes de generalización de la Capa de Negocio — las abstracciones de comportamiento estructural común entre componentes lógicos de negocio. Como se establece en §7.3.2, estos componentes no implementan lógica efectiva de ningún

concepto de dominio concreto — abstraen lógica contextual estructural que es compartida por varios componentes lógicos de la misma capa, liberando a los componentes lógicos concretos para que se concentren exclusivamente en su lógica efectiva.

Al igual que en la Capa de Acceso, la generalización en la Capa de Negocio puede operar en múltiples niveles de especificidad, formando cadenas de herencia donde cada nivel abstrae un aspecto más concreto del comportamiento estructural. Los patrones estructurales más habituales en esta capa son los siguientes:

Business — componente de generalización base que abstrae el comportamiento estructural más general de cualquier componente lógico de negocio. Puede contener lógica común a todos los negocios del artefacto — registro de trazas, gestión de errores comunes, patrones de inicialización.

StatefulBusiness<T> — componente de generalización que abstrae el patrón de observación y notificación de cambios de estado mutable. Los componentes lógicos que lo heredan — **TemperatureBusiness**, **PetBusiness** — tienen datos observables cuya mutación notifica automáticamente a los observadores registrados. Este patrón es independiente de **ScheduledBusiness** — un componente lógico puede heredar de uno, de ambos o de ninguno según sus necesidades.

ScheduledBusiness — componente de generalización que abstrae el patrón de ejecución periódica sin implicar necesariamente estado observable. Los componentes lógicos que lo heredan ejecutan lógica de forma automática en intervalos de tiempo definidos — sincronización de datos, actualización de cachés, comprobaciones periódicas de estado.

BroadcastBusiness — componente de generalización que abstrae el patrón de coordinación de datos entre múltiples fuentes — por ejemplo, persistir un dato simultáneamente en caché local y en servidor remoto, eligiendo la fuente más actualizada en cada lectura.

Una cadena de herencia típica en la Capa de Negocio podría ser la siguiente:

```

1 Business (comportamiento base de cualquier componente logico de negocio)
2 |--- StatefulBusiness<T> (gestion de estado observable y notificacion a
    |   observadores)
3 |   |--- TemperatureBusiness (logico -- gestion del concepto Temperature)
4 |   |--- ScheduledBusiness   (ejecucion periodica automatica)
5 |   |--- BroadcastBusiness   (coordinacion de datos entre multiples fuentes)
```

Código 6. Patrones canónicos de generalización en la Capa de Negocio

Estos patrones son ilustrativos del contenido típico de **/general** en la Capa de Negocio, no una lista prescriptiva. El contenido concreto de esta carpeta depende de los patrones estructurales que el artefacto necesite abstraer y del grado de reutilización que el equipo considere apropiado.

Es importante recordar que el principio de aislamiento entre capas aplica con la misma fuerza a los componentes de generalización de negocio que a los componentes lógicos. Los patrones estructurales que son necesarios tanto en la Capa de Negocio como en la Capa de Acceso — como el patrón de observación de estado o el patrón de tarea periódica — deben implementarse de forma independiente en cada capa. **StatefulBusiness** y **StatefulRepository** son implementaciones independientes del mismo patrón — no comparten código entre capas.

/logic — Componentes lógicos La carpeta `/logic` contiene los Negocios del artefacto — los componentes lógicos de la Capa de Negocio. Cada negocio modela exactamente un concepto del dominio del artefacto y define las operaciones que sobre ese concepto son posibles, satisfaciendo las responsabilidades prescritas en §7.2.2: implementación de la lógica efectiva del dominio, gestión de precondiciones, delegación hacia la Capa de Acceso y propagación de excepciones de negocio.

La lógica efectiva de un componente lógico de negocio — las operaciones que justifican su existencia en el artefacto — es lo que lo distingue de un componente de generalización en esta capa. Un componente que solo define comportamiento estructural genérico sin modelar ningún concepto concreto del dominio es un componente de generalización, no un componente lógico.

La granularidad de los componentes lógicos de negocio — cuántos conceptos del dominio se agrupan en un único componente o se separan en varios — es una decisión del implementador que el modelo XF no prescribe. Como se estableció en §7.2.2, tanto un único `UserBusiness` que gestiona todos los aspectos del usuario como la separación en `UserBusiness`, `UserProfileBusiness` y `UserSessionBusiness` son igualmente válidos. Lo que el modelo prescribe es que todos los componentes lógicos de negocio sean accesibles exclusivamente a través del componente de inyección B y que cada uno modele un concepto del dominio con responsabilidad bien definida.

La organización interna de `/logic` — con o sin subdivisiones — sigue las prescripciones de §7.2.2.

/transfers — Componentes de transferencia La carpeta `/transfers` contiene los componentes de transferencia definidos en la Capa de Negocio — las estructuras de datos que modelan la información en su forma semántica de dominio, después de las transformaciones estructurales aplicadas por la Capa de Negocio sobre los datos recibidos de la Capa de Acceso.

En el contexto de la Capa de Negocio, los componentes de transferencia típicos modelan los conceptos del dominio tal como son gestionados por la lógica de negocio del artefacto. Cuando la Capa de Negocio necesita una estructura con una definición distinta a la que provee la Capa de Acceso — eliminando atributos sensibles, añadiendo atributos calculados o renombrando atributos para ajustarlos a la semántica del dominio — define esa nueva estructura en esta carpeta.

Como se prescribe en §7.3.5, la obligación de definir una nueva estructura se activa únicamente cuando se modifica la definición estructural — añadiendo o eliminando atributos. La modificación de valores sobre una estructura existente no requiere la definición de una nueva estructura. Las estructuras definidas en esta carpeta pueden ser referenciadas directamente por componentes de la Capa de Interacción siempre que no sean modificadas estructuralmente.

Es igualmente habitual encontrar en esta carpeta las excepciones de negocio — componentes de transferencia que se transmiten típicamente como excepciones del lenguaje y modelan condiciones de error propias del dominio del artefacto: `TemperatureOutOfRangeException`, `SessionExpiredException`, `NetworkUnavailableException`.

/utils — Componentes de utilidad La carpeta `/utils` contiene los componentes de utilidad de la Capa de Negocio — operaciones auxiliares sin estado que sirven de soporte a los componentes lógicos de negocio para la consecución de su lógica efectiva sin pertenecer directamente al dominio del artefacto.

En el contexto de la Capa de Negocio, los componentes de utilidad típicos proveen operaciones de transformación y conversión sobre los conceptos del dominio — `TemperatureUtils` para la conversión entre unidades de medida, `LocationUtils` para el cálculo de distancias, `CipherUtils` para el cifrado de datos, `ScheduleUtils` para la gestión de franjas horarias. Estas operaciones son auxiliares en el sentido de que no modelan ningún concepto del dominio por sí solas, pero son necesarias para que los componentes lógicos puedan implementar su lógica efectiva.

A diferencia de los componentes de utilidad de la Capa de Acceso que operan sobre tipos primitivos — `StringUtils`, `DateUtils` — los componentes de utilidad de la Capa de Negocio operan sobre conceptos del dominio y tienen alcance local a esta capa. No deben ser referenciados por componentes de la Capa de Acceso.

B — Componente de inyección El componente de inyección B se ubica en la raíz de la carpeta `/business` y constituye el único punto de acceso a los componentes lógicos de negocio desde cualquier otro componente del artefacto — incluyendo otros componentes lógicos de la misma capa. Su definición, propiedades y ciclo de vida se desarrollan en §7.3.3.

En el contexto de la Capa de Negocio, B es el punto a través del cual la Capa de Interacción invoca todas las operaciones de dominio del artefacto — toda invocación desde un componente de interacción sigue el patrón `B.<negocio>.<operación>()`. Del mismo modo, todo acceso entre componentes lógicos de negocio se realiza a través de B — un `UserBusiness` que necesita invocar una operación de `SessionBusiness` lo hace siempre a través de `B.session.<operación>()`, nunca mediante referencia directa entre componentes.

7.2.3 Capa de Interacción

La Capa de Interacción constituye el nivel de abstracción más alto de la Arquitectura XF y es la única capa del modelo que tiene contacto directo con los consumidores del artefacto — usuarios humanos a través de interfaces gráficas, o sistemas externos a través de protocolos de comunicación definidos. Su posición en la estratificación es igualmente consecuencia directa de la derivación formal establecida en §5.5: la etapa de interacción de todo proceso formal es aquella en la que el proceso recibe la orden de ser ejecutado y verifica que se satisfacen las precondiciones necesarias para su inicio.

La Capa de Interacción existe en todo artefacto XF como consecuencia del modelado formal de procesos — todo proceso formal tiene una etapa de interacción. Sin embargo, la presencia de componentes lógicos en esta capa no es obligatoria en todos los artefactos. Un artefacto que no permite interacción externa — una tarea periódica autónoma, un servicio de notificación, un proceso batch — tiene una Capa de Interacción estructuralmente presente pero vacía de componentes lógicos. La ausencia de componentes lógicos en la Capa de Interacción no constituye una

violación del modelo — es la expresión arquitectónica correcta de un artefacto que no expone puntos de entrada externos.

El modelo XF supera la distinción tradicional entre artefactos backend y frontend al reconocer que ambos tienen Capa de Interacción — simplemente con tipos de componentes distintos. Un artefacto backend expone su Capa de Interacción a través de servicios — endpoints, contratos de API, comandos de consola. Un artefacto frontend expone su Capa de Interacción a través de vistas — pantallas, formularios, elementos gráficos interactivos. Un artefacto que combina ambos tipos de interacción — un termostato con pantalla integrada y API de control remoto — tiene componentes de ambos tipos en su Capa de Interacción. En todos los casos, la responsabilidad de la capa es la misma: definir los puntos de entrada al artefacto y establecer las condiciones bajo las cuales las operaciones de negocio pueden ser invocadas.

A diferencia de las capas de Acceso y Negocio — donde la generalización forma un único árbol bajo una sola raíz abstracta —, la Capa de Interacción exhibe **dos sub-árboles paralelos** bajo dos bases abstractas hermanas: **Service** para consumidores sistémicos y **View** para consumidores humanos. Ambos sub-árboles son independientes y coexisten con identidad propia: ningún nodo de uno hereda del otro. La forma concreta de cada sub-árbol es decisión del implementador y depende de la naturaleza de los consumidores del artefacto. Un artefacto que solo expone endpoints sistémicos puede tener únicamente el sub-árbol de **Service**; un artefacto que solo presenta interfaz gráfica puede tener únicamente el de **View**; un artefacto que combina ambos los desarrolla en paralelo. Las prescripciones invariantes son las mismas que en las capas anteriores: herencia estrictamente intra-capas, ningún nodo intermedio con un único descendiente y ningún componente con lógica de otra capa. La materialización canónica de ambos sub-árboles se desarrolla más adelante en esta misma cláusula.

Responsabilidad y límites Esta cláusula delimita la responsabilidad de la Capa de Interacción y sus fronteras: el límite inferior con la Capa de Negocio y el límite superior con los consumidores del artefacto.

Responsabilidad La responsabilidad de la Capa de Interacción es definir los puntos de entrada al artefacto — cómo sus consumidores pueden invocar las operaciones de negocio que implementa — y establecer las condiciones formales bajo las cuales esa invocación es válida. Esta responsabilidad tiene tres dimensiones que el modelo XF prescribe:

La primera dimensión es la **definición del protocolo de interacción**: todo componente lógico de la Capa de Interacción define el mecanismo a través del cual un consumidor puede invocar las operaciones de negocio del artefacto. Para los servicios, ese mecanismo es un protocolo sistémico — HTTP, WebSocket, comandos de consola, mensajes de cola. Para las vistas, ese mecanismo es una interfaz gráfica — pantallas, formularios, elementos interactivos. En ambos casos, el componente de interacción es responsable de definir el protocolo con precisión y de garantizar que las invocaciones que lo satisfacen puedan ser procesadas correctamente por la Capa de Negocio.

La segunda dimensión es la **validación de las condiciones de interacción**: antes

de delegar la ejecución a la Capa de Negocio, todo componente de interacción debe verificar que los datos recibidos satisfacen las precondiciones formales del punto de interacción. Esta validación cubre todo lo que no pertenece al modelo de dominio del artefacto — verificaciones de tipo, formato, presencia de campos obligatorios y restricciones formales del protocolo. La verificación de que un valor tiene correspondencia con un concepto del dominio — que un identificador pertenece a una entidad existente, que un valor satisface una regla de negocio — pertenece a la Capa de Negocio, no a la Capa de Interacción.

La tercera dimensión es la **representación de los resultados**: todo componente de interacción es responsable de transformar los resultados de las operaciones de negocio en la forma que el consumidor del artefacto espera recibir. Para los servicios, esa forma es la respuesta del protocolo — códigos de estado, estructuras de respuesta, formatos de error. Para las vistas, esa forma es la representación visual — qué elementos gráficos se muestran, en qué estado y con qué datos.

Límite inferior — Capa de Negocio El límite inferior de la Capa de Interacción es la Capa de Negocio, cuyas operaciones invoca a través del componente de inyección B. La Capa de Interacción no conoce cómo la Capa de Negocio obtiene o procesa los datos — su responsabilidad comienza en el momento en que recibe una solicitud de un consumidor y termina en el momento en que entrega la respuesta correspondiente. Todo lo que ocurre entre esos dos momentos — la lógica de dominio, el acceso a datos externos — es responsabilidad de las capas inferiores.

Límite superior — Consumidores del artefacto El límite superior de la Capa de Interacción son los consumidores del artefacto — usuarios humanos o sistemas externos. La Capa de Interacción es la frontera entre el artefacto y su entorno de consumo, del mismo modo que la Capa de Acceso es la frontera entre el artefacto y su entorno de datos. Esta simetría es intencional — el modelo XF trata la interacción con consumidores y el acceso a sistemas externos como dos instancias del mismo principio de abstracción de frontera, aplicado en direcciones opuestas.

Modelo de proceso El modelo de proceso de interacción define qué caracteriza funcionalmente a la lógica que pertenece a la Capa de Interacción, de forma que todo desarrollador pueda determinar sin ambigüedad si una pieza de lógica concreta pertenece a esta capa o a otra. Al igual que en §7.2.1 y §7.2.2, el modelo prescribe responsabilidades y restricciones — no implementaciones concretas. La orientación de implementación se recoge en el Anexo C.

La Capa de Interacción reconoce dos tipos de componentes lógicos con modelos de proceso distintos: los **servicios** — componentes de interacción sistémica — y las **vistas** — componentes de interacción gráfica. Aunque ambos tipos satisfacen las mismas responsabilidades prescritas de la capa, su modelo de proceso interno tiene características propias que el modelo XF describe con precisión. El modelo XF introduce en esta cláusula una terminología de referencia para describir los elementos de cada tipo con la misma función que la terminología de negocio cumple en §7.2.2 — proveer un vocabulario común que permita a los equipos comunicarse sobre la lógica de interacción con precisión y sin ambigüedad, independientemente de la tecnología empleada.

Responsabilidades prescritas Todo componente lógico de la Capa de Interacción — tanto servicios como vistas — **debe** satisfacer las siguientes responsabilidades:

La primera responsabilidad es la **definición del protocolo de interacción**. Todo componente de interacción define con precisión el mecanismo a través del cual un consumidor puede invocar las operaciones de negocio del artefacto. Este mecanismo debe ser suficientemente preciso para que cualquier consumidor que lo satisfaga pueda ser atendido correctamente, y suficientemente restrictivo para que las invocaciones que no lo satisfagan sean rechazadas antes de alcanzar la Capa de Negocio.

La segunda responsabilidad es la **validación de las condiciones de interacción**. Todo componente de interacción debe verificar que los datos recibidos satisfacen las precondiciones formales del punto de interacción antes de delegar la ejecución a la Capa de Negocio. Esta validación cubre exclusivamente lo que no pertenece al modelo de dominio: verificaciones de tipo, formato, presencia de campos obligatorios y restricciones formales del protocolo. La verificación de que un valor tiene correspondencia con un concepto del dominio pertenece a la Capa de Negocio.

La tercera responsabilidad es la **delegación a la Capa de Negocio**. Toda lógica de dominio debe ser delegada a los componentes lógicos de negocio a través del componente de inyección B. Un componente de interacción no implementa lógica de dominio — invoca `B.<negocio>.<operación>()` y recibe el resultado como un componente de transferencia o una excepción de negocio.

La cuarta responsabilidad es la **representación del resultado**. Todo componente de interacción es responsable de transformar el resultado de las operaciones de negocio en la forma que el consumidor espera recibir — la respuesta del protocolo en los servicios, la representación visual en las vistas. Esta transformación puede implicar la definición de nuevos componentes de transferencia en la Capa de Interacción cuando la estructura del resultado difiere de la estructura provista por la Capa de Negocio.

Responsabilidades excluidas Un componente de interacción **no debe** contener lógica de dominio. Las reglas de negocio, las transformaciones semánticas de datos y las operaciones sobre conceptos del dominio pertenecen a la Capa de Negocio y deben ser invocadas a través de B.

Un componente de interacción **no debe** contener lógica de acceso a sistemas externos. Toda comunicación con sistemas externos pertenece a la Capa de Acceso y debe ser accedida a través de R. Un componente de interacción que invoca directamente un repositorio viola el principio de aislamiento establecido en §6.2.2.

Un componente de interacción **no debe** verificar condiciones que pertenecen al dominio del artefacto. La verificación de que un valor tiene correspondencia con un concepto del dominio — que un identificador pertenece a una entidad existente, que un valor satisface una regla de negocio — pertenece a la Capa de Negocio.

Criterio de clasificación Una pieza de lógica pertenece a la Capa de Interacción si y solo si satisface simultáneamente las tres condiciones siguientes:

La primera condición es que **define o gestiona un punto de entrada al artefacto** — establece cómo un consumidor puede invocar una operación de negocio o recibir

información del artefacto.

La segunda condición es que **no conoce ninguna regla de negocio del dominio** — no toma decisiones basadas en el significado de los datos, solo en las precondiciones formales del protocolo de interacción.

La tercera condición es que **su resultado es una representación para el consumidor** — una respuesta de protocolo, una estructura de presentación o un estado visual — no un dato de dominio procesado.

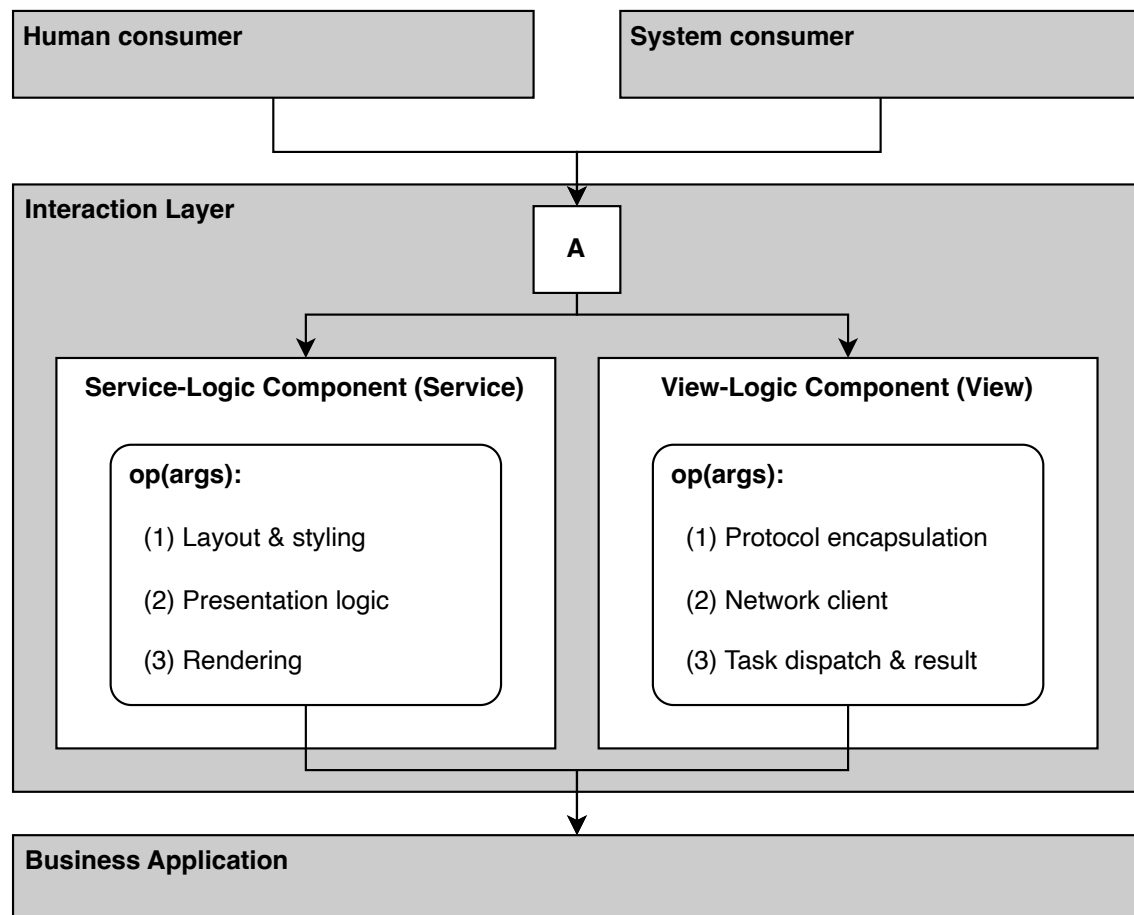


Figure 9. Contenido prescrito de un componente lógico de la Capa de Interacción. Recibe invocaciones del consumidor externo — sistema externo en el caso del servicio, usuario humano en el caso de la vista —, valida las precondiciones formales del protocolo de interacción, delega la lógica de dominio en la Capa de Negocio a través del componente de inyección B y transforma el resultado en la representación que el consumidor espera. Ningún protocolo de acceso a sistemas externos ni regla de negocio reside en este componente.

Terminología de referencia El modelo XF define los siguientes términos para describir con precisión los elementos que componen la lógica de interacción de un artefacto. Esta terminología constituye un vocabulario común que facilita la comunicación entre desarrolladores, analistas y revisores sobre la lógica de interacción de cualquier artefacto XF con independencia de la tecnología empleada. Los términos se agrupan en dos conjuntos correspondientes a los dos tipos de componentes lógicos de la capa: servicios (consumidor sistémico) y vistas (consumidor humano).

Se denomina **protocolo de servicio** a la definición formal del mecanismo de interacción de un servicio con su consumidor sistémico — la ruta de acceso al recurso, el método de invocación, el protocolo de transporte y el formato de respuesta. El protocolo de servicio es la descripción completa de cómo un sistema externo puede comunicarse con el servicio.

Se denomina **validación de entrada** al conjunto de verificaciones que un servicio realiza sobre los datos recibidos antes de delegar a la Capa de Negocio. La validación de entrada cubre las restricciones formales del protocolo — presencia de parámetros obligatorios, tipos de datos correctos, formatos válidos — sin entrar en validaciones de dominio.

Se denomina **invocación de negocio** a la delegación de la ejecución a los componentes lógicos de negocio a través de B. La invocación de negocio es la operación central del servicio — todo lo que precede a ella es preparación, y todo lo que la sucede es representación del resultado.

Se denomina **respuesta del servicio** a la transformación del resultado de la invocación de negocio en la forma que el protocolo de transporte requiere — códigos de estado, estructuras de respuesta, formatos de error. La respuesta del servicio incluye tanto el caso de éxito como el tratamiento de las excepciones de negocio recibidas de la Capa de Negocio, que deben ser transformadas en respuestas de error del protocolo apropiadas.

Se denomina **trazado de la solicitud** al registro de la invocación de un servicio y su resultado para fines de auditoría, depuración o monitorización. El trazado es una responsabilidad opcional del servicio que no afecta al flujo principal de la invocación.

Se denomina **composición** a la definición de la jerarquía de elementos gráficos que forman una vista — qué elementos la componen, cómo se organizan entre sí y qué relaciones jerárquicas existen entre ellos. La composición es la estructura de la vista — el equivalente al HTML en el desarrollo web o a la jerarquía de widgets en Flutter.

Se denomina **estilo** a la definición del aspecto visual con el que se representa una vista — márgenes, colores, tipografía, distribución espacial de los elementos. El estilo es independiente de la composición en su definición conceptual, aunque en muchos marcos de desarrollo ambos se expresan de forma conjunta.

Se denomina **lógica de presentación** al conjunto de algoritmos y comprobaciones que determinan cómo debe comportarse una vista ante los datos que recibe y los eventos que se producen sobre ella. La lógica de presentación es el núcleo funcional de la vista — define qué debe mostrarse, cuándo y en qué estado — y comprende las decisiones sobre representación de resultados pendientes, vacíos, invocaciones de negocio en respuesta a eventos del usuario y reacciones a cambios de estado observados.

Se denomina **estado de presentación** al conjunto de datos que determina el comportamiento visual observable de una vista en un momento dado — qué datos se están mostrando, si hay una operación en curso, si el formulario está en modo edición. El estado de presentación es estado en el sentido formal que se define en §7.3.1 — determina el comportamiento observable del componente ante las operaciones que lo invocan — pero su alcance está limitado a la Capa de Interacción y no tiene

correspondencia directa con información del dominio de negocio: modela el contexto dinámico de la interacción gráfica.

Modelo de proceso de los servicios Un servicio define un punto de interacción sistémica — un mecanismo a través del cual otro sistema puede invocar operaciones del artefacto mediante un protocolo de comunicación definido. Su modelo de proceso sigue una secuencia fija: el sistema externo invoca un endpoint definido por el protocolo de servicio; el servicio aplica la validación de entrada sobre los datos recibidos; cuando la validación se satisface, realiza la invocación de negocio a través de B; el resultado — éxito o excepción de negocio — se transforma en la respuesta del servicio según el formato del protocolo de transporte. El trazado de la solicitud opera de forma transversal a la secuencia, sin afectar al flujo principal.

Modelo de proceso de las vistas Una vista define un punto de interacción gráfica — un elemento visual a través del cual un usuario humano puede invocar operaciones del artefacto o recibir información de él. Su modelo de proceso combina tres dimensiones simultáneas que el modelo XF recomienda encarecidamente encapsular en un único componente lógico, aunque reconoce que algunos marcos de desarrollo pueden requerir su distribución en ficheros separados: la composición define la jerarquía estática de elementos gráficos; el estilo determina el aspecto visual; la lógica de presentación gobierna el comportamiento dinámico. La vista mantiene un estado de presentación que captura el contexto observable de la interacción y se actualiza ante datos del dominio observados a través de B.

A título orientativo, son ejemplos típicos de lógica de presentación los siguientes:

- La lógica que determina qué debe representarse mientras se espera el resultado de una operación de negocio — un indicador de progreso, un esqueleto de contenido, un mensaje de espera (**loading**).
- La lógica que determina qué debe representarse cuando la operación de negocio devuelve un resultado vacío o sin datos relevantes (**emptyState**).
- La lógica que determina qué operación de negocio debe invocarse a través de B cuando el usuario interactúa con un elemento de la vista — pulsación de un botón, introducción de datos, gestos (**onClick**, **onInput**, **onSwipe**).
- La lógica que determina qué operaciones deben ejecutarse en cada fase del ciclo de vida de la vista — qué datos cargar al crearse, qué estado persistir al pausarse, qué recursos liberar al destruirse (**onCreate**, **onPause**, **onDestroy**).
- La lógica que determina cómo debe actualizarse la representación visual de la vista cuando el estado de un componente de negocio observable cambia, mediante el patrón de observación definido en §6.2.2 (**onStateChange**).

Estos ejemplos son orientativos — no constituyen una lista exhaustiva ni prescriptiva. La lógica de presentación de una vista concreta estará determinada por las necesidades de interacción del artefacto. El criterio para identificarla es el siguiente: toda lógica que determine qué se muestra o qué se ejecuta en respuesta a un dato o un evento, sin aplicar reglas de dominio ni acceder a sistemas externos, es lógica de presentación y pertenece a la Capa de Interacción.

Un componente de interacción **no debe** derivar su estado de presentación directa-

mente del estado de dominio de un componente de negocio sin mediación — debe obtenerlo a través de **B** y transformarlo en estado de presentación si es necesario. El modelo XF recomienda gestionar el estado de presentación mediante el componente de generalización `StatefulView<T>`, que abstrae el mecanismo de actualización del estado y el re-renderizado de la vista. La implementación autónoma del estado de presentación en el propio componente lógico es igualmente válida.

Criterio de distinción entre servicio y vista El criterio para clasificar un componente lógico de interacción como servicio o como vista es el tipo de consumidor al que está dirigido. Un componente es un servicio si su consumidor es un sistema externo que se comunica con el artefacto mediante un protocolo sistémico definido. Un componente es una vista si su consumidor es un usuario humano que interactúa con el artefacto mediante una interfaz gráfica. Esta distinción es recomendada — facilita la organización interna de la Capa de Interacción — pero no prescriptiva. Ambos tipos de componentes son igualmente conformes al modelo XF independientemente de cómo se organicen en la estructura de carpetas.

Subdivisiones de componentes lógicos La carpeta `/logic` de la Capa de Interacción contiene todos los componentes lógicos de interacción — servicios y vistas — del artefacto. En artefactos de pequeña escala, esta carpeta puede contener un número reducido de componentes que no requieren organización adicional. Sin embargo, a medida que el artefacto crece en funcionalidad y el número de puntos de entrada aumenta, la carpeta `/logic` puede llegar a contener un número de componentes suficientemente elevado como para dificultar su navegación y comprensión.

Para estos casos, el modelo XF permite — y recomienda — la creación de subdivisiones internas dentro de la carpeta `/logic` que agrupen los componentes lógicos según el criterio que el equipo de desarrollo considere más apropiado para su artefacto concreto. Estas subdivisiones son una decisión de diseño del equipo — no una prescripción normativa del modelo — y su implementación no afecta a la conformidad del artefacto.

Subdivisión recomendada El modelo XF propone como subdivisión de referencia la división entre interacciones gráficas y sistémicas, materializada en dos subcarpetas dentro de `/logic`:

- `/api/logic/view` — componentes lógicos que definen puntos de interacción gráfica — vistas — a través de los cuales un usuario humano puede invocar operaciones del artefacto o recibir información de él mediante una interfaz visual. Ejemplos típicos: `MainView`, `LoginView`, `TemperatureView`, `ScheduleView`.
- `/api/logic/service` — componentes lógicos que definen puntos de interacción sistémica — servicios — a través de los cuales un sistema externo puede invocar operaciones del artefacto mediante un protocolo de comunicación definido. Ejemplos típicos: `TemperatureService`, `AuthService`, `ScheduleService`.

La utilidad de esta subdivisión es clara en artefactos que combinan ambos tipos de interacción — por ejemplo un termostato con pantalla integrada y API de control remoto — donde la separación entre vistas y servicios facilita la orientación en la estructura del artefacto. En artefactos puramente backend o puramente frontend, la

subdivisión puede resultar redundante — una de las dos carpetas estará vacía — y el equipo puede optar por no implementarla.

El criterio concreto de clasificación de un componente como gráfico o sistémico sigue el criterio de distinción establecido en §7.2.3: el tipo de consumidor al que está dirigido. En casos ambiguos — por ejemplo, un componente que gestiona una notificación push que tiene tanto representación visual como protocolo sistémico — la clasificación es decisión del implementador.

Criterio de subdivisión El criterio de subdivisión legítimo en la Capa de Interacción es el que se deriva de su responsabilidad técnica: la resolución del tipo de entrada que el artefacto expone a sus consumidores. Toda subdivisión interna de `/api/logic` debe agrupar los componentes lógicos según propiedades del tipo de entrada que resuelven — naturaleza del consumidor (humano o sistémico), tipo de protocolo de invocación, tipo de interfaz — y no según el dominio funcional del artefacto. Un artefacto con múltiples protocolos de servicio puede subdividir por protocolo — `/api/logic/service/rest`, `/api/logic/service/websocket`. Un artefacto con varios tipos de interfaz gráfica puede subdividir por naturaleza de la interfaz — por ejemplo, separando vistas principales de componentes reutilizables — siempre que el criterio sea técnico, no de dominio.

Subdivisión por dominio: no conforme La subdivisión de `/api/logic` por dominio funcional del artefacto — por ejemplo, agrupar vistas o servicios en subcarpetas cuyo nombre coincide con conceptos del dominio modelado por la Capa de Negocio — no es conforme al modelo XF. La razón es estructural: la lógica de interacción resuelve tipo de entrada, no dominio. Varios dominios del artefacto pueden compartir el mismo tipo de interfaz gráfica o el mismo protocolo de servicio, y subdividir por dominio implicaría replicar la misma resolución de entrada en varias subcarpetas con el riesgo de redundancia y de mezcla de responsabilidades. El dominio es criterio de subdivisión legítimo exclusivamente en la Capa de Negocio (§7.2.2).

Condiciones formales Toda subdivisión es válida siempre que se respeten tres condiciones. La primera es que todos los componentes lógicos de la Capa de Interacción estén ubicados dentro de la carpeta `/api/logic`, directamente o en alguna de sus subcarpetas. La segunda es que ninguna subcarpeta contenga componentes de un tipo distinto al lógico — los componentes de generalización, utilidad, transferencia e inyección deben permanecer en sus carpetas canónicas correspondientes. La tercera es que el criterio de subdivisión adoptado sea del vocabulario técnico de la capa — propiedades del tipo de entrada — y no del vocabulario del dominio modelado por la Capa de Negocio.

Ausencia de subdivisión Un artefacto que no implemente ninguna subdivisión dentro de `/logic` es totalmente conforme al modelo XF. La ausencia de subdivisión es la organización más simple y es apropiada para artefactos con un número reducido de componentes lógicos o para artefactos que solo implementan un tipo de interacción — únicamente vistas o únicamente servicios.

```
1 // Sin subdivisión - válido para artefactos de pequeña escala
2 /api/logic
3 |-- MainView
```

```

4  |-- LoginView
5  |-- TemperatureService
6
7  // Con subdivisión recomendada - válido para artefactos con ambos tipos
8  /api/logic
9  |-- /view
10     |-- MainView
11     |-- LoginView
12  |-- /service
13     |-- TemperatureService
14
15 // Con subdivisión por protocolo de servicio - igualmente válido
16 /api/logic
17 |-- /view
18     |-- MainView
19     |-- LoginView
20 |-- /service
21     |-- /rest
22         |-- TemperatureService
23     |-- /websocket
24         |-- LiveUpdateService

```

Código 7. Ausencia de subdivisión en la Capa de Interacción

Organización interna La Capa de Interacción se materializa en el sistema de ficheros del proyecto bajo la carpeta canónica `/api`, cuya estructura interna sigue la organización que se prescribe en §7.4. Esta cláusula consolida esa organización aplicada al contexto específico de la Capa de Interacción, describiendo la función de cada subcarpeta y su relación con las prescripciones del modelo de proceso definido en §7.2.3.

```

1  /api
2  |-- /general (Componentes de generalizacion)
3  |-- /logic (Componentes logicos)
4      |-- /view (Interacciones graficas -- recomendado)
5      |-- /service (Interacciones sistemicas -- recomendado)
6  |-- /transfers (Componentes de transferencia)
7  |-- /utils (Componentes de utilidad)
8  |-- A (Componente de inyeccion)

```

Código 8. Organización interna de la Capa de Interacción

/general — Componentes de generalización La carpeta `/general` contiene los componentes de generalización de la Capa de Interacción — las abstracciones de comportamiento estructural común entre componentes lógicos de interacción. Como se establece en §7.3.2, estos componentes no implementan lógica efectiva de ningún punto de interacción concreto — abstraen lógica contextual estructural compartida por varios componentes lógicos de la misma capa, liberando a los componentes lógicos concretos para que se concentren exclusivamente en su lógica efectiva.

Al igual que en las capas anteriores, la generalización en la Capa de Interacción puede operar en múltiples niveles de especificidad. Los patrones estructurales más habituales en esta capa son los siguientes:

Para los componentes de vista, los patrones de generalización típicos abstraen comportamientos estructurales comunes entre vistas. `StatefulView<T>` abstrae el mecanismo de gestión del estado de presentación y el re-renderizado de la vista cuando ese estado cambia — es el equivalente de `StatefulBusiness<T>` en la Capa de Negocio, aplicado al estado de presentación. `FormView` abstrae la lógica común

de los formularios — validación de campos, gestión del estado de edición, envío de datos. `FullscreenView` abstrae la estructura común de las pantallas completas — gestión de la navegación, ciclo de vida de pantalla completa.

Para los componentes de servicio, los patrones de generalización típicos abstraen protocolos de interacción comunes entre servicios. `RestService` abstrae la lógica común de todos los endpoints REST del artefacto — deserialización de la solicitud, serialización de la respuesta, tratamiento de errores del protocolo. `FormalService` abstrae la estructura de respuesta propia del artefacto — por ejemplo, una terna { code, description, data } que todos los servicios del artefacto utilizan como envoltorio de respuesta. `WebSocketService` abstrae la gestión del canal WebSocket y el ciclo de vida de la conexión.

Las cadenas de herencia típicas en la Capa de Interacción materializan los dos sub-árboles paralelos presentados en §7.2.3:

```

1 Service (comportamiento base de cualquier punto de entrada sistémico)
2   |-- RestService (protocolo HTTP/REST generico)
3       |-- FormalService (estructura de respuesta propia del artefacto)
4           |-- TemperatureService (logico -- endpoints de temperatura)
5           |-- AuthService         (logico -- endpoints de autenticacion)
6
7 View (comportamiento base de cualquier punto de entrada grafico)
8   |-- ScreenView (patron de pantalla completa)
9       |-- TemperatureView (logico -- pantalla de visualizacion de temperatura)
10  |-- AtomicView (patron de elemento grafico atomico reutilizable)
11      |-- ButtonView      (logico -- elemento de accion)
12      |-- TextInputView   (logico -- elemento de captura de texto)

```

Código 9. Patrones canónicos de generalización en la Capa de Interacción

Del mismo modo que en las capas anteriores, los componentes lógicos concretos — `TemperatureService`, `AuthService`, `TemperatureView`, `ButtonView`, `TextInputView` — heredan toda la lógica contextual del protocolo o del patrón gráfico de sus generalizaciones e implementan únicamente su lógica efectiva: los endpoints o elementos concretos, las validaciones específicas de cada recurso y la invocación de las operaciones de negocio correspondientes.

La presencia y el contenido de esta carpeta dependen de la complejidad de los protocolos de interacción del artefacto y del grado de reutilización que el equipo considere apropiado. Un artefacto con un único punto de interacción puede no requerir ningún componente de generalización.

/logic — Componentes lógicos La carpeta `/logic` contiene los servicios y vistas del artefacto — los componentes lógicos de la Capa de Interacción. Cada componente lógico define exactamente un punto de entrada al artefacto y la lógica de interacción asociada a ese punto, satisfaciendo las responsabilidades prescritas en §7.2.3: definición del protocolo de interacción, validación de las condiciones de interacción, delegación a la Capa de Negocio y representación del resultado.

La lógica efectiva de un componente lógico de interacción es lo que lo distingue de un componente de generalización en esta capa. Un componente que solo define comportamiento estructural genérico de protocolo sin implementar ningún punto de entrada concreto es un componente de generalización, no un componente lógico. Un componente que define el endpoint `/temperature` con sus validaciones específicas

y su invocación a `B.temperature.update()` es un componente lógico — esa es su lógica efectiva.

La organización interna de `/logic` — con o sin subdivisiones — sigue las prescripciones de §7.2.3.

/transfers — Componentes de transferencia La carpeta `/transfers` contiene los componentes de transferencia definidos en la Capa de Interacción — las estructuras de datos que modelan la información en la forma que los consumidores del artefacto reciben o envían.

En el contexto de la Capa de Interacción, los componentes de transferencia típicos modelan los datos de entrada y salida de los puntos de interacción del artefacto. Para los servicios, son habituales las estructuras que modelan los cuerpos de las solicitudes recibidas y las respuestas enviadas — con los atributos que el consumidor necesita ver, eliminando los que son internos al dominio. Para las vistas, son habituales las estructuras que modelan el estado de presentación de cada vista — qué datos se muestran, qué opciones están disponibles, qué errores deben representarse.

Como se prescribe en §7.3.5, la obligación de definir una nueva estructura en esta capa se activa únicamente cuando se modifica la definición estructural de una estructura proveniente de la Capa de Negocio. Si la estructura de negocio puede ser presentada al consumidor sin modificación estructural, puede ser referenciada directamente sin necesidad de definir una nueva estructura en esta carpeta.

Es igualmente habitual encontrar en esta carpeta las excepciones de interacción — componentes de transferencia que se transmiten típicamente como excepciones del lenguaje y modelan los errores propios del protocolo de interacción: `InvalidParameterException`, `UnauthorizedException`, `ResourceNotFoundException`.

/utils — Componentes de utilidad La carpeta `/utils` contiene los componentes de utilidad de la Capa de Interacción — operaciones auxiliares sin estado que sirven de soporte a los componentes lógicos de interacción para la consecución de su lógica efectiva sin pertenecer directamente al dominio del artefacto ni al protocolo de acceso.

En el contexto de la Capa de Interacción, los componentes de utilidad típicos proveen operaciones de validación formal de parámetros de entrada — `ValidationUtils` para la verificación de formatos, `FormUtils` para la validación de formularios —, operaciones de transformación de presentación — `ColorUtils` para el cálculo de colores derivados, `AnimationUtils` para la gestión de transiciones —, y operaciones de construcción de respuestas de protocolo — `ResponseUtils` para la construcción de respuestas HTTP estandarizadas.

Estos componentes de utilidad tienen alcance local a la Capa de Interacción. No deben ser referenciados por componentes de las capas inferiores.

A — Componente de inyección El componente de inyección `A` se ubica en la raíz de la carpeta `/api` y constituye el único punto de acceso a los componentes lógicos

de interacción desde cualquier otro componente de la misma capa. Su definición, propiedades y ciclo de vida se desarrollan en §7.3.3.

En el contexto de la Capa de Interacción, A agrega todos los servicios y vistas del artefacto bajo un punto de acceso centralizado. A diferencia de R y B — que son accedidos frecuentemente desde capas superiores — A es típicamente el componente de inyección menos invocado desde otras capas, dado que la Capa de Interacción es el nivel más alto de la arquitectura y sus componentes no son invocados por capas superiores. Su función principal es la gestión del ciclo de vida de los componentes lógicos de interacción — su inicialización en el orden correcto y su terminación controlada — y el acceso entre componentes lógicos de la misma capa cuando sea necesario.

7.3 Dimensión funcional: los tipos de componentes

El modelo XF define cinco tipos funcionales de componentes que, combinados con las tres capas de abstracción, producen el espacio de clasificación completo del modelo. Las definiciones de esta cláusula son normativas: establecen con precisión qué es cada tipo, qué puede contener, qué restricciones le aplican y cómo debe relacionarse con los demás componentes del artefacto.

Para cada tipo se establece: su definición, sus propiedades normativas, sus restricciones, su nomenclatura canónica y su organización en la estructura de carpetas del artefacto.

7.3.1 Componentes lógicos

Definición Un componente lógico es la unidad funcional principal de una capa. Implementa la **lógica efectiva** de la capa a la que pertenece — entendida como el conjunto mínimo de operaciones que justifican su existencia en el artefacto: aquellas que no pueden ser abstraídas en ningún otro tipo de componente sin perder la especificidad del concepto que el componente modela.

La lógica efectiva se distingue de la lógica contextual en que es inherente al concepto que el componente modela y no puede existir en ningún otro componente sin perder su identidad. Todo lo que puede ser compartido con otros componentes, reutilizado o expresado de forma genérica es lógica contextual — y pertenece a los componentes de generalización, utilidad o transferencia según su naturaleza. Los demás tipos de componentes de la capa proveen esa lógica contextual como soporte a la consecución de la lógica efectiva, pero no la reemplazan ni la contienen.

Pragmáticamente, un componente lógico es la respuesta a la pregunta: *¿qué hace este artefacto con este concepto del dominio?* Un componente lógico de negocio que gestiona la autenticación de usuarios define las operaciones “cotejar credenciales”, “refrescar sesión” y “caducar sesión” — estas son operaciones efectivas porque resuelven directamente el problema de autenticación. El cifrado en tránsito de las credenciales, el protocolo de acceso al servidor de identidades o el almacenamiento de la sesión en disco son lógica contextual que pertenece a otros tipos de componentes de la capa correspondiente.

Estructura interna de un componente lógico Un componente lógico se estructura internamente en tres elementos que el modelo XF define con precisión: su estado, sus operaciones y las sentencias que componen cada operación.

El **estado** de un componente lógico es el conjunto de datos que tienen correspondencia directa con información del dominio de negocio y que determina el comportamiento observable del componente ante las operaciones que lo invocan. El estado no debe confundirse con los atributos operativos internos del componente — una referencia a una conexión de base de datos, un flag de inicialización o un contador de reintentos son atributos, no estado. El estado describe el contexto dinámico del concepto que el componente modela: la temperatura actual registrada en `TemperatureBusiness`, los datos de la sesión activa en `SessionBusiness`, o la lista de mascotas cargadas en `PetBusiness` son ejemplos de estado. Los componentes lógicos son los únicos componentes del artefacto que pueden mantener estado en este sentido formal.

Una **operación** es un subconjunto de la lógica efectiva del componente que se ejecuta de forma atómica ante un evento o invocación, produciendo una transformación del estado del componente, un resultado para el invocante, o ambos. Cada operación de un componente lógico corresponde a exactamente una acción bien definida sobre el concepto que el componente modela — “actualizar temperatura”, “refrescar sesión”, “obtener mascotas por categoría”. Una operación que no pueda describirse como una acción sobre el concepto del dominio es un indicio de que la lógica que implementa es contextual y pertenece a otro tipo de componente.

Una **sentencia** es cada uno de los pasos lógicos que componen una operación. Las sentencias de una operación son ejecutadas de forma ordenada y su conjunto constituye la implementación completa de la acción que la operación modela. Una sentencia puede ser una transformación de datos, una invocación a un componente lógico de una capa inferior a través del componente de inyección correspondiente, una evaluación de una condición de negocio, o una mutación del estado del componente. El modelo XF recomienda que el número de sentencias por operación sea el mínimo necesario y suficiente para la consecución del objetivo de la operación — una operación con un número excesivo de sentencias es un indicio de que está asumiendo responsabilidades que deberían ser delegadas a otros componentes, otras operaciones, o a capas inferiores.

Una **condición** es una sentencia específica que evalúa el estado del componente o los parámetros de entrada de una operación antes de ejecutar la lógica que la operación modela. Las condiciones constituyen las precondiciones de ejecución de la operación en el sentido de Meyer [31]: cuando una condición no se satisface, la operación típicamente propaga un componente de transferencia de excepción al invocante, aunque otras formas de gestión — valor por defecto, reintento, lógica de fallback definida — son admisibles según la semántica del proceso. El modelo XF recomienda que las condiciones se evalúen tan pronto como sea posible en la operación, idealmente al inicio cuando la naturaleza del proceso lo permita, de forma que la condición anómala se transmita sin ejecutar lógica innecesaria. Los procesos cuyas precondiciones dependen de datos obtenidos por delegación previa son excepción legítima a esta recomendación.

El fragmento de código siguiente ilustra esta estructura interna en un componente lógico de negocio:

```

1  // Componente lógico de negocio - ejemplo de implementación
2  class TemperatureBusiness extends StatefulBusiness<Temperature> {
3
4      // Estado: la temperatura actual gestionada por el componente
5      Temperature current;
6
7      // Operación: actualizar la temperatura
8      void update(Temperature temperature) {
9
10         // Condiciones - precondiciones de la operación
11         if (TemperatureUtils.toCelsius(temperature) < 14)
12             throw new TemperatureOutOfRangeException();
13
14         if (TemperatureUtils.toCelsius(temperature) > 32)
15             throw new TemperatureOutOfRangeException();
16
17         if (B.network.signal < Network.GOOD)
18             throw new NetworkUnavailableException();
19
20         // Sentencias - pasos de la operación
21         R.server.saveTemperature(temperature); // delega en capa inferior
22         this.current = temperature; // muta el estado
23         super.update(temperature); // notifica a observadores
24     }
25 }

```

Código 10. Estructura interna de un componente lógico

Propiedades normativas La primera propiedad es la **responsabilidad única**: cada componente lógico modela exactamente un concepto del dominio de negocio, exactamente un protocolo de acceso a un sistema externo, o exactamente un punto de interacción del artefacto. Un componente que modele más de un concepto independiente debe ser descompuesto en tantos componentes lógicos como conceptos independientes contiene. Esta propiedad es condición necesaria para la correcta aplicación de la tipificación cerrada y exhaustiva establecida en §6.2.4 — un componente con responsabilidades mixtas no puede ser clasificado de forma unívoca en el espacio $L \times T$.

La segunda propiedad es la **capacidad de estado**: los componentes lógicos son los únicos componentes del artefacto que pueden mantener estado en el sentido formal definido en esta cláusula. Esta propiedad no implica que todo componente lógico deba mantener estado — un componente lógico sin estado es perfectamente válido en el modelo — sino que ningún otro tipo de componente puede mantenerlo.

La tercera propiedad es la **derivabilidad formal**: las operaciones que implementa un componente lógico se derivan típicamente de la especificación formal del proceso que el artefacto automatiza — diagramas de secuencia, diagramas de estados o especificaciones de casos de uso. El modelo recomienda esta trazabilidad como práctica de diseño, pero no la prescribe normativamente: artefactos sin especificación formal previa siguen siendo conformes. Una operación que no pueda trazarse hasta la especificación del proceso, cuando esta existe, es un indicio de que la lógica que implementa es contextual y pertenece a otro tipo de componente.

La cuarta propiedad es la **delegación hacia capas inferiores**: cuando una operación de un componente lógico requiere información o capacidades que residen en sistemas externos al artefacto, esa lógica debe ser delegada a componentes lógicos de capas de menor nivel de abstracción a través del componente de inyección correspondiente.

Un componente lógico de negocio no accede directamente a bases de datos, APIs externas o ficheros del sistema — delega ese acceso en el componente lógico de acceso correspondiente a través de R. Un componente lógico de interacción no implementa lógica de dominio — delega esa lógica en el componente lógico de negocio correspondiente a través de B. Esta propiedad es la expresión operativa del principio de aislamiento entre capas establecido en §6.2.2.

Restricciones Un componente lógico **debe** estar clasificado en exactamente una capa. La lógica que implementa debe corresponder exclusivamente a la responsabilidad de esa capa. La presencia de lógica correspondiente a otra capa constituye una violación del principio de aislamiento establecido en §6.2.2.

Un componente lógico **debe** implementar sus operaciones en términos de sentencias sobre su estado, condiciones sobre sus precondiciones y delegaciones a capas inferiores a través del componente de inyección. No debe implementar lógica que pueda ser clasificada como lógica contextual de otro tipo de componente.

Un componente lógico **puede** heredar de un componente de generalización de su misma capa. **No debe** heredar de otro componente lógico ni de un componente de generalización de otra capa.

Un componente lógico **puede** referenciar componentes de transferencia de su misma capa o de capas de menor nivel de abstracción. No debe referenciar componentes de transferencia de capas de mayor nivel de abstracción. La regla aplica uniformemente a las transferencias del propio artefacto y a las heredadas de artefactos externos integrados: todas son componentes de transferencia del dominio único $\mathbb{C}_T$.

Un componente lógico **puede** invocar operaciones de componentes lógicos de capas de menor nivel de abstracción, siempre a través del componente de inyección de la capa correspondiente. No debe invocar operaciones de componentes lógicos de capas de mayor nivel de abstracción.

Ciclo de vida del componente lógico Un componente lógico **debe** exponer una operación invocable `init()` y una operación invocable `terminate()`, que el componente de inyección de su capa orquesta en el arranque y la terminación del artefacto (§8.2). Su constructor se restringe a la inicialización trivial de los atributos propios del componente: toda lógica de arranque que dependa de recursos del entorno o que invoque a otros componentes pertenece a `init()`, no al constructor.

Nomenclatura canónica Los componentes lógicos de la Capa de Acceso **deben** llevar el sufijo `Repository` — por ejemplo, `DatabaseRepository`, `IdentityRepository`, `FileRepository`.

Los componentes lógicos de la Capa de Negocio **deben** llevar el sufijo `Business` — por ejemplo, `UserBusiness`, `SessionBusiness`, `TemperatureBusiness`.

Los componentes lógicos de la Capa de Interacción **deben** llevar el sufijo `Service` cuando implementan interacciones sistémicas — por ejemplo, `TemperatureService`, `AuthService` — o el sufijo `View` cuando implementan interacciones gráficas — por ejemplo, `LoginView`, `MainView`, `ThermometerView`.

Organización en la estructura de carpetas Los componentes lógicos se agrupan bajo la subcarpeta `/logic` dentro de la carpeta de su capa correspondiente:

```
1 /src
2 |--- /repository/logic (Componentes logicos de Acceso)
3 |--- /business/logic (Componentes logicos de Negocio)
4 |--- /api/logic (Componentes logicos de Interaccion)
```

Código 11. Organización en carpetas: componente lógico

7.3.2 Componentes de generalización

Definición Un componente de generalización es un componente que abstrae comportamiento estructural común entre dos o más componentes lógicos de la **misma capa**, permitiendo que ese comportamiento sea heredado por los componentes lógicos que lo requieran sin necesidad de duplicarlo en cada uno de ellos. A diferencia de los componentes lógicos, un componente de generalización no resuelve por sí solo ninguna lógica efectiva del artefacto — no modela ningún concepto del dominio, no implementa ninguna operación de negocio y no tiene utilidad independiente de los componentes lógicos que lo heredan. Su función es exclusivamente la de proveer una estructura reutilizable sobre la que los componentes lógicos construyen su lógica efectiva.

Pragmáticamente, un componente de generalización es la respuesta a la pregunta: *¿qué comportamiento estructural comparten varios componentes lógicos de esta capa?* Cuando varios componentes lógicos de la Capa de Negocio necesitan observar y notificar cambios de estado mutable, ese comportamiento observable es estructuralmente idéntico en todos ellos independientemente del concepto de negocio que cada uno modele. El componente de generalización `StatefulBusiness<T>` encapsula ese comportamiento, y cada componente lógico que lo herede — `TemperatureBusiness`, `UserBusiness`, `SessionBusiness` — lo especializa con el estado concreto que gestiona. Del mismo modo, cuando varios componentes lógicos de la Capa de Acceso implementan el protocolo REST, el componente de generalización `RestRepository` encapsula la lógica común del protocolo y cada repositorio concreto hereda de él para especializarlo con el recurso al que accede.

Distinción respecto a los componentes lógicos: paramétrica respecto al dominio La distinción entre un componente de generalización y un componente lógico no siempre es inmediata, y su correcta aplicación es uno de los criterios de diseño más relevantes del modelo XF. El criterio definitorio es el siguiente: un componente es de generalización si su lógica puede aplicarse a más de un concepto del dominio sin modificación — es decir, si es **paramétrico respecto al dominio**. Un componente es lógico si su lógica está **ligada a un concepto específico del dominio** y no puede aplicarse a otro sin modificación.

`StatefulBusiness<T>` es de generalización porque su lógica — mantener un estado observable y notificar a los observadores cuando cambia — es independiente del tipo de estado `T` que gestiona. `TemperatureBusiness` es lógico porque su lógica — validar umbrales de temperatura, sincronizar con el servidor, convertir unidades — está ligada al concepto de temperatura y no puede aplicarse a ningún otro concepto sin modificación.

Cuando un componente que se ha clasificado como de generalización comienza a contener referencias a conceptos concretos del dominio — por ejemplo, una clase base que hace referencia explícita a `Temperature` o a `User` — es un indicio de que la lógica que contiene ha dejado de ser estructural y ha pasado a ser efectiva, y por tanto debe ser trasladada al componente lógico concreto que modela ese concepto.

Atributos mutables y estado Los componentes de generalización pueden tener atributos mutables en tanto que son subconjuntos lógicos reproducibles de los componentes lógicos que los heredan. Sin embargo, existe una distinción normativa importante entre los atributos de un componente de generalización y el estado de un componente lógico.

Un componente de generalización puede definir atributos mutables que den soporte al comportamiento estructural que abstrae — por ejemplo, la lista de observadores registrados en `StatefulBusiness<T>`, o la referencia al cliente HTTP en `RestRepository`. Estos atributos son operativos: sirven al funcionamiento interno del patrón estructural, pero no tienen correspondencia directa con información del dominio de negocio.

El estado en el sentido formal del modelo — el conjunto de datos con correspondencia directa con información del dominio — pertenece siempre al componente lógico concreto que hereda del componente de generalización, nunca al componente de generalización en sí. Un componente de generalización no debe definir atributos que modelen información del dominio de negocio — esa responsabilidad es exclusiva de los componentes lógicos.

```

1 // Componente de generalización - atributos operativos, sin estado de dominio
2 abstract class StatefulBusiness<T> {
3
4     // Atributo operativo: lista de observadores registrados
5     List<Observer<T>> observers = new ArrayList<>();
6
7     // Atributo operativo: referencia al estado actual (genérico, sin
8     // semántica de dominio)
9     T state;
10
11     void observe(Observer<T> observer) {
12         observers.add(observer);
13     }
14
15     void update(Partial<T> partial) {
16         this.state = partial.applyOn(this.state);
17         observers.forEach(obs -> obs.notify(this.state));
18     }
19 }
20
21 // Componente lógico - estado con semántica de dominio concreta
22 class TemperatureBusiness extends StatefulBusiness<Temperature> {
23
24     // El estado de dominio es Temperature, definido en el componente lógico
25     void update(Temperature temperature) {
26
27         if (TemperatureUtils.toCelsius(temperature) < 14)
28             throw new TemperatureOutOfRangeException();
29
30         R.server.saveTemperature(temperature);
31
32         // delega la notificación al componente de generalización
33         super.update(temperature);
34     }
35 }

```

Código 12. Atributos mutables y estado

El principio de aislamiento aplicado a la generalización El principio de aislamiento entre capas establecido en §6.2.2 aplica a los componentes de generalización con la misma fuerza que a los componentes lógicos. Un componente de generalización está clasificado en una capa específica y su alcance está estrictamente limitado a esa capa: ningún componente lógico puede heredar de un componente de generalización de una capa distinta a la suya.

Esta restricción tiene una consecuencia que el modelo XF prescribe de forma explícita: cuando el mismo patrón estructural es necesario en más de una capa, cada capa debe implementar su propio componente de generalización de forma independiente. El modelo prescribe esta duplicación controlada porque la alternativa — un componente de generalización transversal a varias capas — introduciría una dependencia estructural entre capas que comprometería el aislamiento y la resistencia al cambio local del artefacto.

Los patrones estructurales que más frecuentemente requieren implementación independiente por capa son los siguientes:

El patrón de observación de estado mutable — necesario cuando los componentes de capas superiores necesitan reaccionar a cambios de estado en componentes de capas inferiores sin que estos los invoquen directamente — se implementa mediante una generalización propia en cada capa que lo requiera, por ejemplo, `StatefulRepository`, `StatefulBusiness` y `StatefulView` en las capas de Acceso, Negocio e Interacción respectivamente (esta última aplicada al estado de presentación). Las tres implementaciones encarnan el mismo patrón, pero son componentes de generalización independientes y no comparten código entre capas.

El patrón de tarea periódica — necesario cuando un componente necesita ejecutar lógica de forma automática en intervalos de tiempo definidos — se implementa mediante una generalización propia en cada capa que lo requiera, por ejemplo, `ScheduledRepository`, `ScheduledBusiness` y `ScheduledView` respectivamente. Del mismo modo, las tres implementaciones son independientes entre sí.

El patrón de comunicación por protocolo se materializa en dos generalizaciones complementarias, una en cada capa frontera del artefacto. En la Capa de Acceso — por ejemplo, `RestRepository`, `WebSocketRepository` o `PostgresRepository` — abstrae la lógica común de *consumir* protocolos publicados por sistemas externos. En la Capa de Interacción — por ejemplo, `RestService`, `WebSocketService` o `FormalService` — abstrae la lógica común de *exponer* los protocolos del propio artefacto a sus consumidores. Ambas son generalizaciones distintas pese a compartir nombre de protocolo: una consume y la otra expone, y cada una respeta el aislamiento de su capa.

El comportamiento compartido no trivial entre capas La generalización entre capas no es admisible: el comportamiento compartido por componentes de distintas capas se encapsula como componente lógico de la **capa más baja** que lo requiere, invocable por las superiores en dirección descendente.

Propiedades normativas La primera propiedad es la **neutralidad de dominio**: un componente de generalización no debe contener referencias a conceptos concretos del dominio de negocio del artefacto. Su lógica debe ser aplicable a cualquier componente lógico de su capa que satisfaga las precondiciones estructurales del patrón que abstrae. La presencia de referencias a conceptos concretos del dominio en un componente de generalización es un indicio de que parte de su lógica debe ser trasladada al componente lógico concreto.

La segunda propiedad es la **dependencia de los componentes lógicos**: en el contexto de un artefacto final (no librería), un componente de generalización no tiene utilidad independiente — existe en función de los componentes lógicos que lo heredan. Esta propiedad se enuncia como criterio de diseño y no como regla normativa: las librerías de utilidad pueden exportar componentes de generalización pensados para ser consumidos por artefactos terceros, sin que ello suponga una violación de conformidad del artefacto-librería.

La tercera propiedad es el **aislamiento de capa**: el alcance de un componente de generalización está estrictamente limitado a la capa en la que está definido. No debe ser heredado por componentes — lógicos o de generalización — de otras capas, ni debe referenciar componentes de inyección de capa alguna — ni la propia ni cualquier otra — ya que el acceso a las instancias de los componentes lógicos a través de los componentes de inyección está reservado a otros componentes lógicos (§7.3.3).

Restricciones Un componente de generalización **debe** estar clasificado en exactamente una capa y su alcance está limitado a esa capa.

Un componente de generalización **debe** ser abstracto — no puede ser instanciado directamente. Su instanciación se produce siempre a través de los componentes lógicos concretos que lo heredan.

Un componente de generalización **puede** tener atributos mutables que soporten el comportamiento estructural que abstrae, pero no debe definir atributos que modelen información del dominio de negocio.

Un componente de generalización **no debe** contener referencias a conceptos concretos del dominio de negocio del artefacto.

Un componente de generalización **no debe** ser heredado por componentes — lógicos o de generalización — de una capa distinta a la suya, ni por componentes de tipo distinto al lógico o de generalización.

Un componente de generalización **no debe** invocar operaciones de componentes lógicos a través de los componentes de inyección, en ninguna capa — ni la propia ni cualquier otra —. El acceso a las instancias de los componentes lógicos a través de las inyecciones está reservado a otros componentes lógicos (§7.3.3). Si una generalización necesita acceder a lógica encapsulada en un componente lógico, esa necesidad es un indicio de que la lógica que contiene es efectiva y pertenece al componente lógico concreto, no al componente de generalización.

Nomenclatura canónica Los componentes de generalización **deben** llevar el nombre del patrón estructural que abstraen seguido del sufijo canónico de la capa

a la que pertenecen (`Repository`, `Business`, `Service` o `View` según corresponda) — por ejemplo, `StatefulBusiness`, `ScheduledBusiness`, `BroadcastBusiness` en la Capa de Negocio; `RestRepository`, `PostgresRepository`, `StatefulRepository`, `ScheduledRepository` en la Capa de Acceso; `StatefulView`, `ScheduledView`, `FormView`, `RestService` en la Capa de Interacción.

Ciclo de vida del componente de generalización Un componente de generalización **debe** satisfacer las mismas obligaciones de ciclo de vida que el componente lógico que lo hereda: declarar (o heredar) una operación invocable `init()` y una operación invocable `terminate()` (§8.2), y restringir su constructor a la inicialización trivial de atributos estructurales propios de la abstracción. Toda lógica de arranque que dependa de recursos del entorno o que invoque otros componentes pertenece a `init()`, no al constructor.

Organización en la estructura de carpetas Los componentes de generalización se agrupan bajo la subcarpeta `/general` dentro de la carpeta de su capa correspondiente. La pertenencia de un componente a la categoría “generalización en capa *l*” se determina por su ubicación bajo `/src/l/general/`; la clasificación es total por construcción (§7.4).

```

1 /src
2 |--- /repository/general (Componentes de generalizacion de Acceso)
3 |--- /business/general (Componentes de generalizacion de Negocio)
4 |--- /api/general (Componentes de generalizacion de Interaccion)

```

Código 13. Organización en carpetas: componente de generalización

7.3.3 Componentes de inyección

Definición Un componente de inyección es el único punto de acceso a los componentes lógicos de su capa desde cualquier otro componente del artefacto. Agrega las únicas instancias de todos los componentes lógicos de su capa bajo un punto de acceso centralizado, garantizando que cada componente lógico tenga exactamente una instancia durante toda la ejecución del artefacto, y que esa instancia sea accesible de forma directa y predecible desde cualquier punto del artefacto que la requiera.

El componente de inyección implementa el patrón **Singleton Gathering** — una extensión del patrón Singleton [14] que agrega en un único punto las instancias únicas de un conjunto de componentes relacionados, en lugar de gestionar la unicidad de un único componente. Esta extensión es la que confiere al componente de inyección su doble función: por un lado, garantiza la unicidad de instancia de cada componente lógico de la capa; por otro, provee un punto de acceso nombrado y estable a cada una de esas instancias, haciendo que la estructura de componentes lógicos de la capa sea visible e inspeccionable sin necesidad de herramientas adicionales.

Pragmáticamente, el componente de inyección es la respuesta a la pregunta: *¿qué componentes lógicos existen en esta capa, cómo se inicializan, y en qué orden?* Un desarrollador que consulte el componente de inyección de la Capa de Negocio — **B** — obtiene de forma inmediata la lista completa de componentes lógicos de negocio disponibles en el artefacto, sus nombres canónicos y el orden en que se inicializan.

Esta propiedad de autodocumentación es uno de los beneficios más prácticos del modelo para los equipos de desarrollo.

El modelo XF define exactamente **tres componentes de inyección** en todo artefacto, uno por capa:

- **R** — componente de inyección de la Capa de Acceso, que agrega las instancias únicas de los componentes lógicos de tipo **Repository**.
- **B** — componente de inyección de la Capa de Negocio, que agrega las instancias únicas de los componentes lógicos de tipo **Business**.
- **A** — componente de inyección de la Capa de Interacción, que agrega las instancias únicas de los componentes lógicos de tipo **Service** y **View**.

Naturaleza del componente de inyección El componente de inyección es una clase **no instanciable** — no puede ser objeto de instanciación por parte de ningún componente del artefacto. Nadie crea un `new B()` ni un `new R()`. Esta propiedad es intencional y tiene dos consecuencias normativas directas.

La primera es que el acceso a los componentes lógicos a través del componente de inyección es siempre **estático** — las referencias a las instancias de los componentes lógicos son atributos estáticos públicos del componente de inyección, accesibles directamente mediante `B.session`, `R.database`, `A.temperatureService` sin necesidad de instanciar el componente de inyección.

La segunda es que el componente de inyección **no mantiene estado** — no tiene atributos de instancia, no tiene operaciones de mutación de estado y no puede ser observado. Su única función es la de agregar referencias estáticas a instancias de componentes lógicos y gestionar su ciclo de vida a través de las operaciones de inicialización y terminación.

Las referencias estáticas que el componente de inyección mantiene son **inmutables post-inicialización**: se asignan una única vez durante la operación `init()` del componente de inyección y no deben ser reasignadas durante la ejecución del artefacto. Antes de que `init()` haya sido invocado, las referencias pueden estar en estado no inicializado — el modelo XF prescribe que ningún componente del artefacto acceda a las referencias de un componente de inyección antes de que su operación `init()` haya completado. El incumplimiento de esta prescripción puede producir condiciones de carrera o accesos a referencias no inicializadas cuyo comportamiento es indefinido.

El ciclo de vida gestionado por el componente de inyección El componente de inyección gestiona el ciclo de vida completo de los componentes lógicos de su capa mediante una **instanciación inicial** — que establece el estado inicial σ_0 de cada lógico — y **dos operaciones invocables** — inicialización y terminación — que el modelo XF prescribe con precisión:

La **instanciación inicial** es la declaración, por parte del componente de inyección, de las instancias únicas de los componentes lógicos de la capa como atributos estáticos públicos. No es una operación invocable: es el efecto, en el estado del artefacto, de la inicialización de campos estáticos del lenguaje al cargar la clase del singleton — antes de que cualquier operación del componente de inyección sea invocada —, resultado de invocar el constructor de cada componente lógico. Esta

instanciación cumple dos funciones: establece qué componentes lógicos existen en la capa y con qué nombre canónico son accesibles y, simultáneamente, establece el **estado inicial** σ_0 **de cada componente lógico** — el estado en que cada lógico se encuentra inmediatamente después de su construcción, antes de invocar `init()` —. El componente de inyección no posee estado mutable propio: σ_0 es propiedad del lógico instanciado, no del componente de inyección que lo instancia. Es importante distinguir que la **instanciación** — la creación del objeto en memoria mediante el constructor — es una operación del lenguaje de programación que puede ejecutar lógica de inicialización de atributos internos del lógico, pero **no debe** ejecutar lógica que dependa de recursos externos ni invocar operaciones de otros componentes. Cualquier lógica de arranque que dependa de recursos externos debe ser delegada a la operación `init()` del componente lógico.

```

1 // Componente de inyección B - instanciación inicial de los componentes lógicos
2 final abstract class B {
3
4     // Estado inicial sigma_0 - instanciación de los componentes
5     // lógicos de la capa al cargar la clase del singleton
6
7     static readonly SessionBusiness session = new SessionBusiness();
8
9     static readonly UserBusiness user = new UserBusiness();
10
11    static readonly TemperatureBusiness temperature = new
12        TemperatureBusiness();
13
14    static readonly NetworkBusiness network = new NetworkBusiness();
15 }

```

Código 14. Ciclo de vida (I): instanciación inicial de los componentes lógicos

La **inicialización** es la ejecución de la lógica de arranque de cada componente lógico de la capa, delegada a través de la operación `init()` de cada componente lógico. La inicialización es responsabilidad del modelo XF — es la operación que prepara cada componente lógico para procesar las operaciones para las que fue diseñado, estableciendo conexiones con sistemas externos, arrancando temporizadores de tareas programadas, cargando configuración desde fuentes externas o estableciendo el estado inicial del componente.

```

1 // Componente de inyección B - composición completa con operación init()
2 final abstract class B {
3
4     static readonly SessionBusiness session = new SessionBusiness();
5
6     static readonly UserBusiness user = new UserBusiness();
7
8     static readonly TemperatureBusiness temperature = new
9         TemperatureBusiness();
10
11    static readonly NetworkBusiness network = new NetworkBusiness();
12
13    // Inicialización - en orden de dependencias entre componentes
14    static void init() {
15        B.network.init(); // NetworkBusiness no depende de otros componentes
16        B.session.init(); // SessionBusiness puede depender de NetworkBusiness
17        B.user.init(); // UserBusiness puede depender de SessionBusiness
18        B.temperature.init(); // TemperatureBusiness puede depender de ambos
19    }
20 }

```

Código 15. Ciclo de vida (II): composición completa con la operación `init()`

La **terminación** es la ejecución de la lógica de cierre de cada componente lógico de la capa, delegada a través de la operación `terminate()` de cada componente lógico en orden inverso al de inicialización. La terminación libera los recursos adquiridos durante la inicialización — cierra conexiones con sistemas externos, detiene temporizadores de tareas programadas, persiste el estado que deba ser preservado entre ejecuciones y deja el sistema en un estado de reposo definido para la siguiente ejecución del artefacto. Como buena práctica, el artefacto invoca `terminate()` antes de la finalización de su proceso siempre que el entorno de ejecución lo permita — tanto en condiciones de cierre controlado como ante la detección de excepciones no controladas —; la invocación efectiva depende del entorno y la política del modelo ante esta situación se desarrolla en §8.3.

```

1 // Componente de inyección B - operación terminate() en orden inverso
2 final abstract class B {
3
4     // ... definición e inicialización ...
5
6     // Terminación - en orden inverso al de inicialización
7     static void terminate() {
8         B.temperature.terminate();
9         B.user.terminate();
10        B.session.terminate();
11        B.network.terminate();
12    }
13 }

```

Código 16. Ciclo de vida (III): terminación en orden inverso

El patrón de acceso a componentes lógicos Todo acceso a un componente lógico desde cualquier punto del artefacto — incluyendo componentes de la misma capa — se realiza exclusivamente a través del componente de inyección de la capa a la que pertenece el componente lógico invocado, siguiendo el patrón:

```

1 <componente_de_inyeccion>.<componente_logico>.<operacion>(<parametros>)

```

Código 17. Patrón de acceso canónico: forma general

Este patrón aplica sin excepción. Un componente lógico de negocio que necesite invocar una operación de otro componente lógico de negocio lo hace a través de B, no mediante referencia directa entre componentes:

```

1 // Uso del componente de inyección - patrón de acceso canónico desde un
2 // componente lógico
3 class UserBusiness {
4     User getUser(String token) {
5
6         // Acceso a SessionBusiness a través del componente de inyección B
7         Session session = B.session.decode(token);
8         return session.user;
9     }
10 }

```

Código 18. Patrón de acceso canónico: ejemplo de uso

La prohibición de instanciación directa de componentes lógicos fuera del componente de inyección es absoluta. La creación de una instancia de un componente lógico fuera del componente de inyección — mediante `new UserBusiness()` en cualquier punto del artefacto distinto del componente de inyección B — constituye una violación

del modelo independientemente del contexto en que se produzca. Esta restricción garantiza que la unicidad de instancia prescrita por el modelo sea verificable de forma estática por las herramientas de análisis de conformidad.

Concurrencia y unicidad de instancia El modelo XF prescribe unicidad lógica de instancia dentro del espacio de ejecución del artefacto: existe exactamente una instancia de cada componente lógico por contexto de ejecución. En entornos de ejecución concurrente — múltiples hilos accediendo simultáneamente al artefacto — la coordinación concreta entre la inicialización de las referencias estáticas y el resto de la lógica del artefacto es decisión del implementador y depende del modelo de concurrencia del lenguaje de implementación; el modelo XF no prescribe mecanismos de exclusión mutua, atomicidad ni orden de visibilidad entre hilos, por mantenerse agnóstico respecto del modelo de concurrencia.

Cuando el artefacto se ejecuta simultáneamente en múltiples instancias con espacios de memoria disjuntos, cada instancia desplegada constituye un **artefacto XF independiente**, con su propio conjunto de instancias singleton gestionadas por sus componentes de inyección. La arquitectura interna de cada uno de esos artefactos es plenamente conforme al modelo y está sujeta a todas sus prescripciones. La coherencia entre instancias distintas — sincronización de estado, propagación de eventos, consenso — pertenece a la infraestructura de comunicación entre artefactos, no a la estructura interna de ninguno de ellos. Es esa coherencia inter-artefacto, y no el caso de ejecución distribuida en sí, la que queda fuera del alcance del modelo XF, en aplicación directa del principio de agnosticismo respecto a la infraestructura de despliegue (§6.2.1).

Propiedades normativas La primera propiedad es la **unicidad**: existe exactamente un componente de inyección por capa en cada raíz de artefacto XF. No deben existir múltiples componentes de inyección para una misma capa, ni debe existir un componente de inyección que agregue componentes lógicos de más de una capa.

La segunda propiedad es la **exhaustividad funcional**: el componente de inyección agrega las instancias de **todos** los componentes lógicos de su capa que deban ser accesibles desde otros componentes del artefacto. Un componente lógico que no esté registrado en el componente de inyección de su capa no es accesible desde ningún otro componente del artefacto. Esta propiedad se enuncia como criterio de diseño y no como regla normativa: pueden existir componentes lógicos declarados pero deliberadamente no expuestos a través de la inyección (por ejemplo, lógicos internos de una librería que solo se usan dentro de su propio paquete).

La tercera propiedad es la **centralidad de acceso**: el componente de inyección es el único punto de acceso a los componentes lógicos de su capa, y este acceso queda reservado a otros componentes lógicos. La prescripción se sigue por inferencia directa de dos propiedades establecidas en §7.3.1: (P1) los componentes lógicos son los únicos componentes del artefacto que encapsulan la lógica efectiva del dominio, y (P2) la lógica efectiva solo puede ser ejecutada por los componentes lógicos que la encapsulan. Dado que el acceso a un componente lógico a través de su componente de inyección — el patrón canónico `<inyección>.<componente>.<operación>()` — es por definición una invocación de la lógica efectiva encapsulada en ese componente lógico, (C)

tales referencias solo pueden originarse en componentes lógicos. Componentes de generalización, utilidad, transferencia o de otra inyección no deben referenciar miembros slot de ninguna inyección. No existe ningún mecanismo alternativo de acceso a un componente lógico que sea compatible con el modelo XF.

La cuarta propiedad es la **inmutabilidad post-inicialización**: las referencias a los componentes lógicos son asignadas durante la definición e inicialización del componente de inyección y no deben ser modificadas durante la ejecución del artefacto. La constancia estructural del artefacto a nivel global durante la fase de ejecución se garantiza adicionalmente como propiedad formal del modelo, sin operacionalización mecánica: la prescripción es la propiedad observable, no un mecanismo concreto.

Restricciones Las restricciones del componente de inyección se agrupan en dos bloques: las estructurales, que prescriben la composición de la clase y los miembros declarables, y las de ciclo de vida, que prescriben las operaciones invocables y las condiciones de su uso.

Restricciones estructurales Un componente de inyección **no debe** ser instanciable. Su implementación debe impedir explícitamente la creación de instancias — mediante clase abstracta final, constructor privado u otro mecanismo equivalente provisto por el lenguaje de programación.

Las referencias estáticas a componentes lógicos declaradas en el componente de inyección **deben** tener visibilidad pública, de forma que sean accesibles desde cualquier punto del artefacto mediante el patrón canónico `<inyección>.<componente>.<operación>()`. La aplicabilidad de la regla es trivial en lenguajes que no diferencian visibilidad de miembros estáticos.

Un componente de inyección **no debe** declarar miembros distintos de referencias estáticas tipadas a componentes lógicos de su capa y de las operaciones estáticas `init()` y `terminate()` que orquestan el ciclo de vida de esos lógicos. Cualquier otro miembro — atributo no tipado a un lógico de la capa, operación estática distinta de `init()` o `terminate()`, lógica de dominio incrustada en cualquier parte de la clase — constituye violación del modelo.

Un componente de inyección **no debe** ser accedido desde una capa de menor nivel de abstracción que la capa a la que pertenece. El componente de inyección A de la Capa de Interacción no debe ser accedido desde la Capa de Negocio ni desde la Capa de Acceso. El componente de inyección B de la Capa de Negocio no debe ser accedido desde la Capa de Acceso. El componente de inyección R de la Capa de Acceso puede ser accedido desde la Capa de Negocio y desde la Capa de Interacción.

Un componente de inyección **no debe** heredar de ningún componente, interno o externo al artefacto. La inyección es un singleton único por capa cuyo contrato está fijado por el modelo — conjunto cerrado de miembros, no instanciable, puramente estático, con dos operaciones `init()` y `terminate()` prescritas por el modelo —, y la herencia introduciría miembros adicionales no admisibles o entraría en conflicto con la unicidad estructural del componente de inyección. La inyección no participa en jerarquías de herencia.

Restricciones de ciclo de vida Un componente de inyección **debe** declarar la operación estática `init()`. Esta operación orquesta la inicialización ordenada de los componentes lógicos de su capa; su ausencia impide al artefacto invocar la inicialización de la capa.

Un componente de inyección **debe** declarar la operación estática `terminate()`. Esta operación libera de forma controlada los recursos adquiridos durante `init()`; su ausencia impide al artefacto invocar la terminación controlada de la capa, independientemente de si el entorno de ejecución garantiza su invocación.

El cuerpo de la operación `init()` de un componente de inyección **no debe** contener sentencias distintas de invocaciones a la operación `init()` de los componentes lógicos agregados como slots en esa misma inyección. La inyección es orquestador puro de la inicialización de su capa: cualquier otra sentencia — invocaciones a operaciones distintas de `init()`, invocaciones a otras inyecciones, lógica de procesamiento, E/S contra el entorno, acceso a recursos externos — excede el rol formal del Singleton Gathering y constituye violación.

El cuerpo de la operación `terminate()` de un componente de inyección **no debe** contener sentencias distintas de invocaciones a la operación `terminate()` de los componentes lógicos agregados como slots en esa misma inyección. Constituye la restricción simétrica, sobre la fase de terminación, de la prescrita para la operación `init()`.

Las operaciones `init()` y `terminate()` de un componente de inyección **deben** ser simétricas en su delegación: todo componente lógico cuyo `init()` sea invocado por `init()` de la inyección **debe** tener su `terminate()` invocado por `terminate()` de esa misma inyección, en orden inverso. La asimetría — componentes inicializados sin terminación correspondiente, o terminados sin haber sido inicializados — constituye violación de la propiedad formal de simetría de recursos del ciclo de vida (§8.2).

La invocación de las operaciones `init()` y `terminate()` de un componente lógico **queda reservada** al componente de inyección de su capa. Ningún otro componente del artefacto — ni lógicos de la misma o de otra capa, ni generalizaciones, ni utilidades, ni transferencias, ni otras inyecciones — puede invocar `init()` ni `terminate()` sobre un componente lógico. La orquestación del ciclo de vida del lógico está confinada al componente de inyección de su capa.

La invocación de las operaciones `init()` y `terminate()` de un componente de inyección **queda reservada** al punto de inicio de ejecución del artefacto. Ningún componente clasificable del artefacto — lógico, de generalización, de utilidad, de transferencia o de otra inyección — puede invocar `R.init()`, `B.init()`, `A.init()` ni sus simétricas `terminate()`.

La existencia de al menos un orden de invocación válido entre los componentes lógicos inyectados es propiedad derivada del modelo: como consecuencia de aplicar el principio de aislamiento entre capas (§6.2.2), el grafo de dependencias inter-capas resulta acíclico (propiedad derivada del modelo). La elección concreta del orden, entre los topológicamente válidos, queda a discreción del implementador. El orden de invocación **entre** las inyecciones R, B y A respeta la dependencia descendente entre capas: `R.init()` precede a `B.init()`, que precede a `A.init()`, y `terminate()` se invoca en orden inverso.

Nomenclatura canónica Los tres componentes de inyección del modelo XF tienen nombres canónicos fijos que no admiten variación:

- R — componente de inyección de la Capa de Acceso.
- B — componente de inyección de la Capa de Negocio.
- A — componente de inyección de la Capa de Interacción.

Organización en la estructura de carpetas Cada componente de inyección se ubica en la raíz de la carpeta de su capa correspondiente, al mismo nivel que las subcarpetas `/logic`, `/general`, `/utils` y `/transfers`:

```

1 /src
2   |-- /repository
3     |-- R (Componente de inyeccion de la Capa de Acceso)
4   |-- /business
5     |-- B (Componente de inyeccion de la Capa de Negocio)
6   |-- /api
7     |-- A (Componente de inyeccion de la Capa de Interaccion)

```

Código 19. Organización en carpetas: componente de inyección

7.3.4 Componentes de utilidad

Definición Un componente de utilidad es un componente que provee operaciones auxiliares de alcance local a su capa, que sirven de soporte contextual a los componentes lógicos de esa capa para la consecución de su lógica efectiva, pero que no resuelven por sí solos ningún problema relacionado directamente con el dominio del artefacto. Su función es encapsular algoritmos y transformaciones de propósito general que, de no existir, deberían duplicarse en múltiples componentes lógicos de la misma capa.

Pragmáticamente, un componente de utilidad es la respuesta a la pregunta: *¿qué operaciones auxiliares necesitan los componentes lógicos de esta capa para ejecutar su lógica efectiva, que no pertenecen al dominio del artefacto?* Un componente lógico de negocio que gestiona la temperatura necesita convertir valores entre unidades de medida — Celsius, Fahrenheit, Kelvin — antes de aplicar sus reglas de negocio. Esa conversión no es lógica de negocio — no modela ningún concepto del dominio, no modifica el estado de ningún componente lógico y no depende de ninguna regla de negocio específica. Es lógica auxiliar de propósito general que puede encapsularse en `TemperatureUtils` y ser reutilizada por cualquier componente lógico de negocio que la necesite.

Distinción respecto a los componentes lógicos: ausencia de modelado y de efectos laterales La distinción entre un componente de utilidad y un componente lógico es operativa y verificable mediante dos criterios complementarios.

El primero es el criterio de **ausencia de modelado del dominio**: un componente de utilidad puede operar sobre conceptos del dominio del artefacto — filtrar, mapear, agrupar, transformar estructuralmente sus instancias, leer sus atributos — pero no debe implementar las reglas que definen sus transiciones, sus transformaciones semánticas ni el modelado conceptual del dominio. `TemperatureUtils` puede convertir valores numéricos entre escalas (Celsius, Fahrenheit, Kelvin) porque la conversión

es una operación matemática aplicable a cualquier magnitud térmica, no una regla del dominio del artefacto. Si en cambio aplicara umbrales definidos por el negocio, decidiera si una temperatura es “crítica” o “segura” o ejecutara la lógica de respuesta del termostato ante una variación, esas operaciones serían reglas del dominio del artefacto y pertenecerían a un componente lógico — no a una utilidad.

El segundo es el criterio de **ausencia de efectos laterales**: un componente de utilidad no modifica el estado de ningún componente del artefacto, no invoca operaciones de componentes lógicos a través de los componentes de inyección, no realiza E/S directa contra el sistema o la red, no muta sus argumentos por referencia y no depende de estado externo mutable. Las operaciones de un componente de utilidad son funciones puras en el sentido de la programación funcional — dada la misma entrada, producen siempre la misma salida. Si una operación produce cualquiera de los efectos anteriores, pertenece a un componente lógico.

Propiedades estructurales Los componentes de utilidad tienen un conjunto de propiedades estructurales que los distinguen de los demás tipos de componentes y que el modelo XF prescribe de forma explícita.

Los componentes de utilidad **no son instanciables**. No tienen constructor público y no pueden ser objeto de instanciación por ningún componente del artefacto. Sus operaciones son estáticas — accesibles directamente mediante el nombre del componente de utilidad sin necesidad de instanciarlo: `TemperatureUtils.toCelsius(temperature)`, `JsonUtils.serialize(object)`, `DateUtils.format(date, pattern)`.

Los componentes de utilidad **son extensibles**. Una utilidad puede ser base de otra utilidad que añade operaciones estáticas a su catálogo. La extensibilidad es ortogonal a la no instanciabilidad: la no instanciabilidad de una utilidad deriva del carácter estático de sus operaciones, no de un modificador `abstract`; ni la utilidad base ni la derivada necesitan declararse abstractas para que la herencia sea legítima. Lo único que la extensión exige es que la declaración de la utilidad base la permita (no final, no sellada o equivalente según el lenguaje) y que la utilidad derivada respete el mismo contrato del tipo — operaciones estáticas, sin estado mutable, no instanciable. Esta extensibilidad es lo que permite que el ecosistema de librerías XF publique utilidades base (§10.3) que los artefactos consumidores extienden con operaciones propias sin duplicar el catálogo importado.

Los componentes de utilidad **no mantienen estado**. No tienen atributos de instancia ni de clase que sean mutados durante la ejecución del artefacto. Todos sus atributos, si los tienen, son constantes — valores inmutables que parametrizan el comportamiento de sus operaciones sin constituir estado en sentido formal.

Estas tres propiedades conjuntamente sitúan a los componentes de utilidad en un paradigma de programación imperativo o funcional, a diferencia del paradigma orientado a objetos que caracteriza a los demás tipos de componentes del modelo XF. Esta distinción no es casual — refleja la naturaleza de la lógica que encapsulan: operaciones sin estado, sin efectos laterales y sin dependencias entre ellas.

Alcance local y la excepción de los tipos primitivos El alcance de un componente de utilidad está restringido a la capa en la que está definido. Un componente lógico de la Capa de Negocio puede usar `BusinessUtils` o cualquier componente de utilidad de la Capa de Negocio, pero no debe usar directamente un componente de utilidad de la Capa de Interacción — esa dependencia violaría el principio de aislamiento entre capas establecido en §6.2.2.

Sin embargo, el modelo XF reconoce una excepción explícita y bien delimitada a esta restricción de alcance local: los componentes de utilidad de la Capa de Acceso que operan sobre **tipos primitivos**.

Los **tipos primitivos** se definen en §3.1.15 en el sentido de ISO/IEC/IEEE 24765:2017 [20]. Las utilidades que operan sobre tipos primitivos — `StringUtils`, `DateUtils`, `NumberUtils`, `ArrayUtils` — no dependen de ningún concepto del dominio del artefacto, no asumen ninguna semántica de negocio y son aplicables en cualquier capa sin que su uso implique conocimiento de la lógica de acceso que caracteriza a la Capa de Acceso.

Por estos motivos, el modelo XF prescribe que los componentes de utilidad sobre tipos primitivos se definan en la Capa de Acceso y pueden ser referenciados por componentes de cualquier capa sin que ello constituya una violación del principio de aislamiento. Esta excepción está justificada porque los tipos primitivos son, en el sentido del modelo OSI, anteriores a la estratificación XF — no pertenecen a ninguna capa en particular sino que son la materia prima sobre la que todas las capas operan.

```
1 // Uso válido desde cualquier capa: una utilidad sobre tipo primitivo
2 // definida en la Capa de Acceso puede referenciarse desde Negocio
3 class TemperatureBusiness {
4
5     void update(Temperature temperature) {
6         // StringUtils está definido en la Capa de Acceso.
7         String formatted = StringUtils.format(temperature.value, 2);
8         ...
9     }
10
11 }
```

Código 20. Alcance local y la excepción de los tipos primitivos

Los componentes de utilidad que no operan sobre tipos primitivos — `TemperatureUtils`, `QueryUtils`, `ColorUtils`, `ValidationUtils` — están sujetos a la restricción de alcance local y solo pueden ser referenciados por componentes de su misma capa.

Propiedades normativas La primera propiedad es la **ausencia de modelado del dominio**: un componente de utilidad puede operar sobre conceptos del dominio del artefacto pero no debe implementar las reglas que los gobiernan — transiciones de estado del dominio, transformaciones semánticas, decisiones basadas en umbrales del negocio, lógica de modelado conceptual —. Esa responsabilidad pertenece exclusivamente a los componentes lógicos.

La segunda propiedad es la **pureza funcional**: las operaciones de un componente de utilidad son funciones puras — no tienen efectos laterales, no modifican el estado de ningún componente, no invocan inyecciones, no realizan E/S directa, no mutan

sus argumentos por referencia y producen siempre el mismo resultado ante la misma entrada.

La tercera propiedad es la **localidad de capa**: el alcance de un componente de utilidad está restringido a la capa en la que está definido, con la excepción de los componentes de utilidad sobre tipos primitivos de la Capa de Acceso, que pueden ser referenciados desde cualquier capa.

Restricciones Un componente de utilidad **no debe** ser instanciable. Su implementación debe impedir explícitamente la instanciación mediante los mecanismos que provea el lenguaje de programación — constructor inaccesible que lanza al invocarse, clase estática, objeto singleton, módulo funcional u equivalente. Esta prescripción controla exclusivamente la no instanciabilidad — propiedad derivada del carácter estático de las operaciones del tipo — y no su abstractness ni su heredabilidad. La extensión por herencia es admisible cuando la declaración de la utilidad base lo permite, sin que esto exija declarar la base como abstracta; la utilidad derivada queda sujeta al mismo contrato del tipo.

Un componente de utilidad **no debe** declarar miembros de instancia — atributos o métodos no estáticos —. La utilidad es puramente estática: opera en paradigma imperativo o funcional, no orientado a objetos. Sus únicos miembros admisibles son operaciones estáticas y constantes estáticas inmutables.

Un componente de utilidad **no debe** mantener estado mutable. Todos sus atributos, si los tiene, deben ser constantes inmutables.

Un componente de utilidad **no debe** implementar lógica que excede su rol auxiliar: ni reglas del modelado del dominio del artefacto, ni invocaciones a componentes lógicos a través de los componentes de inyección, ni operaciones que produzcan efectos laterales observables — E/S directa contra el sistema o la red, mutación de argumentos por referencia, dependencias de estado externo mutable —. Si una operación produce cualquiera de esos efectos o implementa cualquiera de esas reglas, pertenece a un componente lógico.

Un componente de utilidad **no debe** ser referenciado por componentes de capas de menor nivel de abstracción que la capa en la que está definido, salvo en el caso de los componentes de utilidad sobre tipos primitivos de la Capa de Acceso.

Un componente de utilidad **no debe** heredar de ningún componente distinto de otro componente de utilidad de su misma capa. La utilidad es un agrupador de operaciones estáticas puras y no participa en jerarquías de herencia heterogéneas; las cadenas de herencia entre utilidades de la misma capa (*p. ej.* extender el catálogo de operaciones de una utilidad base) son admisibles, las cadenas que cruzan tipos o capas son violación del modelo.

Un componente de utilidad **debe** llevar el sufijo `Utils` en su nombre, de forma que su naturaleza sea identificable de forma inmediata por cualquier desarrollador que consulte la estructura del artefacto.

Nomenclatura canónica Todos los componentes de utilidad llevan el sufijo `Utils` — por ejemplo, `TemperatureUtils`, `QueryUtils`, `JsonUtils`, `ColorUtils`,

ValidationUtils, StringUtils, DateUtils, NumberUtils.

Organización en la estructura de carpetas Los componentes de utilidad se agrupan bajo la subcarpeta `/utils` dentro de la carpeta de su capa correspondiente:

```

1 /src
2 --- /repository/utils (Componentes de utilidad de Acceso)
3     --- StringUtils (Utilidad sobre tipo primitivo)
4     --- DateUtils (Utilidad sobre tipo primitivo)
5     --- NumberUtils (Utilidad sobre tipo primitivo)
6     --- QueryUtils (Utilidad de acceso)
7 --- /business/utils (Componentes de utilidad de Negocio)
8     --- TemperatureUtils (Utilidad de negocio)
9 --- /api/utils (Componentes de utilidad de Interaccion)
10    --- ColorUtils (Utilidad de interaccion)

```

Código 21. Organización en carpetas: componente de utilidad

7.3.5 Componentes de transferencia

Definición Un componente de transferencia es una estructura de datos que modela la forma de la información en un punto concreto del flujo de procesamiento del artefacto, y que es utilizada por los componentes lógicos para intercambiar información entre sí, tanto dentro de una misma capa como entre capas. A diferencia de los componentes lógicos — que *mantienen* un estado que evoluciona en correspondencia con el proceso de negocio que modelan —, un componente de transferencia *es* la propia estructura de datos y no modela ningún proceso de negocio. Puede declarar operaciones, siempre que sean **autocontenidas**: operan exclusivamente sobre los datos de la propia estructura — consultarlos, transformarlos, derivarlos o combinarlos — y no acceden a ningún otro componente del artefacto. Su función es dar a esos datos una forma verificable, nombrada y reutilizable, junto con las operaciones que la estructura ofrece sobre sí misma.

Pragmáticamente, un componente de transferencia es la respuesta a la pregunta: *¿qué forma tienen los datos en este punto del flujo de procesamiento?* Cuando un componente de acceso recupera datos de temperatura de una API externa, el resultado de esa recuperación es un dato con una forma concreta — un valor numérico, una unidad de medida y una marca temporal. Esa forma se modela como un componente de transferencia **Temperature** que puede ser utilizado por cualquier componente del artefacto que necesite trabajar con datos de temperatura, independientemente de la capa en la que esté clasificado.

Distinción respecto a los demás tipos: ausencia de modelado de negocio

La distinción entre un componente de transferencia y un componente lógico no es la presencia de operaciones, sino su **naturaleza**: las operaciones de un componente de transferencia son autocontenidas — transforman, consultan o derivan sus propios datos — y no modelan un proceso de negocio del dominio ni orquestan a otros componentes; esa lógica efectiva del dominio es competencia exclusiva del componente lógico. Sus atributos describen el concepto que modelan con la precisión necesaria para que cualquier componente que los consuma pueda interpretarlos sin información adicional.

Históricamente, la industria ha utilizado múltiples términos para referirse a estructuras de datos según su contexto de uso — DTO, Entity, Model, POJO, Record, Schema, VO, Struct — cada uno con connotaciones de uso distintas que han generado la fragmentación semántica descrita en §5.4. El modelo XF unifica todos estos conceptos bajo un único tipo: el componente de transferencia. Esta unificación no implica que todas las estructuras de datos sean equivalentes en su semántica — implica que todas ellas comparten la misma naturaleza estructural en el modelo, y que las diferencias de semántica son gestionadas por los componentes lógicos que las procesan, no por la clasificación del componente de transferencia en sí.

Competencia en la taxonomía y proyección de estructuras de marco Esta caracterización fija con precisión la competencia del componente de transferencia dentro de la taxonomía. Frente al **componente lógico**, la transferencia no modela un proceso de negocio ni orquesta otros componentes; frente al **componente de utilidad** — que agrupa operaciones puras y estáticas, sin instancia ni datos propios —, la transferencia es una estructura *instanciable* cuyas operaciones actúan sobre los datos que ella misma porta. En consecuencia, las estructuras de datos “ricas” — las que combinan datos y operaciones autocontenidas sobre esos datos — son componentes de transferencia, no componentes lógicos.

Esta competencia cierra la brecha semántica entre el modelo y el marco de desarrollo: las estructuras que los marcos y las bibliotecas estándar ya proveen con esta naturaleza — colecciones, estructuras de fecha y hora, futuros y promesas, controles de interfaz, temporizadores, envoltorios de resultado — **proyectan** a componentes de transferencia del modelo XF, pues portan operaciones sin modelar ningún proceso de negocio del artefacto. El implementador las clasifica como transferencias en la capa correspondiente al concepto que modelan, sin necesidad de reescribirlas: el modelo reconoce su naturaleza estructural y las integra en la matriz $L \times T$ como cualquier otra transferencia.

El siguiente fragmento ilustra un componente de transferencia con operaciones autocontenidas:

```

1 // /src/repository/transfers/Temperature
2 // Transferencia con datos y operaciones autocontenidas sobre esos datos.
3 class Temperature {
4     double celsius;           // dato propio
5     DateTime measuredAt;     // dato propio
6
7     // operaciones autocontenidas: consultan o derivan los propios datos
8     double toFahrenheit() { return this.celsius * 9 / 5 + 32; }
9     double toKelvin()       { return this.celsius + 273.15; }
10    int compareTo(Temperature other) { return Double.compare(this.celsius,
11        other.celsius); }
12    boolean isAbove(Temperature other) { return this.celsius > other.celsius; }
13
14    // NO accede a R, B, A ni a otros componentes; NO modela proceso de
15    // negocio.
16    // Decidir si encender la calefacción según la temperatura SÍ es proceso
17    // de negocio:
18    // vive en un componente lógico (Business), no en la transferencia
19    // Temperature.
20 }

```

Código 22. Componente de transferencia con operaciones autocontenidas

Unificación de estructuras como capacidad del modelo El modelo XF **habilita** la unificación de estructuras como capacidad emergente: un mismo componente de transferencia puede ser compartido por todas las capas que trabajen con el mismo concepto del dominio del artefacto sin necesidad de duplicarlo por capa. El dominio del artefacto incluye conceptos de las tres capas — no solo del negocio — y la capacidad aplica a todos por igual: a un concepto de negocio como `User` o `Order`, a un concepto de acceso como `Query`, `HttpRequest` o `Cursor`, y a un concepto de interacción como `Button`, `MouseEvent` o `Viewport`. Esta capacidad contrasta con el enfoque tradicional de definir estructuras distintas por capa para un mismo concepto — una `UserEntity` en la Capa de Acceso, un `UserModel` en la Capa de Negocio y un `UserDTO` en la Capa de Interacción — que produce proliferación de tipos redundantes, duplicación de atributos y operaciones de mapeo entre estructuras que no aportan valor al artefacto.

La unificación es *capacidad*, no prescripción: el modelo no obliga al implementador a unificar transferencias del mismo concepto en distintas capas, y dos transferencias estructuralmente equivalentes en capas distintas constituyen una redundancia, no una violación. El modelo la recomienda como buena práctica de diseño porque reduce la complejidad del artefacto, elimina la necesidad de operaciones de mapeo entre capas y hace que la evolución del modelo de datos sea un cambio localizado en un único punto. La unificación es *posible* porque las referencias a componentes de transferencia, aun cruzando fronteras de capa, lo hacen siempre en la dirección descendente prescrita por el principio de aislamiento — una capa superior consume estructuras definidas en capas inferiores, nunca al revés — sujeto a las condiciones de direccionalidad y de no modificación que se desarrollan a continuación.

Direccionalidad y no modificación de las transferencias entre capas El componente de transferencia presenta una **referenciabilidad directa entre capas** de la que carecen los demás tipos de componente. Para el resto de tipos, el acceso a otra capa se canaliza a través del componente de inyección de la capa inmediatamente inferior (§7.3.3); un componente de transferencia, en cambio, puede ser referenciado de forma directa desde otra capa, salvando incluso más de una frontera. Esta referenciabilidad no constituye una excepción al principio de aislamiento: conserva su dirección descendente — un componente de transferencia **puede** ser referenciado por componentes de su propia capa o de capas de mayor nivel de abstracción, y **no debe** ser referenciado por componentes de capas de menor nivel de abstracción. La matriz completa de referencias permitidas entre capas se desarrolla en §9.4.

La **no modificación** es una **directriz de diseño**, no una prescripción: la reutilización directa de un componente de transferencia entre capas es apropiada cuando la estructura se consume sin modificar su definición; cuando una capa necesita atributos añadidos o eliminados respecto a la estructura recibida de una capa inferior, la opción canónica es definir un nuevo componente de transferencia en su propia capa. La modificación de los valores de los atributos de una estructura existente no requiere definir una nueva. El modelo no opera mecánicamente esta directriz: distinguir una transformación estructural de una mera reutilización no es decidible por análisis estático, sino que requiere juicio del dominio (§9.4). La directriz puede enunciarse del siguiente modo:

Si una capa **consume** una estructura de una capa inferior sin modificar su definición,

puede referenciarla directamente. Si una capa **transforma** la definición de una estructura de una capa inferior —añadiendo o eliminando atributos—, la opción canónica es **definir un nuevo componente de transferencia** en su propia capa que represente el resultado de esa transformación.

Esta directriz favorece que cada capa sea responsable de la definición de las estructuras que produce, preservando la trazabilidad de las transformaciones de datos a lo largo del flujo de procesamiento del artefacto.

El fragmento de código siguiente ilustra la aplicación de ambas restricciones:

```

1  // Componente de transferencia definido en la Capa de Acceso
2
3  // /src/repository/transfers/User
4  class User {
5      long id;
6      String name;
7      String email;
8      String passwordHash; // atributo sensible - no debe exponerse en la Capa
                           // de Interacción
9      DateTime createdAt; // atributo de auditoría - no relevante en la Capa de
                           // Interacción
10     boolean isActive;
11 }
12
13 // La Capa de Negocio consume User sin modificarlo - puede reutilizarlo
   // directamente
14 class UserBusiness {
15
16     User getUser(String token) {
17         Session session = B.session.decode(token);
18         return R.identity.fetchUser(session.userId); // retorna User de la
               // Capa de Acceso
19     }
20 }

```

Código 23. Transferencia definida en la Capa de Acceso

```

1  // Componente de transferencia de la Capa de Interacción - transformación
   // estructural
2  // La Capa de Interacción transforma User - debe definir su propio componente
   // de transferencia
3
4  // /src/api/transfers/UserResponse
5  class UserResponse {
6      long id;
7      String name;
8      String email;
9      // passwordHash eliminado - dato sensible
10     // createdAt eliminado - no relevante para el consumidor
11     // isActive eliminado - no relevante para el consumidor
12 }
13
14 // El componente lógico de interacción aplica la transformación
15 class UserService {
16
17     UserResponse getUser(String token) {
18         User user = B.user.getUser(token);
19         return new UserResponse(user.id, user.name, user.email); // transforma
               // User en UserResponse
20     }
21 }

```

Código 24. Transformación estructural de la transferencia en la Capa de Interacción

Propiedades normativas La primera propiedad es la **autocontención**: un componente de transferencia puede declarar operaciones, pero estas operan exclusi-

vamente sobre los datos de la propia estructura y no modelan procesos de negocio ni acceden a otros componentes del artefacto. El modelado de la lógica efectiva del dominio es competencia exclusiva de los componentes lógicos.

La segunda propiedad es la **independencia semántica del marco**: los atributos de un componente de transferencia describen el concepto del dominio del artefacto, no las convenciones del marco de desarrollo o del ecosistema tecnológico de implementación. Las anotaciones, decoradores, atributos de tipo u otros constructos sintácticos del lenguaje que el marco requiera por motivos de serialización, persistencia o despliegue son admisibles cuando sean estrictamente necesarios para esos fines y no alteren la semántica de la estructura modelada. El modelo no prescribe la verificación mecánica de esta propiedad: la pertenencia de un atributo al dominio del artefacto es decisión del implementador, y las desviaciones que comprometan la portabilidad — tipos de atributo ligados a una tecnología, nombres de campo alterados por anotaciones de serialización — son detectables a partir del nombre del propio componente o quedan como cuestión de revisión arquitectónica.

La tercera propiedad es la **direccionalidad de referencia**: un componente de transferencia solo puede ser referenciado por componentes de capas de mayor o igual nivel de abstracción que la capa en la que está definido. La regla aplica uniformemente a transferencias del propio artefacto y de artefactos externos integrados, sin distinción.

Restricciones Las operaciones de un componente de transferencia **deben** ser autocontenidas: operan exclusivamente sobre los datos de la propia estructura. Un componente de transferencia **no debe** acceder a ningún otro componente del artefacto desde sus operaciones — ni a los componentes de inyección **R**, **B** y **A**, ni a componentes lógicos, de generalización o de utilidad de ninguna capa.

Un componente de transferencia **no debe** modelar un proceso de negocio del dominio. Toda operación que implemente la lógica efectiva del dominio, coordine varios componentes o produzca efectos fuera de la propia estructura corresponde a un componente lógico, no a una transferencia: es esa frontera — y no la mera presencia de operaciones — la que separa la transferencia del componente lógico.

El constructor de un componente de transferencia, como el resto de sus operaciones, **no debe** acceder a otros componentes del artefacto — en particular, no debe acceder a componentes lógicos a través de los componentes de inyección — ni depender de recursos del entorno: inicializa los datos de la propia estructura a partir de sus parámetros, de valores constantes o de operaciones autocontenidas sobre esos datos.

Un componente de transferencia, si hereda, **debe** heredar exclusivamente de otro componente de transferencia. La heredabilidad entre transferencias está permitida — es el mecanismo natural cuando una transferencia especializada amplía con atributos propios una estructura base publicada por una librería XF (§10.3) sin perder la capacidad de ser tratada como instancia del tipo base — siempre que la subclase preserve el modelado estructural sin función y respete las restricciones de contenido prescritas para el tipo. La direccionalidad de la herencia entre capas se gobierna adicionalmente por el principio de aislamiento (§6.2.2).

Un componente de transferencia **no debe** ser referenciado por componentes de capas de menor nivel de abstracción que la capa en la que está definido.

Nomenclatura canónica Los componentes de transferencia **no deben** llevar sufijo añadido al concepto del dominio que modelan: su nombre describe directamente ese concepto, sin etiqueta adicional de tipo de estructura de datos. Ejemplos canónicos: `User`, `Temperature`, `Schedule`, `Session`, `Product`. La verificación es semántica y depende del juicio sobre qué constituye el “concepto del dominio” en el artefacto bajo análisis.

Los componentes de transferencia que típicamente se transmiten como excepciones del lenguaje — `TemperatureOutOfRangeException`, `NetworkUnavailableException`, `AuthenticationFailedException`, `DatabaseConnectionException` — pueden llevar el sufijo `Exception` cuando el lenguaje o la convención del ecosistema lo favorezcan, pero el modelo no lo prescribe: la elección del constructo sintáctico de transmisión es competencia del implementador (§9.5) y no genera asimetría estructural en el dominio formal $\mathbb{C}_T$.

Organización en la estructura de carpetas Los componentes de transferencia se agrupan bajo la subcarpeta `/transfers` dentro de la carpeta de la capa en la que están definidos:

```

1 /src
2 |--- /repository/transfers (Componentes de transferencia de Acceso)
3 |   |--- User (Estructura de datos de usuario -- origen en Capa de Acceso)
4 |   |--- Temperature (Estructura de datos de temperatura)
5 |   |--- Wrapper (Estructura de envoltura de respuesta de API)
6 |
7 |--- /business/transfers (Componentes de transferencia de Negocio)
8 |   |--- Session (Estructura de sesion -- definida y transformada en Capa de
9 |       Negocio)
10 |
11 |--- /api/transfers (Componentes de transferencia de Interaccion)
12 |   |--- UserResponse (Transformacion de User para exposicion en Capa de
13 |       Interaccion)
14 |   |--- TemperatureOutOfRangeException (Excepcion de interaccion)

```

Código 25. Organización en carpetas: componente de transferencia

La [tabla siguiente](#) contrasta el modelado tradicional con la unificación de estructuras del modelo XF.

Table 4. Contraste entre el modelado tradicional con múltiples estructuras por capa y el modelado del modelo XF con una única estructura compartida que circula entre capas. Cada fila representa la misma información conceptual (**User**) tal como se modela en cada paradigma. El modelo XF habilita la unificación de estructuras como capacidad de diseño; la definición de una nueva transferencia ante una transformación estructural es directriz, no prescripción.

Capa donde se usa	Modelado tradicional	Modelado XF
Acceso	UserEntity (con @Entity , anotaciones JPA, mapping a tabla)	User (única estructura)
Negocio	UserModel (objeto enriquecido con métodos de negocio, validaciones)	User (la misma; sin redefinición)
Interacción	UserDTO / UserViewModel / UserResponse (campos serializables, sin lógica)	User (la misma; salvo si se transforma estructuralmente, en cuyo caso es UserResponse en Interacción)
Operaciones de mapeo entre capas	UserMapper , UserAssembler , conversores manuales o automáticos	Ninguna; las capas comparten directamente la estructura
Coste de evolución (añadir un atributo)	Modificar cada estructura por capa y todos los mappers afectados	Modificar la estructura única en su capa de origen

7.4 Estructura de carpetas canónica

Definición La estructura de carpetas canónica es la materialización física de la matriz de clasificación $L \times T$ en el sistema de ficheros. Cada elemento — carpeta de capa, subcarpeta de tipo y componente de inyección — corresponde biunívocamente a un elemento del modelo: las carpetas de primer nivel representan las capas de abstracción, las subcarpetas de segundo nivel los tipos funcionales dentro de cada capa, y los componentes de inyección residen en la raíz de la carpeta de su capa como punto de acceso centralizado a sus componentes lógicos.

Toda estructura de carpetas cuyas subcarpetas organizan así la matriz $L \times T$ constituye una **raíz de artefacto**: la materialización estructural de un **artefacto** en el sentido de §3.1.1 — pieza de código autocontenida que formaliza un proceso, se estructura en las tres capas de abstracción y se ejecuta en un único espacio de ejecución. La correspondencia define estructuralmente al artefacto y es biunívoca: **un artefacto se materializa en exactamente una raíz de artefacto, y toda raíz de artefacto materializa exactamente un artefacto**. Una raíz de artefacto es, en estos términos, cualquier punto del árbol de ficheros desde el cual cuelga una matriz $L \times T$ completa.

Lo normativo es esa organización matricial y los nombres canónicos de sus carpetas; la *ubicación física* de la raíz no lo es. La ubicación recomendada es la carpeta **/src** en la raíz del proyecto, pero normativamente una raíz de artefacto es **cualquier** estructura de carpetas que organice la matriz $L \times T$, sea cual sea la ruta que la contenga. Esta abstracción es la que permite que lenguajes y ecosistemas con convenciones de ubicación distintas se proyecten sobre el modelo sin excepción.

Esa organización matricial tiene rango normativo — no de mera recomendación — por la misma razón que motiva la tipificación cerrada y exhaustiva (§6.2.4):

una estructura de carpetas que refleja fielmente la matriz $L \times T$ del artefacto es documentación estructural embebida en el propio proyecto, inspeccionable sin necesidad de herramientas adicionales. Un desarrollador que accede por primera vez a un artefacto XF bien formado se orienta con la misma rapidez con independencia del dominio, el lenguaje o el marco — la organización es siempre la misma.

Estructura prescrita En su ubicación recomendada bajo `/src`, la estructura de carpetas de un artefacto XF es la siguiente:

```

1 /projectName
2 --- /src
3     --- /api (Capa de Interaccion)
4         --- /general (Componentes de generalizacion)
5         --- /logic (Componentes logicos)
6         --- /transfers (Componentes de transferencia)
7         --- /utils (Componentes de utilidad)
8         --- A (Componente de inyeccion)
9
10    --- /business (Capa de Negocio)
11        --- /general (Componentes de generalizacion)
12        --- /logic (Componentes logicos)
13        --- /transfers (Componentes de transferencia)
14        --- /utils (Componentes de utilidad)
15        --- B (Componente de inyeccion)
16
17    --- /repository (Capa de Acceso)
18        --- /general (Componentes de generalizacion)
19        --- /logic (Componentes logicos)
20        --- /transfers (Componentes de transferencia)
21        --- /utils (Componentes de utilidad)
22        --- R (Componente de inyeccion)
23 --- [...] (Resto de ficheros del proyecto)

```

Código 26. Estructura prescrita

Convención de nombres de carpeta y su justificación normativa Los nombres de las tres carpetas de capa — `/repository`, `/business`, `/api` — y de las cuatro subcarpetas de tipo — `/general`, `/logic`, `/transfers`, `/utils` — son **términos industriales consolidados**, adoptados por alineación con el tipo de componente que alojan: `/repository` (del patrón *Repository* [12], el término más reconocible para el rol de la Capa de Acceso), `/business` (término universal para la lógica de dominio; alineado con el sufijo **Business* y la inyección B), `/api` (interfaz pública del artefacto: cubre los puntos de entrada sistémicos **Service* y las vistas gráficas **View*; alineado con la inyección A), `/general` (del componente de generalización; se prefiere a *base* o *abstract* por evitar colisiones semánticas), `/logic` (componentes lógicos, donde reside la lógica efectiva de la capa), `/transfers` (componentes de transferencia; se prefiere a *dto*, *model*, *entity* o *vo* por no presuponer sus connotaciones) y `/utils` (componentes de utilidad; coincide con el sufijo **Utils*).

Una vez adoptados, estos nombres son **contrato normativo verificable e inmutable**: la conformidad opera sobre los nombres exactos — un artefacto con `/access` en lugar de `/repository` no se reconoce como conforme — y se conservan en cualquier traducción, adaptación o extensión futura del modelo. Su invariabilidad es la expresión física del principio de agnosticismo tecnológico (§6.2.1).

Convención de nombres de fichero Todo componente XF **debe** definirse en un fichero cuyo nombre, sin extensión, coincida con el nombre canónico de la clase del

componente declarada en ese fichero. Esta correspondencia biunívoca entre fichero y componente es la base sobre la que opera la verificación estructural del modelo: la verificación identifica los componentes por el nombre del fichero — sin necesidad de inspeccionar su contenido — y resuelven la pertenencia tipológica y de capa a partir de la ubicación del fichero en la estructura canónica. La prescripción es trivial en lenguajes que la imponen por especificación sobre clases públicas y queda exigida por el modelo en cualquier lenguaje, con independencia del mecanismo que el lenguaje provea para hacerla efectiva.

Elementos del árbol canónico Cuando una capa contiene componentes, éstos **deben** residir bajo la carpeta canónica de primer nivel correspondiente a su capa — `/api`, `/business` o `/repository`; las carpetas de capa sin componentes no se exigen. Los tres componentes de inyección — A, B y R — **deben** residir en la raíz de la carpeta de su capa cuando la capa contiene al menos un componente lógico.

Las cuatro subcarpetas de segundo nivel — `/general`, `/logic`, `/transfers` y `/utils` — son los **espacios canónicos** de los cuatro tipos de componentes con carpeta propia. No se exige materializarlas vacías: una capa que no contiene componentes de un tipo no necesita crear su subcarpeta. Cuando sí los contiene, esos componentes **deben** residir en la subcarpeta canónica de su tipo. La conformidad se evalúa por la ubicación de los componentes existentes, no por la presencia de las carpetas; la pertenencia tipológica de cada componente queda determinada por su ubicación bajo la subcarpeta canónica correspondiente.

Las subdivisiones internas de la carpeta `/logic` dentro de cada capa, cuya organización específica responde al criterio propio de cada capa y se define en §7.2 junto con la descripción detallada de cada capa, **se recomiendan** pero no se exigen; un artefacto que no las implemente sigue siendo conforme al modelo XF.

Localización del punto de inicio de ejecución El **punto de inicio de ejecución** del artefacto — fichero `main.ts`, `App.java`, `main.go` o equivalente según el lenguaje o marco, cuando éste no lo gestiona implícitamente — **debe residir fuera de la raíz del artefacto**, típicamente en la raíz del proyecto. No es un componente clasificable en la matriz $L \times T$ — ni lógico, ni de generalización, ni de utilidad, ni de transferencia, ni de inyección —, y la raíz del artefacto contiene exclusivamente componentes XF clasificados; por tanto queda fuera de la estructura canónica. Su definición y su papel en la inicialización del artefacto se desarrollan en §8.1.

Correspondencia con la matriz $L \times T$ La [tabla siguiente](#) establece la correspondencia entre cada elemento de la estructura de carpetas y su categoría en la matriz de clasificación $L \times T$:

Table 5. Correspondencia entre la estructura de carpetas canónica y la matriz de clasificación $L \times T$.

Carpeta	Capa (L)	Tipo (T)
/repository/logic	Acceso	Lógico
/repository/general	Acceso	Generalización
/repository/transfers	Acceso	Transferencia
/repository/utils	Acceso	Utilidad
/repository/R	Acceso	Inyección
/business/logic	Negocio	Lógico
/business/general	Negocio	Generalización
/business/transfers	Negocio	Transferencia
/business/utils	Negocio	Utilidad
/business/B	Negocio	Inyección
/api/logic	Interacción	Lógico
/api/general	Interacción	Generalización
/api/transfers	Interacción	Transferencia
/api/utils	Interacción	Utilidad
/api/A	Interacción	Inyección

Componentes fuera de la estructura canónica La presencia de componentes fuera de la estructura canónica de su raíz de artefacto — en carpetas no reconocidas por el modelo, directamente bajo la raíz de artefacto o en ubicaciones arbitrarias del proyecto — constituye una violación del modelo.

En artefactos en proceso de migración hacia el modelo XF, los componentes aún no migrados coexisten con componentes XF correctamente clasificados. El modelo XF no prescribe una forma específica de organizar los componentes no migrados, pero recomienda que su ubicación sea claramente distinguible de la estructura XF para facilitar el seguimiento del proceso de migración.

Proyección de la raíz de artefacto en distintos ecosistemas Como la raíz de artefacto se define por la organización de la matriz $L \times T$ y no por su ruta física, se proyecta sin fricción sobre las convenciones de ubicación de cualquier ecosistema o lenguaje. El modelo no se acomoda a estos entornos: los reconoce como materializaciones distintas de una misma abstracción. La raíz de artefacto efectiva es, en cada caso, el punto del árbol de ficheros desde el cual cuelga la matriz $L \times T$ completa; a partir de ese punto, la sub-estructura — carpetas de capa, subcarpetas de tipo `/general`, `/logic`, `/transfers`, `/utils` y componentes de inyección `R`, `B`, `A` — y los nombres canónicos aplican sin variación.

- **Raíz de fuentes propia del ecosistema.** Maven o Gradle en la JVM imponen una raíz de fuentes (`src/main/java/`, `src/main/kotlin/`); la matriz $L \times T$ cuelga de ella — `src/main/java/repository/`, `.../business/`, `.../api/`.
- **Prefijo de paquete corporativo.** Lenguajes que exigen un paquete corporativo como prefijo del *namespace* (`com.<empresa>.<artefacto>` en Java) anidan la matriz bajo ese subárbol — `src/main/java/com/empresa/artefacto/repository/`, etcétera.

- **Proyecto multi-artefacto.** Un mismo árbol de ficheros puede contener varias raíces de artefacto independientes — mono-repos, proyectos multi-módulo, agrupaciones de microservicios —, cada una con su propia matriz $L \times T$, como se desarrolla a continuación.

La proyección no relaja ninguna prescripción: el modelo identifica la raíz de artefacto por la presencia de la matriz $L \times T$, no por una ruta literal, y sobre cada raíz los nombres canónicos, la clasificación $L \times T$ total y exhaustiva y las reglas de conformidad aplican íntegramente.

Raíces de artefacto múltiples Un mismo árbol de fuentes puede contener **varias raíces de artefacto** independientes — esto es, varios artefactos coexistiendo en un mismo árbol. Esta disposición es habitual en proyectos contemporáneos: mono-repos (Nx, Turborepo, pnpm workspaces), proyectos multi-módulo (Maven/Gradle, .NET solutions, Go workspaces), arquitecturas con separación librería / aplicación, sistemas organizados por contextos delimitados de DDD, o agrupaciones de microservicios en un mismo repositorio.

Cada raíz de artefacto se detecta por la presencia de su matriz $L \times T$ — al menos una carpeta canónica de capa (**repository**, **business** o **api**) como hijo directo — y es **maximal por inclusión**: ninguna raíz es descendiente de otra raíz detectada en el mismo árbol.

El conjunto de raíces detectadas constituye el conjunto de artefactos XF a verificar. **La verificación se ejecuta independientemente sobre cada raíz**: la verificación de conformidad se aplica a cada raíz por separado, con su propia matriz $L \times T$, su propio triple $R / B / A$ y su propia detección de violaciones. La cardinalidad “una inyección por capa” (§7.3.3) aplica por raíz, no sobre el árbol: dos raíces pueden coexistir con sus propios triples sin violación.

Código fuera de toda raíz detectada queda fuera del alcance del modelo.

El árbol de fuentes puede contener directorios no-XF — ficheros de configuración, scripts de construcción, código auxiliar, librerías externas vendorizadas, código en migración, tests fuera de la jerarquía canónica — sin que el modelo intervenga. El modelo XF gobierna únicamente lo que estructuralmente se declara XF (es decir, lo que vive dentro de alguna raíz detectada).

La frontera entre raíces es estructural (no física): dos raíces en el mismo árbol son tan independientes como dos artefactos en repositorios distintos.

El siguiente ejemplo ilustra una disposición conforme con dos raíces de artefacto y una carpeta no-XF coexistiendo bajo una raíz de fuentes Maven / Gradle:

```

1  src/main/java
2  |-- com/empresa/app          <-- raíz de artefacto "app"
3      |-- repository/
4      |-- business/
5      |-- api/
6  |-- com/empresa/core        <-- raíz de artefacto "core"
7      |-- repository/
8      |-- business/
9      |-- api/
10 |-- com/empresa/shared      <-- no es raíz: sin carpetas de capa
11    |-- helpers.java         <-- fuera del modelo, no verificado

```

Código 27. Raíces de artefacto múltiples

Las raíces `app` y `core` se verifican de forma independiente. La carpeta `com/empresa/shared` no es raíz (no contiene `repository`, `business` ni `api` como hijo directo) y su contenido está fuera del alcance del modelo, independientemente de si los nombres de los ficheros que contiene coinciden o no con sufijos canónicos.

8 Instanciación de la arquitectura

La instanciación de la arquitectura es el proceso por el cual un artefacto XF pasa de ser una estructura estática de componentes clasificados a ser un sistema en ejecución capaz de procesar interacciones. Este proceso no es exclusivo del modelo XF — todo artefacto software requiere un mecanismo de arranque que inicialice sus componentes en el orden correcto antes de que el sistema pueda operar. Lo que el modelo XF aporta es una forma de entender ese mecanismo en términos del modelado formal de procesos, dotándolo de sentido conceptual independientemente del marco de desarrollo o entorno de ejecución empleado.

La filosofía del modelo XF en relación con la instanciación se fundamenta en un principio que es necesario enunciar con precisión antes de desarrollar las prescripciones concretas: el modelo prescribe la estructura conceptual del proceso de instanciación — qué debe ocurrir, en qué orden y por qué — pero no la forma física en que ese proceso se implementa en cada marco de desarrollo o entorno de ejecución.

Todo marco de desarrollo implementa mecanismos automáticos de gestión del ciclo de vida de sus componentes — contenedores de inyección de dependencias, gestores de ciclo de vida de vistas, schedulers de tareas periódicas, consumers de mensajes. Estos mecanismos automáticos no violan el modelo XF — lo implementan, con su propia nomenclatura y sin que el desarrollador los haya codificado explícitamente. El desarrollador que conoce el modelo XF entiende por qué el marco de desarrollo gestiona esos ciclos de vida de la forma en que lo hace: una `Activity` de Android es una vista cuyo ciclo de vida gestiona el sistema operativo; un `@Scheduled` de Spring es un `BusinessTask` cuya ejecución periódica gestiona el contenedor; un `Consumer` de Kafka es un servicio cuyo ciclo de invocación gestiona el entorno de microservicios. En todos estos casos, el marco de desarrollo está implementando el espíritu del modelo — el desarrollador solo necesita reconocer esa correspondencia y asegurarse de que la lógica que implementa dentro de esos mecanismos respeta las prescripciones de la capa y el tipo de componente correspondientes.

El código que el desarrollador escribe explícitamente sí está sujeto a las prescripciones normativas del modelo en su totalidad. Lo que el marco de desarrollo gestiona antes, durante o después de ese punto de entrada ya satisface el espíritu del modelo desde la perspectiva del entorno de ejecución.

La cláusula se organiza del siguiente modo. La cláusula §8.1 describe el punto de inicio de ejecución y su relación con la inicialización de los componentes de inyección. La cláusula §8.2 describe el ciclo de vida completo del artefacto en cuatro fases — instanciación, inicialización, ejecución y terminación —, sus prescripciones normativas y la representación formal como autómata. La cláusula §8.3 describe el elemento del modelo XF que actúa de punto de control centralizado de la instanciación, el elemento punto de inicio XF.

8.1 Punto de inicio de ejecución

Definición El **punto de inicio de ejecución** de un artefacto es el primer fragmento del código del desarrollador que el entorno de ejecución invoca al cargar el artefacto. Es el origen causal de toda la actividad observable del artefacto en memoria y el lugar en que el control cruza del entorno de ejecución al artefacto: en ese instante el artefacto existe como estructura estática de componentes clasificados — construidos en memoria con su estado inicial σ_0 , pero todavía no operacionales —. Hacer del artefacto un sistema activo, capaz de procesar las interacciones para las que fue diseñado, es función de la operación de inicialización que el punto de inicio invoca, no del punto de inicio en sí.

El punto de inicio es un concepto del modelo, no una entidad fija del lenguaje: se manifiesta físicamente de formas distintas según el entorno de ejecución — función global de arranque, módulo de bootstrap, fichero designado por la configuración del entorno, función handler invocada por la infraestructura, miembro estático de una clase, u otro mecanismo del entorno subyacente. El modelo XF formaliza la función arquitectónica del punto de inicio, pero su ámbito normativo no lo alcanza: las prescripciones del modelo comienzan en los componentes y, en su caso, en el elemento de punto de inicio XF (§8.3). El punto de inicio pertenece al entorno de ejecución; el modelo describe su papel sin imponer sobre su código prescripciones de conformidad.

Función arquitectónica El punto de inicio cumple dos funciones arquitectónicas inseparables:

1. **Frontera con el entorno de ejecución.** El punto de inicio es el único lugar en que el control cruza del entorno de ejecución al artefacto. En ese instante el artefacto está cargado en memoria pero no operacional: sus componentes existen como objetos cuyo estado inicial σ_0 aún no ha sido validado por la lógica de arranque. El paso a operacional no lo produce el punto de inicio, sino la operación de inicialización que se invoca desde él.
2. **Origen de la orquestación del ciclo de vida.** El punto de inicio es el único lugar desde el cual la **secuencia canónica de inicialización** se invoca legítimamente. Esta secuencia hace operacional el artefacto y **debe** completarse antes de cualquier otra actividad de la lógica del artefacto.

Prescripción normativa El modelo XF prescribe que la **inicialización del artefacto** se materializa como la secuencia canónica de invocaciones `R.init()`; `B.init()`; `A.init()`; en orden ascendente de abstracción, y su **terminación** como la secuencia `A.terminate()`; `B.terminate()`; `R.terminate()`; en orden inverso. La inicialización es la primera operación que se ejecuta sobre el artefacto y la terminación la última: ninguna operación de la lógica del artefacto puede preceder a la inicialización ni suceder a la terminación.

Estas secuencias se invocan desde el punto de inicio de ejecución, que queda fuera del ámbito normativo del modelo: la prescripción recae sobre los componentes de inyección R, B y A — que **deben** exponer las operaciones `init()` y `terminate()` — y sobre el orden de la secuencia, no sobre el código del punto de inicio. El registro de la terminación con el mecanismo de cierre del entorno, cuando éste lo provea, ocurre

en el punto de inicio; el modelo no prescribe la disponibilidad de tales mecanismos en todos los entornos.

La prescripción opera sobre la secuencia canónica como totalidad ordenada; el modelo no prescribe el sitio físico donde esta se invoca, ni si las tres invocaciones se realizan directamente desde el punto de inicio o quedan encapsuladas en una única operación que las orqueste.

Frontera externa del artefacto Como consecuencia de que la inicialización **debe** completar antes de cualquier otra actividad de la lógica del artefacto, ningún consumidor externo puede invocar las operaciones de la Capa de Interacción del artefacto antes de que la inicialización haya completado. Hasta ese momento, los puntos de entrada del artefacto no son operacionales: la Capa de Interacción no ha sido inicializada y el artefacto no está preparado para procesar interacciones.

La frontera no aplica a las invocaciones internas entre componentes lógicos durante la propia inicialización: en `init()` los componentes lógicos ya están instanciados — su instanciación se completa atómicamente como efecto de la carga de las clases de los componentes de inyección `R`, `B` y `A` (§8.2) — y pueden invocarse entre sí a través de los componentes de inyección siempre que respeten la jerarquía descendente entre capas establecida en §6.2.2. La única invocación prohibida en todos los casos es la que se produce desde el constructor de un componente: durante la instanciación, las referencias estáticas pueden no estar todavía resueltas y su acceso produce comportamiento indefinido.

Mecanismos automáticos del entorno de ejecución Los entornos de ejecución modernos implementan mecanismos automáticos de gestión del ciclo de vida que pueden preceder, suceder o reemplazar parcialmente al punto de inicio del desarrollador: contenedores de inyección de dependencias, gestores de ciclo de vida de vistas, schedulers de tareas periódicas, consumidores de mensajes. El modelo XF prescribe **qué tareas deben realizarse** — la existencia de las operaciones canónicas `init()` y `terminate()`, su invocación en el orden prescrito, la disciplina arquitectónica de cada componente — y **no prescribe a quién corresponde realizarlas**. La atribución concreta de cada tarea entre el desarrollador y el entorno de ejecución es responsabilidad compartida cuyos límites determinan el lenguaje, el marco de desarrollo y la decisión del integrador: cuando el entorno asume la instanciación, la configuración o la activación de un componente, esa porción del ciclo queda satisfecha por construcción; cuando el entorno no la asume, **debe** ser codificada por el desarrollador conforme a la secuencia canónica de §8.2. Las prescripciones del modelo aplican íntegramente al código que el desarrollador escribe *dentro* de cada componente — su clasificación $L \times T$, su responsabilidad funcional, sus reglas de capa y tipo —, con independencia de qué actor active el componente desde fuera.

8.2 Ciclo de vida del artefacto

El ciclo de vida de un artefacto XF describe la secuencia de estados por los que transita el artefacto desde el momento en que el entorno de ejecución carga sus clases hasta el momento en que el proceso del artefacto finaliza. El modelo XF formaliza este ciclo de vida en cuatro fases bien diferenciadas — **instanciación**, **inicialización**,

ejecución y terminación — cada una con responsabilidades precisas y con una transición bien definida entre ellas. La primera fase es *no invocable* — es efecto de la carga de las clases del singleton —; las tres restantes se gobiernan mediante operaciones explícitas del componente de inyección o del elemento de arquitectura XF.

Fase 0 — Instanciación La fase de instanciación comprende la construcción en memoria de los tres componentes de inyección R, B y A y, como efecto de la inicialización de sus campos estáticos, la instanciación de todos los componentes lógicos del artefacto. Su objetivo es dejar el artefacto en el estado **defined**: cada componente lógico existe como objeto en memoria, accesible a través de su identificador canónico en el componente de inyección correspondiente, y se encuentra en su estado inicial σ_0 — el estado posterior a la ejecución del constructor y anterior a la invocación de `init()`. El componente de inyección no posee estado mutable propio: σ_0 es propiedad del lógico instanciado, no del componente de inyección que lo instancia.

La fase 0 es *no invocable*: ocurre como efecto de la carga de las clases del singleton por parte del entorno de ejecución, antes de que el punto de inicio del artefacto invoque ninguna operación. El desarrollador no la gobierna mediante una llamada; el modelo prescribe únicamente las propiedades estructurales del código que la fase materializa y las restricciones sobre el contenido de los constructores que la fase ejecuta. Al término de la fase 0, ningún recurso externo ha sido todavía adquirido por el artefacto.

```

1 // La fase 0 es código declarativo: la inicialización de los
2 // campos estáticos al cargar las clases R, B y A construye
3 // todos los lógicos y los deja en su estado inicial sigma_0.
4 // No requiere invocación alguna.
5
6 final abstract class R {
7     public static final DatabaseRepository database = new DatabaseRepository();
8     public static final IdentityRepository identity = new IdentityRepository();
9     // ...
10 }
11
12 final abstract class B {
13     public static final SessionBusiness session = new SessionBusiness();
14     public static final UserBusiness user = new UserBusiness();
15     public static final TemperatureBusiness temperature = new
16         TemperatureBusiness();
17     // ...
18 }
19
20 final abstract class A {
21     public static final TemperatureService temperatureService = new
22         TemperatureService();
23     public static final TemperatureView temperatureView = new
24         TemperatureView();
25     // ...
26 }

```

Código 28. Fase 0 — Instanciación

Todo componente lógico del artefacto que requiera ser accesible desde otros componentes **debe** estar declarado en el componente de inyección de su capa. Un componente lógico que no está declarado en su componente de inyección no es accesible desde ningún otro componente del artefacto. La presencia de lógicos no expuestos no es violación normativa: pueden existir componentes lógicos internos de un paquete o librería que deliberadamente no se expongan a consumidores externos.

Prescripciones normativas de la fase 0:

Las referencias estáticas de los componentes de inyección **deben** ser inmutables — declaradas como `readonly`, `final` o el equivalente en el lenguaje de programación empleado. Su valor se asigna en la declaración mediante la instanciación del componente lógico y no puede ser modificado durante la ejecución del artefacto.

El componente de inyección **no debe** declarar referencias a componentes lógicos de otras capas. **B** declara únicamente componentes lógicos de la Capa de Negocio. **R** declara únicamente componentes lógicos de la Capa de Acceso. **A** declara únicamente componentes lógicos de la Capa de Interacción. La declaración cruzada constituye violación del principio de aislamiento (§6.2.2).

El constructor de un componente lógico puede inicializar sus atributos internos pero **no debe** establecer comunicación con sistemas externos, invocar operaciones de otros componentes ni ejecutar lógica que dependa de recursos del entorno de ejecución. Toda lógica de arranque que dependa de recursos externos pertenece a la operación `init()` del lógico, no a su constructor.

Fase 1 — Inicialización La fase de inicialización comprende todas las operaciones que el artefacto debe completar antes de que pueda procesar su primera interacción externa. Su objetivo es llevar el artefacto desde el estado `DEFINED` establecido en la fase 0 hasta el estado **initialized** — todos los componentes están inicializados, sus dependencias están resueltas y el artefacto está preparado para procesar interacciones.

El modelo XF prescribe que la fase de inicialización se realiza mediante la invocación secuencial de la operación `init()` de los tres componentes de inyección en orden ascendente de abstracción:

```

1 R.init() // Inicialización de la Capa de Acceso
2
3 B.init() // Inicialización de la Capa de Negocio
4
5 A.init() // Inicialización de la Capa de Interacción

```

Código 29. Fase 1 — Inicialización: orden ascendente entre capas

Cada `init()` de un componente de inyección **delega** en la operación `init()` de los componentes lógicos de su capa, invocados en un orden compatible con sus dependencias internas; la elección del orden concreto es decisión del implementador:

```

1 // Cuerpo típico de B.init(): delega en cada business.init()
2 // en un orden compatible con las dependencias internas (la
3 // elección del orden concreto es decisión del implementador)
4
5 public static void init() {
6     B.session.init();
7     B.user.init();
8     B.temperature.init();
9     // ...
10 }

```

Código 30. Fase 1 — Inicialización: cuerpo típico de la operación `init()`

Este orden no es arbitrario — es consecuencia directa del principio de dependencia entre capas establecido en §6.2.2. La Capa de Negocio depende de la Capa de Acceso — sus componentes lógicos pueden invocar operaciones de repositorios durante su

inicialización, y esos repositorios deben estar inicializados previamente. Del mismo modo, la Capa de Interacción depende de la Capa de Negocio — sus componentes lógicos pueden invocar operaciones de negocio durante su inicialización, y esos negocios deben estar inicializados previamente. Invertir este orden produciría accesos a componentes no inicializados cuyo comportamiento es indefinido.

La fase de inicialización es atómica desde la perspectiva del consumidor del artefacto: el artefacto no debe procesar ninguna interacción externa hasta que `A.init()` haya completado satisfactoriamente. Si cualquier operación de inicialización falla — una conexión a base de datos que no puede establecerse, un fichero de configuración que no puede leerse, un recurso del sistema que no está disponible — el artefacto no puede alcanzar el estado `INITIALIZED` en un estado consistente y debe terminar de forma controlada, propagando el error al entorno de ejecución. El modelo XF no prescribe cómo debe gestionarse ese error — prescribe que no debe silenciarse ni ignorarse.

Prescripciones normativas de la fase 1:

Todo componente lógico que requiera lógica de arranque **debe** implementar una operación `init()` separada de su constructor. La prohibición de alojar esa lógica en el constructor corresponde a la fase 0.

El cuerpo de la operación `init()` del componente de inyección **debe** estar limitado a invocaciones a la operación `init()` de los componentes lógicos declarados como slots de la propia inyección. La existencia de un orden topológico que respete las dependencias internas entre los lógicos es propiedad derivada del modelo — no prescripción independiente —: la aciclicidad del grafo de dependencias se sigue del principio de aislamiento entre capas (§6.2.2), y la existencia de una ordenación topológica respetuosa con las capas se sigue de esa aciclicidad; el orden concreto en que la inyección invoca a sus lógicos queda a decisión del implementador, siempre que sea compatible con dichas dependencias.

Fase 2 — Ejecución La fase de ejecución comprende el periodo durante el cual el artefacto procesa activamente la lógica para la que fue diseñado. Esta fase comienza en el momento en que `A.init()` completa satisfactoriamente y termina en el momento en que el artefacto recibe una señal de cierre o produce una excepción no controlada.

Durante la fase de ejecución, el modelo XF no prescribe ningún comportamiento adicional más allá de las prescripciones de las capas y los tipos de componentes definidas en las cláusulas 6 y 7. Los componentes lógicos procesan las operaciones que les son invocadas, mantienen su estado, se comunican entre capas a través de los componentes de inyección y propagan excepciones según las reglas establecidas. El ciclo de vida de los componentes individuales durante la ejecución es responsabilidad de cada componente lógico y de su componente de inyección — no del ciclo de vida del artefacto.

Una propiedad importante de la fase de ejecución es la **inmutabilidad estructural del artefacto**: durante la ejecución, el conjunto de componentes lógicos activos — definidos en los componentes de inyección — no varía. No se crean nuevas instancias de componentes lógicos, no se destruyen las existentes y no cambian las referencias estáticas de los componentes de inyección. El artefacto opera sobre el conjunto

de componentes que fueron definidos e inicializados durante el arranque; cualquier modificación estructural requiere una terminación controlada y un nuevo arranque.

Prescripciones normativas de la fase 2:

La fase 2 no introduce prescripciones propias. La inmutabilidad estructural descrita no es una prescripción independiente: se sigue de la inmutabilidad de las referencias estáticas de los componentes de inyección y de la prescripción de declarar en ellos todo componente lógico accesible (fase 0). El resto de propiedades aplicables durante la ejecución — aislamiento entre capas, direccionalidad de las referencias, disciplina de cada tipo de componente — son las prescripciones de las cláusulas §6, §7.2 y §7.3, que aplican íntegramente sin reformulación específica de fase.

Fase 3 — Terminación La fase de terminación comprende todas las operaciones que el artefacto debe completar antes de que su proceso finalice, con el objetivo de liberar los recursos adquiridos durante la inicialización y dejar el sistema en un estado de reposo definido para la siguiente ejecución. Su objetivo es la inversa simétrica de la fase de inicialización: llevar el artefacto desde el estado INITIALIZED hasta el estado TERMINATED de forma ordenada y controlada.

El modelo XF prescribe que todo artefacto XF **debe** implementar el mecanismo de terminación controlada mediante la operación `terminate()` de cada componente de inyección, invocadas en orden inverso al de inicialización:

```
1 A.terminate() // Terminación de la Capa de Interacción
2
3 B.terminate() // Terminación de la Capa de Negocio
4
5 R.terminate() // Terminación de la Capa de Acceso
```

Código 31. Fase 3 — Terminación: orden descendente entre capas

Simétricamente, cada `terminate()` del componente de inyección **delega** en la operación `terminate()` de los componentes lógicos de su capa, invocados en orden *inverso* al de su inicialización:

```
1 // Cuerpo típico de B.terminate(): delega en cada business.terminate()
2 // en orden inverso al de B.init()
3
4 public static void terminate() {
5     B.temperature.terminate();
6     B.user.terminate();
7     B.session.terminate();
8     // ...
9 }
```

Código 32. Fase 3 — Terminación: cuerpo típico de la operación `terminate()`

Este orden inverso es igualmente consecuencia del principio de dependencia entre capas: la Capa de Interacción debe terminar antes que la Capa de Negocio para garantizar que no llegan nuevas interacciones mientras los negocios están siendo terminados. Del mismo modo, la Capa de Negocio debe terminar antes que la Capa de Acceso para garantizar que ninguna operación de negocio intenta acceder a un repositorio que ya ha cerrado sus conexiones.

La prescripción de implementar el mecanismo de terminación es normativa y no admite excepción: la ausencia de `terminate()` en los componentes de inyección de

un artefacto es una violación del modelo, independientemente de si el entorno de ejecución garantiza su invocación. Un `terminate()` que implementa la liberación de recursos pero que el entorno no invoca en determinados escenarios es un problema del entorno — no del artefacto. Un `terminate()` que no existe sí es un problema del artefacto.

El registro efectivo del mecanismo de terminación controlada que el entorno de ejecución provea — handlers de señal `SIGTERM` en sistemas Unix, `onDestroy` en Android, `onApplicationStop` en Spring, hooks de cierre en Node.js — depende del entorno y del modelo de inicialización del artefacto; el modelo XF no lo prescribe ni lo recomienda, por considerarlo fuera del alcance estructural del modelo arquitectónico. El modelo XF reconoce explícitamente que en determinados entornos de ejecución — funciones serverless, contenedores destruidos sin previo aviso, procesos interrumpidos por el sistema operativo — la invocación de la terminación controlada no puede ser garantizada y depende del entorno de infraestructura. El artefacto **debe** implementar el mecanismo; el entorno determina si puede invocarlo.

Prescripciones normativas de la fase 3:

Todo componente de inyección **debe** implementar la operación `terminate()`. La ausencia de `terminate()` en un componente de inyección constituye una violación del modelo independientemente de si el entorno de ejecución garantiza su invocación.

El orden de invocación entre las inyecciones A, B, R es el inverso del de inicialización.

Adicionalmente, todo componente lógico que haya adquirido recursos del sistema durante su inicialización **debe** implementar una operación `terminate()` que los libere de forma controlada.

Simetría de recursos. La relación entre las operaciones `init()` y `terminate()` se expresa formalmente como una simetría de recursos: todo recurso adquirido durante `init()` debe ser liberado durante `terminate()`, y ningún recurso debe ser liberado en `terminate()` que no haya sido adquirido en `init()`. Esta simetría es la propiedad formal que garantiza que el artefacto no produce fugas de recursos — memory leaks, conexiones no cerradas, ficheros bloqueados — entre ejecuciones sucesivas. Expresado en términos del autómata: la transición `DEFINED` $\rightarrow$ `INITIALIZED` adquiere recursos; la transición `INITIALIZED` $\rightarrow$ `TERMINATED` los libera. El estado `TERMINATED` es funcionalmente equivalente al estado `DEFINED` en cuanto a los recursos del sistema que el artefacto ocupa.

Representación formal del ciclo de vida El ciclo de vida del artefacto puede representarse formalmente como una máquina de estados con tres estados y cinco transiciones, según recoge la [figura siguiente](#):

- **defined:** estado inicial del artefacto. Los componentes existen como estructuras de código pero no están inicializados. Ningún componente lógico puede ser accedido.
- **initialized:** estado activo del artefacto. Todos los componentes están inicializados y el artefacto procesa interacciones. La estructura de componentes es inmutable.

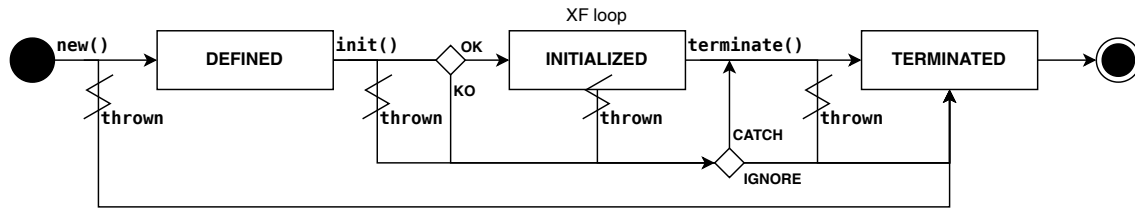


Figure 10. Autómata del ciclo de vida del artefacto: tres estados — DEFINED, INITIALIZED y TERMINATED — y sus transiciones, disparadas por los eventos new, init y terminate o por la ocurrencia de una excepción. Una excepción capturada (CATCH) lleva a TERMINATED de forma controlada — ejecutando terminate —; una no capturada (IGNORE), o cualquier excepción surgida en la instanciación o en la propia terminación, de forma abrupta. Las transiciones no representadas no están definidas y constituyen condiciones de error.

- **terminated:** estado final del artefacto; el proceso puede finalizar. Se alcanza en uno de dos modos — **controlado**, ejecutando terminate y liberando de forma ordenada los recursos adquiridos, o **abrupto**, sin ejecutar terminate. Estado absorbente: una vez alcanzado, el artefacto no puede volver al estado INITIALIZED sin un nuevo arranque.

Las transiciones del autómata se disparan por cuatro tipos de evento — new, init, terminate y la ocurrencia de una excepción — y producen cinco transiciones formales (el evento de excepción dispara una transición a TERMINATED desde DEFINED y otra desde INITIALIZED):

- **new (instanciación):** (sin estado previo) → DEFINED. Transición de entrada al autómata: la carga en memoria de los componentes lógicos por el entorno de ejecución (*Fase 0*) lleva al artefacto al estado inicial DEFINED.
- **XF.init() exitoso:** DEFINED → INITIALIZED. Requiere que todas las operaciones init() de todos los componentes de inyección completen satisfactoriamente.
- **XF.init() fallido:** DEFINED → TERMINATED. Alguna operación init() de los componentes de inyección no se completa satisfactoriamente — por reportar fallo (KO) o por lanzar una excepción — antes de alcanzar INITIALIZED.
- **XF.terminate() controlado:** INITIALIZED → TERMINATED. Disparado por la señal de cierre del entorno o por una excepción capturada (CATCH); ejecuta la secuencia terminate() y libera los recursos de forma ordenada.
- **Excepción en ejecución:** INITIALIZED → TERMINATED. Excepción surgida mientras el artefacto procesa interacciones; su modo de terminación depende de si es capturada (véase *Modos de terminación*).

Modos de terminación. El artefacto alcanza TERMINATED en uno de dos modos, según la excepción sea capturada por el lazo de control del artefacto:

- **controlado (CATCH):** la excepción se captura y el artefacto ejecuta la secuencia terminate(), liberando de forma ordenada los recursos adquiridos. Es también el modo de la terminación normal disparada por la señal de cierre del entorno.
- **abrupto (IGNORE):** la excepción se propaga al entorno de ejecución y el proceso concluye sin ejecutar terminate().

Una excepción surgida durante la instanciación (fase 0) o durante la propia terminación (fase 3) conduce siempre a una terminación abrupta: en la fase 0 el lazo de control todavía no está activo para capturarla; en la fase 3 el artefacto ya está terminando y la terminación no admite recuperación.

Esta representación formal es la base sobre la que se construyen las reglas de conformidad del ciclo de vida. Comprender estos flujos ayuda a entender las precondiciones de estado en los que está permitido ejecutar las operaciones de inicialización y terminación de los componentes.

Table 6. Autómata del ciclo de vida del artefacto: transiciones entre los tres estados DEFINED, INITIALIZED y TERMINATED disparadas por los eventos **new**, **init** y **terminate** y por la ocurrencia de excepciones. El modo de terminación — controlado (ejecuta terminate) o abrupto — depende de si la excepción es capturada. Las transiciones no listadas no están definidas y constituyen condiciones de error.

Estado origen	Evento	Estado destino	Condición operacional y modo
(sin estado previo)	new	DEFINED	El entorno de ejecución carga las clases de los componentes de inyección e instancia los lógicos como campos estáticos (fase 0). Una excepción en esta fase termina el artefacto de forma <i>abrupta</i> .
DEFINED	init (éxito)	INITIALIZED	init _R , init _B e init _A completan satisfactoriamente en orden ascendente.
DEFINED	init (fallo)	TERMINATED	Alguna init _L no completa — fallo (KO) o excepción — antes de alcanzar INITIALIZED. <i>Controlado</i> si se captura y ejecuta terminate; <i>abrupto</i> si se propaga.
INITIALIZED	terminate	TERMINATED	Señal de cierre del entorno o excepción capturada (CATCH); terminate _A , terminate _B y terminate _R completan en orden inverso (<i>controlado</i>). Una excepción durante terminate concluye igualmente en TERMINATED, de forma <i>abrupta</i> .
INITIALIZED	excepción no capturada	TERMINATED	La excepción se propaga al entorno (IGNORE); el proceso concluye sin ejecutar terminate (<i>abrupto</i>).
DEFINED	terminate	(sin transición)	No definida: no se puede terminar un artefacto que no ha sido inicializado.
INITIALIZED	init	(sin transición)	No definida: no se puede re-inicializar un artefacto en operación.
TERMINATED	(cualquiera)	(sin transición)	Estado final absorbente: el ciclo de vida del artefacto ha concluido.

8.3 Elemento de punto de inicio XF

El elemento XF es el **elemento de punto de inicio de la arquitectura XF**: el constructo del artefacto que materializa, mediante dos operaciones estáticas, las transiciones de estado del ciclo de vida agregado del artefacto. Su presencia en un artefacto es la declaración explícita de que ese artefacto implementa el modelo XF.

El elemento XF **no es un componente**: no se clasifica en ninguna celda de la matriz

$L \times T$ del modelo y queda fuera del dominio de la función de clasificación ϕ . Es un elemento estructural del artefacto — análogo a las carpetas canónicas — que el modelo reconoce por su nombre canónico y por su ubicación bajo `/src`, no por una asignación tipológica. Su función estructural consiste en declarar dos operaciones — inicialización y terminación — que orquestan la secuencia canónica del ciclo de vida agregado, y, en su caso, alojar el punto de inicio de ejecución del artefacto cuando el lenguaje permita que éste sea miembro estático de una clase.

Opcionalidad La presencia del elemento XF en el artefacto es **opcional**. El modelo no obliga a declararlo: el punto de inicio de ejecución (§8.1) puede invocar directamente la secuencia canónica de inicialización (`R.init(); B.init(); A.init();`) y, simétricamente, la secuencia canónica de terminación (`A.terminate(); B.terminate(); R.terminate();`), sin que el elemento XF esté presente.

Cuando el implementador opta por declararlo, el elemento XF queda obligado por las prescripciones que las cláusulas siguientes establecen: su nombre canónico, su ubicación, la declaración obligatoria de `init()` y `terminate()` y la canonicidad del cuerpo de esas dos operaciones. La presencia del componente no aporta capacidad estructural adicional sobre la materialización sin él, pero centraliza la orquestación del ciclo de vida en un único punto de invocación, declara explícitamente la conformidad del artefacto con el modelo, y simplifica el contenido del punto de inicio.

Ubicación canónica El fichero del elemento XF reside en la raíz de `/src` al mismo nivel que las carpetas de las tres capas. Esta ubicación queda verificada por construcción: cualquier fichero llamado XF ubicado fuera de la raíz canónica constituye una violación de las prescripciones de ubicación según el lugar concreto donde se ubique, sin necesidad de una prescripción específica para el elemento XF.

```

1 /src
2 |--- /api
3 |--- /business
4 |--- /repository
5 |--- XF

```

Código 33. Ubicación canónica

Operaciones obligatorias El elemento XF **debe** declarar dos operaciones estáticas: `init()` y `terminate()`. La ausencia de cualquiera de ellas en un elemento XF presente en el artefacto constituye violación del modelo.

El cuerpo de `XF.init()` materializa la transición `DEFINED → INITIALIZED` del ciclo de vida del artefacto (§8.2, Fase 1) y **está cerrado** a la secuencia canónica de invocaciones `R.init(); B.init(); A.init();` en orden ascendente de abstracción.

El cuerpo de `XF.terminate()` materializa la transición `INITIALIZED → TERMINATED` del ciclo de vida del artefacto (§8.2, Fase 3) y **está cerrado** a la secuencia canónica de invocaciones `A.terminate(); B.terminate(); R.terminate();` en orden inverso.

```

1 // Contenido obligatorio del elemento XF
2 class XF {
3     static void init()      { R.init(); B.init(); A.init(); }
4     static void terminate() { A.terminate(); B.terminate(); R.terminate(); }
5 }

```

Código 34. Operaciones obligatorias

Apertura estructural de la clase El modelo **no cierra** el conjunto de miembros del elemento XF, ni restringe la herencia, ni restringe la instanciabilidad de la clase. La clase puede declarar miembros adicionales y participar en jerarquías de herencia o ser instanciable cuando el entorno de ejecución lo requiera. Esta apertura permite que XF acomode dos configuraciones de uso normativamente equivalentes:

1. **XF no contiene el punto de inicio de ejecución del artefacto.** El punto de inicio reside fuera de XF (§8.1), e invoca `XF.init()` como primera operación del código del desarrollador y, opcionalmente, registra `XF.terminate()` con el mecanismo de cierre del entorno. En esta configuración, XF contiene exclusivamente las dos operaciones obligatorias.
2. **XF contiene el punto de inicio de ejecución del artefacto.** Cuando el lenguaje permite que el punto de inicio sea miembro estático de una clase, el implementador puede declararlo dentro de XF — como miembro adicional, distinto de `init()` y `terminate()` — e invocar desde él `init()` y `terminate()` internamente. En esta configuración, XF puede además incluir las anotaciones, decoradores, herencia o configuración estática que el entorno de ejecución prescriba para reconocer a la clase como punto de inicio.

La elección entre ambas configuraciones la determina el lenguaje y el entorno de ejecución, y dentro de lo que el entorno permite, el implementador. La disciplina arquitectónica sobre los miembros adicionales que se declaren queda cubierta transversalmente por el resto del modelo: instanciar componentes lógicos fuera de las inyecciones, invocar operaciones de lógicos fuera del patrón canónico o referenciar componentes no clasificados constituyen violaciones del modelo.

Visibilidad de `init()` y `terminate()` El modelo no prescribe una visibilidad concreta para `init()` y `terminate()`. La visibilidad efectiva queda determinada por la accesibilidad necesaria para sus invocadores legítimos, y por tanto por el lenguaje subyacente: si el punto de inicio reside fuera de XF, ambas operaciones deben ser accesibles desde ese punto y su visibilidad será pública (o equivalente en el lenguaje); si el punto de inicio reside dentro de XF, las operaciones son invocadas internamente y su visibilidad puede ser privada. Este matiz no aparece como dos casos prescriptivos distintos: es la generalización única de que la visibilidad es la mínima necesaria para que la invocación legítima sea posible.

Exclusividad de invocación Las operaciones `XF.init()` y `XF.terminate()` **deben** ser invocadas **únicamente** desde el propio elemento XF — cuando éste contiene el punto de inicio de ejecución del artefacto — o desde el punto de inicio externo del artefacto cuando éste reside fuera de XF. Ningún componente clasificable del artefacto — lógico, de generalización, de inyección, de utilidad o de transferencia — puede invocar `XF.init()` ni `XF.terminate()`. La orquestación del ciclo de vida agregado del artefacto está reservada en exclusiva al punto de inicio de ejecución, sea cual sea su materialización. En el caso del punto de inicio externo, el invocador no es un componente clasificable y queda fuera del alcance del modelo por construcción.

El elemento XF como contrato de conformidad La presencia del elemento XF en un artefacto es la declaración explícita de que ese artefacto implementa el modelo XF. Un artefacto que contiene el elemento XF está declarando que su estructura está clasificada según la taxonomía del modelo, que sus dependencias respetan el principio de aislamiento entre capas y que su ciclo de vida agregado se gobierna desde un único punto de control. Esta propiedad semántica justifica que el elemento XF sea el primer elemento que las herramientas de análisis de conformidad inspeccionan al evaluar un artefacto: su presencia o ausencia es el indicador más inmediato de la implementación del modelo.

9 Flujos de datos

El flujo de datos describe cómo la información se transmite y transforma entre los componentes de un artefacto XF durante su ejecución. Este flujo no es una propiedad que el modelo XF inventa — es una propiedad intrínseca de todo proceso formal y, por tanto, de todo artefacto software. El modelo XF formaliza esa propiedad, establece las restricciones que garantizan la coherencia del flujo con el principio de aislamiento entre capas y provee las prescripciones que permiten verificar su corrección de forma estática.

La cláusula se organiza del siguiente modo. La cláusula §9.1 describe el modelo de transmisión de información como propiedad intrínseca del software. La cláusula §9.2 formaliza los tipos de estructuras de datos que el modelo reconoce. La cláusula §9.3 desarrolla el principio de unificación de estructuras de datos. La cláusula §9.4 prescribe las reglas de direccionalidad y de transformación estructural de los componentes de transferencia. La cláusula §9.5 formaliza la vía de comunicación de los componentes de transferencia como concepto único del modelo y prescribe la propiedad de homogeneidad estructural sobre $\mathbb{C}_T$.

9.1 Modelo de transmisión de información

La transmisión y transformación de datos entre componentes es una propiedad intrínseca de todo proceso formal automatizado mediante software. Cuando un componente ejecuta su lógica efectiva, puede requerir datos de entrada para operar — producidos por otro componente — y produce datos de salida como resultado — consumidos por otro componente. Este intercambio de información entre componentes es el mecanismo fundamental mediante el cual los componentes de un artefacto cooperan para cumplir el propósito del sistema.

Este mecanismo no está determinado por el lenguaje de programación ni por el marco de desarrollo — es una propiedad común a toda gestión de procesos de negocio, independiente de las herramientas de modelado. Un componente que necesita datos para operar los recibe como parámetros de entrada. Un componente que produce datos como resultado de su operación los devuelve como valor de retorno. Esta dualidad entrada-salida es universal y está presente en cualquier paradigma de programación — orientado a objetos, funcional, reactivo o imperativo.

En el contexto del modelo XF, la transmisión de información entre componentes se caracteriza por una única dimensión: la **dirección del flujo** — los datos pueden

fluir en sentido descendente, de capas superiores hacia capas inferiores, o en sentido ascendente, de capas inferiores hacia capas superiores. La vía de comunicación por la que se materializa cada transmisión concreta (§9.5) es un único concepto formal del modelo, con independencia del constructo sintáctico que el lenguaje proporcione para entregar el dato al invocador.

El modelo XF reconoce dos tipos de flujo según la dirección, ambos bien identificados en la práctica del software:

- **Flujo descendente:** una capa superior invoca una operación de una capa inferior y recibe el resultado por la vía de comunicación de la operación invocada. Es el flujo más frecuente — la Capa de Interacción invoca a la Capa de Negocio, que invoca a la Capa de Acceso. El dato entregado por la vía de comunicación puede ser un resultado previsto o una condición anómala; en ambos casos es un componente de transferencia (§9.5).
- **Flujo ascendente:** una capa inferior notifica un cambio de estado o una condición a una capa superior; esta notificación **nunca** emplea invocación directa ascendente y se realiza típicamente mediante el patrón de observación. El componente de la capa inferior notifica a sus observadores registrados; cada observador recibe la notificación a través de la vía de comunicación del método de callback que él mismo expuso; el dato transportado por el callback es asimismo un componente de transferencia (§9.5).

Nota sobre el aislamiento entre capas. La existencia del flujo ascendente no contradice el principio de aislamiento descendente prescrito en §6.2.2. La relación de dependencia entre capas se define en términos de **conocimiento estructural**: un componente *A* depende de un componente *B* si y solo si *A* conoce la definición de *B* — esto es, declara su identificador, importa su clase o referencia su tipo en su código fuente. El principio de aislamiento prohíbe únicamente este tipo de dependencia en sentido ascendente: ningún componente de una capa inferior puede conocer la definición de un componente de una capa superior. El flujo ascendente preserva esta prohibición porque se materializa típicamente mediante el **patrón de observación** y la **delegación de eventos**, descritos en §6.2.2: la capa superior conoce al componente observable de la capa inferior y se registra en él; la capa inferior emite notificaciones a través de un mecanismo genérico — una lista de observadores tipados, un canal de eventos — sin conocer la identidad, el tipo ni la existencia de sus consumidores. Por tanto, el flujo de información asciende mientras la dependencia estructural permanece descendente.

El vehículo de transmisión de información en todos estos flujos es siempre el mismo: el **componente de transferencia** — la estructura de datos que modela la forma de la información en un punto concreto del flujo de procesamiento. Las reglas que gobiernan el uso de los componentes de transferencia en cada uno de estos flujos son el objeto de las cláusulas siguientes.

9.2 Tipos de estructuras de datos

El modelo XF reconoce dos categorías de estructuras de datos que se gestionan a nivel de bits en la capa donde se trabajan — categorización consistente con los fundamentos de la teoría de tipos en lenguajes de programación [40] y que el modelo

adopta sin modificación.

La primera categoría son los **tipos primitivos**, definidos en §3.1.15 en el sentido de ISO/IEC/IEEE 24765:2017 [20]. Constituyen la materia prima sobre la que operan todos los componentes de transferencia.

La segunda categoría son las **estructuras** — agregaciones de tipos primitivos y de otras estructuras previamente definidas, que permiten modelar información de mayor complejidad. En el modelo XF, estas estructuras son, en sentido estricto, los componentes de transferencia: las clases de datos que modelan los conceptos del dominio del artefacto en sus distintos estados de procesamiento; pueden portar operaciones autocontenidas sobre sus propios datos (§7.3.5).

El modelo XF no introduce distinción adicional dentro de las estructuras más allá de la que ya establece su clasificación en la matriz $L \times T$ — la capa en la que están definidas. La proliferación de categorías semánticas que caracteriza a las arquitecturas tradicionales — DTO, Entity, Model, POJO, Record, VO, Schema — no tiene lugar en el modelo XF. Todas las estructuras son componentes de transferencia, con independencia de su contexto de uso, y su rol en el flujo de datos es gestionado por los componentes lógicos que las procesan, no por su clasificación.

9.3 Unificación de estructuras de datos

Definición y alcance del dominio del artefacto El modelo XF **habilita la unificación de estructuras de datos** como capacidad emergente del marco de transferencias: un mismo componente de transferencia puede ser compartido por todas las capas que trabajen con el mismo concepto del dominio del artefacto, sin necesidad de duplicarlo por capa. El *dominio del artefacto* comprende tanto los conceptos de negocio que el artefacto modela (User, Order, Temperature, Schedule) como los conceptos de acceso a sus sistemas externos (Query, HttpRequest, Cursor, Connection) y los conceptos de interacción con sus consumidores (MouseEvent, Viewport, HttpResponse). La unificación es posible en cualquiera de estos tres subdominios.

Contraste con el modelado tradicional Esta capacidad contrasta directamente con el enfoque predominante en las arquitecturas tradicionales, donde cada capa define sus propias estructuras de datos para representar el mismo concepto en función de su dirección de flujo y su nivel de abstracción. Con este enfoque, un único concepto de negocio puede requerir hasta seis estructuras distintas en un artefacto con tres capas y flujo bidireccional — una estructura de entrada y una de salida por cada capa — cada una con sus propios atributos, sus propias operaciones de transformación y su propio mantenimiento independiente.

En el modelado tradicional los desarrolladores mantienen estructuras distintas por capa, llegando en el extremo a esquemas como `UserRequestDTO`, `UserRequestModel`, `UserRequestEntity` para el flujo de entrada, y `UserResponseDTO`, `UserResponseModel`, `UserResponseEntity` para el flujo de salida — seis estructuras estructuralmente equivalentes para el mismo concepto `User`, con cinco operaciones de conversión entre cada par adyacente. En el modelado XF, un único componente de transferencia puede circular por cualquier capa en

sentido ascendente. Las operaciones de transformación — $f(C_T) \rightarrow C'_T$ — no producen un componente de un tipo distinto sino una modificación de los datos de un mismo componente, con los atributos ajustados según las necesidades del componente lógico que lo procesa.

Naturaleza de la unificación: capacidad, no prescripción La unificación es una *capacidad* que el modelo habilita, no una prescripción normativa que el modelo imponga. El modelo no obliga al implementador a unificar transferencias del mismo concepto en distintas capas: dos transferencias estructuralmente equivalentes en capas distintas constituyen una redundancia, no una violación. Pragmáticamente, una única estructura por concepto reduce la complejidad del artefacto, elimina las operaciones de mapeo entre estructuras equivalentes y hace que la evolución del modelo de datos sea un cambio localizado en un único punto; el modelo recomienda la unificación como buena práctica, pero la decisión queda en el dominio del diseño, no en el dominio normativo. La unificación es posible porque los componentes de transferencia están exentos de la restricción estricta de aislamiento entre capas: pueden ser referenciados desde capas de mayor abstracción sin violar el principio de aislamiento, sujetos a las restricciones de direccionalidad de las transferencias y a la regla de transformación estructural (§9.4).

9.4 Direccionalidad y transformación de los componentes de transferencia

Un componente de transferencia **puede ser referenciado** por componentes de su propia capa o de capas de mayor nivel de abstracción, y **no debe ser referenciado** por componentes de capas de menor nivel de abstracción; la referencia preserva así la dirección descendente común a toda dependencia del modelo XF. A diferencia del resto de tipos de componente — cuyo acceso desde otra capa se canaliza a través del componente de inyección de la capa inmediatamente inferior (§7.3.3) —, la referencia a un componente de transferencia es directa y puede cruzar más de una frontera de capa. La unificación de estructuras (§9.3) opera dentro de esta direccionalidad, no contra ella.

La [tabla siguiente](#) resume la direccionalidad permitida de las referencias a transferencias entre capas.

Table 7. Matriz de direccionalidad permitida de referencias a componentes de transferencia entre capas: la fila indica la capa que referencia; la columna, la capa donde se define la transferencia. “Permitido” indica referencia admitida; “Prohibido”, referencia no admitida.

Componente \ Transferencia definida en	Interacción	Negocio	Acceso
Componente de Interacción	Permitido	Permitido	Permitido
Componente de Negocio	Prohibido	Permitido	Permitido
Componente de Acceso	Prohibido	Prohibido	Permitido

Esta restricción preserva la secuenciación de las capas — la información fluye de capas de menor abstracción hacia capas de mayor abstracción, nunca en sentido

inverso. Una estructura definida en la Capa de Interacción no puede ser referenciada por componentes de la Capa de Negocio ni de la Capa de Acceso — su existencia es invisible para las capas inferiores. Una estructura definida en la Capa de Negocio puede ser referenciada por componentes de la Capa de Interacción pero no por componentes de la Capa de Acceso. Una estructura definida en la Capa de Acceso puede ser referenciada por componentes de cualquier capa.

La restricción de direccionalidad aplica indistintamente a toda transferencia, con independencia del constructo sintáctico con el que el lenguaje materialice su transmisión: las transferencias entregadas como excepciones del lenguaje son componentes de transferencia del mismo dominio $\mathbb{C}_T$ que las entregadas como retorno ordinario (§9.5). Una transferencia definida en la Capa de Acceso puede ser referenciada por la Capa de Negocio y por la Capa de Interacción, con independencia del constructo habitual de transmisión. Una transferencia definida en la Capa de Negocio puede ser referenciada por la Capa de Interacción pero no por la Capa de Acceso.

Esta propiedad tiene una implicación práctica directa: la Capa de Interacción es la única capa que puede conocer y gestionar el conjunto completo de transferencias de error del artefacto — tanto las definidas en la Capa de Acceso como las definidas en la Capa de Negocio y las propias. Esto es coherente con la responsabilidad de la Capa de Interacción de representar los resultados al consumidor del artefacto — incluyendo los resultados de error — de forma apropiada para el protocolo de interacción empleado.

Transformación estructural: directriz de diseño La direccionalidad determina *desde qué capas* puede referenciarse un componente de transferencia, pero no resuelve por sí sola en qué condiciones la capa receptora puede reusar la transferencia recibida o, alternativamente, definir una nueva. El modelo XF establece la siguiente directriz de diseño — no prescripción normativa — en términos del criterio que distingue ambas opciones: la **reutilización directa** es apropiada cuando la capa receptora consume el componente sin modificar su **definición estructural** (su conjunto de atributos); la **definición de una nueva transferencia** en la capa receptora es la opción canónica cuando esa capa necesita una estructura con atributos añadidos o eliminados respecto al componente recibido. Toda nueva estructura del artefacto queda clasificada como componente de transferencia (prescripción estructural sobre los componentes de transferencia y sobre la estructura de carpetas).

La modificación de los **valores** de los atributos — calcular un valor derivado, formatear una cadena, convertir una unidad de medida — no requiere definir un nuevo componente: la transferencia original puede reutilizarse con valores ajustados, siempre que su definición estructural no cambie. Solo cuando la **definición** de la estructura cambia — atributos añadidos o eliminados — la capa receptora necesita una nueva transferencia para modelar el resultado.

9.5 Homogeneidad de las transferencias y vía de comunicación

Los lenguajes de programación ofrecen distintos constructos sintácticos para que un componente transmita información al componente que lo invoca: el retorno

explícito mediante `return`, el lanzamiento de excepciones mediante `throw` o `raise`, el retorno multi-valor del tipo `(T, error)`, los tipos algebraicos como `Result<T, E>` o `Either`, los códigos de retorno y las variables globales como `errno`. Por debajo de esta variedad sintáctica, todos estos constructos materializan el mismo concepto subyacente: la **vía de comunicación** de la operación — el medio por el que un componente entrega información a su invocador durante la ejecución. Esta cláusula formaliza la vía de comunicación como concepto único del modelo y prescribe la propiedad de homogeneidad sobre la que reposa la coherencia de las prescripciones del modelo frente a la variedad sintáctica del lenguaje.

Materialización sintáctica en el lenguaje La vía de comunicación de una operación se materializa de forma distinta según el paradigma del lenguaje de implementación:

- **Lenguajes con manejo estructurado de excepciones** — Java, C#, C++, Python, JavaScript, Kotlin, Swift, Ruby — proveen dos primitivas sintácticamente diferenciadas: la primitiva de retorno (`return` y equivalentes) y la primitiva de lanzamiento (`throw`, `raise`) capturada con `try/catch` o equivalente.
- **Lenguajes con tipos algebraicos** — Rust, Haskell, Scala — materializan toda salida sobre la misma primitiva de retorno, distinguiendo el propósito del dato transmitido mediante el tipo del valor retornado: `Result<T, E>` con los constructores `Ok(T)` y `Err(E)`, o `Either E T` con `Right` y `Left`.
- **Lenguajes con retorno multi-valor** — Go — materializan la salida como un retorno tuplado: `(T, nil)` cuando la operación se completa según lo previsto y `(zero, error)` cuando se produce una condición anómala.
- **Lenguajes basados en convención** — C tradicional — materializan la salida mediante el valor de retorno y, cuando se produce una condición anómala, mediante variables globales (`errno`) o códigos de retorno reservados.

Esta variedad de constructos sintácticos no introduce variedad conceptual: en todos los casos, un componente entrega información a su invocador a través de su vía de comunicación, y el dato entregado — sea un resultado previsto o una condición anómala — es siempre un componente de transferencia.

De los constructos del lenguaje al componente de transferencia XF La equivalencia conceptual de los constructos sintácticos de transmisión es la base formal sobre la que el modelo XF prescribe la homogeneidad de los componentes de transferencia. Si los constructos del lenguaje son materializaciones sintácticas distintas del mismo concepto subyacente, los datos que circulan por ellos son entidades de la misma naturaleza estructural. La estructura ordinaria que una operación retorna y la excepción que una operación lanza no son entidades de tipo distinto en el modelo XF: ambas son componentes de transferencia — pertenecen al conjunto único $\mathbb{C}_T$, se clasifican en la misma matriz $L \times T$, y se rigen por las mismas reglas de direccionalidad y de transformación estructural (§9.4). La diferencia entre una y otra es exclusivamente el constructo sintáctico con el que se materializa la transmisión en una invocación concreta, no su categoría en el modelo. La formalización del modelo absorbe los distintos paradigmas sintácticos sin asimetría: la vía de comunicación

se modela como una única firma de salida de la operación, y el tipo transportado pertenece siempre al mismo $\mathbb{C}_T$.

Homogeneidad estructural de las transferencias El modelo XF establece como propiedad normativa que **toda transferencia que circula por la vía de comunicación de una operación pertenece al mismo dominio único $\mathbb{C}_T$** . No existen “transferencias de resultado normal” y “transferencias de error” como dos clases disjuntas: existe el conjunto único de componentes de transferencia $\mathbb{C}_T$ del artefacto, y cada transferencia es admisible por la vía de comunicación con independencia del propósito del dato que transporta y del constructo sintáctico que el lenguaje use para entregarla al invocador. La distinción entre el propósito de los datos transmitidos — resultado previsto o condición anómala — es exclusivamente *de implementación* — qué primitiva sintáctica del lenguaje materializa la transmisión — y **no altera la naturaleza estructural, la clasificación en $L \times T$, ni las reglas que gobiernan al componente de transferencia**.

Esta homogeneidad tiene cuatro implicaciones directas, todas ellas corolarios de reglas o directrices ya prescritas:

1. *Direccionalidad uniforme.* La direccionalidad descendente (§9.4) se aplica a toda transferencia que aparezca en la firma de salida de una operación, con independencia del constructo sintáctico que materialice la transmisión. Una transferencia de la Capa de Acceso puede entregarse a la Capa de Interacción mediante `return`, `throw`, `Result<T, E>` o cualquier otro constructo equivalente — la dirección permitida es la misma.
2. *Transformación estructural uniforme.* La directriz de §9.4 — definir nueva transferencia cuando una capa transforma la definición estructural recibida — opera con la misma fuerza sobre las transferencias materializadas como excepciones del lenguaje. Si una transferencia de error de Acceso se transforma al pasar por Negocio, el integrador define una nueva transferencia en Negocio.
3. *Unificación uniforme.* La capacidad de unificación habilitada en §9.3 permite modelar una transferencia que representa el mismo concepto del dominio con un único componente y entregarla mediante cualquier constructo sintáctico sin duplicación. Duplicar la misma transferencia conceptual — uno para el constructo de retorno y otro para el de excepción — es contrario a la directriz de unificación salvo que representen conceptos efectivamente distintos del dominio; la elección entre unificación y duplicación es decisión de diseño, no prescripción del modelo.
4. *Libertad sintáctica del constructo.* El modelo no atribuye constructos sintácticos a transferencias. Una transferencia con sufijo `Exception` puede retornarse mediante `return` (típicamente para reificar el error como valor procesable por el invocador), y una transferencia ordinaria puede lanzarse como excepción cuando el lenguaje y la semántica lo justifiquen. La elección del constructo para cada transferencia es competencia del diseñador del artefacto.

Convención de nomenclatura para transferencias de error El modelo XF no prescribe convenciones de nomenclatura adicionales sobre las transferencias que se transmiten habitualmente como condiciones anómalas. El sufijo `Exception` en su identificador canónico — útil para legibilidad y para herramientas de análisis —

es una convención que el implementador puede adoptar libremente; un artefacto que entregue errores como valores ordinarios mediante tipos suma `Result<T, E>`, o que utilice nombres ad-hoc para transferencias de error, es plenamente conforme al modelo. El propósito del dato transmitido no genera asimetría estructural en $\mathbb{C}_T$.

La [tabla siguiente](#) recoge las transferencias recurrentes que se transmiten como excepciones en artefactos XF.

Table 8. Transferencias recurrentes transmitidas habitualmente como excepciones del lenguaje, con la capa donde se originan y su contexto típico. Son componentes de transferencia como cualquier otro: el sufijo **Exception** es convención de nomenclatura, no requisito estructural. Tabla informativa y no exhaustiva.

Excepción canónica	Capa de origen	Contexto típico
<code>NetworkException</code>	Acceso	Fallo en la comunicación con un sistema remoto: timeout, conexión cerrada, host no resoluble.
<code>AuthenticationException</code>	Acceso	Credenciales inválidas o expiradas al autenticarse contra un sistema externo.
<code>AuthorizationException</code>	Acceso o Negocio	Operación no permitida para el sujeto autenticado.
<code>ResourceNotFoundException</code>	Acceso	Recurso solicitado al sistema remoto no existe (HTTP 404, fila ausente).
<code>ResourceConflictException</code>	Acceso	Conflicto de estado al modificar un recurso (HTTP 409, optimistic locking).
<code>ServerErrorException</code>	Acceso	Error del sistema remoto (HTTP 5xx, fallo de BD, etc.).
<code>ValidationException</code>	Negocio	Datos de entrada no satisfacen las reglas estructurales del dominio.
<code>BusinessException</code>	Negocio	Regla semántica del dominio violada (saldo insuficiente, estado inválido, etc.).
<code>ConflictException</code>	Negocio	Estado del dominio incompatible con la operación solicitada.
<code>InvalidArgumentException</code>	Interacción	Parámetro de entrada al artefacto malformado o ausente.
<code>UnsupportedOperationException</code>	Interacción	Operación expuesta no implementada por el componente.

Excepciones nativas del lenguaje Las excepciones nativas provistas por el lenguaje de implementación (**Error** en JavaScript/TypeScript, **Exception** y subclases en Java, Kotlin o C#, **BaseException** y subclases en Python, etc.) son, en la axiomatización XF, componentes de transferencia. El modelo no obliga a redefinirlas como transferencias propias del artefacto: cuando un componente lógico lanza una

excepción nativa, esa excepción es a todos los efectos una transferencia que viaja por la vía de comunicación — heredada del lenguaje, pero parte del mismo dominio formal. La compatibilidad con transferencias heredadas se desarrolla en la [cláusula §10](#).

Decisión del constructo de transmisión como responsabilidad del implementador El modelo XF no prescribe las condiciones bajo las cuales un componente debe transmitir información mediante un constructo sintáctico u otro. Esta es una decisión de implementación que depende de las reglas de negocio del dominio que el artefacto modela. Por ejemplo, si se introduce un valor inconsistente como entrada de una operación de negocio, el componente puede retornar una estructura ajustada mediante `return` — indicando el error como parte del resultado — o lanzar una excepción — interrumpiendo la ejecución normal. Ambas opciones son válidas en el modelo XF — la elección pertenece al implementador.

Lo que el modelo sí prescribe es que, cualquiera que sea el constructo elegido, el vehículo de transmisión es siempre un componente de transferencia — una estructura de datos bien definida, nombrada y clasificada en la capa correspondiente — nunca un error sin tipificar ni un mensaje de texto sin estructura formal.

Nota histórica La nomenclatura clásica de los modelos de propagación de errores en software distingue habitualmente entre “canal principal” (el medio por el que se entrega el resultado de una operación cuando se completa según lo previsto) y “canal de excepción” (el medio por el que se transmiten las condiciones anómalas). Esta dualidad refleja la separación sintáctica entre `return` y `throw` característica de los lenguajes con manejo estructurado de excepciones, y se ha trasladado a los modelos arquitectónicos como una propiedad del sistema. El modelo XF reconoce el uso extendido de esta nomenclatura, pero no la incorpora a su formalización: la vía de comunicación es un único concepto unificado, y la elección del constructo sintáctico para cada transferencia queda completamente en el dominio del implementador.

10 Compatibilidad e interoperabilidad

El modelo XF deriva de una propiedad estructural del software establecida en [§5.1](#): todo artefacto software, sea XF o no, modela un proceso formal. Esta caracterización implica que todo componente de cualquier artefacto desempeña una de las tres responsabilidades funcionales invariantes del proceso formal — interacción, procesamiento o acceso ([§5.2](#)). La clasificación $L \times T$ del modelo XF no es propietaria: es la formalización de esa propiedad universal del software. Por la propiedad de completitud ([§5.5](#)), todo componente de cualquier artefacto es clasificable en exactamente una celda de la matriz, con independencia de si el artefacto que lo aloja se construyó bajo el modelo XF o bajo una arquitectura tradicional.

De esta universalidad estructural se deriva la noción de compatibilidad del modelo XF: integrar un componente externo no consiste en convertirlo al modelo ni en relajar las prescripciones de la capa que lo recibe, sino en **reconocer la responsabilidad funcional** que el componente desempeña, **asignarle su celda** en la matriz $L \times T$ del artefacto integrador y **aplicar el mecanismo prescrito** por el modelo para esa

celda y ese tipo de componente. La cláusula se organiza en tres cláusulas internas. La §10.1 establece el principio de compatibilidad y formaliza los cuatro casos posibles según si cada artefacto implementa o no el modelo. La §10.2 cataloga los mecanismos prescritos por tipo de componente integrado. La §10.3 caracteriza el ecosistema de librerías XF como consecuencia natural del modelo.

10.1 Principio de compatibilidad

Universalidad de la clasificación $L \times T$ Todo proceso formal admite una descomposición canónica en tres responsabilidades invariantes (§5.2); las clasifique explícitamente o no, todo artefacto software desempeña esas tres responsabilidades. La matriz $L \times T$ del modelo XF es la formalización de esa descomposición: organiza las tres capas como expresión de las tres responsabilidades del proceso formal, y los cinco tipos funcionales como expresión de los roles estructurales con los que los componentes contribuyen a cada capa. Por la propiedad de completitud derivada en §5.5, toda lógica de programa pertenece a exactamente una de las tres capas: no existe componente de ningún artefacto que escape a la clasificación $L \times T$. La diferencia entre un artefacto XF y un artefacto tradicional no es que el primero tenga clasificación y el segundo no la tenga — ambos la tienen por la naturaleza del software —; es que el primero la hace explícita por construcción y el segundo la mantiene implícita en su nomenclatura propietaria.

Definición operativa de la compatibilidad Cuando un artefacto XF A integra un componente de otro artefacto B, el implementador del integrador realiza tres operaciones cognitivas en orden:

1. **Reconocimiento:** identificar la responsabilidad funcional — interacción, procesamiento o acceso — que el componente integrado desempeña según el criterio de clasificación de las tres condiciones (§7.2.1, §7.2.2, §7.2.3).
2. **Asignación:** clasificar el componente en la celda correspondiente de la matriz $L \times T$ del artefacto A.
3. **Aplicación del mecanismo:** integrar el componente mediante el mecanismo prescrito en §10.2 para esa celda y ese tipo de componente.

Las dos primeras operaciones — reconocimiento y asignación — son juicios arquitectónicos del integrador, no propiedades estructurales del artefacto, y por tanto no se verifican como prescripciones independientes; el modelo verifica sus consecuencias materiales (capa correcta del componente XF resultante, ajuste de su responsabilidad funcional a la de su capa) mediante las prescripciones aplicables a su tipo y al aislamiento entre capas. La tercera operación — aplicación del mecanismo — se materializa en la conformidad estructural del componente XF resultante con las prescripciones de su celda $L \times T$.

El reconocimiento y la asignación los realiza el integrador, no el integrado. Esta es la base de la **bidireccionalidad** y la **no invasividad** de la compatibilidad: ningún artefacto integrado se modifica para hacer posible su integración; cada artefacto conserva la autonomía de su propio espacio de clasificación y solo el integrador es responsable de las decisiones de compatibilidad sobre las dependencias que adopta.

Niveles de integración La compatibilidad opera en dos niveles según el espacio de ejecución de los artefactos involucrados. Se define el **espacio de ejecución** de un artefacto como el contexto computacional en el que sus componentes operan — memoria, recursos del sistema y contexto de proceso. Dos artefactos comparten espacio de ejecución si y solo si sus componentes pueden referenciarse directamente en memoria sin serialización ni transmisión a través de un canal externo. La distinción es binaria: o se comparte o no se comparte.

Según compartan o no su espacio de ejecución, la integración entre dos artefactos se clasifica en dos niveles:

- *Integración a nivel de aplicación* cuando los espacios de ejecución son disjuntos — procesos independientes, comunicación serializada por protocolo. Desde la perspectiva del artefacto XF, el artefacto externo es un sistema externo al que la Capa de Acceso accede según las prescripciones de §7.2.1; los mecanismos de compatibilidad de §10.2 no aplican.
- *Integración a nivel de artefacto* cuando los artefactos comparten espacio de ejecución — mismo proceso, referencias directas en memoria. Los componentes del integrado son candidatos a integración a través de los mecanismos de §10.2.

La [tabla siguiente](#) resume ambos niveles de integración.

Table 9. Niveles de integración entre artefactos según el espacio de ejecución. La integración a nivel de aplicación es independiente de la arquitectura interna de cada artefacto: dos artefactos son compatibles a este nivel desde el momento en que conocen y satisfacen mutuamente su protocolo de comunicación. La integración a nivel de artefacto activa las prescripciones de compatibilidad del modelo XF.

Propiedad	Integración a nivel de aplicación	Integración a nivel de artefacto
Espacio de ejecución	Disjunto (procesos independientes)	Compartido (mismo proceso)
Mecanismo de comunicación	Protocolo de red (HTTP, gRPC, WebSocket, mensajería)	Referencias directas en memoria, invocación local
Activación de las prescripciones de compatibilidad XF	No — el integrado es un sistema externo accedido vía Capa de Acceso	Sí — los mecanismos de §10.2 aplican
Ejemplo típico	Microservicio invocando otro vía REST	Artefacto importando una librería XF en el mismo runtime

Matriz de casos A nivel de artefacto, la integración admite cuatro combinaciones posibles según si el artefacto integrador y el artefacto integrado implementan o no el modelo XF. Las cuatro combinaciones se desarrollan a continuación y se resumen en la [Tabla 10](#).

Caso (A) — Integrador XF, integrado XF La clasificación $L \times T$ de cada componente del integrado está hecha por construcción del propio integrado. El integrador no reclasifica; respeta la celda que el integrado asignó a cada componente. Los mecanismos del catálogo de §10.2 aplican según el tipo del componente que se

integra — **inyección** para los componentes lógicos, **herencia** para los componentes de generalización, **herencia o uso directo** para los componentes de utilidad y **uso directo** para los componentes de transferencia. La elección entre las variantes que cada mecanismo prevé queda a discreción del integrador.

Caso (B) — Integrador XF, integrado no-XF El integrado no expone clasificación $L \times T$ por construcción; el integrador **debe** suplir esa clasificación a través del ciclo de reconocimiento $\rightarrow$ asignación $\rightarrow$ aplicación. El componente XF resultante puede tomar contacto con el integrado mediante cualquier mecanismo que el lenguaje o el marco de desarrollo provean — encapsulación (delegación a una instancia interna), herencia (extensión de una clase del marco de desarrollo), anotación o decoración (metadatos procesados por el marco de desarrollo), implementación de interfaz (conformidad a un contrato del marco de desarrollo), entre otros. La elección del mecanismo es decisión del integrador y no altera la clasificación del componente XF resultante en su matriz $L \times T$: el componente sigue siendo XF y su celda la dicta su responsabilidad funcional. Las prescripciones específicas de cada mecanismo se desarrollan en §10.2.

Caso (C) — Integrador no-XF, integrado XF El modelo XF no prescribe sobre artefactos que no lo implementan; el integrador no está sujeto a las prescripciones del modelo. El integrado XF, por el principio de aislamiento entre capas (§6.2.2) y la prescripción de acceso canónico (§7.3.3), expone sus componentes lógicos a través de sus componentes de inyección con firmas estables e independientes de los detalles de implementación interna. El integrador no-XF accede a esas interfaces directamente. Aunque el modelo no prescribe comportamiento al integrador, se recomienda que A respete el orden de inicialización del integrado (§8.2) — invocar `R.init()`, después `B.init()`, después `A.init()` — para no producir comportamiento indefinido en B; el incumplimiento de este orden no constituye violación del modelo por parte de A, que no lo sigue.

Caso (D) — Integrador no-XF, integrado no-XF Cuando ninguno de los dos artefactos implementa el modelo, la integración queda íntegramente fuera del ámbito prescriptivo del modelo XF. El modelo no enuncia ninguna prescripción sobre esta combinación. Su inclusión en la matriz responde solo a la exhaustividad del cuadro.

Table 10. Matriz de casos de integración a nivel de artefacto. Las filas indican si el artefacto integrador implementa el modelo XF; las columnas, si lo implementa el artefacto integrado. Cada celda enuncia qué prescribe el modelo: el caso $\text{XF} \times \text{XF}$ activa la integración directa porque la clasificación $L \times T$ del integrado es inferible por construcción; el caso $\text{XF} \times \text{no-XF}$ requiere que el integrador reconozca la responsabilidad funcional y aplique la reformulación prescrita; los casos en los que el integrador no es XF quedan fuera del ámbito prescriptivo del modelo.

	Integrado XF	Integrado no-XF
Integrador XF	(A) Clasificación inferida por construcción; el mecanismo lo determina el tipo del componente integrado (§10.2)	(B) El integrador debe reconocer la responsabilidad, asignar celda y reformular la funcionalidad en un componente XF propio (§10.2.1)
Integrador no-XF	(C) Posible sin criterio normativo del modelo; el integrado garantiza interfaces estables por construcción	(D) Fuera del ámbito prescriptivo del modelo XF

10.2 Mecanismos prescritos

La fase de aplicación del ciclo de compatibilidad (§10.1) materializa la integración del componente reconocido y asignado en una dependencia efectiva entre el artefacto integrador y el artefacto integrado. La presente cláusula establece los mecanismos que el modelo prescribe para esa materialización y delimita su ámbito de aplicación.

La estructura de los mecanismos de integración refleja una distinción que el modelo eleva a criterio rector: el componente integrado puede estar ya normalizado en el vocabulario XF o no estarlo. La diferencia no afecta al principio — el ciclo reconocimiento → asignación → aplicación es invariante — pero sí condiciona la fase de aplicación. Cuando el integrado expone su clasificación $L \times T$ por construcción, el mecanismo lo determina exclusivamente el tipo del componente y el modelo lo prescribe sin ambigüedad. Cuando el integrado no expone esa clasificación, el integrador **debe** definir un componente XF propio que asume la responsabilidad funcional reconocida; el mecanismo sintáctico con el que ese componente toma contacto con la funcionalidad ajena es libre dentro de lo que el lenguaje y el marco de desarrollo ofrezcan.

Esta libertad sintáctica en el segundo caso no es una concesión: es consecuencia directa del principio de agnosticismo tecnológico (§6.2.1). Un modelo con vocación de aplicarse uniformemente sobre un ecosistema de marcos de desarrollo heterogéneos no puede legislar sobre la primitiva concreta — anotación, herencia, decorador, implementación de interfaz, delegación por atributo — que cada marco de desarrollo exige para activarse. Lo prescriptivo es el resultado: la conformidad del componente XF resultante con las reglas de su celda $L \times T$ y con los axiomas del modelo. Cualquier sintaxis del integrado es admisible mientras ese resultado se preserve.

La [tabla siguiente](#) recoge el mecanismo prescrito en cada caso.

Table 11. Mecanismo prescrito por el modelo XF según la normalización en XF del componente integrado. Cuando el integrado no está normalizado en XF, la prescripción opera sobre el resultado — la correcta clasificación del componente XF propio que asume la responsabilidad — y no sobre la sintaxis del contacto, que queda a discreción del integrador. Cuando el integrado ya está normalizado en XF, el mecanismo lo determina el tipo del componente y el modelo lo prescribe normativamente.

Normalización en XF	Tipo del componente	Mecanismo XF
No normalizado en XF	(cualquier funcionalidad)	Reformulación en componente XF propio (§10.2.1)
Normalizado en XF	Lógico	Inyección (§10.2.2)
Normalizado en XF	Generalización	Herencia (§10.2.3)
Normalizado en XF	Utilidad	Herencia o uso directo (§10.2.3 / §10.2.4)
Normalizado en XF	Transferencia	Uso directo (§10.2.4)

Una propiedad transversal aplica a los cuatro mecanismos: las restricciones de capa, de tipo y de direccionalidad del modelo (§6.2.2, §6.2.4, §9.4) operan con la misma fuerza sobre dependencias entre artefactos que sobre dependencias internas a un artefacto. La frontera entre artefactos no es una zona de relajación normativa; es una frontera de visibilidad, no de prescripción.

10.2.1 Reformulación de funcionalidad no normalizada en XF

Cuando el integrado no expone clasificación $L \times T$ — marco de desarrollo, kit de desarrollo del sistema operativo, librería tradicional, módulo legado, código generado por una herramienta que no conoce el modelo XF — el integrador reconoce la responsabilidad funcional que la funcionalidad ajena va a desempeñar dentro de su artefacto y define un componente XF propio clasificado en la celda $L \times T$ correspondiente que toma contacto con el comportamiento ajeno mediante el mecanismo sintáctico que el lenguaje o el marco de desarrollo ofrezcan. El tipo del componente XF resultante — lógico, generalización, utilidad o transferencia — es decisión del integrador según la naturaleza del comportamiento que la dependencia tradicional aporta; un componente de generalización es el lugar canónico para abstraer dependencias tecnológicas comunes a varios lógicos de una misma capa (*p. ej.* un **RestRepository** que encapsula un cliente HTTP genérico del que heredan los lógicos de Acceso del integrador). El modelo no añade reglas específicas para verificar la corrección de la asignación: las consecuencias materiales — capa correcta del componente XF, ajuste de su responsabilidad funcional a la de su capa, ausencia de filtración de primitivas del marco de desarrollo integrado — las captan las prescripciones aplicables a su tipo, que operan con la misma fuerza sobre el componente independientemente de si su comportamiento procede de implementación propia del integrador o de la encapsulación de una dependencia externa.

El criterio de asignación de capa es el mismo que rige cualquier clasificación lógica del modelo, sin atenuación por el origen externo: una funcionalidad que conoce un

protocolo de comunicación con un sistema externo y no conoce reglas de negocio se asigna a la **Capa de Acceso**; una funcionalidad que conoce conceptos del dominio del artefacto y no conoce protocolos externos ni mecanismos de presentación se asigna a la **Capa de Negocio**; una funcionalidad que define un punto de entrada al artefacto o produce una representación para el consumidor se asigna a la **Capa de Interacción**. Cuando una funcionalidad ajena implementa responsabilidades de más de una capa, esas responsabilidades **deben** descomponerse y reformularse en componentes XF distintos, uno por capa.

Libertad sintáctica del contacto El mecanismo sintáctico con el que el componente XF resultante toma contacto con la funcionalidad ajena es decisión del integrador. La delegación por atributo — el componente XF mantiene una instancia de la clase del marco de desarrollo como atributo interno y delega operaciones en ella — es la realización más directa y autosuficiente, pero no es la única ni necesariamente la más adecuada. La herencia de una clase base del marco de desarrollo, la decoración con anotaciones procesadas por el contenedor del marco de desarrollo, la implementación de una interfaz declarada por el marco de desarrollo, o cualquier combinación de las anteriores son igualmente admisibles cuando el lenguaje o el marco de desarrollo lo requieren para activar su infraestructura — gestión automática de transacciones, descubrimiento por reflexión, inyección de dependencias del marco de desarrollo, integración en la jerarquía de componentes del sistema operativo.

Lo que el modelo prescribe es el resultado, no la primitiva: el componente XF resultante **debe** cumplir todas las prescripciones de su celda $L \times T$, y en particular su responsabilidad funcional — incluyendo la interfaz pública que expone — debe limitarse a la de su capa (§7.3.1). La superficie expuesta al resto del artefacto es el componente XF, no el integrado: las capas superiores no conocen la existencia de la funcionalidad ajena, solo conocen el componente XF que la reformula. La filtración de primitivas del marco de desarrollo integrado (*p. ej.* un **Repository** que expone una **Connection** JDBC a sus consumidores) es un caso particular de exceso de responsabilidad funcional: el contrato del componente XF resultante deja de ser “ofrecer la abstracción de datos del dominio” para convertirse en “exponer el driver”, lo que sale de la responsabilidad de la Capa de Acceso correctamente entendida.

```

1 // Delegación por atributo: el componente XF mantiene una instancia
2 // de la librería como atributo y delega operaciones en ella.
3
4 class DatabaseRepository extends Repository {
5
6     FrameworkClient client;
7
8     void init(String url) {
9         this.client = new FrameworkClient(url);
10    }
11
12    List<Record> get(String endpoint) {
13        return this.client.call("GET", endpoint);
14    }
15 }
```

Código 35. Contacto por delegación en atributo

```

1 // Decoración con anotación del marco de desarrollo: el componente XF se
2 // declara
3 // con metadatos que el contenedor procesa para activar gestión de
4 // transacciones, pooling u otros servicios del marco de desarrollo.
```

```

4
5 @Repository
6 class IdentityRepository extends Repository {
7     // ...
8 }

```

Código 36. Contacto por decoración con anotación del marco de desarrollo

```

1 // Herencia de una clase base del marco de desarrollo: el componente XF queda
2 // integrado en la jerarquía del marco de desarrollo conservando su rol XF
3 // en la matriz  $L \times T$  del artefacto integrador. Aquí un componente de la Capa
4 // de Interacción (sufijo View) hereda del Widget base de Flutter para
5 // integrarse en su árbol de render, sin dejar de cumplir su rol XF: presentar
6 // el estado del dominio al consumidor del artefacto.
7
8 class TemperatureView extends StatelessWidget {
9
10     // build() es el punto de extensión que exige el Widget de Flutter;
11     // el View accede al dominio por la inyección (Interacción -> Negocio)
12     // y compone la vista con primitivas del marco.
13     Widget build(BuildContext context) {
14         Temperature t = B.temperature.current();
15         return Text(t.toFahrenheit() + " F");
16     }
17 }

```

Código 37. Contacto por herencia de una clase base del marco de desarrollo

Los tres ejemplos ilustran sintaxis distintas; ninguno agota las posibilidades ni constituye la única forma admisible para su caso. La elección la dicta el marco de desarrollo integrado y el criterio de diseño del integrador.

Transferencias heredadas: clasificación $L \times T$ sin canonización Al reformular funcionalidad ajena, el componente XF recibe, devuelve o propaga tipos de datos definidos por el marco de desarrollo, el kit del sistema operativo o la librería integrada — `Connection` de un driver JDBC, `Response` de un cliente HTTP, `IOException` de un SDK, `Promise<T>` del runtime, `Date` del lenguaje. El modelo los llama **transferencias heredadas** y los trata como a cualquier otro componente de transferencia: pertenecen a $\mathbb{C}_T$ y la axiomatización no reconoce un dominio externo (§9.5). De ello se siguen dos consecuencias prácticas:

- **Se clasifican en $L \times T$.** El integrador **debe** asignar cada transferencia heredada que aparezca en la firma de sus operaciones a la capa de menor abstracción en la que se introduce (criterio de §6.2.4): una `IOException` que proviene de un driver de base de datos es de la Capa de Acceso; un `Date` que emplea la lógica de negocio, de la Capa de Negocio. La regla alcanza a toda transferencia heredada, sin excepción.
- **No se canonizan.** Conservan el identificador y la estructura de origen: el modelo **no** prescribe renombrarlas con sufijos canónicos ni reubicarlas. El vocabulario canónico (§7.3.5) obliga solo a las transferencias que el integrador define por sí mismo.

10.2.2 Inyección de componentes lógicos XF

Cuando el integrado expone componentes lógicos ya normalizados en XF, el integrador los accede siguiendo el patrón canónico `<inyección>.<componente>.<operación>()` prescrito en §7.3.3. El patrón

queda satisfecho tanto si la inyección referenciada es la del propio integrado (*p. ej.* `com.authxf.B.user.getCurrent()`) como si el integrador incorpora previamente el lógico ajeno en su propia inyección y accede a través de ella (`B.user.getCurrent()`). Ambas formas son conformes al modelo; el modelo no prescribe la elección.

La incorporación en la inyección propia — cuando el integrador opta por ella — está sujeta a las reglas de tipado de los slots: un componente lógico de la Capa de Negocio del integrado solo se puede incorporar en el componente de inyección B del integrador; un lógico de Acceso, en R; un lógico de Interacción, en A. La capa es propiedad del componente, no del artefacto que lo provee, y la incorporación no la altera. Esta restricción se aplica per-slot sobre el componente de inyección propio.

La incorporación admite varias variantes según cómo combine el integrador la **inicialización del ciclo de vida** — delegada al integrado o asumida por el integrador — y la **referenciación** — alias a la instancia gestionada por el integrado o instancia nueva. El modelo no prescribe la variante: la elección la dicta la cobertura del catálogo ajeno que el integrador necesita exhibir, si el estado del componente debe compartirse o ser independiente, y si la presencia en memoria del catálogo completo del integrado es admisible. Las combinaciones derivadas son válidas y pueden coexistir en una misma inyección con componentes lógicos propios del integrador.

La única proscripción deriva directamente del principio de unicidad del componente de inyección por capa (§7.3.3): ningún componente lógico — propio o ajeno — puede instanciarse fuera de un componente de inyección, ni declararse en una inyección de capa distinta a la suya. El modelo cubre el primer aspecto (la creación debe ocurrir dentro de algún R, B o A) y el segundo (el slot debe estar en la inyección de la capa correspondiente), con la misma fuerza sobre componentes integrados que sobre componentes propios.

El orden de inicialización (§8.2) aplica dentro de cada artefacto sobre sus propios componentes. La dependencia entre artefactos se resuelve en el orden de invocación: si un componente lógico propio del integrador depende de un componente lógico del integrado durante su inicialización, el integrador **debe** ordenar las invocaciones de `init` respetando esa dependencia.

```

1  // Variante con estado compartido: la inyección propia mantiene un
2  // alias a la instancia gestionada por el integrado y combina ese
3  // alias con componentes lógicos propios del integrador.
4  final abstract class B {
5
6      // Componentes lógicos propios del integrador.
7      static readonly NetworkBusiness network = new NetworkBusiness();
8
9      // Componentes lógicos del integrado incorporados por referencia.
10     static readonly TemperatureBusiness temperature =
11         com.myappxf.B.temperature;
12     static readonly UserBusiness user = com.myappxf.B.user;
13     static readonly SessionBusiness session = com.myappxf.B.session;
14
15     static void init() {
16         // el integrado inicializa sus propios componentes.
17         com.myappxf.B.init();
18
19         // el integrador inicializa los suyos.
20         B.network.init();
21     }
22
23     static void terminate() {
24         B.network.terminate();
25     }
26 }

```

```

24     com.myappxf.B.terminate();
25 }
26 }

```

Código 38. Inyección por referencia de componentes lógicos ajenos (estado compartido)

```

1  // Variante con estado aislado: la inyección propia instancia
2  // directamente el componente lógico del integrado y asume su
3  // ciclo de vida.
4  final abstract class B {
5
6      static readonly TemperatureBusiness temperature =
7          new com.myappxf.TemperatureBusiness();
8
9      static void init() {
10         B.temperature.init();
11     }
12
13     static void terminate() {
14         B.temperature.terminate();
15     }
16 }

```

Código 39. Inyección por instanciación de un componente lógico ajeno (estado aislado)

10.2.3 Herencia de componentes

La herencia es el mecanismo prescrito para integrar componentes de generalización publicados por un artefacto XF. El integrador define un componente propio en la capa equivalente y especializa su comportamiento.

La restricción de capa deriva directamente del principio de aislamiento entre capas (§6.2.2): un componente clasificado en una capa del integrado solo puede ser heredado por componentes de la **misma capa** del integrador. La herencia entre capas distintas constituye violación del modelo.

```

1  // Herencia de una generalización XF publicada por una librería.
2  class IdentityRepository extends com.restxf.RestRepository {
3
4      void init() {
5          super.init("https://identity.xforg.com");
6      }
7
8      User fetchUser(String id) {
9          return get("/users/" + id);
10     }
11 }

```

Código 40. Herencia de una generalización XF publicada por una librería

```

1  // Herencia de una utilidad XF publicada por una librería: la base
2  // es extensible (no final) y no instanciable por el carácter estático
3  // de sus operaciones; la derivada añade operaciones estáticas
4  // al catálogo sin alterar el contrato del tipo.
5  class TemperatureUtils extends com.unitsxf.MeasurementUtils {
6
7      static Temperature normalize(Temperature t) {
8          // operación propia que extiende el catálogo de MeasurementUtils
9      }
10 }

```

Código 41. Herencia de una utilidad XF publicada por una librería

10.2.4 Uso directo

El uso directo es el mecanismo prescrito para integrar componentes de utilidad y de transferencia publicados por un artefacto XF. El integrador importa el componente y lo usa en su código sin envolverlo ni declararlo en ningún componente de inyección: instancia las transferencias con `new` cuando lo necesita e invoca las operaciones de utilidad estáticamente.

Las restricciones de direccionalidad de §9.4 operan sobre las transferencias del integrado con la misma fuerza que sobre las propias: una transferencia del integrado definida en la Capa de Acceso puede ser referenciada por componentes de cualquier capa del integrador; una transferencia de Negocio, por componentes de Negocio o Interacción del integrador, pero no por componentes de Acceso. La directriz de transformación estructural de §9.4 aplica indistintamente como criterio de diseño: si el integrador necesita una estructura con definición distinta a la que el integrado publica — añadiendo o eliminando atributos — la opción canónica es definir su propio componente de transferencia en la capa correspondiente. La heredabilidad de la transferencia (§7.3.5) es una alternativa cuando el integrador requiere preservar el linaje estructural y a la vez extender atributos; el uso directo y la herencia son ambos admisibles según convenga.

Las restricciones de localidad de las utilidades (§7.3.4) aplican igualmente: una utilidad definida en la Capa de Negocio del integrado solo puede ser referenciada por componentes de la Capa de Negocio del integrador, salvo la excepción ya nombrada de las utilidades sobre tipos primitivos.

```
1 // Uso directo de la estructura Temperature y la utilidad
2 // TemperatureUtils publicadas por la librería xf-temperature.
3 import com.temperaturexf.repository.transfers.Temperature;
4 import com.temperaturexf.business.utils.TemperatureUtils;
5
6 class TemperatureBusiness extends StatefulBusiness<Temperature> {
7
8     void update(Temperature temperature) {
9         if (TemperatureUtils.toCelsius(temperature) < 14)
10             throw new TemperatureOutOfRangeException();
11         R.server.saveTemperature(temperature);
12         super.update(temperature);
13     }
14 }
```

Código 42. Uso directo de una transferencia y una utilidad XF publicadas por una librería

Combinación de mecanismos Los cuatro mecanismos no son mutuamente excluyentes. Un mismo artefacto integrador puede simultáneamente reformular funcionalidades de varios marcos de desarrollo tradicionales en sus propios componentes XF, inyectar componentes lógicos de una librería XF en su inyección, heredar componentes de generalización de otra, y usar directamente sus transferencias. Cada mecanismo opera bajo sus prescripciones propias sin restricciones adicionales por la combinación. Lo único transversal es la regla rectora: cada componente XF resultante, sea cual sea el mecanismo empleado para construirlo, conserva su clasificación $L \times T$ y debe satisfacer las reglas de su celda.

10.3 Ecosistema de librerías XF

Definición formal de librería XF Se define una **librería XF** como un artefacto software que implementa el modelo XF y que expone, a través de sus interfaces públicas, uno o más de sus componentes para su consumo por otros artefactos XF. Una librería XF puede combinar libremente componentes de cualquier tipo — generalizaciones, transferencias, utilidades y, cuando modela un dominio concreto, también componentes lógicos a través de sus componentes de inyección.

Una librería XF se distingue de un artefacto XF convencional en su propósito: mientras que un artefacto XF convencional automatiza un proceso formal completo — con sus tres capas activas y su punto de entrada propio — una librería XF provee componentes que otros artefactos incorporan para construir su propia lógica.

Dos propiedades caracterizan la calidad de una librería XF sin formar parte del cuerpo prescriptivo del modelo. El modelo no las prescribe como reglas de conformidad: una librería que no las satisface sigue siendo conforme al modelo XF si su estructura lo es; lo que aportan o restan es calidad del ecosistema, no conformidad arquitectónica.

La primera es la **idempotencia funcional**: las interfaces que la librería expone deberían producir resultados consistentes y predecibles ante las mismas entradas, independientemente del artefacto que las consuma o del contexto de ejecución en el que operen. Una librería con interfaces no idempotentes es estructuralmente XF si respeta capas, tipos, nomenclatura y ciclo de vida, pero su utilidad como pieza reutilizable se degrada porque sus consumidores no pueden razonar sobre su comportamiento de forma local.

La segunda es la **uniformidad entre tecnologías**: cuando una librería XF se publica en varias tecnologías (*p. ej.* una variante Java y una variante TypeScript de la misma abstracción), las firmas de sus operaciones deberían ser conceptualmente equivalentes para minimizar la curva de aprendizaje del consumidor que se mueve entre lenguajes. Una librería que no satisface esta propiedad — por razones idiomáticas del lenguaje, por convenciones del marco de desarrollo de destino o por decisión del autor — sigue siendo conforme al modelo; lo que aporta es complejidad de heterogeneidad sobre el ecosistema, no incumplimiento normativo.

Tipos de librerías XF La única distinción doctrinalmente significativa entre librerías XF es la del compromiso semántico con un dominio concreto. La distinción no es taxonómica al modo del tipo de componente — una librería XF puede combinar libremente componentes de cualquier tipo y normalmente lo hace — sino estructural y funcional, derivada de una sola pregunta: *¿la librería implementa la semántica de algún dominio concreto?* La presencia o ausencia de componentes lógicos es la marca sintáctica de la respuesta, y tiene consecuencia inmediata sobre el ciclo de vida que la librería impone al consumidor. Esta distinción es descriptiva: enriquece el vocabulario con que la norma XF discute el ecosistema, sin alterar las prescripciones de conformidad del modelo ni los mecanismos de integración de §10.2, comunes a ambos tipos.

Librerías de utilidad. No modelan ningún dominio. Su contribución son patrones, estructuras y operaciones reutilizables sin compromiso semántico — una generalización `RestRepository` que abstrae el protocolo HTTP genérico, una utili-

dad **StringUtils** que opera sobre tipos primitivos, una transferencia **HttpResponse** que normaliza la representación del resultado de una invocación REST, una utilidad **ValidationUtils** que verifica formatos universales como UUID o IBAN. Al no contener componentes lógicos no exponen punto de entrada propio: no requieren orquestación de `init()` ni `terminate()` por parte del consumidor, y no mantienen estado entre invocaciones. Su valor crece linealmente con el número de artefactos que las usan sin imponerles ninguna decisión de dominio, lo que las hace candidatas naturales a infraestructura compartida transversal a la industria — una buena librería XF de cliente HTTP, de criptografía, de manejo temporal o de validación de formatos universales tiene potencial de adopción en cualquier artefacto, sea cual sea su dominio.

Librerías de dominio. Modelan un subconjunto de dominio concreto y lo encapsulan como artefacto reusable — un proveedor de identidad federada, un esquema de autenticación corporativa, un protocolo de telemetría, una librería de termorregulación. Contienen componentes lógicos cuyas instancias singleton viven en los componentes de inyección de la librería y, en consecuencia, exponen su ciclo de vida al consumidor, que orquesta su `init()` y `terminate()` respetando las dependencias con el resto del artefacto. Su valor se concentra en los artefactos que comparten ese dominio, y su impacto se mide por la convergencia que producen entre implementaciones heterogéneas sobre una misma representación del mismo dominio — el punto de inflexión donde la industria deja de reimplementar lógica de identidad, telemetría o pago en cada artefacto y empieza a componer librerías de dominio probadas.

Las propiedades del ecosistema El ecosistema de librerías XF tiene cuatro propiedades que lo distinguen de los ecosistemas de librerías tradicionales. A diferencia de las propiedades de calidad enunciadas antes, descriptivas, estas propiedades del ecosistema son consecuencias derivables de las prescripciones del modelo: emergen cuando el modelo se aplica a un conjunto suficientemente amplio de artefactos y librerías, y reflejan la estructura del propio modelo, no aspiraciones del autor.

La primera es la **no redundancia entre librerías**. En los ecosistemas de librerías tradicionales, la misma funcionalidad coexiste replicada en múltiples librerías con interfaces distintas — es habitual encontrar varios clientes HTTP para Java, varias librerías de validación para JavaScript o varias implementaciones del patrón observable para Dart. La redundancia se atribuye en parte al acoplamiento de cada librería a un marco de desarrollo o a un paradigma concreto, además de a factores como la competencia entre proyectos y la evolución temporal del ecosistema. En el ecosistema XF, una librería que implementa **RestRepository** conforme al modelo es la implementación de referencia para ese patrón en esa tecnología — no hay razón para que existan múltiples implementaciones incompatibles del mismo patrón en la misma tecnología, porque el modelo prescribe la interfaz que todas deben satisfacer.

La segunda es la **minimización del tamaño de los artefactos**. Cuando los patrones estructurales más comunes están publicados como librerías XF, los artefactos que los consumen no necesitan implementarlos internamente. Un artefacto que hereda **RestRepository**, **StatefulBusiness** y **FormView** de librerías XF solo necesita implementar la lógica efectiva de sus propios componentes — los endpoints concretos, las reglas de negocio específicas, la composición visual particular. La lógica contextual — el protocolo, el patrón de observación, la gestión del formulario —

ya está en las librerías. Esto reduce el tamaño del código específico de cada artefacto y concentra el esfuerzo de desarrollo en lo que realmente diferencia a cada solución.

La tercera es la **portabilidad entre tecnologías**. Una librería XF que implementa `RestRepository` en Java y su equivalente en TypeScript exponen la misma abstracción con las mismas responsabilidades. Un desarrollador que conoce la abstracción XF de `RestRepository` puede usar cualquiera de las dos implementaciones sin curva de aprendizaje adicional — solo necesita aprender la sintaxis del lenguaje, no el concepto. Esta propiedad es la que permite que el ecosistema XF reduzca el coste de cambio de contexto entre tecnologías caracterizado en §1.2.

La cuarta es la **verificabilidad de conformidad**. Dado que el modelo XF prescribe la interfaz que los componentes de cada tipo deben satisfacer, es posible verificar de forma estática si una librería es XF-compatible — si sus componentes de generalización respetan las restricciones de capa, si sus componentes de transferencia no contienen lógica de comportamiento, si sus componentes de utilidad son puros y no instanciables. Las herramientas de análisis de conformidad pueden verificar tanto los artefactos como las librerías que consumen.

11 Conformidad del modelo XF

La conformidad de un artefacto con el modelo XF es la propiedad que determina en qué medida ese artefacto satisface las prescripciones normativas establecidas en las cláusulas anteriores. La presente cláusula operacionaliza esas prescripciones — las convierte en condiciones verificables que permiten calcular de forma determinista el grado de conformidad de cualquier artefacto software con el modelo XF, independientemente del lenguaje de programación, el marco de desarrollo o el dominio de negocio del artefacto.

La naturaleza formal del modelo XF hace posible que la conformidad sea calculable algorítmicamente por herramientas de análisis estático de código. Dado un artefacto y el catálogo de reglas de esta cláusula, el nivel de conformidad es una función del artefacto — no depende de la interpretación del analista sino de la aplicación mecánica del algoritmo definido en §11.4. Esta propiedad habilita la verificación automática y determinista de la conformidad de cualquier artefacto, con independencia de la tecnología y de la arquitectura de referencia que el artefacto proyecte sobre el modelo.

La cláusula se organiza del siguiente modo. La cláusula §11.1 define los conceptos específicos de la conformidad — elemento verificable, violación, nivel de conformidad y verificabilidad de las reglas. La cláusula §11.2 describe los cinco niveles de conformidad. La cláusula §11.3 presenta el catálogo completo de reglas verificables, organizadas en nueve grupos temáticos derivados de las cláusulas normativas anteriores. La cláusula §11.4 define el algoritmo de determinación del nivel de conformidad y las condiciones de aplicabilidad de las reglas.

11.1 Modelo de conformidad

La verificación de la conformidad de un artefacto con el modelo XF requiere un conjunto de conceptos precisos que permitan describir, clasificar y cuantificar las desviaciones del artefacto respecto a las prescripciones del modelo. El concepto

central es la **regla de conformidad**: una condición verificable que operacionaliza una prescripción normativa del modelo en forma evaluable sobre un elemento verificable del artefacto. El conjunto cerrado y exhaustivo de reglas que el modelo prescribe constituye el **catálogo** desarrollado en §11.3; cada regla del catálogo lleva un identificador canónico en formato kebab-case y se caracteriza por su **verificabilidad** — estructural o semántica —, que determina si la regla puede evaluarse de forma determinista a partir de propiedades sintácticas observables del código fuente (estructural) o requiere interpretación semántica de la responsabilidad funcional (semántica). La verificabilidad delimita honestamente el alcance de las herramientas de análisis estático y se trata en §11.1.4. Toda regla del catálogo tiene el mismo estatus normativo: su incumplimiento produce una violación que afecta al nivel de conformidad del artefacto.

La presente cláusula define los conceptos auxiliares del modelo de conformidad — elemento verificable, violación, nivel de conformidad y verificabilidad de las reglas — con la precisión necesaria para que su aplicación sea inequívoca.

11.1.1 Elemento verificable

Un **elemento verificable** es todo elemento del artefacto sobre el que una regla de conformidad puede ser evaluada. El modelo XF reconoce dos tipos de elementos verificables, que son disjuntos entre sí — ningún elemento pertenece simultáneamente a más de un tipo.

Un **elemento de código** es toda unidad de código con identidad propia — clases, módulos o ficheros de funciones cohesionadas según se define en §12. Comprende dos clases: el **componente** — elemento de código clasificado en la matriz $L \times T$, el caso más frecuente, sobre el que opera la mayoría de las reglas del catálogo — y el **elemento de punto de inicio XF** — elemento de código que orquesta el ciclo de vida del artefacto y no se clasifica en $L \times T$, sobre el que operan las reglas del Grupo 8.

Un **elemento estructural** es toda carpeta o subcarpeta del sistema de ficheros del artefacto que forma parte de su estructura de directorios. Las reglas del Grupo 1 del catálogo operan exclusivamente sobre elementos estructurales — verifican la existencia y organización de la estructura de carpetas canónica sin necesidad de inspeccionar el contenido de ningún fichero.

Toda regla del catálogo de §11.3 opera sobre exactamente un tipo de elemento verificable. Esta clasificación está indicada en la definición de cada regla del catálogo (§11.3).

11.1.2 Violación

Una **violación** es la constatación de que un elemento verificable del artefacto no satisface la condición prescrita por una regla del catálogo de conformidad. Toda violación está asociada a exactamente una regla y exactamente un elemento verificable — si diez componentes violan la misma regla, existen diez violaciones independientes. La presencia de cualquier violación es incompatible con la conformidad plena del modelo y, por tanto, impide que el artefacto alcance el nivel 4.

Una regla es **aplicable** a un artefacto cuando existe al menos un elemento verificable del artefacto sobre el que la regla puede ser evaluada. Una regla no aplicable no produce violaciones — ni satisfechas ni violadas. La restricción del dominio de cada regla a su tipo de elemento verificable garantiza por construcción que las reglas tipo-específicas no generan violaciones sobre elementos fuera de su clasificación canónica.

11.1.3 Nivel de conformidad

El **nivel de conformidad** es una propiedad del artefacto que determina en qué medida satisface las prescripciones normativas del modelo XF. El nivel de conformidad es un valor discreto en el conjunto $\{0, 1, 2, 3, 4\}$ — calculable de forma determinista a partir de dos propiedades del artefacto: la condición de totalidad del código y el conjunto de violaciones.

La **condición de totalidad del código** se satisface cuando todo componente del artefacto está clasificado en la matriz $L \times T$ mediante la función de clasificación ϕ . El elemento XF, cuando se declara, no es componente y queda fuera del predicado de totalidad por construcción. Un artefacto que no satisface esta condición tiene componentes no clasificados que coexisten con componentes XF.

Las condiciones que determinan cada nivel se definen en §11.2.

11.1.4 Verificabilidad de las reglas

Las reglas del catálogo de conformidad se clasifican según su **verificabilidad**, que determina el tipo de análisis necesario para evaluar la regla.

Una regla es **estructuralmente verificable** cuando su evaluación puede realizarse de forma determinista a partir de propiedades sintácticas observables del código fuente del artefacto — nombres de archivos, ubicación en la estructura de carpetas, declaraciones de importación entre módulos, sufijos y prefijos de identificadores, grafo de dependencias entre componentes. Las reglas estructuralmente verificables son las que las herramientas de análisis estático pueden evaluar automáticamente con resultado preciso e independiente del analista que ejecute la herramienta.

Una regla es **semánticamente verificable** cuando su evaluación requiere comprender la responsabilidad funcional del código — no solo su forma sintáctica — para determinar si la lógica implementada se corresponde con la responsabilidad de la categoría a la que el componente está clasificado. Las reglas semánticamente verificables exigen análisis adicional al de la herramienta de análisis estático: revisión arquitectónica humana, análisis de flujo de datos, análisis de intención del código, o combinaciones de los anteriores. Su evaluación no es trivialmente automatizable.

Las reglas del catálogo se clasifican según su **verificabilidad**: **estructural** (decidible de forma determinista por análisis estático sobre propiedades sintácticas observables del código fuente) o **semántica** (requiere interpretación de la responsabilidad funcional). La distinción delimita el alcance de las herramientas de análisis estático: las reglas estructurales son evaluables algorítmicamente; las reglas semánticas no.

Implicaciones para el cálculo de conformidad La verificabilidad de una violación — estructural o semántica — tiene efecto **normativo** sobre el nivel de conformidad: determina el techo que el artefacto puede alcanzar. Una violación estructural impide superar el nivel 2; en ausencia de violaciones estructurales, una violación semántica impide el nivel 4 pero permite el nivel 3. La distinción conserva además su lado metodológico — la violación estructural se detecta algorítmicamente por análisis estático, la semántica requiere intervención humana.

La función de conformidad es **determinista** — dado el conjunto completo de violaciones, el nivel queda unívocamente determinado —, pero su **aplicación completa no es automatizable**: las reglas semánticas del catálogo requieren interpretación de la responsabilidad funcional. De ello se sigue una frontera precisa para el análisis estático: una herramienta puede determinar por sí sola si un artefacto alcanza el **nivel 3** — que depende únicamente de violaciones estructurales —, pero **no** puede afirmar el **nivel 4**, cuya distinción respecto al nivel 3 reposa en violaciones semánticas. Las herramientas del ecosistema XF **no deben** reportar el nivel 4 como definitivo sin la cualificación explícita de que las reglas semánticas del catálogo requieren evaluación humana.

11.2 Niveles de conformidad

El modelo XF define cinco niveles de conformidad discretos, ordenados y mutuamente excluyentes. Todo artefacto software pertenece a exactamente uno de estos niveles en un momento dado. Los niveles son ordenados — un nivel superior implica la satisfacción de todas las condiciones de los niveles inferiores más condiciones adicionales. La transición entre niveles es siempre el resultado de corregir violaciones o de clasificar componentes no clasificados — no requiere ninguna acción adicional fuera del alcance normativo del modelo.

11.2.1 Nivel 0 — No conforme

Un artefacto es de nivel 0 cuando no contiene ningún elemento clasificado en la matriz $L \times T$. No existe estructura XF reconocible — ningún componente de inyección, ninguna carpeta canónica, ningún componente tipificado según la taxonomía del modelo. Las reglas del catálogo de §11.3 no son aplicables porque no hay estructura XF sobre la que evaluarlas.

El nivel 0 es el estado natural de todo artefacto tradicional — desarrollado bajo cualquier arquitectura o patrón que no sea XF. No es un estado de error sino el estado de partida de cualquier artefacto antes de la adopción del modelo.

11.2.2 Nivel 1 — Parcialmente conforme

Un artefacto es de nivel 1 cuando contiene al menos un componente clasificado en la matriz $L \times T$ y accesible a través de un componente de inyección, pero no satisface la condición de totalidad del código — existen componentes no clasificados que coexisten con componentes XF. Las reglas del catálogo aplican sobre la parte XF del artefacto — los componentes no clasificados no activan reglas de conformidad porque no forman parte de la estructura XF del artefacto.

El nivel 1 es el estado típico de un artefacto en proceso de migración progresiva hacia el modelo XF. Los componentes se clasifican y migran de forma incremental sin necesidad de refactorizar el artefacto completo de una vez — cada componente que se clasifica correctamente reduce el conjunto de componentes no clasificados y acerca el artefacto a la condición de totalidad necesaria para alcanzar el nivel 2.

11.2.3 Nivel 2 — Imperfectamente conforme

Un artefacto es de nivel 2 cuando satisface la condición de totalidad del código — todo componente está clasificado en la matriz $L \times T$ — pero contiene al menos una violación **estructural**. El artefacto ha adoptado completamente la estructura XF, pero incumple al menos una prescripción estructural; corregir todas las violaciones estructurales lo eleva al nivel 3.

El nivel 2 es el estado típico de un artefacto en proceso de refinamiento — la tipificación es completa pero quedan violaciones estructurales por corregir. Las herramientas de análisis de conformidad son especialmente valiosas en este nivel porque pueden identificar con precisión qué violaciones deben corregirse y en qué componentes se producen.

11.2.4 Nivel 3 — Estructuralmente conforme

Un artefacto es de nivel 3 cuando satisface la condición de totalidad del código y no contiene ninguna violación estructural, aunque contenga al menos una violación semántica. Su estructura — clasificación, direccionalidad entre capas, nomenclatura, ubicación y unicidad de los componentes de inyección — es íntegramente correcta; las inconsistencias que restan pertenecen a la responsabilidad funcional de algún componente, cuya evaluación exige interpretación semántica.

El nivel 3 es el mayor nivel que una herramienta de análisis estático puede certificar por sí sola: depende exclusivamente de violaciones estructurales, decidibles de forma determinista sobre el código fuente. Un artefacto en nivel 3 es un **artefacto bien formado** — satisface por construcción las condiciones estructurales de las que el modelo hace depender sus propiedades formales, como la aciclicidad inter-capas del grafo de dependencias. El desarrollo formal de esa equivalencia como teoría queda como objeto de evoluciones futuras del modelo.

11.2.5 Nivel 4 — Perfectamente conforme

Un artefacto es de nivel 4 cuando satisface la condición de totalidad del código y no contiene ninguna violación, ni estructural ni semántica. Satisface todas las prescripciones normativas del modelo — estructurales y semánticas — y puede ser analizado, mantenido y evolucionado por cualquier desarrollador formado en el modelo con independencia de la tecnología empleada.

El nivel 4 es el estado objetivo de todo artefacto XF en producción. Su certificación no es completamente automatizable: requiere, sobre un artefacto ya verificado en nivel 3 por análisis estático, la evaluación humana de las reglas semánticas del catálogo.

La [tabla siguiente](#) resume los cinco niveles con sus condiciones determinantes:

Table 12. Comparativa de los cinco niveles de conformidad del modelo XF: condiciones determinantes, semántica operacional, qué garantiza el artefacto y qué herramientas de análisis estático lo detectan.

Nivel	Denominación	Totalidad	Violaciones	Semántica / detección
$\lambda = 0$	No conforme	Sin elementos XF	N/A	Artefacto tradicional; ninguna garantía estructural.
$\lambda = 1$	Parcialmente conforme	No satisfecha	N/A	Coexisten componentes clasificados y no clasificados; el catálogo no aplica hasta resolver la totalidad.
$\lambda = 2$	Imperfectamente conforme	Satisfecha	≥ 1 estructural	Incumple al menos una prescripción estructural; detectable de forma determinista por análisis estático.
$\lambda = 3$	Estructuralmente conforme	Satisfecha	0 estructurales; ≥ 1 semántica	Artefacto bien formado; estructura íntegramente correcta. Máximo nivel certificable por una herramienta de análisis estático.
$\lambda = 4$	Perfectamente conforme	Satisfecha	0 totales	Todas las prescripciones, estructurales y semánticas, satisfechas; equivalente al artefacto idealizado. La distinción $3 \rightarrow 4$ exige evaluación semántica humana.

11.3 Catálogo de reglas

El catálogo de reglas de conformidad es el conjunto exhaustivo y cerrado de condiciones verificables que operacionalizan las prescripciones normativas del modelo XF. Cada regla del catálogo deriva directamente de una prescripción establecida en las cláusulas 6 a 10 — no introduce requisitos nuevos sino que expresa cada prescripción en forma de condición verificable por herramientas de análisis estático.

El catálogo está organizado en nueve grupos temáticos. El criterio de agrupación es el **tipo de componente o elemento estructural** sobre el que opera la regla — coherente con el orden expositivo de §7.3 y §7.2, que describen las prescripciones normativas en el contexto de cada tipo. La clase de prescripción (nomenclatura, dependencia, herencia, estado, pureza, ciclo de vida) queda codificada como segundo segmento del identificador canónico (por ejemplo, `logic-naming-repository` pertenece al Grupo 3 por tipo de componente y al subgrupo *nomenclatura* por clase de prescripción), permitiendo el filtrado por clase desde herramientas de análisis sin necesidad de reagrupar.

Los grupos no son independientes entre sí: una misma desviación arquitectónica puede activar reglas de múltiples grupos simultáneamente. La restricción de dominio de cada regla (§11.4.2) garantiza que las reglas tipo-específicas no generan violaciones sobre elementos fuera de su clasificación canónica, sin necesidad de mecanismo de propagación independiente.

Cada regla se identifica mediante un **identificador canónico** en formato kebab-case — el nombre único con el que las herramientas de análisis referencian la regla — y se describe mediante una frase precisa que indica qué condición verifica. Su violación afecta al nivel de conformidad del artefacto. La definición de cada regla — su condición de violación, el tipo de elemento verificable sobre el que opera y su referencia normativa — se desarrolla en los grupos del catálogo que siguen.

El presente catálogo es normativo y cerrado en la versión actual del modelo. Futuras versiones del modelo podrán extender el catálogo con nuevas reglas derivadas del documento de compatibilidad por marco de desarrollo y de la evolución del ecosistema de librerías XF, siempre que cada nueva regla derive de una prescripción normativa del modelo y no introduzca requisitos no contemplados en el cuerpo normativo.

Table 13. Catálogo maestro de reglas de conformidad del modelo XF: identificador canónico (kebab-case), grupo temático, dominio de aplicabilidad sobre el tipo de elemento verificable (**fld** = Folder, **cod** = elemento de código), verificabilidad (**str** = Structural — decidible por análisis estático determinista —, **sem** = Semantic — requiere interpretación de la responsabilidad funcional) y referencia a la cláusula que la origina. El incumplimiento de cualquier regla produce violación que afecta al nivel de conformidad.

Identificador canónico	Grupo	Dom.	Verif.	Origen normativo
structure-layer-mismatch	1	fld	str	§7.4, §8.1
structure-type-mismatch	1	fld	str	§7.4
structure-injection-missing	1	fld	str	§7.3.3
structure-injection-multiplicity	1	fld	str	§7.3.3
structure-component-naming	1	fld	str	§7.4
structure-domain-subdivision	1	fld	sem	§7.2.1, §7.2.3
layer-reference	2	cod	str	§6.2.2
layer-inheritance	2	cod	str	§6.2.2
layer-skip	2	cod	str	§6.2.2
logic-naming-repository	3	cod	str	§7.3.1, §7.2.1
logic-naming-business	3	cod	str	§7.3.1, §7.2.2
logic-naming-service	3	cod	str	§7.3.1, §7.2.3
logic-naming-view	3	cod	str	§7.3.1, §7.2.3
logic-mismatch-repository	3	cod	sem	§7.2.1
logic-mismatch-business	3	cod	sem	§7.2.2
logic-mismatch-api	3	cod	sem	§7.2.3
logic-initialization-missing	3	cod	str	§7.3.1, §8.2
logic-termination-missing	3	cod	str	§7.3.1, §8.2
logic-constructor-mismatch	3	cod	str	§8.2
logic-inheritance	3	cod	str	§7.3.1
general-naming-repository	4	cod	str	§7.3.2, §7.2.1
general-naming-business	4	cod	str	§7.3.2, §7.2.2
general-naming-service	4	cod	str	§7.3.2, §7.2.3
general-naming-view	4	cod	str	§7.3.2, §7.2.3
general-mismatch-repository	4	cod	sem	§7.3.2, §7.2.1

Continúa en la página siguiente

Tabla 13 (continuación)

Identificador canónico	Grupo	Dom.	Verif.	Origen normativo
general-mismatch-business	4	cod	sem	§7.3.2, §7.2.2
general-mismatch-api	4	cod	sem	§7.3.2, §7.2.3
general-injection-reference	4	cod	str	§7.3.2
general-domain-state	4	cod	str	§7.3.1, §7.3.2
general-instantiable	4	cod	str	§7.3.2
general-initialization-missing	4	cod	str	§7.3.2, §8.2
general-termination-missing	4	cod	str	§7.3.2, §8.2
general-constructor-mismatch	4	cod	str	§8.2
general-inheritance	4	cod	str	§7.3.2
injection-naming-r	5	cod	str	§7.3.3
injection-naming-b	5	cod	str	§7.3.3
injection-naming-a	5	cod	str	§7.3.3
injection-non-repository	5	cod	str	§7.3.3
injection-non-business	5	cod	str	§7.3.3
injection-non-api	5	cod	str	§7.3.3
injection-mismatch	5	cod	str	§7.3.3
injection-instantiable	5	cod	str	§7.3.3
injection-member-mutable	5	cod	str	§7.3.3
injection-member-public	5	cod	str	§7.3.3
injection-init-missing	5	cod	str	§7.3.3, §8.2
injection-terminate-missing	5	cod	str	§7.3.3, §8.2
injection-init-mismatch	5	cod	str	§7.3.3
injection-terminate-mismatch	5	cod	str	§7.3.3
injection-lifecycle-symmetry	5	cod	str	§8.2
injection-inheritance	5	cod	str	§7.3.3
utility-naming	6	cod	str	§7.3.4
utility-mismatch	6	cod	sem	§7.3.4
utility-instantiable	6	cod	str	§7.3.4
utility-member-instance	6	cod	str	§7.3.4
utility-mutable-state	6	cod	str	§7.3.4
utility-inheritance	6	cod	str	§7.3.4
transfer-naming	7	cod	sem	§7.3.5
transfer-dependency	7	cod	str	§7.3.5
transfer-business-logic	7	cod	sem	§7.3.5
transfer-inheritance	7	cod	str	§7.3.5
xf-init-missing	8	cod	str	§8.3
xf-terminate-missing	8	cod	str	§8.3
xf-init-mismatch	8	cod	str	§8.3
xf-terminate-mismatch	8	cod	str	§8.3
lifecycle-logic-instantiation	9	cod	str	§7.3.3
lifecycle-logic-init	9	cod	str	§7.3.3, §8.3
lifecycle-logic-terminate	9	cod	str	§8.3
lifecycle-injection-init	9	cod	str	§7.3.3, §8.3
lifecycle-injection-terminate	9	cod	str	§8.3
lifecycle-xf-init	9	cod	str	§8.3
lifecycle-xf-terminate	9	cod	str	§8.3

11.3.1 Grupo 1 — Estructura de carpetas

Las reglas de este grupo verifican la existencia y organización de la estructura de carpetas canónica prescrita en §7.4. Son las reglas de más alto nivel del catálogo — su verificación no requiere inspección del contenido de ningún fichero, solo análisis

de la estructura de directorios del artefacto.

La existencia material de las cuatro subcarpetas de tipo — `/general`, `/logic`, `/transfers` y `/utils` — no se enumera como regla independiente: una capa que no contiene componentes de un tipo no requiere materializar la subcarpeta correspondiente como carpeta vacía. Cuando una capa contiene componentes de un tipo, esos componentes deben residir en la subcarpeta canónica de ese tipo — prescripción verificada por las reglas de ubicación de los grupos 3, 4, 6 y 7 sobre el componente individual, no sobre la presencia de la carpeta.

Reglas

1. **structure-layer-mismatch** (*estructural*) — Existe un elemento (archivo o carpeta) directamente bajo una raíz de artefacto cuyo nombre no es canónico para ese nivel (§7.4, §8.1).
2. **structure-type-mismatch** (*estructural*) — Existe un elemento (archivo o carpeta) directamente bajo una carpeta de capa cuyo nombre no es canónico para esa capa (§7.4).
3. **structure-injection-missing** (*estructural*) — Una carpeta de capa no contiene el archivo canónico del componente de inyección que le corresponde (§7.3.3).
4. **structure-injection-multiplicity** (*estructural*) — Existe más de un archivo con el nombre canónico de un componente de inyección bajo la raíz de la capa correspondiente (§7.3.3).
5. **structure-component-naming** (*estructural*) — Un archivo declara un componente XF cuyo nombre canónico no coincide con el nombre del archivo sin extensión (§7.4).
6. **structure-domain-subdivision** (*semántica*) — La subdivisión interna de `/repository/logic` o de `/api/logic` agrupa componentes lógicos por dominio funcional del artefacto, en lugar de por el criterio de subdivisión legítimo de su capa (§7.2.1, §7.2.3).

11.3.2 Grupo 2 — Aislamiento entre capas

Las reglas de este grupo verifican el cumplimiento del principio de aislamiento entre capas (§6.2.2) y de la regla de direccionalidad de las transferencias (§9.5) sobre cualquier componente del artefacto, independientemente de su tipo. Operan homogéneamente sobre la relación de referencia entre componentes: dado un par (c, c') tal que c referencia a c' , las reglas verifican la admisibilidad del par en términos de la diferencia de capa de sus extremos. La verificación se realiza sobre el componente que importa — no sobre el importado —, lo que permite atribuir violaciones al archivo concreto que las introduce.

Reglas

1. **layer-reference** (*estructural*) — Un componente referencia a otro componente de una capa de mayor nivel de abstracción que la suya (§6.2.2).
2. **layer-inheritance** (*estructural*) — Un componente hereda de otro componente clasificado en una capa distinta a la suya, en cualquier dirección (§6.2.2).

3. **layer-skip** (*estructural*) — Un componente referencia a otro componente de una capa de menor nivel de abstracción saltando la capa intermedia. Se exceptúan los componentes de transferencia (§9.5) y los componentes de utilidad sobre tipos primitivos definidos en `/src/repository/utils` (§7.3.4).

11.3.3 Grupo 3 — Componentes lógicos

Verifican el cumplimiento de las prescripciones normativas sobre componentes lógicos (§7.3.1, §7.2.1, §7.2.2, §7.2.3). Aplican a las clases ubicadas en `/src/capa/logic/` o con sufijo canónico `Repository`, `Business`, `Service` o `View`.

Reglas de nomenclatura

1. **logic-naming-repository** (*estructural*) — Un componente lógico de la Capa de Acceso no lleva el sufijo canónico `Repository`.
2. **logic-naming-business** (*estructural*) — Un componente lógico de la Capa de Negocio no lleva el sufijo canónico `Business`.
3. **logic-naming-service** (*estructural*) — Un componente lógico de la Capa de Interacción que implementa un punto de interacción sistémica no lleva el sufijo canónico `Service`.
4. **logic-naming-view** (*estructural*) — Un componente lógico de la Capa de Interacción que implementa un punto de interacción gráfica no lleva el sufijo canónico `View`.

Reglas de contenido

1. **logic-mismatch-repository** (*semántica*) — Un componente lógico clasificado en la Capa de Acceso contiene lógica que no corresponde a la responsabilidad funcional de esa capa (§7.2.1).
2. **logic-mismatch-business** (*semántica*) — Un componente lógico clasificado en la Capa de Negocio contiene lógica que no corresponde a la responsabilidad funcional de esa capa (§7.2.2).
3. **logic-mismatch-api** (*semántica*) — Un componente lógico clasificado en la Capa de Interacción contiene lógica que no corresponde a la responsabilidad funcional de esa capa (§7.2.3).

Reglas de ciclo de vida

1. **logic-initialization-missing** (*estructural*) — Un componente lógico no declara ni hereda una operación invocable `init()` (§7.3.1, §8.2).
2. **logic-termination-missing** (*estructural*) — Un componente lógico no declara ni hereda una operación invocable `terminate()` (§7.3.1, §8.2).
3. **logic-constructor-mismatch** (*estructural*) — Un componente lógico implementa lógica de inicialización no trivial dentro de su constructor en lugar de en una operación `init()` separada (§8.2).

Reglas de herencia

1. **logic-inheritance** (*estructural*) — Un componente lógico hereda de un componente que no es un componente lógico ni un componente de generalización de su misma capa (§7.3.1, §7.3.2).

11.3.4 Grupo 4 — Componentes de generalización

Verifican el cumplimiento de las prescripciones normativas sobre componentes de generalización (§7.3.2, §7.4). Aplican a las clases ubicadas en `/src/capa/general/`.

Reglas de nomenclatura

1. **general-naming-repository** (*estructural*) — Un componente de generalización de la Capa de Acceso no termina con el sufijo canónico **Repository** (§7.3.2, §7.2.1).
2. **general-naming-business** (*estructural*) — Un componente de generalización de la Capa de Negocio no termina con el sufijo canónico **Business** (§7.3.2, §7.2.2).
3. **general-naming-service** (*estructural*) — Un componente de generalización de la Capa de Interacción que abstrae comportamiento para puntos de interacción sistémica no termina con el sufijo canónico **Service** (§7.3.2, §7.2.3).
4. **general-naming-view** (*estructural*) — Un componente de generalización de la Capa de Interacción que abstrae comportamiento para puntos de interacción gráfica no termina con el sufijo canónico **View** (§7.3.2, §7.2.3).

Reglas de contenido

1. **general-mismatch-repository** (*semántica*) — Un componente de generalización clasificado en la Capa de Acceso contiene lógica que no corresponde a la responsabilidad funcional de esa capa (§7.2.1).
2. **general-mismatch-business** (*semántica*) — Un componente de generalización clasificado en la Capa de Negocio contiene lógica que no corresponde a la responsabilidad funcional de esa capa (§7.2.2).
3. **general-mismatch-api** (*semántica*) — Un componente de generalización clasificado en la Capa de Interacción contiene lógica que no corresponde a la responsabilidad funcional de esa capa (§7.2.3).
4. **general-injection-reference** (*estructural*) — Un componente de generalización invoca operaciones de componentes lógicos a través de un componente de inyección (§7.3.2).
5. **general-domain-state** (*estructural*) — Un componente de generalización declara atributos mutables que modelan conceptos del dominio del artefacto (§7.3.1, §7.3.2).

Reglas de instanciación

1. **general-instantiable** (*estructural*) — Un componente de generalización no impide explícitamente su instanciación directa mediante los mecanismos del lenguaje de programación (§7.3.2).

Reglas de ciclo de vida

1. **general-initialization-missing** (*estructural*) — Un componente de generalización no declara ni hereda una operación invocable `init()` (§7.3.2, §8.2).
2. **general-termination-missing** (*estructural*) — Un componente de generalización no declara ni hereda una operación invocable `terminate()` (§7.3.2, §8.2).
3. **general-constructor-mismatch** (*estructural*) — Un componente de generalización implementa lógica no trivial dentro de su constructor en lugar de en una operación `init()` separada (§8.2).

Reglas de herencia

1. **general-inheritance** (*estructural*) — Un componente de generalización hereda de un componente XF interno del artefacto que no es un componente de generalización de su misma capa (§7.3.2).

11.3.5 Grupo 5 — Componentes de inyección

Verifican el cumplimiento de las prescripciones normativas sobre los componentes de inyección R, B y A (§7.3.3, §8.2). Aplican a los ficheros con nombre canónico R, B o A ubicados directamente en la raíz de `/src/repository`, `/src/business` y `/src/api`, respectivamente.

Reglas de nomenclatura

1. **injection-naming-r** (*estructural*) — El componente de inyección de la Capa de Acceso no lleva el nombre canónico R (§7.3.3).
2. **injection-naming-b** (*estructural*) — El componente de inyección de la Capa de Negocio no lleva el nombre canónico B (§7.3.3).
3. **injection-naming-a** (*estructural*) — El componente de inyección de la Capa de Interacción no lleva el nombre canónico A (§7.3.3).

Reglas de contenido

1. **injection-non-repository** (*estructural*) — El componente de inyección R declara como atributo estático un componente que no es un componente lógico de la Capa de Acceso (§7.3.3).
2. **injection-non-business** (*estructural*) — El componente de inyección B declara como atributo estático un componente que no es un componente lógico de la Capa de Negocio (§7.3.3).
3. **injection-non-api** (*estructural*) — El componente de inyección A declara como atributo estático un componente que no es un componente lógico de la Capa de Interacción (§7.3.3).
4. **injection-mismatch** (*estructural*) — Un componente de inyección declara cualquier miembro que no sea una referencia estática tipada a un componente lógico de su capa o las operaciones estáticas `init()` o `terminate()` (§7.3.3).

Reglas de estructura de la clase

1. `injection-instantiable` (*estructural*) — Un componente de inyección no impide explícitamente su instanciación mediante los mecanismos del lenguaje de programación (§7.3.3).
2. `injection-member-mutable` (*estructural*) — Las referencias estáticas a los componentes lógicos en el componente de inyección no son inmutables (§7.3.3).
3. `injection-member-public` (*estructural*) — Las referencias estáticas a los componentes lógicos en el componente de inyección no tienen visibilidad pública (§7.3.3).

Reglas de ciclo de vida

1. `injection-init-missing` (*estructural*) — Un componente de inyección no declara una operación estática invocable `init()` (§7.3.3, §8.2).
2. `injection-terminate-missing` (*estructural*) — Un componente de inyección no declara una operación estática invocable `terminate()` (§7.3.3, §8.2).
3. `injection-init-mismatch` (*estructural*) — El cuerpo de la operación `init()` de un componente de inyección contiene sentencias distintas de invocaciones a `init()` de los componentes lógicos agregados como slots en esa inyección (§7.3.3).
4. `injection-terminate-mismatch` (*estructural*) — El cuerpo de la operación `terminate()` de un componente de inyección contiene sentencias distintas de invocaciones a `terminate()` de los componentes lógicos agregados como slots en esa inyección (§7.3.3).
5. `injection-lifecycle-symmetry` (*estructural*) — Existe un slot *s* de un componente de inyección tal que *s*.`init()` se invoca en el cuerpo de `init()` y *s*.`terminate()` no se invoca en el cuerpo de `terminate()`, o viceversa (§8.2).

Reglas de herencia

1. `injection-inheritance` (*estructural*) — Un componente de inyección hereda de cualquier componente, interno o externo al artefacto (§7.3.3).

11.3.6 Grupo 6 — Componentes de utilidad

Verifican el cumplimiento de las prescripciones normativas sobre componentes de utilidad (§7.3.4). Aplican a las clases ubicadas en `/src/capa/utils/` o con sufijo canónico `Utils`.

Reglas de nomenclatura

1. `utility-naming` (*estructural*) — Un componente de utilidad no lleva el sufijo `Utils` (§7.3.4).

Reglas de contenido

1. `utility-mismatch` (*semántica*) — Un componente de utilidad implementa reglas del modelado del dominio del artefacto, invoca componentes lógicos a

través de los componentes de inyección, o produce efectos laterales observables (§7.3.4).

Reglas de estructura de la clase

1. `utility-instantiable` (*estructural*) — Un componente de utilidad no impide explícitamente su instanciación mediante los mecanismos del lenguaje de programación (§7.3.4).
2. `utility-member-instance` (*estructural*) — Un componente de utilidad declara miembros de instancia (atributos o métodos no estáticos) (§7.3.4).
3. `utility-mutable-state` (*estructural*) — Un componente de utilidad declara atributos mutables (§7.3.4).

Reglas de herencia

1. `utility-inheritance` (*estructural*) — Un componente de utilidad hereda de un componente que no es un componente de utilidad de su misma capa (§7.3.4).

11.3.7 Grupo 7 — Componentes de transferencia

Verifican el cumplimiento de las prescripciones normativas sobre componentes de transferencia (§7.3.5, §9). Aplican a las clases ubicadas en `/src/capa/transfers/`.

Reglas de nomenclatura

1. `transfer-naming` (*semántica*) — Un componente de transferencia lleva un sufijo añadido al concepto del dominio que modela (§7.3.5).

Reglas de estructura y autocontención

1. `transfer-dependency` (*estructural*) — Un componente de transferencia referencia un componente de inyección, lógico, de generalización o de utilidad (§7.3.5).
2. `transfer-business-logic` (*semántica*) — Una operación de un componente de transferencia modela un proceso de negocio del dominio (§7.3.5).
3. `transfer-inheritance` (*estructural*) — Un componente de transferencia hereda de un componente que no es de transferencia (§7.3.5).

11.3.8 Grupo 8 — Elemento punto de inicio XF

Las reglas de este grupo verifican las prescripciones normativas sobre el **elemento punto de inicio XF** y la orquestación del ciclo de vida agregado del artefacto que su declaración representa: la presencia de las operaciones `init()` y `terminate()` y la canonicidad de sus cuerpos — secuencia de invocaciones `R.init()`; `B.init()`; `A.init()`; en orden ascendente y `A.terminate()`; `B.terminate()`; `R.terminate()`; en orden inverso. Las cuatro reglas comparten el prefijo `xf-` porque verifican el mismo elemento de código: el elemento de punto de inicio XF. La exclusividad de la orquestación del ciclo de vida sobre los componentes lógicos — qué

componentes pueden instanciarlos e invocar `init()`/`terminate()` sobre ellos — se verifica per-componente en el Grupo 9.

Reglas del elemento XF

1. `xf-init-missing` (*estructural*) — El elemento XF no declara una operación estática invocable `init()` (§8.3).
2. `xf-terminate-missing` (*estructural*) — El elemento XF no declara una operación estática invocable `terminate()` (§8.3).
3. `xf-init-mismatch` (*estructural*) — El cuerpo de la operación `init()` del elemento XF contiene sentencias distintas de las invocaciones `R.init()`, `B.init()`, `A.init()` en ese orden (§8.3).
4. `xf-terminate-mismatch` (*estructural*) — El cuerpo de la operación `terminate()` del elemento XF contiene sentencias distintas de las invocaciones `A.terminate()`, `B.terminate()`, `R.terminate()` en ese orden inverso (§8.3).

11.3.9 Grupo 9 — Exclusividad de la orquestación del ciclo de vida

Las reglas de este grupo verifican la exclusividad de las operaciones de ciclo de vida en tres sentidos paralelos a la jerarquía de orquestación del artefacto: la creación de instancias de componentes lógicos y la invocación de `init()`/`terminate()` sobre lógicos están reservadas a los componentes de inyección; la invocación de `init()`/`terminate()` sobre las inyecciones está reservada al elemento XF; y la invocación de `init()`/`terminate()` sobre el propio XF está reservada al XF mismo o al punto de inicio de ejecución externo del artefacto. Cuantifican sobre todo componente del artefacto y reportan como testigo de la violación el componente invocador, no el invocado.

Reglas sobre componentes lógicos

1. `lifecycle-logic-instantiation` (*estructural*) — Un componente del artefacto distinto de los componentes de inyección instancia un componente lógico mediante `new` (§7.3.3).
2. `lifecycle-logic-init` (*estructural*) — Un componente del artefacto distinto del componente de inyección de la capa del lógico invoca `init()` sobre un componente lógico (§7.3.3, §8.2).
3. `lifecycle-logic-terminate` (*estructural*) — Un componente del artefacto distinto del componente de inyección de la capa del lógico invoca `terminate()` sobre un componente lógico (§8.2).

Reglas sobre componentes de inyección

1. `lifecycle-injection-init` (*estructural*) — Un componente del artefacto distinto del elemento XF invoca `init()` sobre un componente de inyección (§7.3.3, §8.3).
2. `lifecycle-injection-terminate` (*estructural*) — Un componente del artefacto distinto del elemento XF invoca `terminate()` sobre un componente de inyección (§8.3).

Reglas sobre el elemento XF

1. `lifecycle-xf-init` (*estructural*) — Un componente del artefacto distinto del propio XF invoca `init()` sobre el elemento XF (§8.3).
2. `lifecycle-xf-terminate` (*estructural*) — Un componente del artefacto distinto del propio XF invoca `terminate()` sobre el elemento XF (§8.3).

11.3.10 Cierre del catálogo

El catálogo enumera **71 reglas** distribuidas en **9 grupos temáticos**. El incumplimiento de cualquiera de ellas constituye violación y afecta al nivel de conformidad λ del artefacto. Las cifras agregadas por dominio (**fld**, **cmp**) y por verificabilidad (**str**, **sem**) son derivables de la tabla maestra y se reportan, cuando proceda, por las herramientas de análisis.

El catálogo es cerrado en la versión 1.0 del modelo. Futuras versiones podrán extenderlo mediante adición de nuevas reglas conforme al procedimiento normativo de evolución, siempre que cada nueva regla derive de una prescripción establecida en las cláusulas 6 a 10 y no introduzca requisitos no contemplados en el cuerpo normativo. La eliminación de una regla existente requiere justificación normativa explícita y queda registrada en el Anexo F (Historial de revisiones).

11.4 Determinación del nivel de conformidad

La determinación del nivel de conformidad de un artefacto es un procedimiento algorítmico que produce un resultado único y determinista a partir de dos entradas: el artefacto a evaluar y el catálogo de reglas de §11.3. El procedimiento no requiere interpretación — dadas las mismas entradas, produce siempre el mismo resultado independientemente de quién lo ejecute o de la herramienta que lo implemente.

11.4.1 Algoritmo de determinación

El algoritmo de determinación del nivel de conformidad se ejecuta en cuatro etapas secuenciales. Cada etapa produce un resultado que condiciona la ejecución de las etapas siguientes. La presente cláusula prescribe el comportamiento que toda herramienta de análisis estático conforme con el modelo XF debe implementar.

Etap 1 — Inventario de elementos verificables El analizador recorre el sistema de ficheros del artefacto y construye el inventario completo de elementos verificables (§3.1.22): los elementos estructurales (carpetas y subcarpetas bajo `/src`) y los elementos de código — clases, módulos o ficheros de funciones cohesionadas, incluido el elemento de punto de inicio XF —, clasificando los componentes por su ubicación en la estructura de carpetas y por su nomenclatura. El inventario retiene los componentes para los que la clasificación no está definida — componentes no clasificados — a efectos de la etapa siguiente.

Etap 2 — Verificación de la condición de totalidad El analizador verifica si el artefacto satisface la condición de totalidad del código definida en §11.1.3: todo componente del artefacto está clasificado en la matriz $L \times T$. El elemento XF, cuando se declara, queda fuera del predicado de totalidad por no ser componente.

- Si la condición se satisface, el algoritmo continúa en la Etapa 3.
- Si no se satisface y existe al menos un componente clasificado, el artefacto es de **nivel 1** (parcialmente conforme) y el algoritmo termina.
- Si no se satisface y no existe ningún componente clasificado, el artefacto es de **nivel 0** (no conforme) y el algoritmo termina.

Etapa 3 — Evaluación del catálogo de reglas El analizador evalúa el catálogo de reglas de §11.3 sobre el inventario de elementos verificables, respetando las condiciones de aplicabilidad de §11.4.2. Cada par regla-elemento cuya regla resulta aplicable y no se satisface se registra como violación.

Etapa 4 — Determinación del nivel El analizador clasifica las violaciones registradas según su verificabilidad (§11.1.4) y aplica las condiciones de §11.2:

- Si existe al menos una violación **estructural**, el artefacto es de **nivel 2** (imperfectamente conforme).
- Si no existe ninguna violación estructural pero existe al menos una violación **semántica**, el artefacto es de **nivel 3** (estructuralmente conforme).
- Si no existe ninguna violación, el artefacto es de **nivel 4** (perfectamente conforme).

El resultado final del algoritmo es $\Lambda(\mathfrak{A}) \in \{0, 1, 2, 3, 4\}$ que caracteriza completamente el estado de conformidad del artefacto respecto al modelo XF. Las propiedades formales del algoritmo son su determinismo y su completitud.

El árbol de decisión de la Figura 11 resume las cuatro etapas en forma operativa.

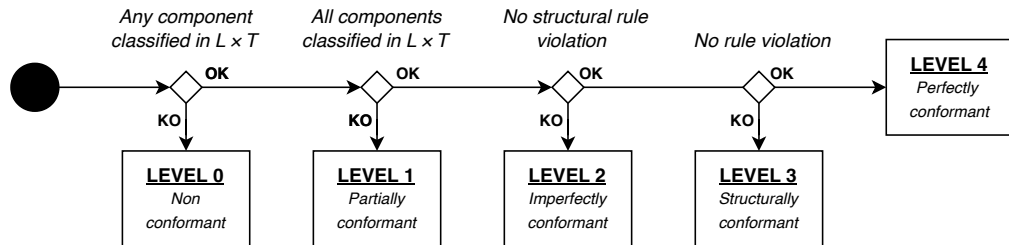


Figure 11. Árbol de decisión que materializa el algoritmo de determinación del nivel de conformidad $\Lambda(\mathfrak{A}) \in \{0, 1, 2, 3, 4\}$. Cada nodo de decisión evalúa una condición sucesiva — presencia de al menos un componente clasificado, satisfacción de la condición de totalidad, ausencia de violaciones estructurales y ausencia de violaciones semánticas — y cada rama terminal asigna el nivel correspondiente. La secuencia es la operacionalización determinista del algoritmo de determinación del nivel de conformidad.

11.4.2 Aplicabilidad de reglas

Una regla es aplicable a un artefacto cuando existe al menos un elemento verificable del artefacto sobre el que la regla puede ser evaluada de forma significativa. Una regla no aplicable no contribuye al conjunto de violaciones: una regla cuyas condiciones de aplicabilidad no se satisfacen no genera violaciones. Por ejemplo, un artefacto sin componentes de generalización no produce violaciones de **general-mismatch-***

porque no existe ningún elemento sobre el que evaluarlas; las reglas correspondientes simplemente no son aplicables al artefacto concreto.

La presente cláusula del cuerpo normativo prescribe el comportamiento operacional que toda herramienta de análisis estático conforme con el modelo XF debe implementar.

12 Ontología de la arquitectura

La ontología de la arquitectura XF establece el vocabulario formal del modelo — el conjunto de términos con definición precisa y normativa que permite a los equipos de desarrollo, analistas, revisores y herramientas comunicarse sobre la arquitectura de un artefacto XF con la misma comprensión, independientemente de la tecnología empleada. Este vocabulario es la materialización del objetivo central del modelo: eliminar la ambigüedad terminológica que caracteriza al ecosistema de desarrollo de software actual.

La cláusula se organiza en dos cláusulas internas. La cláusula §12.1 presenta el mapa conceptual del modelo — una representación visual de las relaciones entre los conceptos fundamentales. La cláusula §12.2 presenta el diccionario de términos — la definición precisa de cada término del modelo, organizada según el orden expositivo del documento, con agrupación temática local cuando favorece la legibilidad.

12.1 Mapa conceptual

El mapa conceptual de la Figura 12 reúne en un único grafo los conceptos fundamentales del modelo XF y las relaciones semánticas que los vinculan, desde el artefacto y el proceso que formaliza hasta la sentencia y el estado que una operación transforma. Sirve de índice visual al diccionario de términos de §12.2, donde cada concepto recibe su definición normativa precisa.

12.2 Diccionario de términos

Conceptos del proceso formal y de la industria

Proceso formal Secuencia de actividades con semántica precisa, entradas y salidas bien definidas y reglas de transición verificables, que puede ser descrita con precisión suficiente para ser automatizada mediante software. Todo proceso formal admite una descomposición canónica en tres etapas con estatus formal distinto: el *procesamiento* como etapa estructuralmente necesaria, y la *interacción* y el *acceso* como vías estructuralmente opcionales pero exhaustivas y mutuamente excluyentes de comunicación con el entorno.

Referencia: §5.1, §5.2.

Etapas de interacción Primera etapa invariante de todo proceso formal. Comprende la recepción de la orden de ejecución del proceso y la verificación de que se satisfacen las precondiciones necesarias para su inicio. En el modelo XF, esta etapa se corresponde biunívocamente con la Capa de Interacción del artefacto.

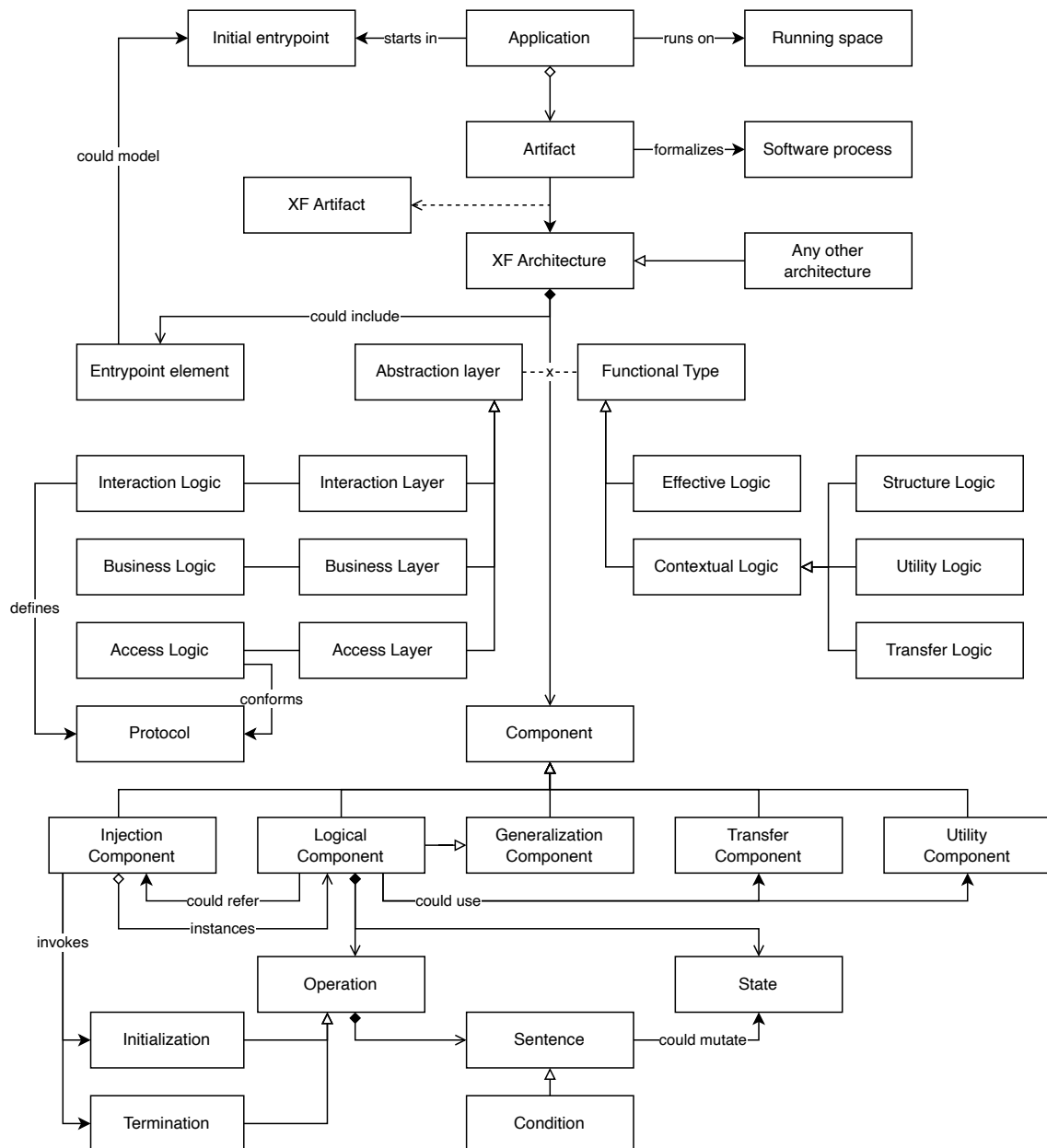


Figure 12. Mapa conceptual del modelo XF: sus conceptos fundamentales — artefacto, arquitectura, capas, tipos, componentes, operaciones, sentencias y estado — y las relaciones semánticas que los vinculan.

Referencia: §5.2.

Etapas de procesamiento Segunda etapa invariante de todo proceso formal. Comprende la ejecución ordenada y determinista de las operaciones que transforman las entradas en el resultado esperado. En el modelo XF, esta etapa se corresponde biunívocamente con la Capa de Negocio del artefacto.

Referencia: §5.2.

Etapas de acceso Tercera etapa invariante de todo proceso formal. Comprende la delegación de parte del procesamiento en sistemas externos al proceso, a través de

un protocolo de comunicación definido. En el modelo XF, esta etapa se corresponde biunívocamente con la Capa de Acceso del artefacto.

Referencia: §5.2.

Sinonimia heterotécnica Fenómeno descriptivo del problema que el modelo XF aborda: el uso de nombres distintos para designar el mismo concepto arquitectónico en marcos de desarrollo diferentes — por ejemplo, **Service** en una tecnología y **Manager** en otra para el mismo tipo de componente. La sinonimia heterotécnica imposibilita la comunicación arquitectónica entre equipos formados en tradiciones tecnológicas distintas y constituye una de las raíces de la fragmentación del ecosistema.

Referencia: §5.4.

Homonimia interframework Fenómeno descriptivo del problema que el modelo XF aborda: el uso del mismo nombre para designar conceptos arquitectónicos distintos en marcos de desarrollo diferentes — por ejemplo, **Service** con acepción de componente de lógica de negocio en una tecnología y con acepción de componente de exposición sistémica en otra. La homonimia interframework introduce ambigüedad en las discusiones arquitectónicas entre equipos y constituye una de las raíces de la fragmentación del ecosistema.

Referencia: §5.4.

Acoplamiento semántico al marco de desarrollo Fenómeno descriptivo del problema que el modelo XF aborda: la dependencia estructural de la arquitectura del artefacto respecto a las convenciones, nomenclaturas y mecanismos del marco de desarrollo empleado. Un artefacto con acoplamiento semántico al marco de desarrollo no puede ser portado a otra tecnología sin reconceptualización arquitectónica completa, y su revisión exige conocimiento específico de las convenciones del marco. El modelo XF se construye precisamente para suprimir este acoplamiento, manteniendo la arquitectura como capa estable por encima del marco.

Referencia: §5.4, §6.2.3.

Convergencia implícita Fenómeno descriptivo observado por el modelo XF: la tendencia espontánea de la industria a converger hacia el mismo conjunto de categorías de componente y la misma estratificación de capas, con independencia del marco de desarrollo empleado. La convergencia implícita es la evidencia empírica que justifica la viabilidad de un modelo de referencia arquitectónico agnóstico: si la industria converge de hecho, la formalización del patrón de convergencia es no solo posible sino económicamente eficiente. El modelo XF hace explícita y normativa esta convergencia.

Referencia: §5.4.

Artefacto, capas y matriz $L \times T$

Artefacto En el modelo XF, un artefacto es la unidad de software que automatiza un proceso formal completo. Todo artefacto tiene exactamente tres capas de

abstracción — Acceso, Negocio e Interacción — y un único espacio de ejecución. El término “artefacto” en el modelo XF es más preciso que el término de uso general “aplicación” o “sistema” — un artefacto es una unidad arquitectónica autocontenida que puede ser instanciada, inicializada y terminada de forma independiente.

Referencia: §6.1.

Aplicación Unidad ejecutable resultante de cargar uno o más artefactos en un mismo espacio de ejecución, típicamente un único proceso del sistema operativo. El término *aplicación* designa la unidad funcional ejecutable; el término *artefacto* designa la unidad estructural autocontenida que la aplicación carga y compone. Una aplicación puede componerse de un único artefacto XF o integrar varios — propios o externos — conforme a los mecanismos prescritos en §10.2.

Referencia: §10.2.

Modelo de referencia arquitectónico Conjunto de decisiones de diseño aplicables a un dominio de problemas recurrentes que, al ser adoptadas, establecen restricciones sobre la arquitectura de los sistemas que las implementan y favorecen determinadas cualidades del sistema resultante. El modelo XF es un modelo de referencia arquitectónico en este sentido — no es un patrón de diseño ni un estilo arquitectónico, sino una abstracción conceptual que establece un vocabulario común, una estructura de organización universal y un conjunto de principios rectores aplicables a cualquier artefacto software.

Referencia: §6.

Capa de abstracción División horizontal del artefacto que agrupa los componentes que implementan la misma etapa del proceso formal que el artefacto automatiza. Cada capa tiene responsabilidades exclusivas, límites bien definidos y dependencias unidireccionales respecto a las capas adyacentes. El modelo XF define exactamente tres capas de abstracción — Acceso, Negocio e Interacción — que estratifican la estructura interna del artefacto en una dimensión ortogonal a las siete capas del modelo OSI (ISO/IEC 7498-1).

Referencia: §6.1, §6.2.2, §5.5.

Capa de Acceso Capa de abstracción del modelo XF correspondiente a la etapa de acceso del proceso formal. Contiene todos los componentes que encapsulan la comunicación con sistemas externos al artefacto. Es la única capa del modelo XF que tiene contacto directo con la Capa de Presentación del modelo OSI (Nivel 6). Su responsabilidad es la abstracción de protocolo, la normalización de datos y la propagación de errores de comunicación.

Referencia: §7.2.1.

Capa de Negocio Capa de abstracción del modelo XF correspondiente a la etapa de procesamiento del proceso formal. Contiene todos los componentes que implementan la lógica de dominio del artefacto. Es la capa que define la identidad funcional del artefacto — qué hace, qué conceptos gestiona y qué operaciones sobre

esos conceptos son posibles. Es la única capa del modelo que puede mantener estado de dominio.

Referencia: §7.2.2.

Capa de Interacción Capa de abstracción del modelo XF correspondiente a la etapa de interacción del proceso formal. Contiene todos los componentes que definen los puntos de entrada al artefacto — cómo los usuarios y sistemas externos pueden invocar sus operaciones de negocio. Existe en todo artefacto XF aunque puede estar vacía de componentes lógicos en artefactos sin interacción externa.

Referencia: §7.2.3.

Componente Unidad atómica de clasificación del modelo XF. Toda unidad de código con identidad propia, nombre único dentro del artefacto y clasificación definida en la matriz $L \times T$. En paradigmas orientados a objetos, un componente XF se corresponde con una clase. En paradigmas funcionales, con un módulo. Todo componente pertenece a exactamente una capa y exactamente un tipo.

Referencia: §7.1, §11.2.1.

Tipo funcional Clasificación de un componente según la función que desempeña dentro de su capa. El modelo XF define exactamente cinco tipos funcionales — Lógico, Generalización, Inyección, Utilidad y Transferencia — que forman un sistema de clasificación cerrado y exhaustivo.

Referencia: §6.2.4, §7.3.

Matriz $L \times T$ Estructura de clasificación bidimensional del modelo XF formada por el producto cartesiano del conjunto de capas $L = \{\text{Acceso, Negocio, Interacción}\}$ y el conjunto de tipos funcionales $T = \{\text{Lógico, Generalización, Inyección, Utilidad, Transferencia}\}$. Produce quince categorías de clasificación posibles. Todo componente de un artefacto XF pertenece a exactamente una celda de la matriz.

Referencia: §7.1.

Dimensión de abstracción Primera dimensión ortogonal de la matriz $L \times T$ del modelo XF. Estratifica el artefacto en tres capas — Acceso, Negocio e Interacción — derivadas de las etapas invariantes del proceso formal que el artefacto automatiza. Es la dimensión que ordena los componentes según el nivel de cercanía a las fronteras del artefacto.

Referencia: §6.1, §6.2.2.

Dimensión funcional Segunda dimensión ortogonal de la matriz $L \times T$ del modelo XF. Tipifica los componentes de cada capa en cinco categorías funcionales — Lógico, Generalización, Inyección, Utilidad y Transferencia — con responsabilidades disjuntas. Es la dimensión que ordena los componentes según la función que desempeñan dentro de su capa.

Referencia: §6.2.4, §7.3.

Función de clasificación ϕ Función que asigna a cada componente del artefacto su posición en la matriz $L \times T$. Se denota $\phi: \mathbb{C} \rightarrow L \times T$. La función ϕ es total sobre el conjunto de componentes del artefacto — todo componente tiene exactamente una clasificación; ningún componente pertenece a más de una categoría.

Referencia: §6.2.4, §11.2.2.

Componentes lógicos y lógica

Componente lógico Tipo funcional que implementa la lógica efectiva de la capa a la que pertenece. Es la unidad funcional principal de cada capa — el único tipo que puede mantener estado en el sentido formal del modelo y el único cuyas instancias son gestionadas por el componente de inyección. Nomenclatura canónica: sufijo **Repository** en la Capa de Acceso, **Business** en la Capa de Negocio, **Service** o **View** en la Capa de Interacción.

Referencia: §7.3.1.

Repository (componente lógico) Componente lógico de la Capa de Acceso. Encapsula la comunicación con un sistema externo concreto del que el artefacto delega parte de su procesamiento. Aplica abstracción de protocolo y normalización de datos sobre la información que cruza la frontera del artefacto. Nomenclatura canónica: sufijo **Repository**. El modelo XF amplía la acepción del patrón *Repository* de Fowler [12] para cubrir todo acceso a sistema externo — persistente, remoto, de mensajería, de hardware o de otro entorno —, no solo el acceso a persistencia (§7.4).

Referencia: §7.3.1, §7.2.1.

Negocio (componente lógico) Componente lógico de la Capa de Negocio. Implementa la lógica efectiva de un concepto del dominio que el artefacto automatiza — sus operaciones de negocio, sus condiciones de negocio y el estado de dominio asociado al concepto —. Nomenclatura canónica: sufijo **Business**.

Referencia: §7.3.1, §7.2.2.

Servicio (componente lógico) Componente lógico de la Capa de Interacción que define un punto de interacción sistémica — un mecanismo a través del cual un sistema externo puede invocar operaciones del artefacto mediante un protocolo de comunicación definido. Nomenclatura canónica: sufijo **Service**.

Referencia: §7.3.1, §7.2.3.

Vista (componente lógico) Componente lógico de la Capa de Interacción que define un punto de interacción gráfica — un elemento visual a través del cual un usuario humano puede invocar operaciones del artefacto o recibir información de él. Combina composición, estilo y lógica de presentación. Nomenclatura canónica: sufijo **View**.

Referencia: §7.3.1, §7.2.3.

Lógica efectiva Conjunto mínimo de operaciones que justifican la existencia de un componente lógico en el artefacto — aquellas que no pueden ser abstraídas en ningún otro tipo de componente sin perder la especificidad del concepto que el componente modela. La lógica efectiva es inherente al concepto que el componente modela y no puede existir en ningún otro componente sin perder su identidad. Se distingue de la lógica contextual en que es irreducible al componente concreto.

Referencia: §7.3.1.

Lógica contextual Lógica auxiliar que sirve de soporte a la consecución de la lógica efectiva de un componente lógico, pero que puede ser compartida con otros componentes, reutilizada o expresada de forma genérica. La lógica contextual pertenece a los componentes de generalización, utilidad o transferencia según su naturaleza — no al componente lógico. Se subdivide formalmente en tres especies según el tipo de componente que la aloja: **lógica estructural** en los componentes de generalización, **lógica de utilidad** en los componentes de utilidad y **lógica de transferencia** en los componentes de transferencia.

Referencia: §7.3.1.

Lógica estructural Especie de lógica contextual encapsulada en un componente de generalización, que abstrae un patrón de comportamiento estructural común entre dos o más componentes lógicos de la **misma capa** y se transmite a éstos por herencia. Es **paramétrica respecto al dominio**: su lógica aplica con independencia del concepto concreto que cada componente lógico que la hereda modela. La presencia en un componente de generalización de lógica ligada a un concepto concreto del dominio del artefacto indica que ha dejado de ser lógica estructural y debe trasladarse al componente lógico al que pertenezca.

Referencia: §7.3.2.

Lógica de utilidad Especie de lógica contextual encapsulada en un componente de utilidad. Provee operaciones auxiliares — funciones puras, sin estado, sin efectos laterales — que pueden ser invocadas por cualquier componente de la capa a la que el componente de utilidad pertenece. A diferencia de la lógica estructural, no se transmite por herencia sino por invocación directa.

Referencia: §7.3.1, §7.3.4.

Lógica de transferencia Especie de lógica contextual encapsulada en un componente de transferencia. Comprende exclusivamente la representación estructural del concepto que el componente modela — declaración de atributos, restricciones de tipo, validaciones sintácticas — y opcionalmente operaciones de transformación entre representaciones equivalentes del mismo concepto. No incluye lógica de comportamiento sobre el dominio.

Referencia: §7.3.1, §7.3.5.

Estado Conjunto de datos de un componente lógico que tienen correspondencia directa con información del dominio de negocio y que determinan el comportamiento

observable del componente ante las operaciones que lo invocan. El estado no incluye atributos operativos internos del componente — referencias a conexiones, flags de control, contadores de reintentos. Solo los componentes lógicos pueden mantener estado en el sentido formal del modelo.

Referencia: §7.3.1.

Operación Subconjunto de la lógica efectiva de un componente lógico que se ejecuta de forma atómica ante un evento o invocación, produciendo una transformación del estado del componente, un resultado para el invocante, o ambos. Cada operación de un componente lógico corresponde a exactamente una acción bien definida sobre el concepto que el componente modela.

Referencia: §7.3.1.

Sentencia Cada uno de los pasos lógicos que componen una operación. Las sentencias se ejecutan de forma ordenada y su conjunto constituye la implementación completa de la acción que la operación modela. Una sentencia puede ser una transformación de datos, una invocación a un componente de capa inferior, una evaluación de una condición, o una mutación del estado del componente.

Referencia: §7.3.1.

Condición Sentencia específica que evalúa el estado del componente o los parámetros de entrada de una operación antes de ejecutar la lógica efectiva. Las condiciones constituyen las precondiciones de ejecución de la operación — si una condición no se satisface, la operación propaga típicamente una excepción al invocante, si bien otras formas de gestión — valor por defecto, reintento — son admisibles cuando la semántica del proceso lo justifica. El modelo XF recomienda que las condiciones se evalúen al inicio de la operación, idealmente antes de cualquier otra sentencia.

Referencia: §7.3.1.

Componentes de generalización, inyección, utilidad y transferencia

Componente de generalización Tipo funcional que abstrae comportamiento estructural común entre dos o más componentes lógicos de la misma capa. No implementa lógica efectiva de ningún concepto concreto del dominio — su lógica es paramétrica respecto al dominio y no tiene utilidad independiente de los componentes lógicos que lo heredan. Su alcance está estrictamente limitado a la capa en la que está definido.

Referencia: §7.3.2.

Singleton Gathering Patrón de diseño implementado por los componentes de inyección del modelo XF. Extensión del patrón Singleton que agrega en un único punto las instancias únicas de un conjunto de componentes relacionados, garantizando la unicidad de instancia de cada componente lógico de la capa y proveyendo un punto de acceso nombrado y estable a cada una de esas instancias.

Referencia: §7.3.3.

Componente de inyección Tipo funcional que implementa el patrón Singleton Gathering para los componentes lógicos de su capa. Es el único punto de acceso a los componentes lógicos de su capa desde cualquier otro componente del artefacto. Es una clase no instanciable con referencias estáticas públicas inmutables a las únicas instancias de los componentes lógicos de su capa. El modelo XF define exactamente tres componentes de inyección — R para la Capa de Acceso, B para la Capa de Negocio, A para la Capa de Interacción.

Referencia: §7.3.3.

R Nombre canónico del componente de inyección de la Capa de Acceso. Agrega las instancias únicas de todos los Repositorios del artefacto y gestiona su ciclo de vida. Es el único punto de acceso a los datos externos del artefacto desde la Capa de Negocio y desde la propia Capa de Acceso.

Referencia: §7.3.3.

B Nombre canónico del componente de inyección de la Capa de Negocio. Agrega las instancias únicas de todos los Negocios del artefacto y gestiona su ciclo de vida. Es el único punto de acceso a la lógica de dominio del artefacto desde la Capa de Interacción y desde la propia Capa de Negocio.

Referencia: §7.3.3.

A Nombre canónico del componente de inyección de la Capa de Interacción. Agrega las instancias únicas de todos los servicios y vistas del artefacto y gestiona su ciclo de vida. Es el único punto de acceso para el consumo del artefacto por otras Aplicaciones y Usuarios humanos.

Referencia: §7.3.3.

Slot (de inyección) Referencia estática pública e inmutable, declarada en un componente de inyección, que apunta a la instancia única de un componente lógico de la capa correspondiente. Es el mecanismo prescrito por el modelo para la materialización del patrón Singleton Gathering: el conjunto de slots de un componente de inyección constituye la enumeración exhaustiva de los componentes lógicos accesibles de su capa.

Referencia: §7.3.3.

Inmutabilidad post-inicialización Propiedad estructural del componente de inyección que prescribe que la composición de slots — identidad de los componentes lógicos referenciados — no puede modificarse una vez que el artefacto ha alcanzado el estado INITIALIZED. La inmutabilidad post-inicialización es la base sobre la que descansan la unicidad de instancia y la estabilidad de las referencias a lo largo del ciclo de vida del artefacto.

Referencia: §7.3.3, §8.2.

Componente de utilidad Tipo funcional que provee operaciones auxiliares de alcance local a su capa. Sus operaciones son funciones puras — sin estado, sin efectos laterales, sin dependencias de otros componentes. No es instanciable ni heredable. Los componentes de utilidad sobre tipos primitivos definidos en la Capa de Acceso constituyen la única excepción al alcance local — pueden ser referenciados desde cualquier capa. Nomenclatura canónica: sufijo **Utils**.

Referencia: §7.3.4.

Tipo primitivo Tipo de dato primitivo en el sentido de ISO/IEC/IEEE 24765:2017 [20]: tipo del que se construyen otros tipos, presente como tipo base en la mayoría de lenguajes de programación. En el modelo XF, los tipos primitivos entran al artefacto a través de la Capa de Acceso — frontera con la Capa de Presentación del modelo OSI (Nivel 6) —, son anteriores a cualquier transformación semántica y constituyen la materia prima sobre la que operan todas las capas. Los formatos de intercambio del sistema operativo o del entorno de ejecución cuya estructura está determinada por la plataforma se equiparan a tipos primitivos a efectos del modelo.

Referencia: §7.3.4, §7.2.1.

Componente de transferencia Tipo funcional que modela la forma de la información en un punto concreto del flujo de procesamiento del artefacto. Puede declarar operaciones autocontenidas sobre sus propios datos, pero no modela procesos de negocio ni accede a otros componentes; sus atributos describen el concepto del dominio del artefacto que modela: un concepto de negocio (**User**, **Order**), un concepto de acceso a un sistema externo (**Query**, **HttpRequest**) o un concepto de interacción con un consumidor (**Button**, **MouseEvent**). Es el único tipo de componente que puede cruzar fronteras de capa, sujeto a las restricciones de direccionalidad y no modificación estructural. Nomenclatura canónica: nombre del concepto sin sufijo; sufijo **Exception** como convención recomendada para transferencias que se transmiten típicamente como excepciones del lenguaje.

Referencia: §7.3.5.

Transferencia heredada Tipo de datos definido por un marco de desarrollo, kit del sistema operativo o librería integrada que un componente XF resultante recibe, devuelve o propaga sin canonizar a una transferencia propia del artefacto. A efectos del modelo, toda transferencia heredada que aparece en una firma de operación XF es un componente de transferencia ($\mathbb{C}_T$) clasificable en la matriz $L \times T$, aunque su identificador no siga la nomenclatura canónica del modelo. Permite alcanzar $\lambda = 3$ aunque el código maneje identificadores externos.

Referencia: §10.2.1.

Estructura En el modelo XF, agrupación de tipos primitivos u otras estructuras que modela un concepto del dominio del artefacto. Sinónimo de componente de transferencia en su acepción estructural. El modelo XF unifica bajo este único término la proliferación de categorías semánticas tradicionales — **DTO**, **Entity**, **Model**, **POJO**, **Record**, **VO**, **Schema** — que designaban el mismo tipo de elemento con distintos nombres según el contexto de uso.

Referencia: §9.2.

Estructura, nomenclatura y principios rectores

Estructura de carpetas canónica Representación física de la matriz $L \times T$ en el sistema de ficheros del proyecto. Prescribe la organización de carpetas y subcarpetas que todo artefacto XF debe implementar, de forma que la arquitectura sea visible e inspeccionable sin necesidad de herramientas adicionales. Su presencia y organización correcta es condición necesaria para la conformidad del artefacto.

Referencia: §7.4.

Raíz de artefacto Carpeta de primer nivel del sistema de ficheros que contiene el artefacto. Es el ancla sobre el que opera la estructura de carpetas canónica del modelo. Su nombre no está prescrito por el modelo. Debe contener exclusivamente la subcarpeta `/src` y el punto de inicio de ejecución, junto con cualquier elemento técnico del marco de desarrollo o del lenguaje empleado.

Referencia: §7.4.

Nomenclatura canónica Conjunto de nombres y patrones de nombrado prescritos por el modelo XF para cada categoría de componente. La nomenclatura canónica es parte del modelo y debe respetarse con independencia de las convenciones de nombrado del lenguaje o marco de desarrollo empleado. Su consistencia es condición necesaria para la interoperabilidad semántica entre equipos.

Referencia: §7.1.

Principio de agnosticismo tecnológico Principio rector del modelo XF que establece que el modelo no impone restricciones sobre el lenguaje de programación, el marco de desarrollo, el paradigma de programación, la plataforma de ejecución ni el dominio de negocio del artefacto. Todo artefacto que automatice un proceso formal puede ser organizado según el modelo XF con independencia de cualquier consideración tecnológica.

Referencia: §6.2.1.

Principio de aislamiento entre capas Principio rector del modelo XF que establece que todo componente de un artefacto XF depende exclusivamente de componentes clasificados en su misma capa o en capas de menor nivel de abstracción. Las dependencias entre capas son estrictamente unidireccionales en sentido descendente. Los componentes de transferencia se rigen por esta misma direccionalidad; la unificación de estructuras entre capas (§9.3) opera dentro del principio de aislamiento, no como excepción a él, sujeta a las restricciones de direccionalidad y no modificación.

Referencia: §6.2.2.

Principio de precedencia de la arquitectura sobre la herramienta Principio rector del modelo XF que establece que la estructura arquitectónica de un artefacto XF deriva de las propiedades del proceso formal que automatiza, y no de las convenciones

del marco de desarrollo, el lenguaje de programación o la plataforma de ejecución empleados. El marco de desarrollo es el instrumento mediante el cual se implementa la arquitectura — no su origen ni su determinante.

Referencia: §6.2.3.

Principio de tipificación cerrada y exhaustiva Principio rector del modelo XF que establece que el sistema de clasificación de componentes es simultáneamente cerrado — el conjunto de tipos reconocidos es finito y no extensible — y exhaustivo — todo componente de cualquier artefacto software puede ser clasificado en exactamente una de las quince categorías de la matriz $L \times T$.

Referencia: §6.2.4.

Estratificación, ciclo de vida y flujos de datos

Dependencia En el modelo XF, toda relación entre dos componentes A y B en la que A requiere conocer la definición o el comportamiento de B para poder ser compilado, instanciado o ejecutado. Abarca invocación de operaciones, herencia, declaración de atributos del tipo B, y referencia al módulo o espacio de nombres de B. Toda dependencia está sujeta al principio de aislamiento entre capas.

Referencia: §6.2.2.

Herencia Relación de dependencia entre un componente lógico y un componente de generalización de la misma capa, mediante la cual el componente lógico adquiere el comportamiento estructural abstraído por el componente de generalización. En el modelo XF, la herencia está estrictamente restringida a componentes de la misma capa — ningún componente puede heredar de un componente de una capa distinta.

Referencia: §7.3.2.

Instanciación Creación de un objeto en memoria mediante el constructor de un componente. En el modelo XF, la instanciación de los componentes lógicos es una operación exclusiva del componente de inyección de su capa — se produce en tiempo de carga de la clase mediante la declaración de las referencias estáticas del componente de inyección. La instanciación se distingue de la inicialización — la instanciación crea el objeto, la inicialización lo prepara para operar.

Referencia: §7.3.3, §8.3.

Inicialización Ejecución de la lógica de arranque de un componente lógico — establecimiento de conexiones con sistemas externos, carga de configuración, arranque de tareas periódicas, establecimiento del estado inicial. La inicialización se implementa en la operación `init()` del componente lógico y es invocada por el componente de inyección de su capa en el orden que las dependencias entre componentes requieran. Se distingue de la instanciación — la inicialización prepara el componente para operar, no lo crea.

Referencia: §7.3.3, §8.3.

Terminación Ejecución de la lógica de cierre de un componente lógico — liberación de recursos adquiridos durante la inicialización, cierre de conexiones, persistencia del estado que deba preservarse entre ejecuciones. La terminación se implementa en la operación `terminate()` del componente lógico y es invocada por el componente de inyección en orden inverso al de inicialización.

Referencia: §7.3.3, §8.3.

Protocolo En el modelo XF, el conjunto de reglas y formatos que rigen la comunicación entre dos artefactos software a través de un canal de comunicación. La gestión de los protocolos es responsabilidad estructural de las dos capas frontera del artefacto, en roles complementarios: la **Capa de Interacción** *define y expone* el protocolo por el que los consumidores externos invocan al artefacto; la **Capa de Acceso** *consume* los protocolos publicados por los sistemas externos a los que el artefacto delega parte de su ejecución. Ningún componente de la Capa de Negocio puede conocer los detalles de ningún protocolo. El término tiene en el modelo XF una acepción más amplia que en la terminología OSI: incluye tanto protocolos de red como protocolos de acceso local — ficheros del sistema, buses de datos, consola de comandos.

Referencia: §7.2.1 y §7.2.3.

Abstracción de protocolo Propiedad de los componentes lógicos de la Capa de Acceso por la cual los detalles del protocolo de comunicación con sistemas externos quedan confinados en el componente y son invisibles para las capas superiores. La abstracción de protocolo es la propiedad que hace posible el principio de agnosticismo tecnológico a nivel de datos — un cambio de protocolo queda confinado en el repositorio correspondiente sin requerir modificaciones en la Capa de Negocio ni en la Capa de Interacción.

Referencia: §7.2.1.

Normalización de datos Transformación sintáctica aplicada por los componentes de la Capa de Acceso sobre los datos recibidos de sistemas externos — convierte el formato de protocolo en componentes de transferencia estructurados e interpretables por la Capa de Negocio. La normalización es exclusivamente sintáctica — no aplica transformaciones semánticas sobre los datos.

Referencia: §7.2.1.

Ontología del negocio Modelo conceptual que define el conjunto estructurado de conceptos clave tratados por el artefacto, sus relaciones, las operaciones permitidas sobre ellos y las reglas que los gobiernan. La ontología del negocio es la descripción formal del dominio que el artefacto automatiza y está representada en el artefacto XF por el conjunto de componentes lógicos de la Capa de Negocio.

Referencia: §7.2.2.

Operación de negocio Subconjunto de la lógica efectiva de un componente lógico de negocio que se ejecuta de forma atómica ante un evento o invocación, produciendo

una transformación del estado del componente, un resultado para el invocante, o ambos.

Referencia: §7.2.2.

Condición de negocio Sentencia de una operación de negocio que evalúa el estado del componente o los parámetros de entrada antes de ejecutar la lógica efectiva. Constituye la precondition de ejecución de la operación. Si no se satisface, la operación propaga una excepción de negocio.

Referencia: §7.2.2.

Excepción de negocio Componente de transferencia que modela una condición de error propia del dominio del artefacto — una condición de negocio no satisfecha o una dependencia de la Capa de Acceso no resuelta — y que se transmite típicamente como excepción del lenguaje. Por convención del modelo (§9.5, no requisito estructural) lleva el sufijo `Exception`. Las excepciones nativas del lenguaje de implementación (`Error`, `Exception`, `BaseException`, etc.) son, a efectos de la axiomatización XF, componentes de transferencia equivalentes (§10.2.1, transferencias heredadas), y no requieren ser envueltas en un componente propio del artefacto.

Referencia: §7.2.2.

Excepción de comunicación Componente de transferencia originado en la Capa de Acceso que modela una condición de error en la comunicación con un sistema externo — fallo de red, error de protocolo, indisponibilidad del sistema —. Se transmite típicamente como excepción del lenguaje y debe ser interceptada por la Capa de Negocio, que decide si propagarla, transformarla en excepción de negocio o intentar una recuperación dentro de la lógica de dominio. Su propagación sin intermediación hasta la Capa de Interacción no es admisible bajo el principio de aislamiento entre capas (§6.2.2).

Referencia: §7.2.2, §9.5.

Punto de entrada En el modelo XF, todo mecanismo a través del cual un consumidor del artefacto — usuario humano o sistema externo — puede invocar las operaciones de negocio que el artefacto implementa. Los puntos de entrada son responsabilidad exclusiva de la Capa de Interacción. El término tiene en el modelo XF una acepción más precisa que en la terminología general — no designa el inicio de ejecución del proceso sino los mecanismos de acceso externo al artefacto.

Referencia: §7.2.3.

Lógica de presentación Conjunto de algoritmos y comprobaciones que determinan cómo debe comportarse una vista ante los datos que recibe y los eventos que se producen sobre ella — qué debe mostrarse, cuándo y en qué estado. No incluye reglas de negocio ni acceso a sistemas externos.

Referencia: §7.2.3.

Estado de presentación Conjunto de datos de un componente lógico de la Capa de Interacción que determina su comportamiento visual o de respuesta ante los eventos que recibe. No tiene correspondencia directa con información del dominio de negocio — modela el contexto dinámico de la interacción.

Referencia: §7.2.3.

Punto de inicio de ejecución Fragmento de código que el entorno de ejecución invoca para arrancar el proceso del artefacto — `main`, `App.main()`, `index.ts` o equivalente según el lenguaje. Es el primer código del desarrollador que recibe el control del entorno; su única responsabilidad es delegar la inicialización XF y, al final, la terminación. No es un componente clasificable en la matriz $L \times T$ y reside fuera de la raíz del artefacto. No debe confundirse con el punto de entrada (responsabilidad de la Capa de Interacción) ni con el elemento de punto de inicio XF (constructo opcional interno al artefacto que orquesta la inicialización centralizada).

Referencia: §8.1.

Elemento de punto de inicio XF Elemento de código opcional del artefacto denominado XF que encapsula las operaciones de inicialización y terminación de los tres componentes de inyección en un único punto de control centralizado. *No es un componente*: no se clasifica en la matriz $L \times T$, queda fuera del dominio de la función de clasificación ϕ , pero es elemento verificable de tipo *elemento de código*, sobre el que operan las reglas del Grupo 8 del catálogo (§11.3). Su presencia en un artefacto es una declaración explícita de que ese artefacto implementa el modelo XF.

Referencia: §8.3.

Ciclo de vida del artefacto Secuencia de estados por los que transita un artefacto XF desde que el entorno de ejecución cede el control al código del desarrollador hasta que el proceso del artefacto finaliza. El modelo XF formaliza tres estados — DEFINED, INITIALIZED y TERMINATED — y cinco transiciones entre ellos.

Referencia: §8.2.

Estado defined Estado inicial del artefacto XF. Los componentes existen como estructuras de código pero no están inicializados. Ningún componente lógico puede ser accedido. El artefacto entra en este estado en el momento en que el entorno de ejecución cede el control al código del desarrollador.

Referencia: §8.2.

Estado initialized Estado activo del artefacto XF. Todos los componentes están inicializados, sus dependencias están resueltas y el artefacto procesa interacciones. La estructura de componentes es inmutable durante este estado. El artefacto entra en este estado cuando `XF.init()` completa satisfactoriamente.

Referencia: §8.2.

Estado terminated Estado final del artefacto XF. Los recursos han sido liberados y el proceso puede finalizar. Es un estado absorbente — una vez alcanzado, el artefacto no puede volver al estado INITIALIZED sin un nuevo arranque.

Referencia: §8.2.

Orden de inicialización Secuencia en la que los componentes lógicos de un artefacto XF son inicializados. Está determinada por las relaciones de dependencia entre componentes y debe ser una ordenación topológica del grafo de dependencias que satisface dos restricciones — la restricción de capa y la restricción de dependencia intra-capa.

Referencia: §8.3.

Secuencia canónica de inicialización Orden prescrito por el modelo XF para invocar las operaciones de inicialización de los componentes lógicos de un artefacto. Se determina como ordenación topológica del grafo de dependencias entre componentes restringida por el principio de aislamiento entre capas: los componentes de la Capa de Acceso se inicializan antes que los de la Capa de Negocio, y estos antes que los de la Capa de Interacción. Dentro de cada capa, el orden satisface las dependencias intra-capa.

Referencia: §8.3.

Secuencia canónica de terminación Orden prescrito por el modelo XF para invocar las operaciones de terminación de los componentes lógicos de un artefacto. Es el inverso de la secuencia canónica de inicialización: los componentes de la Capa de Interacción se terminan antes que los de la Capa de Negocio, y éstos antes que los de la Capa de Acceso.

Referencia: §8.3.

Dependencia circular Ciclo en el grafo de dependencias entre componentes que haría imposible determinar un orden de inicialización válido. Las dependencias circulares inter-capa no se manifiestan en un artefacto XF bien formado como consecuencia del principio de aislamiento entre capas (§6.2.2, operacionalizado por el Grupo 2 del catálogo): la aciclicidad inter-capa es propiedad derivada del modelo, no prescripción independiente.

Referencia: §8.3.

Flujo de datos Mecanismo de transmisión de información entre los componentes de un artefacto XF durante su ejecución. El modelo XF reconoce dos tipos de flujo según su dirección — ascendente o descendente — y formaliza la vía de comunicación como concepto único, con independencia del constructo sintáctico que el lenguaje proporcione para entregar el dato al invocador.

Referencia: §9.1.

Flujo descendente Flujo de datos que va de capas de mayor abstracción hacia capas de menor abstracción — la Capa de Interacción invoca a la Capa de Negocio, que invoca a la Capa de Acceso. Es el flujo de invocación prescrito por el principio de aislamiento entre capas.

Referencia: §9.1.

Flujo ascendente Flujo de datos que va de capas de menor abstracción hacia capas de mayor abstracción. En el modelo XF, el flujo ascendente nunca emplea invocación directa ascendente; se realiza típicamente mediante el patrón de observación.

Referencia: §9.1.

Patrón observador Mecanismo característico — recomendado, no exclusivo — para implementar el flujo ascendente entre capas. Un componente de capa superior se registra como observador de un componente de capa inferior; el componente inferior notifica al observador la ocurrencia de eventos relevantes sin conocer su identidad concreta. El observador recibe la notificación a través de una interfaz definida por el componente de capa superior. El patrón observador es la realización habitual de la comunicación ascendente; lo que el modelo prohíbe por el principio de aislamiento es la dependencia estructural ascendente y, con ella, la invocación directa ascendente.

Referencia: §9.1.

Homogeneidad estructural de las transferencias Propiedad normativa según la cual toda transferencia que circula por la vía de comunicación de una operación pertenece al mismo dominio único $\mathbb{C}_T$, con independencia del propósito del dato que transporta — resultado previsto o condición anómala — y del constructo sintáctico que el lenguaje emplee para entregarla. La distinción de propósito es exclusivamente de implementación: no constituye clases de transferencia disjuntas ni altera la clasificación en $L \times T$ ni las reglas que gobiernan al componente de transferencia.

Referencia: §9.5.

Vía de comunicación Medio por el que un componente entrega información a su invocador durante la ejecución de una operación. El modelo XF formaliza la vía de comunicación como concepto único: independientemente del constructo sintáctico que el lenguaje proporcione (`return`, `throw`, `Result<T, E>`, retorno multi-valor, variables globales como `errno`), todos materializan el mismo medio de transmisión de componentes de transferencia $C_T \in \mathbb{C}_T$. La distinción entre el propósito del dato transmitido — resultado previsto o condición anómala — es exclusivamente de implementación; no genera asimetría estructural en el dominio formal y las reglas de direccionalidad y transformación estructural aplican indistintamente con independencia del constructo elegido por el implementador.

Referencia: §9.5.

Unificación de estructuras de datos Capacidad emergente del modelo XF: un mismo componente de transferencia puede ser compartido por todas las capas que trabajen con el mismo concepto del dominio del artefacto sin necesidad de duplicarlo

por capa. Es una capacidad habilitada por la exención del aislamiento estricto entre capas que el modelo otorga a los componentes de transferencia, sujeta a la direccionalidad de las transferencias (§9.4) y a la regla de transformación estructural (§9.4). El modelo recomienda la unificación como buena práctica de diseño, pero no la impone como prescripción del catálogo.

Referencia: §9.3.

Regla de transformación estructural Prescripción del modelo XF que establece que si una capa modifica la definición estructural de un componente de transferencia proveniente de una capa inferior — añadiendo o eliminando atributos — debe definir un nuevo componente de transferencia en su propia capa. La modificación de los valores de los atributos no activa esta obligación.

Referencia: §9.3.

Compatibilidad, conformidad y sufijos canónicos

Espacio de ejecución Contexto computacional en el que los componentes de un artefacto operan — la memoria, los recursos del sistema y el contexto de proceso que el artefacto ocupa durante su ejecución. Dos artefactos comparten el mismo espacio de ejecución si sus componentes pueden referenciarse directamente en memoria sin serialización ni transmisión a través de ningún canal de comunicación externo. El espacio de ejecución es el criterio que determina el nivel de integración entre artefactos.

Referencia: §10.2.

Compatibilidad Propiedad del modelo XF que prescribe cómo un artefacto XF integra componentes de otros artefactos: reconociendo la responsabilidad funcional del componente integrado, asignándole una celda de su propia matriz $L \times T$ y aplicando el mecanismo prescrito para esa celda y ese tipo de componente. Es bidireccional — aplica indistintamente a la integración de componentes externos en un artefacto XF y a la integración de componentes XF en artefactos externos — y no invasiva: ningún artefacto integrado debe modificarse para hacer posible su integración.

Referencia: §10.1, §10.2.

Componente integrado Componente proveniente de un artefacto distinto al artefacto XF que lo integra. Por el principio de compatibilidad, el componente integrado se clasifica en la matriz $L \times T$ del artefacto integrador conforme a su responsabilidad funcional, con independencia de la clasificación que el componente tenga en su artefacto original. La clasificación del componente integrado es una decisión del artefacto integrador, no del componente.

Referencia: §10.1, §10.2.

Reformulación Operación de reorganización que hace conforme al modelo XF, sin alterar el comportamiento funcional observable, bien un artefacto preexistente en su

conjunto, bien una funcionalidad no normalizada en XF que se integra reformulándola en un componente XF propio (§10.2.1). Opera sobre la estructura de ficheros, la nomenclatura, las dependencias declaradas y la clasificación de los componentes; no constituye reescritura del código de los componentes lógicos. Es la vía prescrita para que un artefacto preexistente alcance niveles de conformidad superiores a cero.

Referencia: §10.1, §10.2.1.

Mapeo tecnológico Documento complementario a la especificación normativa del modelo que prescribe cómo se materializan los conceptos del modelo XF en una tecnología concreta — lenguaje, marco de desarrollo o plataforma. Los mapeos tecnológicos resuelven la tensión específica entre las prescripciones del modelo y las restricciones del entorno técnico. Son externos a la especificación normativa del modelo y no extienden el conjunto de conceptos del modelo.

Referencia: §6.2.3.

Integración a nivel de aplicación Forma de integración entre artefactos con espacios de ejecución disjuntos, que intercambian información a través de un protocolo de comunicación establecido. Es independiente de la arquitectura interna de cada artefacto.

Referencia: §10.1.

Integración a nivel de artefacto Forma de integración entre artefactos que comparten espacio de ejecución, comunicándose directamente a través de las interfaces de sus componentes sin intermediación de ningún protocolo de red. Las restricciones de capa del modelo XF aplican sobre las dependencias entre artefactos integrados a este nivel con la misma fuerza que sobre las dependencias internas.

Referencia: §10.1.

Librería XF Artefacto software que implementa el modelo XF y que expone, a través de sus interfaces públicas, componentes de cualquier tipo — generalización, transferencia, utilidad y, cuando modela un dominio concreto, también componentes lógicos a través de sus componentes de inyección — para su consumo por otros artefactos XF. Se recomienda que sus interfaces sean idempotentes e independientes de la tecnología de implementación.

Referencia: §10.3.

Regla de conformidad Condición verificable que operacionaliza una prescripción normativa del modelo XF en forma evaluable sobre un elemento verificable del artefacto. Toda regla tiene un identificador canónico en formato kebab-case y se caracteriza por su **verificabilidad** — estructural (decidible por análisis estático determinista) o semántica (requiere interpretación de la responsabilidad funcional) —. El incumplimiento de cualquier regla del catálogo produce una violación que afecta al nivel de conformidad del artefacto. El catálogo cerrado y exhaustivo se desarrolla en §11.3.

Referencia: §11.1, §11.3.

Identificador canónico Cadena en formato kebab-case que identifica de forma única a una regla del catálogo de conformidad. Constituye la clave de referencia entre la prescripción normativa, las herramientas de análisis estático y los informes de conformidad emitidos sobre un artefacto. Es estable a lo largo del ciclo de vida del modelo: una revisión del catálogo puede añadir reglas con nuevos identificadores pero no reasignar un identificador existente a una regla distinta.

Referencia: §11.1, §11.3.

Catálogo (de reglas) Conjunto cerrado y exhaustivo de reglas de conformidad prescritas por el modelo XF, agrupado por grupos temáticos. Su exposición en lenguaje natural se desarrolla en §11.3. Un artefacto es conforme al modelo si y solo si satisface todas las reglas del catálogo aplicables a su contenido.

Referencia: §11.1, §11.3.

Verificabilidad estructural Propiedad de una regla del catálogo de conformidad que indica que la regla es decidible por análisis estático determinista sobre la estructura de ficheros, la nomenclatura y las dependencias declaradas del artefacto. Las reglas con verificabilidad estructural pueden implementarse de forma agnóstica al lenguaje y al marco de desarrollo. La pertenencia a esta clase es propiedad declarada de cada regla del catálogo.

Referencia: §11.1, §11.3.

Verificabilidad semántica Propiedad de una regla del catálogo de conformidad que indica que la regla requiere interpretación del contenido de los componentes y no es decidible por análisis exclusivamente estructural. Su implementación en herramientas no es agnóstica al lenguaje.

Referencia: §11.1, §11.3.

Elemento verificable Todo elemento del artefacto sobre el que una regla de conformidad puede ser evaluada. El modelo XF reconoce dos tipos de elementos verificables — elemento de código y elemento estructural — que son disjuntos entre sí.

Referencia: §11.1.1.

Elemento estructural Tipo de elemento verificable que corresponde a toda carpeta o subcarpeta del sistema de ficheros del artefacto que forma parte de su estructura de directorios.

Referencia: §11.1.1.

Violación Constatación de que un elemento verificable del artefacto no satisface la condición prescrita por una regla del catálogo de conformidad. Toda violación está asociada a exactamente una regla y exactamente un elemento verificable. La presencia de cualquier violación impide que el artefacto alcance el nivel 4.

Referencia: §11.1.2.

Totalidad del código Condición que un artefacto satisface cuando todo componente del artefacto — excepto el elemento **XF** — está clasificado en la matriz $L \times T$ mediante la función de clasificación ϕ . Es condición necesaria para que el artefacto alcance el nivel de conformidad 2, 3 o 4.

Referencia: §11.1.3.

Función de conformidad Λ Función que asigna a cada artefacto su nivel de conformidad con el modelo XF. Se denota $\Lambda: \mathfrak{A} \rightarrow \{0, 1, 2, 3, 4\}$; su resultado para un artefacto concreto se denota $\lambda = \Lambda(\mathfrak{A})$.

Referencia: §11.4.

Nivel de conformidad Propiedad discreta de un artefacto que determina en qué medida satisface las prescripciones normativas del modelo XF. Toma valores en el conjunto $\{0, 1, 2, 3, 4\}$ — No conforme, Parcialmente conforme, Imperfectamente conforme, Estructuralmente conforme y Perfectamente conforme. Es calculable de forma determinista a partir del conjunto de violaciones del artefacto y de la condición de totalidad del código.

Referencia: §11.2.

Artefacto bien formado Artefacto que satisface estructuralmente las prescripciones mínimas del modelo XF necesarias para que las propiedades formales del modelo — direccionalidad de las capas, totalidad de la función de clasificación, unicidad de los componentes de inyección — estén garantizadas por construcción. Es el supuesto bajo el cual se derivan los teoremas del modelo, por ejemplo, la aciclicidad inter-capas del grafo de dependencias (propiedad derivada del modelo). Un artefacto con nivel de conformidad 3 o 4 es necesariamente bien formado.

Referencia: §8.3, §11.4.

Repository Sufijo canónico prescrito por el modelo XF para el nombre de los componentes lógicos de la Capa de Acceso (§7.3.1). Aplica también a los componentes de generalización que abstraen comportamiento estructural de componentes lógicos de esta capa. Ejemplos: `DatabaseRepository`, `IdentityRepository`, `FileRepository`.

Referencia: §7.3.1, §7.2.1.

Business Sufijo canónico prescrito por el modelo XF para el nombre de los componentes lógicos de la Capa de Negocio (§7.3.1). Aplica también a los componentes de generalización que abstraen comportamiento estructural de componentes lógicos de esta capa. Ejemplos: `UserBusiness`, `SessionBusiness`, `TemperatureBusiness`.

Referencia: §7.3.1, §7.2.2.

Service Sufijo canónico prescrito por el modelo XF para el nombre de los componentes lógicos de la Capa de Interacción que implementan un punto de interacción

sistémica (§7.3.1). Aplica también a los componentes de generalización correspondientes. Ejemplos: `TemperatureService`, `AuthService`, `ScheduleService`.

Referencia: §7.3.1, §7.2.3.

View Sufijo canónico prescrito por el modelo XF para el nombre de los componentes lógicos de la Capa de Interacción que implementan un punto de interacción gráfica (§7.3.1). Aplica también a los componentes de generalización correspondientes. Ejemplos: `LoginView`, `MainView`, `ThermometerView`.

Referencia: §7.3.1, §7.2.3.

Utils Sufijo canónico prescrito por el modelo XF para el nombre de los componentes de utilidad (§7.3.4) de cualquier capa. Operacionaliza la regla `utility-naming` del catálogo de conformidad. Ejemplos: `StringUtilsils`, `DateUtils`, `TemperatureUtils`.

Referencia: §7.3.4.

Exception Sufijo canónico prescrito por el modelo XF para el nombre de los componentes de transferencia (§7.3.5) del subtipo excepción. Aplica a cualquier capa según el origen de la excepción. Ejemplos: `NetworkException`, `AuthenticationException`, `ValidationException`.

Referencia: §7.3.5.

13 Discusión

Las cláusulas internas tienen carácter informativo. Su propósito es analizar las consecuencias prácticas de aplicar el modelo XF a los proyectos software de la industria, organizadas en cinco vectores de ventaja. Las limitaciones del modelo se enuncian en §14.1; la proyección formal de las arquitecturas de referencia consolidadas en la literatura se recoge en §D.3.

13.1 Comunicación y vocabulario común

Todo artefacto software cumple, en su naturaleza profunda, una función comunicativa: comunica al equipo lo que el sistema hace, al desarrollador que llega lo que está construido y a la máquina lo que debe ejecutarse. La fragmentación terminológica documentada en §5.4 — sinonimia heterotécnica y homonimia interframework — compromete sistemáticamente esa función comunicativa: equipos con desarrolladores formados en tradiciones tecnológicas distintas no pueden discutir arquitectura con el mismo vocabulario, aunque la estructura subyacente sea idéntica.

El modelo XF aporta a la comunicación arquitectónica tres niveles de mejora que se refuerzan entre sí. El primero es el **vocabulario único y agnóstico**: cada componente del artefacto pertenece a una celda concreta de la matriz $L \times T$, identificada por nomenclatura canónica. Un desarrollador que diga “componente lógico de la Capa de Negocio con dependencia a un repositorio” está describiendo con precisión la misma realidad arquitectónica con independencia de la tecnología de implementación, eliminando la traducción mental que las nomenclaturas propietarias imponen. La

discusión de diseño deja de cargar el sobre coste de aclarar a qué se refiere cada término en cada marco de desarrollo.

El segundo es la **taxonomía cerrada y exhaustiva**. La industria converge implícitamente hacia las mismas categorías de componente porque los procesos formales que automatiza tienen las mismas etapas invariantes (§5.3); el modelo hace visible esa convergencia y la cierra: la matriz $L \times T$ produce quince categorías posibles, cinco tipos en tres capas, en las que se clasifica cada componente del artefacto. La revisión arquitectónica del código deja de ser un juicio interpretativo y pasa a ser una verificación de pertenencia: ¿está este componente en la celda que su responsabilidad funcional dicta?

El tercero es la **precisión documental por construcción**. La estructura de carpetas canónica (§7.4) y la nomenclatura prescrita constituyen documentación arquitectónica embebida en el propio artefacto, indistinguible del código que documenta y, en la medida en que el artefacto satisface el catálogo, actualizada con él por construcción. Un desarrollador que abre un proyecto XF reconoce su estructura sin necesidad de leer documentación externa: las capas están donde se prescribe, los tipos donde se prescribe, los componentes con los nombres que se prescriben. La documentación tradicional — wikis, READMEs, diagramas — pasa de ser una capa paralela susceptible de quedar desactualizada a una capa complementaria que documenta lo específico del dominio, no lo que el modelo ya prescribe.

Estas tres mejoras tienen un corolario práctico: las descripciones de trabajo en metodologías ágiles — Definition of Work y Definition of Done — ganan en elocuencia y pierden en ambigüedad cuando se expresan en el vocabulario del modelo. Una tarea descrita como “implementar una operación en un componente lógico de la Capa de Negocio que verifica una precondition y delega en un repositorio de la Capa de Acceso” es interpretable de forma unívoca por cualquier desarrollador formado en el modelo, y mensurable contra el catálogo de reglas. La brecha de comunicación entre análisis e implementación — fuente histórica de inconsistencias en proyectos software — queda acotada por el vocabulario formal compartido.

13.2 Reducción de costes operativos

Las consecuencias operativas de la fragmentación terminológica documentadas en §1.1 se traducen en costes concretos: incremento del tiempo de cambio de contexto entre proyectos con tecnologías distintas, incremento del tiempo de incorporación de nuevos desarrolladores, inconsistencias en la implementación de funcionalidades equivalentes y deuda técnica derivada de la dependencia estructural del artefacto respecto al marco de desarrollo. El modelo XF actúa sobre cada uno de esos vectores de coste. La [Tabla 14](#) resume el contraste entre los mecanismos tradicionales y los mecanismos prescritos por el modelo para cada vector, e identifica la propiedad del modelo que lo habilita; las cláusulas siguientes desarrollan en detalle cada uno.

Table 14. Vectores de coste operativo del desarrollo de software y su tratamiento bajo cada paradigma. Para cada vector, se contrasta el mecanismo característico del enfoque tradicional con el mecanismo prescrito por el modelo XF y se identifica la propiedad del modelo que habilita la reducción de coste. La tabla es informativa y opera como mapa de lectura de las cláusulas siguientes.

Vector de coste	Mecanismo tradicional	Mecanismo XF	Propiedad del modelo que lo habilita
Cambio de contexto entre tecnologías	Convenciones específicas del marco de desarrollo; reaprendizaje arquitectónico por proyecto.	Estructura de capas, tipos, nomenclatura y restricciones idénticas con independencia de la tecnología.	Principio de agnosticismo tecnológico (§6.2.1).
Incorporación de nuevos desarrolladores	Tiempo proporcional a la singularidad de las convenciones del proyecto.	Aprendizaje acotado a la lógica de dominio; convenciones arquitectónicas universales.	Nomenclatura canónica y estructura de carpetas canónica (§7.3, §7.4).
Estimación y planificación de costes	Estimación experta o por semejanza, sin unidad arquitectónica bien definida.	Coste agregado como suma de costes unitarios de los componentes clasificados en la matriz $L \times T$.	Tipificación cerrada y exhaustiva (§6.2.4).
Mantenimiento y evolución	Cambios tecnológicos propagan a múltiples capas; deuda técnica acumulativa.	Cambio acotado a la capa correspondiente sin afectar al resto del artefacto.	Principio de aislamiento entre capas (§6.2.2).

Cambio de contexto entre tecnologías Un desarrollador formado en el modelo XF que cambia de proyecto entre tecnologías distintas se enfrenta a una curva de aprendizaje acotada exclusivamente al sustrato técnico — sintaxis del lenguaje, idiomatismos del marco de desarrollo, herramientas de construcción. La estructura arquitectónica es la misma en todos los artefactos XF, con independencia de la tecnología: las mismas tres capas, los mismos cinco tipos, la misma nomenclatura, las mismas restricciones de dependencia. El coste de orientación arquitectónica — históricamente proporcional a la singularidad de las convenciones del proyecto — tiende a anularse porque las convenciones son universales.

Incorporación de nuevos desarrolladores Análogamente, la incorporación de un nuevo miembro al equipo se acelera porque la estructura del artefacto es predecible. El desarrollador no necesita aprender las convenciones específicas del proyecto antes de poder contribuir — necesita aprender la lógica del dominio que el artefacto modela, que es lo único genuinamente nuevo. La capacidad de un equipo para escalar mediante la incorporación de personas formadas en el modelo se desacopla de la curva de aprendizaje específica del proyecto, eliminando un cuello de botella clásico en organizaciones con alta rotación o crecimiento sostenido.

Estimación y planificación de costes de desarrollo La clasificación obligatoria de los componentes del artefacto en la matriz $L \times T$ — producida durante el análisis funcional — abre una vía de estimación de costes que los métodos tradicionales no soportan. Cada componente identificado y clasificado constituye una unidad de trabajo con un coste de implementación estimable de forma independiente; la suma de los costes unitarios produce el coste agregado del artefacto sin recurrir al criterio experto, a la estimación por semejanza ni a la ponderación subjetiva por caso de uso. El mapa de componentes XF se convierte así en un instrumento de planificación con trazabilidad exacta de la cantidad de código desarrollado y pendiente de desarrollar, con una precisión que los métodos tradicionales no alcanzan precisamente porque carecen de una unidad de coste arquitectónicamente bien definida.

Mantenimiento y evolución Siempre que se preserve el contrato del componente de acceso, el principio de aislamiento entre capas (§6.2.2) garantiza por construcción que un cambio localizado en una capa no se propaga ascendentemente: la sustitución de un mecanismo de acceso a un sistema externo no afecta a la lógica de negocio ni a la capa de interacción. Esta propiedad reduce el coste de evolución del artefacto a lo largo de su ciclo de vida: cuando una decisión tecnológica queda obsoleta — un protocolo de comunicación cambia, un sistema externo se reemplaza, una librería se discontinúa — el impacto se acota a la capa correspondiente sin requerir reescritura del artefacto completo. La deuda técnica deja de acumularse de forma estructural y pasa a localizarse en la frontera tecnológica donde se originó.

13.3 Normalización y estandarización

La disciplina de la ingeniería del software, pese a su madurez relativa, carece históricamente de un equivalente a la regulación normativa que caracteriza a las disciplinas de ingeniería consolidadas. La constitución del campo de la *software architecture* como agenda de investigación con vocabulario y problemas propios fue delineada formalmente por Garlan [15]; el modelo XF se inscribe en esa agenda y se posiciona como un paso en la dirección normativa: un modelo de referencia arquitectónico en el sentido de la norma ISO/IEC/IEEE 42010:2022 [22], en continuidad con la tradición de estandarización a la que pertenecen el modelo OSI [17], los vocabularios consolidados de la ingeniería del software [20], los procesos del ciclo de vida del software [19] y los modelos de calidad del software ISO/IEC 25010:2023 [24].

La continuidad con el modelo OSI no es solo formal sino pragmática: lo que OSI aportó a la comunicación entre sistemas — vocabulario común que sustituyó las nomenclaturas propietarias, interfaces independientes de la implementación y validación cruzada que habilitó la enseñanza sistemática — el modelo XF lo aporta a la arquitectura interna del artefacto. Las dos estratificaciones son ortogonales y conceptualmente complementarias: OSI estandariza la comunicación entre artefactos; XF estandariza la estructura interna de cada artefacto.

Meta-modelo agnóstico El modelo XF actúa como meta-modelo respecto a las arquitecturas de referencia previas. La proyección formal de Clean Architecture, Domain-Driven Design, Arquitectura Hexagonal, Arquitectura Onion y arquitecturas estratificadas en la matriz $L \times T$ — desarrollada en §D.3 — demuestra que estas arquitecturas son instancias parciales de la misma estructura subyacente. El modelo

aporta sobre el estado del arte la integración simultánea de tres prescripciones que las arquitecturas previas formulan de forma parcial o independiente: estructura de capas derivada de primeros principios, taxonomía cerrada y exhaustiva de tipos de componente y nomenclatura canónica tecnológicamente agnóstica.

Verificabilidad determinista La conformidad de un artefacto con el modelo es calculable algorítmicamente por herramientas de análisis estático. El catálogo de reglas (§11.3) y el algoritmo de determinación del nivel de conformidad (§11.4) materializan esta verificabilidad: dado un artefacto y el catálogo, el nivel de conformidad es una función del artefacto, no de la interpretación del analista. Esta propiedad es la base sobre la que se construyen los procesos de auditoría arquitectónica reproducibles, requisito recurrente en las disciplinas de ingeniería con tradición normativa consolidada y materializado en marcos como ISO/IEC/IEEE 12207 [19], que prescribe la verificación reproducible en los procesos del ciclo de vida del software.

Independencia tecnológica como criterio de durabilidad Las arquitecturas acopladas a tecnologías concretas envejecen con esas tecnologías. El modelo XF, derivado de propiedades invariantes del proceso formal y no de las convenciones de ningún marco de desarrollo, ofrece a la industria una capa de abstracción cuya validez es independiente del ciclo de vida de cualquier tecnología particular. Esta propiedad lo cualifica como base estable de estandarización a largo plazo: una organización que adopta el modelo no apuesta por la longevidad de un marco concreto sino por una abstracción cuya vigencia está atada a la naturaleza misma del software como automatización de procesos formales.

13.4 Aplicación en modelos generativos de código

La generación de código asistida por sistemas de lenguaje de gran escala ha emergido como un vector de transformación de la industria del software contemporánea. Las propiedades formales del modelo XF — vocabulario único, taxonomía cerrada, verificabilidad determinista — confieren al modelo una relevancia singular en este contexto. Las consecuencias que se desarrollan a continuación son hipótesis derivadas de las propiedades formales del modelo y de las prescripciones de las cláusulas anteriores (§6, §11, §12); su validación empírica controlada queda como línea de trabajo futura.

Contexto arquitectónico de coste constante Los sistemas de generación de código operan sobre una ventana de contexto finita que representa el estado del artefacto sobre el que se trabaja. La calidad del código generado depende directamente de la calidad y la completitud del contexto que el sistema puede mantener. En un artefacto sin convenciones arquitectónicas estandarizadas, el sistema generador debe inferir la estructura, las relaciones entre componentes y las convenciones del equipo antes de producir código coherente — coste de inferencia variable, proporcional a la singularidad del proyecto, sujeto a errores cuando las convenciones son ambiguas o inconsistentes.

En un artefacto conforme al modelo XF, ese coste de inferencia tiende hacia una constante: la estructura de carpetas canónica, la nomenclatura prescrita y las

restricciones de dependencia son las mismas en cualquier artefacto conforme al modelo, con independencia del dominio y la escala. El sistema generador puede dedicar el espacio de contexto disponible a la lógica de dominio — lo único genuinamente específico del artefacto — en lugar de a la inferencia de convenciones arquitectónicas. La consecuencia es directa: mayor coherencia del código generado, menor probabilidad de violaciones estructurales y mayor capacidad efectiva de generación a escala industrial.

Vocabulario de especificación funcional La ontología formal del modelo (§12) actúa como un lenguaje de especificación funcional de alta precisión y baja ambigüedad. Una especificación expresada en el vocabulario del modelo — “un componente lógico en la Capa de Negocio con una operación que verifica una precondition y delega en un repositorio” — describe con exactitud qué existe, dónde reside, qué hace y de qué depende, sin ambigüedad interpretativa.

Esta precisión semántica habilita el paradigma denominado *Specification-Driven Development* [37] — en el que la especificación funcional formal precede y dirige algorítmicamente la generación del código — con un nivel de formalidad que las taxonomías propietarias no admiten. La especificación funcional, expresada en el vocabulario del modelo, deja de ser un texto sujeto a interpretación y pasa a ser un objeto formal transformable algorítmicamente en código. El generador no infiere la estructura — la recibe como parte de la especificación — y se concentra en la lógica del dominio. El analista funcional y el generador comparten el mismo modelo mental porque comparten el mismo vocabulario formal.

Verificación en tiempo de generación La verificabilidad determinista del catálogo (§11.3) cierra el ciclo de generación-verificación-corrección dentro del propio proceso de generación. Un sistema generador que conoce el modelo XF puede evaluar en tiempo de generación si el código producido satisface el catálogo antes de entregarlo al desarrollador, reduciendo las iteraciones humano-en-el-bucle a las dimensiones que requieren juicio arquitectónico de alto nivel y trasladando al sistema generador las verificaciones de adecuación estructural que históricamente han recaído en revisión humana. La especificación, la implementación y la verificación comparten el mismo vocabulario estructural: la transformación entre ellas se reduce a instanciación de ese vocabulario en la sintaxis correspondiente a cada nivel.

13.5 Cierre de brecha conceptual entre humano y código generado

La generación de código por sistemas de inteligencia artificial introduce un desafío de creciente relevancia en la industria: la brecha conceptual entre el código producido por la máquina y la capacidad del desarrollador humano para comprenderlo, revisarlo, mantenerlo y evolucionarlo. Esta brecha no es de competencia técnica del desarrollador sino de homogeneidad estructural del código: cuando cada generación produce un estilo arquitectónico distinto — influenciado por la diversidad de patrones presentes en los datos de entrenamiento y por la ausencia de convenciones explícitas en el contexto — el lector humano debe descifrar las convenciones implícitas de cada fragmento antes de poder operar sobre él. El coste de comprensión sustituye al coste

de redacción que la generación había suprimido, anulando parcialmente la ganancia de productividad.

La consecuencia es un riesgo arquitectónico de naturaleza emergente: a medida que la proporción de código generado en los artefactos crece, la responsabilidad humana sobre su corrección estructural se ve presionada por la incapacidad de inspeccionarlo con rapidez. La auditoría arquitectónica — históricamente una práctica costosa pero acotada a momentos determinados del ciclo de vida — se demanda ahora de forma continua y a velocidades que el flujo humano no puede sostener sin un sustrato estructural que la facilite.

El modelo XF como sustrato común La adopción del modelo XF como contrato arquitectónico del artefacto convierte esa brecha conceptual en una equivalencia operativa entre lector humano y código generado. El humano reconoce la estructura del código generado porque es la estructura canónica del modelo, no una estructura específica del fragmento producido. Las capas están donde el modelo prescribe, los tipos están donde el modelo prescribe, los componentes llevan los nombres que el modelo prescribe. La revisión arquitectónica del código generado deja de requerir descifrar convenciones implícitas y pasa a ser una verificación de pertenencia a celdas conocidas en una matriz conocida. La [Figura 13](#) ilustra esta transformación: a la izquierda, un artefacto sin estructura común sobre el que el revisor humano debe reconstruir las convenciones implícitas de cada fragmento generado; a la derecha, un artefacto conforme al modelo XF cuya estructura canónica es directamente reconocible y reduce la revisión a una verificación de pertenencia a celdas conocidas.

Esta propiedad tiene un paralelismo histórico relevante. La introducción del lenguaje ubicuo en el Domain-Driven Design [11] resolvió la brecha de comunicación entre desarrolladores y expertos del dominio mediante un vocabulario compartido que ambos pudieran utilizar con la misma precisión. El modelo XF resuelve la brecha equivalente entre el desarrollador humano y el sistema generador de código mediante un vocabulario estructural compartido que ambos pueden interpretar con la misma exactitud. El lenguaje ubicuo del DDD operaba sobre el dominio del negocio; el vocabulario del modelo opera sobre el dominio de la arquitectura, completando la cobertura estructural de la comunicación humano-máquina.

Reducción del esfuerzo de auditoría continua La auditoría arquitectónica del código generado se descarga sobre las herramientas de análisis estático de conformidad. La verificación que requiere juicio arquitectónico — la adecuación de la responsabilidad funcional asignada a cada componente, la elección de la celda de clasificación correcta — permanece como tarea humana, pero acotada al subconjunto de decisiones que el catálogo no resuelve algorítmicamente. La verificación estructural — ubicación canónica, nomenclatura, restricciones de dependencia, ciclo de vida — queda completamente automatizada. La razón de auditoría humana/auditoría automatizada se invierte respecto al modelo de revisión tradicional, liberando capacidad humana para las decisiones que genuinamente la requieren.

Mantenimiento y evolución de código no escrito por humanos El reto que la industria empieza a enfrentar de forma creciente — mantener y evolucionar código que ningún humano escribió originalmente — exige que la estructura del código

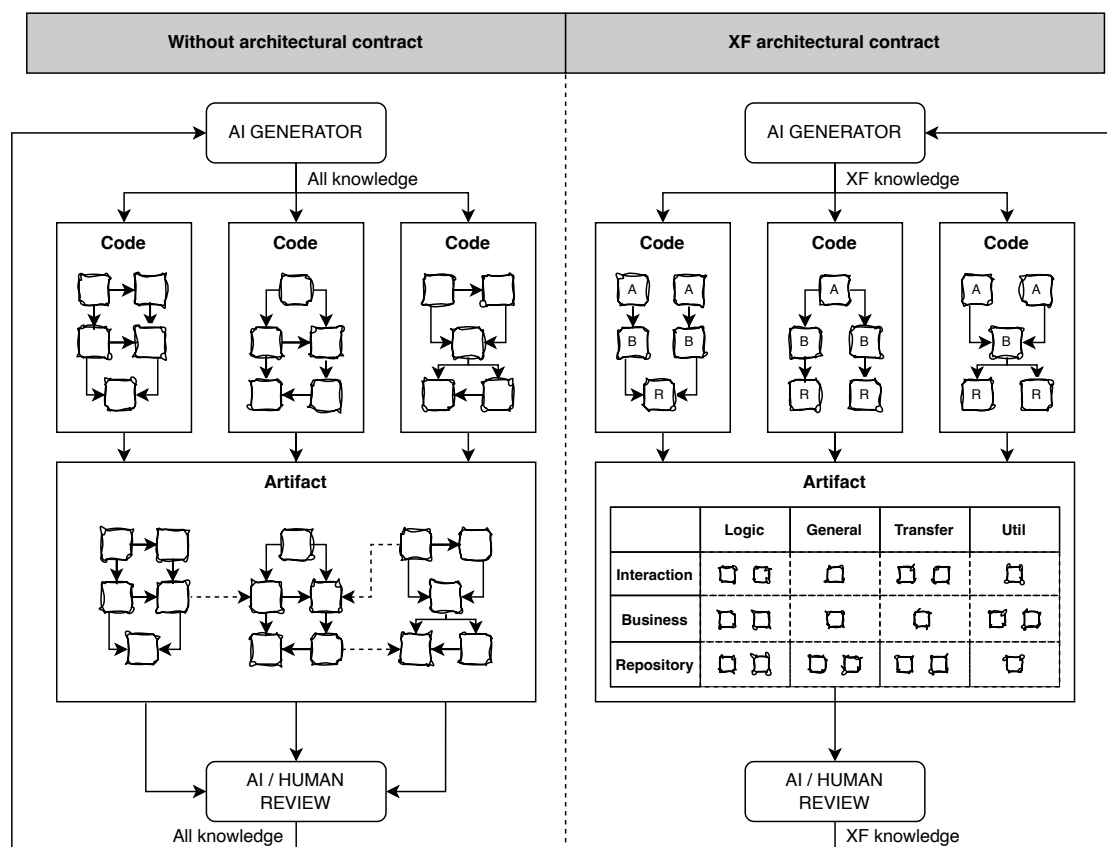


Figure 13. Cierre de la brecha conceptual entre el desarrollador humano y el código generado por sistemas automáticos. A la izquierda, un artefacto sin contrato arquitectónico común: cada fragmento generado adopta convenciones implícitas distintas — nomenclatura, estructura de carpetas, distribución de responsabilidades — y el revisor humano debe descifrarlas antes de poder operar sobre el código, con coste de comprensión variable y proporcional a la heterogeneidad acumulada. A la derecha, un artefacto conforme al modelo XF: las capas, los tipos y la nomenclatura son los canónicos prescritos por el modelo con independencia del fragmento o del sistema generador que lo produjo, y la revisión humana se reduce a verificar la pertenencia de cada componente a su celda correspondiente en la matriz $L \times T$, con coste de comprensión constante.

sea inteligible por construcción, no por la familiaridad acumulada del desarrollador con un estilo específico. El modelo XF aporta esa inteligibilidad estructural: un desarrollador que llega a un artefacto XF generado mayoritariamente por sistemas automáticos puede operar sobre él con la misma facilidad con la que operaría sobre uno escrito por humanos, siempre que ambos satisfagan el contrato arquitectónico del modelo. La autoría deja de ser una propiedad relevante para la mantenibilidad cuando la estructura es uniforme.

Esta propiedad tiene una consecuencia organizativa que merece enunciarse: las decisiones de gobernanza sobre qué código se genera, cómo se revisa y cómo se versiona pueden tomarse sobre bases estructurales objetivas y reproducibles, no sobre criterios cualitativos que dependen de la experiencia del revisor. La adopción del modelo es, en este sentido, una respuesta arquitectónica al riesgo emergente de la generación masiva: convierte en problema técnico verificable lo que actualmente es problema humano sostenido por la pericia individual.

14 Conclusiones

El presente documento ha introducido el modelo de Arquitectura Cross-Framework — XF — como un modelo de referencia arquitectónico que organiza la estructura interna de cualquier artefacto software mediante dos dimensiones ortogonales de clasificación: la dimensión de abstracción, que estratifica el artefacto en tres capas derivadas de las etapas invariantes de todo proceso formal, y la dimensión funcional, que tipifica los componentes de cada capa en cinco categorías con responsabilidades precisas y no solapadas.

La contribución central del modelo puede enunciarse de forma concisa: aborda simultáneamente tres problemas que han caracterizado la fragmentación del ecosistema de desarrollo de software — la ausencia de una estructura de capas universal derivada de primeros principios, la ausencia de una taxonomía de componentes cerrada y exhaustiva, y la ausencia de una nomenclatura canónica común a todas las tecnologías —. Los modelos de referencia previos resuelven cada uno de estos problemas de forma parcial o en dominios tecnológicos específicos; el modelo XF los resuelve de forma integrada, con independencia del lenguaje de programación, del marco de desarrollo, del paradigma de programación, de la plataforma de ejecución y del dominio funcional del artefacto.

Las propiedades formales del modelo — completitud, separación estricta y direccionalidad de las capas; clasificación total de los componentes; prescripción de nomenclatura canónica — hacen posible que la conformidad de un artefacto con el modelo sea calculable de forma determinista por herramientas de análisis estático. El catálogo de reglas (§11.3) y el algoritmo de determinación del nivel de conformidad Λ (§11.4) materializan esa verificabilidad: dado un artefacto y el catálogo, el nivel de conformidad es una función del artefacto, no de la interpretación del analista.

La cláusula de Discusión ha desarrollado las consecuencias prácticas de la adopción del modelo en cinco vectores — comunicación arquitectónica con vocabulario común, reducción de costes operativos, normalización y estandarización, aplicación en modelos generativos de código y cierre de la brecha conceptual entre el desarrollador humano y el código generado por sistemas automáticos — y ha enunciado con honestidad las limitaciones de la especificación actual. La combinación de propiedades formales y consecuencias operativas posiciona al modelo como instrumento de estandarización con vocación industrial y de larga duración, en continuidad con la tradición consolidada de modelos de referencia normativos en ingeniería del software.

14.1 Limitaciones del modelo

El modelo XF, en su versión actual, tiene un conjunto de limitaciones que es necesario enunciar con honestidad. Algunas son limitaciones de la especificación que serán abordadas en versiones futuras; otras son inherentes al enfoque del modelo y no pueden resolverse sin comprometer sus propiedades fundamentales.

Tensión con marcos de inyección de dependencias automática El mecanismo de componentes de inyección del modelo — basado en referencias estáticas inmutables agregadas en un punto único por capa — entra en tensión con los marcos de desarrollo que delegan el ciclo de vida de los componentes a un contenedor del propio marco.

En esos contextos, el contenedor puede crear múltiples instancias, gestionar scopes de visibilidad o resolver dependencias de formas que el modelo no prescribe. La resolución de esta tensión requiere un análisis específico que determine cómo implementar el espíritu del modelo — unicidad de instancia, punto de acceso centralizado, orden de inicialización — dentro de las restricciones del contenedor de cada marco. Este análisis es objeto de documentos de compatibilidad por tecnología, externos a la especificación normativa del modelo.

Verificabilidad semántica de las reglas de contenido El catálogo incluye reglas de contenido — del tipo `logic-mismatch-*` y `general-mismatch-*` — que verifican que un componente no incorpora lógica ajena a su responsabilidad funcional. Estas reglas son conceptualmente precisas pero su verificación automática requiere análisis semántico del código fuente que va más allá del análisis estructural de ficheros y nomenclatura. Su implementación en herramientas requiere conocimiento específico del lenguaje y el ecosistema de librerías empleado y no puede realizarse de forma completamente agnóstica a la tecnología. El modelo reconoce esta limitación y estratifica el catálogo en dos niveles de verificabilidad (estructural y semántica) para que las herramientas puedan reportar la cobertura efectiva del análisis de forma honesta.

Ausencia de validación empírica controlada La validación empírica controlada del impacto de la adopción del modelo no ha sido aún realizada en condiciones reales de desarrollo.

Axiomatización matemática incompleta La versión actual del modelo formaliza los conceptos centrales — la función de clasificación, el cálculo del nivel de conformidad, el orden de inicialización — pero no proporciona una axiomatización exhaustiva de todas las prescripciones.

14.2 Validación empírica pendiente

El modelo XF es una propuesta formal derivada de primeros principios y de la observación sistemática del ecosistema de desarrollo existente. Su fundamentación teórica — el isomorfismo entre proceso formal y artefacto, la derivación de las tres capas a partir de las etapas invariantes, la verificabilidad del catálogo de reglas — ha sido desarrollada con rigor en las cláusulas precedentes. Su validación empírica controlada del impacto de su adopción en contextos reales de desarrollo está, sin embargo, pendiente de realización.

Las hipótesis principales que la validación empírica habría de contrastar son las siguientes.

La **primera hipótesis** es que la adopción del modelo reduce el tiempo de incorporación de nuevos desarrolladores a proyectos existentes. El modelo predice que un desarrollador formado en XF puede orientarse en cualquier artefacto XF con independencia de su dominio tecnológico; si la predicción es correcta, el tiempo de incorporación sería significativamente menor en artefactos XF que en artefactos equivalentes con convenciones propietarias.

La **segunda hipótesis** es que la adopción del modelo reduce la densidad de defectos arquitectónicos en los artefactos. El modelo predice que la clasificación obligatoria de componentes antes de su implementación y la verificación automática de conformidad durante el desarrollo reducen el número de decisiones de diseño incorrectas que se manifiestan como defectos en producción.

La **tercera hipótesis** es que la adopción del modelo reduce el coste de cambio de contexto entre proyectos con tecnologías distintas. El modelo predice que la curva de aprendizaje de un nuevo artefacto XF en una tecnología conocida se limita al aprendizaje de la lógica de dominio y no requiere aprender nuevas convenciones arquitectónicas.

La **cuarta hipótesis** es que la expresión de requisitos funcionales en el vocabulario del modelo reduce el número de iteraciones necesarias entre analistas y desarrolladores para producir una especificación implementable. El modelo predice que la semántica formal del vocabulario XF elimina la ambigüedad interpretativa que caracteriza al lenguaje natural no controlado en la definición de requisitos.

La **quinta hipótesis**, derivada del análisis de la cláusula §13.5, es que la adopción del modelo reduce el coste de auditoría arquitectónica del código generado por sistemas automáticos. El modelo predice que la uniformidad estructural prescrita transforma la revisión humana del código generado en una verificación de pertenencia a celdas conocidas, eliminando el coste de descifrar convenciones implícitas que la generación masiva impone sobre el lector humano.

La [Tabla 15](#) resume las cinco hipótesis con su predicción, una métrica candidata y un diseño de estudio sugerido, como agenda auditable de la línea de validación empírica.

Table 15. Hipótesis de validación empírica del modelo XF, su predicción, una métrica candidata y un diseño de estudio sugerido. La tabla operacionaliza la agenda de investigación empírica del modelo en forma auditable: para cada hipótesis, qué predice el modelo, qué cantidad observable permitiría contrastarla y qué configuración experimental podría producir esa medida.

Hipótesis	Predicción del modelo	Métrica candidata	Diseño de estudio sugerido
H1 Tiempo de incorporación	Reducción significativa del tiempo de orientación de nuevos desarrolladores en artefactos XF respecto a artefactos con convenciones propietarias.	Tiempo medio hasta primera contribución productiva.	Comparativa entre equipos formados y no formados en XF sobre artefactos equivalentes.

Continúa en la página siguiente

Tabla 15 (continuación)

Hipótesis	Predicción del modelo	Métrica candidata	Diseño de estudio sugerido
H2 Densidad de defectos arquitectónicos	La clasificación obligatoria previa y la verificación automática reducen las decisiones de diseño incorrectas que se manifiestan como defectos en producción.	Defectos arquitectónicos por mil líneas de código.	Análisis longitudinal de artefactos XF frente a artefactos con convenciones propietarias en el mismo dominio.
H3 Coste de cambio de contexto	La curva de aprendizaje de un nuevo artefacto XF en una tecnología conocida queda limitada a la lógica de dominio.	Tiempo de orientación arquitectónica al cambiar de proyecto.	Estudio cruzado sobre desarrolladores que se mueven entre artefactos en distintas tecnologías.
H4 Iteraciones analista–desarrollador	La semántica formal del vocabulario XF elimina la ambigüedad interpretativa del lenguaje natural no controlado en la definición de requisitos.	Número de iteraciones por requisito hasta una especificación implementable.	Análisis de tickets de aclaración en proyectos con vocabulario controlado XF frente a proyectos con vocabulario no controlado.
H5 Coste de auditoría del código generado	La uniformidad estructural prescrita transforma la revisión humana del código generado en una verificación de pertenencia a celdas conocidas.	Tiempo de revisión por unidad de código generado.	Estudio comparativo sobre código generado revisado por humanos sobre artefactos con y sin contrato XF.

El diseño de estudios controlados que contrasten estas hipótesis en contextos industriales reales — con grupos de control formados en arquitecturas tradicionales y grupos experimentales formados en el modelo — es la línea de validación empírica de mayor prioridad para el desarrollo futuro del modelo.

14.3 Líneas de trabajo futuro

El modelo XF, en su versión actual, es una especificación normativa completa en los aspectos que cubre, pero deja abiertas líneas de investigación y desarrollo que son objeto de documentos complementarios y de futuras versiones del modelo.

Validación empírica controlada Como se ha desarrollado en la cláusula precedente, la validación empírica controlada del impacto de la adopción del modelo en productividad del equipo, calidad del código producido, coste de incorporación de nuevos desarrolladores, mantenibilidad a largo plazo y reducción de la brecha de auditoría del código generado es la línea de trabajo de mayor prioridad. Los

estudios habrían de diseñarse con rigor metodológico suficiente para ser publicables en revistas de referencia, de forma que la validación empírica pueda citarse junto con la especificación formal como fundamento del modelo. La cuantificación del impacto económico de la fragmentación del ecosistema de desarrollo subraya la relevancia industrial de esta validación.

Investigación sobre el cierre de brecha humano-código generado La brecha conceptual entre el desarrollador humano y el código generado por sistemas automáticos — analizada en la cláusula §13.5 — es un fenómeno emergente cuya magnitud crece con la adopción industrial de la generación asistida. Caracterizarla cuantitativamente, modelar su evolución a medida que la proporción de código generado en los artefactos aumenta y medir el efecto mitigador del modelo XF sobre ese fenómeno constituyen una agenda de investigación independiente y de relevancia inmediata. El cruce entre la teoría de la legibilidad del código, los modelos de carga cognitiva del lector y las propiedades formales del modelo es el espacio natural para esa investigación.

Axiomatización matemática completa La versión actual del modelo incluye expresiones formales para los conceptos centrales pero no proporciona una axiomatización matemática exhaustiva de todas sus prescripciones. La formalización completa en lógica de predicados — que permita demostrar propiedades como la consistencia del sistema de clasificación, la completitud del catálogo de reglas y la corrección del algoritmo de determinación del nivel de conformidad — es una línea de investigación que requiere colaboración con la comunidad de lógica formal e ingeniería del software.

Herramientas de análisis de conformidad El catálogo de reglas y el algoritmo de determinación del nivel de conformidad proveen la especificación completa para la implementación de herramientas de análisis estático de conformidad XF. Las herramientas previstas incluyen analizadores de estructura de carpetas y nomenclatura — implementables de forma tecnológicamente agnóstica —, analizadores de dependencias entre componentes — implementables por lenguaje mediante análisis de importaciones —, analizadores de contenido semántico — implementables mediante análisis de árbol sintáctico abstracto específico por lenguaje —, plugins de entornos de desarrollo que provean retroalimentación en tiempo real y generadores de informes de conformidad que calculen el nivel λ del artefacto.

Proyectos base y generadores de código XF Un conjunto de proyectos base que implementen la estructura de carpetas canónica, los componentes de inyección y las generalizaciones más comunes del modelo para las tecnologías de desarrollo más extendidas, distribuidos como repositorios de código abierto, reduciría el coste de inicio de nuevos artefactos XF y garantizaría que la configuración técnica sea compatible con el modelo desde la primera línea de código. Complementariamente, los generadores de código que reciban una especificación funcional en vocabulario XF y produzcan esqueletos de código conformes al modelo — con componentes correctamente clasificados, ubicados y nombrados, y verificables contra el catálogo — constituyen una línea de desarrollo prometedora especialmente en combinación con sistemas de generación basados en modelos de lenguaje, que aportarían la inferencia de la lógica de dominio sobre el esqueleto arquitectónico prescrito.

Ecosistema de librerías XF El desarrollo de un ecosistema de librerías XF — componentes publicados y distribuidos conforme al modelo, tanto de utilidad como de modelado de dominio (§10.3) — es la condición para que la adopción del modelo sea acumulativa y económicamente eficiente. Las librerías de mayor prioridad son las que abstraen los patrones más recurrentes del desarrollo de software en las tecnologías más extendidas. Su desarrollo como proyectos de código abierto, con verificación de conformidad automática, es la forma más efectiva de impulsar la adopción del modelo en la industria. La trayectoria de referencia es la del modelo OSI: lo que OSI hizo posible para la comunicación entre sistemas — un ecosistema de implementaciones de protocolo interoperables sobre un vocabulario común — el modelo XF lo hace posible para la arquitectura interna del artefacto.

Certificación de conformidad El modelo abre la posibilidad de un sistema de certificación de conformidad — tanto para personas como para artefactos — análogo al que organismos como W3C administran para los estándares web. Una certificación de competencias acreditaría que una persona es capaz de desarrollar artefactos conformes al modelo; un sello de conformidad de artefacto acreditaría que un artefacto satisface las prescripciones del modelo a un nivel determinado. El diseño del sistema de certificación — criterios, organismos certificadores, procesos de auditoría — es una línea de trabajo que trasciende el alcance técnico del modelo y requiere la participación de la comunidad de práctica.

15 Bibliografía

References

- [1] Agha, G. A. (1986). *Actors: A model of concurrent computation in distributed systems*. MIT Press.
- [2] Allen, R., y Garlan, D. (1997). A formal basis for architectural connection. *ACM Transactions on Software Engineering and Methodology*, 6(3), 213–249. <https://doi.org/10.1145/258077.258078>
- [3] Bass, L., Clements, P., y Kazman, R. (2012). *Software architecture in practice* (3.^a ed.). Addison-Wesley Professional.
- [4] Beck, K., Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., Grenning, J., Highsmith, J., Hunt, A., Jeffries, R., Kern, J., Marick, B., Martin, R. C., Mellor, S., Schwaber, K., Sutherland, J., y Thomas, D. (2001). *Manifesto for agile software development*. <https://agilemanifesto.org>
- [5] Bertalanffy, L. von (1968). *General system theory: Foundations, development, applications*. George Braziller.
- [6] Booch, G. (1994). *Object-oriented analysis and design with applications* (2.^a ed.). Benjamin/Cummings.
- [7] Brooks, F. P. (1987). No silver bullet: Essence and accidents of software engineering. *IEEE Computer*, 20(4), 10–19. <https://doi.org/10.1109/MC.1987.1663532>

- [8] Cockburn, A. (2005). *Hexagonal architecture*. alistair.cockburn.us. <https://alistair.cockburn.us/hexagonal-architecture/>
- [9] Cunningham, W. (1992). The WyCash portfolio management system. *ACM SIGPLAN OOPS Messenger*, 4(2), 29–30. <https://doi.org/10.1145/157710.157715>
- [10] Dijkstra, E. W. (1968). The structure of the “THE”-multiprogramming system. *Communications of the ACM*, 11(5), 341–346. <https://doi.org/10.1145/363095.363143>
- [11] Evans, E. (2003). *Domain-driven design: Tackling complexity in the heart of software*. Addison-Wesley Professional.
- [12] Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley Professional.
- [13] Fowler, M. (2004). Inversion of Control Containers and the Dependency Injection pattern. *martinfowler.com*. <https://martinfowler.com/articles/injection.html>
- [14] Gamma, E., Helm, R., Johnson, R., y Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley Professional.
- [15] Garlan, D. (2000). Software architecture: A roadmap. En A. Finkelstein (Ed.), *The Future of Software Engineering* (pp. 91–101). ACM Press. <https://doi.org/10.1145/336512.336537>
- [16] Hoare, C. A. R. (1978). Communicating sequential processes. *Communications of the ACM*, 21(8), 666–677. <https://doi.org/10.1145/359576.359585>
- [17] International Organization for Standardization. (1994). *Information technology — Open Systems Interconnection — Basic Reference Model: The Basic Model* (ISO/IEC 7498-1:1994). ISO/IEC.
- [18] International Organization for Standardization. (2013). *Information technology — Object Management Group Business Process Model and Notation* (ISO/IEC 19510:2013). ISO.
- [19] International Organization for Standardization, International Electrotechnical Commission, e Institute of Electrical and Electronics Engineers. (2017). *Systems and software engineering — Software life cycle processes* (ISO/IEC/IEEE 12207:2017). ISO/IEC/IEEE.
- [20] International Organization for Standardization, International Electrotechnical Commission, e Institute of Electrical and Electronics Engineers. (2017). *Systems and software engineering — Vocabulary* (ISO/IEC/IEEE 24765:2017). ISO/IEC/IEEE.
- [21] International Organization for Standardization. (2021). *ISO/IEC Directives, Part 2: Principles and rules for the structure and drafting of ISO and IEC documents* (9.^a ed.). ISO/IEC.
- [22] International Organization for Standardization, International Electrotechnical Commission, e Institute of Electrical and Electronics Engineers. (2022). *Software*,

- systems and enterprise — Architecture description* (ISO/IEC/IEEE 42010:2022). ISO/IEC/IEEE.
- [23] International Organization for Standardization, International Electrotechnical Commission, e Institute of Electrical and Electronics Engineers. (2023). *Systems and software engineering — System life cycle processes* (ISO/IEC/IEEE 15288:2023). ISO/IEC/IEEE.
- [24] International Organization for Standardization e International Electrotechnical Commission. (2023). *Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model* (ISO/IEC 25010:2023). ISO/IEC.
- [25] Jones, C. (2008). *Applied software measurement: Global analysis of productivity and quality* (3.^a ed.). McGraw-Hill.
- [26] Lewis, J., y Fowler, M. (2014). *Microservices: A definition of this new architectural term*. <https://martinfowler.com/articles/microservices.html>
- [27] Liskov, B. H., y Wing, J. M. (1994). A behavioral notion of subtyping. *ACM Transactions on Programming Languages and Systems*, 16(6), 1811–1841. <https://doi.org/10.1145/197320.197383>
- [28] Mandelbrot, B. B. (1982). *The fractal geometry of nature*. W. H. Freeman.
- [29] Martin, R. C. (2008). *Clean code: A handbook of agile software craftsmanship*. Prentice Hall.
- [30] Martin, R. C. (2017). *Clean architecture: A craftsman's guide to software structure and design*. Prentice Hall.
- [31] Meyer, B. (1992). *Object-oriented software construction* (1.^a ed.). Prentice Hall.
- [32] Milner, R. (1999). *Communicating and mobile systems: The π -calculus*. Cambridge University Press.
- [33] Naur, P., y Randell, B. (Eds.). (1968). *Software engineering: Report on a conference sponsored by the NATO Science Committee*. NATO Scientific Affairs Division.
- [34] Newman, S. (2015). *Building microservices: Designing fine-grained systems*. O'Reilly Media.
- [35] Object Management Group. (2014). *Business process model and notation (BPMN) version 2.0.2* (OMG Document Number: formal/2013-12-09). OMG. <https://www.omg.org/spec/BPMN/2.0.2>
- [36] Object Management Group. (2017). *Unified Modeling Language (UML) specification, version 2.5.1* (OMG Document Number: formal/2017-12-05). OMG. <https://www.omg.org/spec/UML/2.5.1>
- [37] Ostroff, J. S., Makalsky, D., y Paige, R. F. (2004). Agile specification-driven development. En *Extreme Programming and Agile Processes in Software Engineering (XP 2004)* (LNCS 3092, pp. 104–112). Springer.

- [38] Palermo, J. (2008). *The onion architecture*. jeffreypalermo.com. <https://jeffreypalermo.com/2008/07/the-onion-architecture-part-1/>
- [39] Parnas, D. L. (1972). On the criteria to be used in decomposing systems into modules. *Communications of the ACM*, 15(12), 1053–1058. <https://doi.org/10.1145/361598.361623>
- [40] Pierce, B. C. (2002). *Types and programming languages*. MIT Press.
- [41] Shaw, M., y Garlan, D. (1996). *Software architecture: Perspectives on an emerging discipline*. Prentice Hall.
- [42] Szyperski, C. (2002). *Component software: Beyond object-oriented programming* (2.^a ed.). Addison-Wesley Professional.
- [43] Turing, A. M. (1936). On computable numbers, with an application to the Entscheidungsproblem. *Proceedings of the London Mathematical Society*, s2-42(1), 230–265. <https://doi.org/10.1112/plms/s2-42.1.230>
- [44] United States Department of Defense. (1994). *Software development and documentation* (MIL-STD-498). U.S. Department of Defense.
- [45] Wiener, N. (1948). *Cybernetics: Or control and communication in the animal and the machine*. MIT Press.
- [46] Wirth, N. (1971). Program development by stepwise refinement. *Communications of the ACM*, 14(4), 221–227. <https://doi.org/10.1145/362575.362577>
- [47] Yourdon, E., y Constantine, L. L. (1979). *Structured design: Fundamentals of a discipline of computer program and systems design*. Prentice Hall.

A Anexo A — Ejemplos de implementación: Capa de Acceso

Este anexo tiene carácter informativo. Su propósito es ilustrar la aplicación del modelo de Arquitectura XF mediante una implementación de referencia. El contenido de este anexo no constituye requisito normativo.

Este anexo presenta una implementación de referencia de los cinco tipos de componentes de la Capa de Acceso. Los ejemplos pertenecen a un artefacto hipotético de gestión de un termostato conectado: un dispositivo IoT que mide y controla la temperatura ambiente, comunica con un servidor remoto para sincronizar los datos y se gestiona mediante un usuario autenticado. El mismo hilo conductor atraviesa los Anexos B y C. Los ejemplos de los tres anexos (A–C), por simplicidad, no subdividen internamente /`logic`; un artefacto real seguiría las pautas de subdivisión de la capa correspondiente (§7.2.1 para la Capa de Acceso).

Nota sobre el lenguaje de los ejemplos. La elección de TypeScript para los ejemplos de los Anexos A–C es ilustrativa, no normativa: se ha preferido por la elocuencia de su sistema de tipos — interfaces, tipos unión, modificadores de acceso y genéricos — para expresar con precisión la estructura de los componentes, las firmas de las operaciones y las relaciones entre capas sin ruido sintáctico. El modelo XF es agnóstico respecto a la tecnología: cualquier lenguaje orientado a objetos con un

sistema de tipos equivalente materializaría los mismos componentes con idéntica clasificación en la matriz $L \times T$.

A.1 Componentes de transferencia

Los componentes de transferencia de la Capa de Acceso modelan las estructuras de datos tal como las exponen los sistemas externos — antes de cualquier transformación semántica del artefacto — y las excepciones específicas de los fallos de comunicación. Viven en `/src/repository/transfers`.

```
1 // /src/repository/transfers/RawTemperatureRecord.ts
2 export interface RawTemperatureRecord {
3   readonly id: string;
4   readonly value: number;
5   readonly unit: string; // tal como llega del servidor: "C" o "F"
6   readonly recordedAt: string; // ISO-8601 sin transformar
7 }
8
9 // /src/repository/transfers/AuthTokenResponse.ts
10 export interface AuthTokenResponse {
11   readonly token: string;
12   readonly expiresInSeconds: number;
13   readonly refreshToken: string;
14 }
15
16 // /src/repository/transfers/NetworkUnavailableException.ts
17 export class NetworkUnavailableException extends Error {
18   constructor(public readonly endpoint: string, cause?: Error) {
19     super('Network unavailable: ${endpoint}');
20     this.cause = cause;
21   }
22 }
23
24 // /src/repository/transfers/AuthenticationFailedException.ts
25 export class AuthenticationFailedException extends Error {
26   constructor(public readonly reason: string) {
27     super('Authentication failed: ${reason}');
28   }
29 }
```

Código 43. Componentes de transferencia de la Capa de Acceso

A.2 Componentes de utilidad

Las utilidades de la Capa de Acceso operan sobre tipos primitivos heredados de la Capa de Presentación OSI y son, por tanto, accesibles desde cualquier capa del artefacto (§7.3.4). Aquí se ilustra una utilidad de manipulación de cadenas (parseo de fechas) que será reutilizable en todo el artefacto.

```
1 // /src/repository/utis/StringUtils.ts
2 export abstract class StringUtils {
3   private constructor() {}
4
5   static parseIsoDate(s: string): Date {
6     const parsed = new Date(s);
7     if (isNaN(parsed.getTime())) {
8       throw new Error('Invalid ISO-8601 date: ${s}');
9     }
10    return parsed;
11  }
12
13  static isBlank(s: string | null | undefined): boolean {
14    return s === null || s === undefined || s.trim().length === 0;
15  }
16 }
```

Código 44. Componente de utilidad de Acceso sobre tipos primitivos

A.3 Componentes de generalización

Las generalizaciones de la Capa de Acceso abstraen comportamientos comunes entre los Repositorios del artefacto. El ejemplo modela un Repositorio remoto que encapsula el ciclo de petición HTTP, la propagación uniforme de errores de red y la posibilidad de añadir un encabezado de autenticación opcional. Vive en `/src/repository/general`.

```

1 // /src/repository/general/RemoteRepository.ts
2 export abstract class RemoteRepository {
3   protected abstract baseUrl(): string;
4
5   protected async request<T>(<
6     path: string,
7     options: { method: string; body?: unknown; bearerToken?: string } = {
8       method: 'GET' }
9   ): Promise<T> {
10     const url = `${this.baseUrl()}${path}`;
11     const headers: Record<string, string> = { 'Content-Type':
12       'application/json' };
13     if (options.bearerToken) {
14       headers['Authorization'] = `Bearer ${options.bearerToken}`;
15     }
16
17     try {
18       const response = await fetch(url, {
19         method: options.method,
20         headers,
21         body: options.body ? JSON.stringify(options.body) : undefined
22       });
23
24       if (!response.ok) {
25         if (response.status === 401 || response.status === 403) {
26           throw new AuthenticationFailedException(`HTTP ${response.status}`);
27         }
28         throw new NetworkUnavailableException(url);
29       }
30
31       return await response.json() as T;
32     } catch (e) {
33       if (e instanceof AuthenticationFailedException) throw e;
34       if (e instanceof NetworkUnavailableException) throw e;
35       throw new NetworkUnavailableException(url, e as Error);
36     }
37   }
38
39   init(): void {}
40   terminate(): void {}
41 }

```

Código 45. Componente de generalización: RemoteRepository

A.4 Componentes lógicos

Los componentes lógicos de la Capa de Acceso — los Repositorios — encapsulan un protocolo concreto de comunicación con un sistema externo. El ejemplo presenta dos Repositorios: `IdentityRepository` (autenticación contra un servidor de identidad) y `ServerRepository` (sincronización de medidas de temperatura con el servidor de telemetría). Ambos extienden `RemoteRepository`.

La operación `verifyToken` de `IdentityRepository` ilustra el patrón completo: precondiciones (token no vacío), sentencias (delegación a la generalización para la llamada remota, normalización del resultado), retorno tipado por un `AuthTokenResponse`, y transmisión de `AuthenticationFailedException` y `NetworkUnavailableException` como excepciones del lenguaje.

```

1  // /src/repository/logic/IdentityRepository.ts
2  export class IdentityRepository extends RemoteRepository {
3    protected baseUrl(): string {
4      return 'https://identity.example.com/v1';
5    }
6
7    async verifyToken(token: string): Promise<AuthTokenResponse> {
8      // Precondiciones
9      if (StringUtils.isBlank(token)) {
10         throw new AuthenticationFailedException('empty token');
11      }
12
13      // Sentencias: delegación al protocolo remoto
14      const response = await this.request<AuthTokenResponse>('/verify', {
15         method: 'POST',
16         body: { token }
17      });
18
19      // Retorno Transfer
20      return response;
21    }
22
23    async revokeToken(token: string): Promise<void> {
24      if (StringUtils.isBlank(token)) {
25         throw new AuthenticationFailedException('empty token');
26      }
27      await this.request<void>('/revoke', { method: 'POST', body: { token } });
28    }
29  }
30
31  // /src/repository/logic/ServerRepository.ts
32  export class ServerRepository extends RemoteRepository {
33    protected baseUrl(): string {
34      return 'https://telemetry.example.com/v1';
35    }
36
37    async fetchLastTemperature(deviceId: string, bearerToken: string):
38      Promise<RawTemperatureRecord> {
39      if (StringUtils.isBlank(deviceId)) {
40         throw new NetworkUnavailableException('invalid deviceId');
41      }
42      return await this.request<RawTemperatureRecord>(
43        '/devices/${deviceId}/temperature/last',
44        { method: 'GET', bearerToken }
45      );
46    }
47
48    async saveTemperature(
49      deviceId: string,
50      record: RawTemperatureRecord,
51      bearerToken: string
52    ): Promise<void> {
53      await this.request<void>(
54        '/devices/${deviceId}/temperature',
55        { method: 'POST', body: record, bearerToken }
56      );
57    }
58
59    // Retorno primitivo: ejemplo de operación que devuelve un number
60    async countMeasurements(deviceId: string, bearerToken: string):
61      Promise<number> {
62      const result = await this.request<{ count: number }>(
63        '/devices/${deviceId}/temperature/count',
64        { method: 'GET', bearerToken }
65      );
66      return result.count;
67    }
68  }

```

```

65 }
66 }

```

Código 46. Componentes lógicos de Acceso: IdentityRepository y ServerRepository

A.5 Componente de inyección

El componente R agrega las instancias únicas de todos los Repositorios del artefacto y gestiona su ciclo de vida. Es la única superficie de acceso a la Capa de Acceso desde el resto del artefacto.

```

1 // /src/repository/R.ts
2 export abstract class R {
3   // Definición: instancias únicas de los Repositorios del artefacto
4   static readonly identity: IdentityRepository = new IdentityRepository();
5   static readonly server: ServerRepository = new ServerRepository();
6
7   private constructor() {} // no instanciable
8
9   static init(): void {
10    // Orden de dependencias (en este ejemplo no hay dependencias internas)
11    R.identity.init();
12    R.server.init();
13  }
14
15  static terminate(): void {
16    // Orden inverso al de inicialización
17    R.server.terminate();
18    R.identity.terminate();
19  }
20 }

```

Código 47. Componente de inyección de la Capa de Acceso: R

B Anexo B — Ejemplos de implementación: Capa de Negocio

Este anexo tiene carácter informativo. Su propósito es ilustrar la aplicación del modelo de Arquitectura XF mediante una implementación de referencia. El contenido de este anexo no constituye requisito normativo.

Este anexo continúa el hilo del termostato conectado iniciado en el Anexo A y presenta los cinco tipos de componentes de la Capa de Negocio. La Capa de Negocio orquesta la lógica del dominio: gestiona la sesión del usuario, transforma los datos crudos del servidor en estructuras de dominio coherentes, valida las reglas del negocio y mantiene el estado observable de los conceptos del artefacto. El modelo de proceso que materializan estos componentes se especifica en §7.2.2.

B.1 Componentes de transferencia

Los componentes de transferencia de la Capa de Negocio modelan los conceptos del dominio en su forma normalizada y semánticamente significativa. Son distintos de los componentes de transferencia de la Capa de Acceso (que reflejan la forma cruda del servidor): aquí los datos ya tienen tipos del dominio, unidades canónicas y reglas implícitas de coherencia.

```

1 // /src/business/transfers/Temperature.ts
2 export interface Temperature {
3   readonly value: number;
4   readonly unit: 'C' | 'F';
5   readonly recordedAt: Date;
6 }
7
8 // /src/business/transfers/Session.ts
9 export interface Session {
10   readonly token: string;
11   readonly expiresAt: Date;
12   readonly refreshToken: string;
13 }
14
15 // /src/business/transfers/User.ts
16 export interface User {
17   readonly id: string;
18   readonly displayName: string;
19   readonly deviceId: string;
20 }
21
22 // /src/business/transfers/TemperatureOutOfRangeException.ts
23 export class TemperatureOutOfRangeException extends Error {
24   constructor(public readonly value: number, public readonly unit: 'C' | 'F') {
25     super('Temperature ${value}${unit} is out of acceptable range [14C, 32C]');
26   }
27 }
28
29 // /src/business/transfers/SessionExpiredException.ts
30 export class SessionExpiredException extends Error {
31   constructor() {
32     super('Session has expired; refresh required');
33   }
34 }

```

Código 48. Componentes de transferencia de la Capa de Negocio

B.2 Componentes de utilidad

Las utilidades de la Capa de Negocio son funciones puras sobre conceptos del dominio. A diferencia de las utilidades de Acceso (que operan sobre primitivos y son globales), las utilidades de Negocio tienen alcance local: solo se invocan desde componentes de Negocio.

```

1 // /src/business/utils/TemperatureUtils.ts
2 export abstract class TemperatureUtils {
3   private constructor() {}
4
5   static toCelsius(t: Temperature): number {
6     return t.unit === 'F' ? (t.value - 32) * 5 / 9 : t.value;
7   }
8
9   static toFahrenheit(t: Temperature): number {
10    return t.unit === 'C' ? t.value * 9 / 5 + 32 : t.value;
11  }
12
13  static isWithinAcceptableRange(t: Temperature): boolean {
14    const celsius = TemperatureUtils.toCelsius(t);
15    return celsius >= 14 && celsius <= 32;
16  }
17 }

```

Código 49. Componente de utilidad de Negocio: TemperatureUtils

B.3 Componentes de generalización

La generalización de la Capa de Negocio en este ejemplo es `StatefulBusiness<T>`, que abstrae el patrón de Negocio observable: un componente que mantiene un estado y notifica a los observadores registrados cuando el estado cambia, ilustrando la comunicación ascendente sin romper el aislamiento entre capas (§6.2.2).

```

1 // /src/business/general/StatefulBusiness.ts
2 export abstract class StatefulBusiness<T> {
3   private observers: Array<(value: T) => void> = [];
4
5   subscribe(observer: (value: T) => void): void {
6     this.observers.push(observer);
7   }
8
9   unsubscribe(observer: (value: T) => void): void {
10    this.observers = this.observers.filter(o => o !== observer);
11  }
12
13  protected notify(value: T): void {
14    this.observers.forEach(o => o(value));
15  }
16
17  init(): void {}
18  terminate(): void {
19    this.observers = [];
20  }
21 }

```

Código 50. Componente de generalización: `StatefulBusiness<T>`

B.4 Componentes lógicos

Los componentes lógicos de la Capa de Negocio — los Negocios — modelan los conceptos del dominio. El ejemplo presenta tres componentes: `SessionBusiness` (estado de autenticación), `TemperatureBusiness` (estado y reglas de temperatura) y `UserBusiness` (identidad del usuario activo). `TemperatureBusiness` ilustra el patrón completo: precondiciones (rango aceptable, sesión válida), delegación a la Capa de Acceso a través de `R` y traducción de excepciones de Acceso en excepciones de Negocio, mutación de estado y notificación a observadores.

```

1 // /src/business/logic/SessionBusiness.ts
2 export class SessionBusiness extends StatefulBusiness<Session | null> {
3   private current: Session | null = null;
4
5   init(): void {
6     super.init();
7   }
8
9   terminate(): void {
10    this.current = null;
11    super.terminate();
12  }
13
14  async login(token: string): Promise<Session> {
15    // Precondiciones
16    if (StringUtils.isBlank(token)) {
17      throw new SessionExpiredException();
18    }
19
20    // Sentencias: delegación al Acceso + traducción de errores
21    try {
22      const response = await R.identity.verifyToken(token);
23      const session: Session = {
24        token: response.token,
25        expiresAt: new Date(Date.now() + response.expiresInSeconds * 1000),

```

```

26     refreshToken: response.refreshToken
27   };
28   this.current = session;
29   this.notify(session); // notificación a observadores
30   return session;
31 } catch (e) {
32   if (e instanceof AuthenticationFailedException) {
33     throw new SessionExpiredException();
34   }
35   throw e;
36 }
37 }
38
39 // Retorno primitivo: ejemplo de operación que devuelve un boolean
40 isActive(): boolean {
41   return this.current !== null && this.current.expiresAt > new Date();
42 }
43
44 getCurrent(): Session | null {
45   return this.current;
46 }
47 }

```

Código 51. Componente lógico de Negocio: SessionBusiness

```

1 // /src/business/logic/TemperatureBusiness.ts
2 export class TemperatureBusiness extends StatefulBusiness<Temperature> {
3   private current: Temperature | null = null;
4
5   init(): void {
6     super.init();
7   }
8
9   terminate(): void {
10    this.current = null;
11    super.terminate();
12  }
13
14  async update(t: Temperature): Promise<void> {
15    // Precondiciones (transmisión como excepción)
16    if (!TemperatureUtils.isWithinAcceptableRange(t)) {
17      throw new TemperatureOutOfRangeException(t.value, t.unit);
18    }
19    const session = B.session.getCurrent();
20    if (session === null || !B.session.isActive()) {
21      throw new SessionExpiredException();
22    }
23
24    // Sentencias: delegación al Acceso, mutación, notificación
25    const raw: RawTemperatureRecord = {
26      id: crypto.randomUUID(),
27      value: t.value,
28      unit: t.unit,
29      recordedAt: t.recordedAt.toISOString()
30    };
31    try {
32      await R.server.saveTemperature(B.user.getDeviceId(), raw, session.token);
33    } catch (e) {
34      if (e instanceof NetworkUnavailableException) {
35        // El estado local se actualiza incluso si la sincronización remota
36        // falla
37        this.current = t;
38        this.notify(t);
39        throw e;
40      }
41    }
42
43    this.current = t;
44    this.notify(t);
45  }
46
47  // Retorno Transfer

```

```

48     getCurrent(): Temperature | null {
49         return this.current;
50     }
51 }

```

Código 52. Componente lógico de Negocio: TemperatureBusiness

```

1  // /src/business/logic/UserBusiness.ts
2  export class UserBusiness extends StatefulBusiness<User | null> {
3      private current: User | null = null;
4
5      init(): void {
6          super.init();
7      }
8
9      terminate(): void {
10         this.current = null;
11         super.terminate();
12     }
13
14     setActive(user: User): void {
15         this.current = user;
16         this.notify(user);
17     }
18
19     getCurrent(): User | null {
20         return this.current;
21     }
22
23     getDeviceId(): string {
24         if (this.current === null) {
25             throw new SessionExpiredException();
26         }
27         return this.current.deviceId;
28     }
29 }

```

Código 53. Componente lógico de Negocio: UserBusiness

B.5 Componente de inyección

El componente B agrega los Negocios del artefacto y gestiona su ciclo de vida. El orden de inicialización refleja las dependencias internas: `SessionBusiness` primero, después `UserBusiness`, y por último `TemperatureBusiness` (que depende de la sesión activa y del dispositivo del usuario).

```

1  // /src/business/B.ts
2  export abstract class B {
3      // Definición: instancias únicas de los Negocios del artefacto
4      static readonly session: SessionBusiness = new SessionBusiness();
5      static readonly user: UserBusiness = new UserBusiness();
6      static readonly temperature: TemperatureBusiness = new TemperatureBusiness();
7
8      private constructor() {} // no instanciable
9
10     static init(): void {
11         B.session.init();
12         B.user.init();           // puede depender de B.session
13         B.temperature.init();    // depende de B.session y B.user
14     }
15
16     static terminate(): void {
17         B.temperature.terminate();
18         B.user.terminate();
19         B.session.terminate();
20     }
21 }

```

Código 54. Componente de inyección de la Capa de Negocio: B

C Anexo C — Ejemplos de implementación: Capa de Interacción

Este anexo tiene carácter informativo. Su propósito es ilustrar la aplicación del modelo de Arquitectura XF mediante una implementación de referencia. El contenido de este anexo no constituye requisito normativo.

Este anexo cierra el hilo del termostato conectado y presenta los cinco tipos de componentes de la Capa de Interacción. La Capa de Interacción expone los puntos de entrada del artefacto: una interfaz REST para la integración con sistemas externos (un `TemperatureService`) y una vista gráfica para el usuario humano (`MainView`). Cada componente delega exclusivamente en B para la lógica de dominio; ninguno contiene reglas de negocio ni accede directamente a sistemas externos. El modelo de proceso de esta capa se especifica en §7.2.3. Como cierre del hilo conductor, el anexo incluye además el elemento XF (§8.3), que materializa el punto de inicio de ejecución del artefacto.

C.1 Componentes de transferencia

Los componentes de transferencia de la Capa de Interacción modelan los formatos de entrada y salida específicos de los puntos de entrada: cuerpos de petición HTTP, respuestas estandarizadas, eventos de interfaz gráfica. Son distintos de los componentes de transferencia de Negocio: aquí los datos están adaptados al protocolo de entrada concreto, no al dominio. Viven en `/src/api/transfers`.

```

1 // /src/api/transfers/TemperatureUpdateRequest.ts
2 export interface TemperatureUpdateRequest {
3   readonly value: number;
4   readonly unit: 'C' | 'F';
5 }
6
7 // /src/api/transfers/ApiResponse.ts
8 export interface ApiResponse<T> {
9   readonly status: number;
10  readonly data?: T;
11  readonly error?: string;
12 }
13
14 // /src/api/transfers/InvalidParameterException.ts
15 export class InvalidParameterException extends Error {
16   constructor(public readonly field: string, public readonly reason: string) {
17     super('Invalid request: field \'' + field + '\' ' + reason);
18   }
19 }

```

Código 55. Componentes de transferencia de la Capa de Interacción

C.2 Componentes de utilidad

Las utilidades de Interacción son funciones puras sobre conceptos de presentación: formateo de salida, parseo de entrada. Su alcance es local a la Capa de Interacción.

```

1 // /src/api/Utils/FormatUtils.ts
2 export abstract class FormatUtils {
3   private constructor() {}
4
5   static formatTemperature(t: Temperature): string {
6     return `${t.value.toFixed(1)} ${t.unit}`;
7   }
8
9   static formatDate(d: Date): string {
10    return d.toISOString().substring(0, 19).replace('T', ' ');
11  }
12 }

```

Código 56. Componente de utilidad de Interacción: FormatUtils

C.3 Componentes de generalización

La generalización de la Capa de Interacción en este ejemplo modela el patrón común de un servicio REST: la deserialización del cuerpo de la petición, la invocación de la lógica delegada en Negocio, la serialización de la respuesta y la captura uniforme de excepciones para producir códigos de error coherentes.

```

1 // /src/api/general/RestService.ts
2 export abstract class RestService {
3   protected async handle<TRequest, TResponse>(
4     request: TRequest,
5     operation: (req: TRequest) => Promise<TResponse>
6   ): Promise<ApiResponse<TResponse>> {
7     try {
8       const data = await operation(request);
9       return { status: 200, data };
10    } catch (e) {
11      if (e instanceof InvalidParameterException) {
12        return { status: 400, error: e.message };
13      }
14      if (e instanceof SessionExpiredException) {
15        return { status: 401, error: e.message };
16      }
17      if (e instanceof TemperatureOutOfRangeException) {
18        return { status: 422, error: e.message };
19      }
20      if (e instanceof NetworkUnavailableException) {
21        return { status: 503, error: e.message };
22      }
23      return { status: 500, error: 'Internal error' };
24    }
25  }
26
27  init(): void {}
28  terminate(): void {}
29 }

```

Código 57. Componente de generalización: RestService

C.4 Componentes lógicos

Los componentes lógicos de la Capa de Interacción son los servicios (consumidores sistémicos) y las vistas (consumidores humanos). El ejemplo presenta `TemperatureService` (servicio que expone la operación de actualización de temperatura como endpoint REST) y `MainView` (vista que muestra y refresca la temperatura actual en la interfaz gráfica). Ambos delegan en B; ninguno contiene lógica de dominio.

```

1 // /src/api/logic/TemperatureService.ts

```

```

2 export class TemperatureService extends RestService {
3   init(): void {
4     super.init();
5   }
6
7   terminate(): void {
8     super.terminate();
9   }
10
11   // Punto de entrada sistémico: recibe la petición y delega en Negocio
12   async update(request: TemperatureUpdateRequest): Promise<ApiResponse<void>> {
13     return this.handle(request, async (req) => {
14       // Precondiciones de entrada (validación de formato, no de dominio)
15       if (req.unit !== 'C' && req.unit !== 'F') {
16         throw new InvalidParameterException('unit', 'must be C or F');
17       }
18       if (typeof req.value !== 'number' || isNaN(req.value)) {
19         throw new InvalidParameterException('value', 'must be a number');
20       }
21
22       // Delegación a la Capa de Negocio
23       const temperature: Temperature = {
24         value: req.value,
25         unit: req.unit,
26         recordedAt: new Date()
27       };
28       await B.temperature.update(temperature);
29     });
30   }
31
32   // Retorno Transfer: el cliente recibe el estado actual formateado
33   async getCurrent(): Promise<ApiResponse<{ formatted: string }>> {
34     return this.handle({}, async () => {
35       const current = B.temperature.getCurrent();
36       if (current === null) {
37         throw new InvalidParameterException('state', 'no temperature recorded yet');
38       }
39       return { formatted: FormatUtils.formatTemperature(current) };
40     });
41   }
42 }

```

Código 58. Componente lógico de Interacción: TemperatureService

```

1 // /src/api/logic/MainView.ts
2 export class MainView {
3   private displayedValue: string = '--';
4
5   init(): void {
6     // Suscripción al estado observable de la Capa de Negocio.
7     // La View reacciona a los cambios sin invocar al Negocio activamente.
8     B.temperature.subscribe(this.onTemperatureChange);
9   }
10
11   terminate(): void {
12     B.temperature.unsubscribe(this.onTemperatureChange);
13   }
14
15   private onTemperatureChange = (t: Temperature): void => {
16     this.displayedValue = FormatUtils.formatTemperature(t);
17     this.render();
18   };
19
20   // Punto de entrada humano: el usuario solicita refrescar la temperatura
21   async onRefreshClicked(): Promise<void> {
22     try {
23       const current = B.temperature.getCurrent();
24       if (current !== null) {
25         this.displayedValue = FormatUtils.formatTemperature(current);
26         this.render();
27       }
28     } catch (e) {

```

```

29     if (e instanceof SessionExpiredException) {
30         this.displayedValue = 'Session expired. Please log in.';
31         this.render();
32     } else {
33         this.displayedValue = 'Error reading temperature.';
34         this.render();
35     }
36 }
37 }
38
39 // Retorno primitivo: ejemplo de operación que devuelve un string
40 getDisplayedValue(): string {
41     return this.displayedValue;
42 }
43
44 private render(): void {
45     // Renderizado en la plataforma gráfica (omitido por brevedad)
46 }
47 }

```

Código 59. Componente lógico de Interacción: MainView

C.5 Componente de inyección

El componente A agrega los servicios y vistas del artefacto. Es el último componente de inyección que se inicializa en el ciclo de vida del artefacto y el primero que se termina — refleja que la Capa de Interacción depende de la disponibilidad de las capas inferiores ya inicializadas.

```

1  // /src/api/A.ts
2  export abstract class A {
3      static readonly temperatureService: TemperatureService = new
         TemperatureService();
4      static readonly mainView: MainView = new MainView();
5
6      private constructor() {}
7
8      static init(): void {
9          A.temperatureService.init();
10         A.mainView.init(); // se suscribe a B.temperature
11     }
12
13     static terminate(): void {
14         A.mainView.terminate();
15         A.temperatureService.terminate();
16     }
17 }

```

Código 60. Componente de inyección de la Capa de Interacción: A

C.6 Elemento de punto de inicio XF

Como ilustración de cierre, el elemento XF (§8.3) centraliza la inicialización y terminación del artefacto invocando los tres componentes de inyección en orden ascendente y descendente respectivamente, y declara el entry point del artefacto en el mismo fichero cuando el lenguaje lo permite. En TypeScript / Node, donde el entry point se materializa como una función `main()` en el fichero declarado por `package.json`, la convención canónica es declararla en el mismo módulo que XF:

```

1  // /src/XF.ts
2  export abstract class XF {
3      private constructor() {}
4

```

```

5   static init(): void {
6       R.init();
7       B.init();
8       A.init();
9   }
10
11  static terminate(): void {
12      A.terminate();
13      B.terminate();
14      R.terminate();
15  }
16 }
17
18 // Entry point del artefacto -- mismo fichero que XF
19 async function main(): Promise<void> {
20     XF.init();
21     try {
22         // El artefacto está listo: A.mainView observa B.temperature,
23         // que delega en R.server para sincronizar con el sistema externo.
24         // El bucle principal del entorno de ejecución gestiona los eventos.
25         await runApplicationLoop();
26     } finally {
27         XF.terminate();
28     }
29 }
30
31 main();

```

Código 61. Elemento de punto de inicio: XF

D Anexo D — Proyección de vocabularios externos en el modelo XF

Este anexo tiene carácter informativo. Su propósito es mostrar la proyección de los vocabularios de marcos y patrones externos en el modelo de Arquitectura XF. El contenido de este anexo no constituye requisito normativo.

La presente cláusula recoge la proyección de tres clases de vocabulario externo en el modelo XF: la nomenclatura de componente de los marcos de desarrollo más extendidos en la industria (§D.1), los patrones clásicos de inyección de dependencias (§D.2) y los conceptos centrales de las arquitecturas de referencia consolidadas en la literatura (§D.3). Las tres tablas son informativas y sirven como puente terminológico para el lector que llega al modelo desde una tradición tecnológica o arquitectónica específica. La proyección es semántica — el concepto desempeña el mismo rol arquitectónico — aunque la nomenclatura y los detalles de implementación difieren entre vocabularios.

D.1 Equivalencias con la nomenclatura de marcos de desarrollo

Table 16. Equivalencias entre la nomenclatura canónica XF y los términos empleados en marcos de desarrollo populares para cada combinación capa/tipo de componente.

Capa	Tipo XF	Spring (Java)	Angular (Web)	Flutter (Móvil)	Django (Python)	Node/Express
Acceso	Lógico (*Repository)	@Repository	Service (HTTP)	Repository	QuerySet/Manager	Model (DAO)
Acceso	Generalización	abstract class	abstract class	abstract class	abstract Model	abstract class
Acceso	Inyección (R)	ApplicationContext	Injector root	GetIt/Provider	apps registry	container
Acceso	Utilidad	static util class	utility function	static method	utility module	module exports
Acceso	Transferencia	POJO / DTO	interface	class (data)	dict/dataclass	plain object
Negocio	Lógico (*Business)	@Service	Service (@Injectable)	BLoC/Cubit	business in views	business module
Negocio	Generalización	abstract Service	abstract Service	abstract BLoC	abstract logic	abstract module
Negocio	Inyección (B)	@Configuration	Injector domain	Provider tree	services module	container
Negocio	Utilidad	static helpers	utility function	static method	utils module	module exports
Negocio	Transferencia	Entity/Model	interface/class	domain class	Django Model	plain object
Interacción	Lógico (*Service)	@RestController	route component	(no aplica)	View/ViewSet (DRF)	route handler
Interacción	Lógico (*View)	(no aplica)	Component	Widget	TemplateView	(no aplica)
Interacción	Generalización	abstract controller	base Component	abstract Widget	generic view	abstract handler
Interacción	Inyección (A)	WebApplicationContext	Injector UI	Provider root	urls registry	express app
Interacción	Utilidad	template helpers	pipe / formatter	Helper static	template tag/filter	view helper
Interacción	Transferencia	@RequestBody/Response	ApiResponse	DTO class	serializer DRF	req/res body

D.2 Equivalencias con patrones clásicos de inyección de dependencias

Table 17. Equivalencia conceptual entre patrones clásicos de inyección de dependencias [13, 14] y los componentes de inyección del modelo XF. La columna “Diferencia” precisa el matiz de cada patrón respecto a su proyección en el modelo XF.

Patrón clásico	Equivalente XF	Diferencia con el modelo XF
Service Locator [13]	R, B, A	XF tiene exactamente tres locators fijos (uno por capa) accedidos por nombre estático, sin lookup dinámico ni configuración runtime.
Constructor Injection	(no aplica)	XF no inyecta vía constructor: los lógicos acceden a sus dependencias mediante $R.x$, $B.y$, $A.z$ en el cuerpo de las operaciones.
Setter Injection	(no aplica)	XF no muta el grafo de dependencias post-construcción; el modelo garantiza inmutabilidad estructural.
Singleton [14]	Componente lógico bajo inyección	XF garantiza unicidad por construcción del componente de inyección, sin requerir patrón Singleton en la clase.
Service Container (IoC)	$R \cup B \cup A$	XF prescribe tres contenedores con responsabilidad por capa, no un contenedor único expandible.
Factory [14]	Instanciación inicial por el componente de inyección	La instanciación inicial (campos estáticos inicializados al cargar la clase del singleton) construye las instancias lógicas y establece su estado inicial σ_0 ; el modelo no expone factorías invocables ni separa la construcción en una operación con identificador propio.
DI Framework	Comp. de inyección + grafo de dependencias estático	XF resuelve dependencias en compile-time mediante análisis del grafo de dependencias; sin necesidad de un marco de inyección de dependencias en tiempo de ejecución.
Registry Pattern	Función parcial I_l	El $R.id$, $B.id$, $A.id$ es un registry con dominio finito y cerrado conocido en compile-time.

D.3 Proyección de arquitecturas de referencia

El modelo XF actúa como meta-modelo respecto a las arquitecturas de referencia consolidadas en la literatura: provee el vocabulario formal en el que cada arquitectura se proyecta como instancia parcial. Las tablas siguientes muestran cómo los conceptos centrales de cada arquitectura se reformulan en términos de la matriz $L \times T$. La proyección no es una comparación entre alternativas equivalentes — es la reformulación del vocabulario propietario de cada arquitectura en los términos invariantes del modelo, demostrando que las arquitecturas predominantes son instancias parciales de la misma estructura subyacente.

Clean Architecture Clean Architecture [30] prescribe el principio de inversión de dependencias y una organización en capas concéntricas sin prescripción de taxonomía cerrada de componentes ni nomenclatura canónica. Su proyección en el modelo XF se recoge en la [tabla siguiente](#):

Table 18. Proyección de los conceptos de Clean Architecture en el modelo XF: cada constructo se reescribe como su categoría $L \times T$ equivalente.

Concepto Clean Architecture	Capa XF	Tipo XF
Entity	Negocio	Transferencia / Lógico
Use Case (Interactor)	Negocio	Lógico (*Business)
Input Boundary / Output Boundary	Negocio	Generalización
Controller	Interacción	Lógico (*Service)
Presenter	Interacción	Lógico (*View)
Gateway (Repository interface)	Negocio	Generalización
Gateway (Repository implementation)	Acceso	Lógico (*Repository)
Frameworks & Drivers	Acceso	Lógico (*Repository)

La regla de dependencia de Clean Architecture — las dependencias apuntan siempre hacia el interior, hacia las políticas — se manifiesta en el modelo XF como el principio de aislamiento entre capas (§6.2.2): toda dependencia descendente desde Interacción hacia Negocio hacia Acceso. Clean Architecture aporta el principio de inversión de dependencias; el modelo XF lo expresa con tipificación cerrada de los elementos que satisfacen ese principio.

Domain-Driven Design DDD [11] introduce una rica terminología para el modelado del dominio de negocio: Aggregates, Repositories, Domain Services, Application Services, Value Objects, Bounded Contexts. La proyección de este vocabulario en el modelo XF se concentra principalmente en la Capa de Negocio, como recoge la [tabla siguiente](#):

Table 19. Proyección de los building blocks de Domain-Driven Design en el modelo XF: cada uno se reescribe como su categoría $L \times T$ equivalente.

Concepto DDD	Capa XF	Tipo XF
Entity	Negocio	Transferencia (con identidad)
Value Object	Negocio	Transferencia (inmutable)
Aggregate	Negocio	Lógico (*Business)
Aggregate Root	Negocio	Lógico (con estado)
Domain Service	Negocio	Lógico (*Business)
Application Service	Interacción	Lógico (*Service)
Repository (interface)	Acceso	Generalización
Repository (implementation)	Acceso	Lógico (*Repository)
Bounded Context	Negocio	Subdivisión /business/logic/instance
Domain Event	Negocio	Transferencia
Factory	Negocio	Utilidad (*Utils)
Specification	Negocio	Generalización

Los Bounded Contexts de DDD se proyectan en el modelo XF como subdivisiones del directorio **/business/logic/instance** (§7.2.2), región del artefacto donde el modelo reconoce el dominio funcional como criterio legítimo de subdivisión. Las capas de Acceso e Interacción se organizan en cambio por criterios técnicos ortogonales al dominio. DDD aporta el vocabulario rico del modelado del dominio; el modelo

XF lo extiende con tipificación universal aplicable también a las capas de Acceso e Interacción y al conjunto completo del artefacto, con independencia de la complejidad del dominio.

Arquitectura Hexagonal La Arquitectura Hexagonal [8] — también conocida como *Ports and Adapters* — introduce el principio de aislamiento del núcleo de aplicación mediante puertos abstractos que el núcleo expone y adaptadores concretos que los implementan. La proyección de su vocabulario en el modelo XF se recoge en la [tabla siguiente](#):

Table 20. Proyección de la Arquitectura Hexagonal en el modelo XF: puertos y adaptadores se reescriben como categorías $L \times T$.

Concepto Hexagonal	Capa XF	Tipo XF
Application Core (núcleo)	Negocio	Lógico (*Business)
Driving Port (primary port)	Interacción	Generalización (*Service)
Driving Adapter (primary adapter)	Interacción	Lógico (*Service , *View)
Driven Port (secondary port)	Acceso	Generalización (*Repository)
Driven Adapter (secondary adapter)	Acceso	Lógico (*Repository)

El modelo Hexagonal distingue puerto/adaptador por dirección — primary (driving) frente a secondary (driven) — pero los unifica conceptualmente bajo la dicotomía entrada/salida. El modelo XF separa explícitamente la dirección de entrada como Capa de Interacción y la de salida como Capa de Acceso, derivándolas de la propiedad de los procesos formales según la cual la invocación y la presentación de resultados constituyen una etapa estructuralmente distinta del acceso a sistemas externos. La proyección preserva el principio de aislamiento del núcleo de Hexagonal y le añade la prescripción de tipos por capa.

Arquitectura Onion La Arquitectura Onion [38] organiza el sistema en anillos concéntricos donde las dependencias apuntan siempre hacia el centro — el dominio — y nunca hacia los anillos externos. La proyección de sus anillos en el modelo XF se recoge en la [tabla siguiente](#):

Table 21. Proyección de la Arquitectura Onion en el modelo XF: cada anillo se reescribe como su categoría $L \times T$ equivalente.

Anillo Onion (de dentro hacia fuera)	Capa XF	Tipo XF
Domain Model (anillo central)	Negocio	Transferencia + Lógico
Domain Services	Negocio	Lógico (*Business)
Application Services	Interacción	Lógico (*Service)
Infrastructure	Acceso	Lógico (*Repository)
User Interface	Interacción	Lógico (*View)
Tests	(no aplica)	Excluido del modelo

La regla de dependencia hacia el centro de Onion se manifiesta en el modelo XF como la direccionalidad descendente del orden de capas Interacción → Negocio → Acceso.

Onion aporta la topología circular como visualización; el modelo XF la formaliza como matriz $L \times T$ y prescribe los tipos concretos que pueblan cada anillo.

Arquitecturas estratificadas (Bass et al.) Las arquitecturas estratificadas [3] establecen el patrón genérico de organización en capas horizontales con responsabilidades diferenciadas. El modelo XF es una instancia concreta de este patrón con tres capas derivadas de las etapas invariantes de los procesos formales, como recoge la [tabla siguiente](#):

Table 22. Proyección de las arquitecturas en capas en el modelo XF: cada capa tradicional se reescribe como su categoría $L \times T$.

Nomenclatura clásica (Bass et al.)	Capa XF
Presentation Layer	Interacción
Application Layer (Service Layer)	Interacción
Business Logic Layer (Domain Layer)	Negocio
Persistence Layer (Data Layer)	Acceso
Database Layer	Acceso (externo al artefacto)

El patrón genérico de las arquitecturas en capas no prescribe cuántas capas debe tener un sistema ni qué responsabilidades debe tener cada una. El modelo XF aporta esa prescripción — tres capas derivadas de primeros principios — y la complementa con tipificación cerrada y nomenclatura canónica.

Síntesis Las cinco proyecciones anteriores demuestran que el modelo XF no es una alternativa a las arquitecturas previas, sino la formalización del espacio común que todas ellas describen implícitamente con vocabularios distintos. El modelo aporta sobre el estado del arte la integración simultánea de tres prescripciones que los modelos previos formulan de forma parcial o independiente: estructura de capas derivada de primeros principios, taxonomía cerrada y exhaustiva de tipos de componente y nomenclatura canónica tecnológicamente agnóstica. Esta integración es la que permite expresar todas las arquitecturas de referencia en un único vocabulario formal verificable.

E Anexo E — Equivalencias terminológicas con otros vocabularios

Este anexo tiene carácter informativo. Su propósito es recoger las equivalencias terminológicas entre el modelo de Arquitectura XF y otros vocabularios. El contenido de este anexo no constituye requisito normativo.

La presente cláusula recoge las equivalencias y diferencias entre el término *Artefacto* adoptado por el modelo XF (definido en §3.1.1) y los términos análogos empleados por otros vocabularios consolidados en la ingeniería del software. La tabla permite al lector que llega al modelo XF desde otra tradición terminológica reconocer rápidamente las correspondencias y las diferencias semánticas.

La [tabla siguiente](#) sintetiza estas equivalencias terminológicas.

Table 23. Equivalencias terminológicas para el concepto de unidad ejecutable autocontenida.

Vocabulario / Fuente	Término	Diferencia con el Artefacto XF
OMG UML 2.5.1 [36]	<i>Artifact</i>	Supercategoría: incluye ejecutables, modelos, documentos, mensajes y otros productos del proceso. El Artefacto XF es la restricción al subconjunto ejecutable y autocontenido.
ISO/IEC/IEEE 24765:2017 [20]	<i>Software artifact</i>	Definición amplia análoga a UML: cualquier pieza de información producida o consumida por el proceso. No exige ejecutabilidad ni estratificación.
ISO/IEC/IEEE 12207:2017 [19]	<i>Software item</i>	Cualquier elemento del software del sistema (planes, procedimientos, programas, documentación). Más vago que el Artefacto XF.
ISO/IEC/IEEE 12207:2017 [19]	<i>Software unit</i>	Pieza compilable separadamente: función, clase o módulo. Más granular: un artefacto agrupa típicamente decenas o cientos de <i>units</i> .
ISO/IEC/IEEE 15288:2023 [23]	<i>System</i>	Combinación de elementos interactuantes con un propósito declarado. Más amplio: un sistema puede componerse de múltiples artefactos.
MIL-STD-498 [44]	<i>CSCI</i>	Agregación de software que satisface una función de uso final y se gestiona como unidad de configuración separada. Semánticamente próximo, pero terminología militar arcaica.
Szyperski [42]	<i>Software component</i>	Unidad de composición con interfaces contractuales y dependencias de contexto declaradas, desplegable independientemente. Comparte la autocontención, pero exige composición por terceros que XF no impone.
Parnas [39]	<i>Module</i>	Unidad de descomposición que oculta una decisión de diseño. Más granular: un artefacto contiene varios módulos en sentido Parnas.
Lewis y Fowler [26]; Newman [34]	<i>Microservice</i>	Servicio desplegable independiente, modelado sobre un dominio, típicamente HTTP. Captura autocontención y espacio único, pero limita el alcance al estilo distribuido; excluye móvil, scripts y embebidos.
DevOps / Native	Cloud- <i>Deployable unit / artifact</i>	Paquete autocontenido, versionado e inmutable, listo para despliegue. Captura autocontención y operatividad, pero carece de estándar formal y no prescribe estructura interna.
Wirth [46]; Dijkstra [10]	<i>Programa</i>	Composición de algoritmos y estructuras de datos. Concepción pedagógica clásica preservada por el Artefacto XF, al que añade tres condiciones estructurales que el término clásico no prescribe.

Como se desprende de la [tabla anterior](#), **ninguno de los vocabularios recogidos captura simultáneamente las tres condiciones que el modelo XF prescribe para un Artefacto**: autocontención, exactamente tres capas de abstracción derivadas de las etapas invariantes del proceso formal, y un único espacio de ejecución. Esta combinación es la contribución diferencial del vocabulario XF respecto a la literatura previa.

La elección del término *Artefacto* — en lugar de acuñar uno nuevo — es deliberada: el modelo XF se posiciona como especialización del concepto OMG UML 2.5.1, preservando la continuidad con la tradición consolidada de UML, ISO/IEC/IEEE 24765 y

los métodos de proceso unificado, y añadiendo las prescripciones estructurales que constituyen el contenido normativo propio del modelo.

F Anexo F — Historial de revisiones

Este anexo tiene carácter informativo. Su propósito es registrar el historial de revisiones del documento normativo del modelo XF. El contenido de este anexo no constituye requisito normativo.

La presente cláusula registra la cronología de revisiones del documento normativo del modelo XF. Cada entrada de la tabla recoge la versión publicada, su fecha de emisión, una síntesis del alcance de los cambios respecto a la versión anterior y la naturaleza editorial o sustantiva de la revisión.

Table 24. Historial de revisiones del documento: versión, fecha y resumen de los cambios de cada edición.

Versión	Fecha	Alcance de los cambios	Naturaleza
1.0.0	June 9, 2026	Primera edición de referencia. Establece la taxonomía cerrada de tipos de componente, la estratificación en tres capas de abstracción derivadas de las etapas invariantes de los procesos formales, el catálogo de reglas de conformidad y el algoritmo de determinación del nivel de conformidad.	Sustantiva

Las versiones futuras del modelo se registrarán en esta tabla en orden cronológico ascendente (la edición más antigua primero). La política de versionado del modelo se describe en la [fila de Política de cambios de la portada](#). La columna «Naturaleza» toma uno de tres valores: *Sustantiva* (cambios en la taxonomía o los axiomas, que incrementan la versión mayor), *Editorial* (adiciones al catálogo de reglas o reorganizaciones, que incrementan la versión menor) y *Aclaratoria* (precisiones de redacción sin alteración del contenido normativo, que incrementan la versión de parche).

Índice alfabético

- A (componente de inyección), 183
- Abstracción de protocolo, 187
- Acoplamiento semántico al marco de desarrollo, 177
- Agnosticismo tecnológico, 18, 48
- Aislamiento entre capas, 50
- Aplicación, 22, 178
- Artefacto, 21, 177
- Artefacto bien formado, 195
- Artefacto XF, 46

- B (componente de inyección), 183
- Business, 195

- Capa de abstracción, 9, 22, 178
- Capa de Acceso, 64, 178
- Capa de Interacción, 83, 179
- Capa de Negocio, 73, 178
- Catálogo (de reglas), 194
- Catálogo de reglas, 163
- Ciclo de vida del artefacto, 127, 189
- Compatibilidad, 24, 146, 192
- Complejidad accidental, 13
- Componente, 22, 179
- Componente de generalización, 25, 99, 182
- Componente de inyección, 25, 103, 183
- Componente de transferencia, 26, 114, 184
- Componente de utilidad, 25, 110, 184
- Componente integrado, 192
- Componente lógico, 25, 95, 180
- Condición, 22, 182
- Condición de negocio, 188
- Convergencia estructural, 13
- Convergencia implícita, 177

- Dependencia, 24, 186
- Dependencia circular, 190
- Dimensión de abstracción, 63, 179
- Dimensión funcional, 95, 179
- Direccionalidad, 140

- Elemento de punto de inicio XF, 189
- Elemento estructural, 194
- Elemento verificable, 24, 159, 194
- Elemento XF, 134

- Espacio de ejecución, 24, 192
- Estado, 22, 181
- Estado de presentación, 189
- Estado DEFINED, 189
- Estado INITIALIZED, 189
- Estado TERMINATED, 190
- Estratificación en capas, 40
- Estructura, 23, 184
- Estructura de carpetas canónica, 120, 185
- Estructura de datos, 138
- Estructura matricial, 60
- Etapas de acceso, 176
- Etapas de interacción, 175
- Etapas de procesamiento, 176
- Excepción de comunicación, 188
- Excepción de negocio, 188
- Exception, 196

- Flujo ascendente, 191
- Flujo de datos, 137, 190
- Flujo descendente, 191
- Fragmentación terminológica, 11
- Función de clasificación ϕ , 180
- Función de conformidad Λ , 195

- Herencia, 186
- Homogeneidad estructural, 143, 191
- Homonimia interframework, 177

- Identificador canónico, 194
- Inicialización, 23, 186
- Inmutabilidad post-inicialización, 183
- Instanciación, 23, 186
- Integración a nivel de aplicación, 193
- Integración a nivel de artefacto, 193

- Lenguaje ubicuo, 11
- Librería XF, 156, 193
- Lógica contextual, 22, 181
- Lógica de presentación, 188
- Lógica de transferencia, 181
- Lógica de utilidad, 181
- Lógica efectiva, 22, 181
- Lógica estructural, 25, 181

- Mapa conceptual, 175
- Mapeo tecnológico, 193

- Matriz $L \times T$, 46, 60, 179
- Modelo de referencia arquitectónico, 178
- Negocio (componente lógico), 180
- Nivel 0 de conformidad, 161
- Nivel 1 de conformidad, 161
- Nivel 2 de conformidad, 162
- Nivel 3 de conformidad, 162
- Nivel 4 de conformidad, 162
- Nivel de conformidad, 24, 160, 195
- Nomenclatura canónica, 185
- Normalización de datos, 187
- Ontología del negocio, 187
- Operación, 22, 182
- Operación de negocio, 187
- Orden de inicialización, 190
- Patrón observador, 191
- Precedencia de la arquitectura, 53
- Principio de agnosticismo tecnológico, 185
- Principio de aislamiento entre capas, 185
- Principio de precedencia de la arquitectura sobre la herramienta, 185
- Principio de tipificación cerrada y exhaustiva, 186
- Proceso formal, 31, 175
- Protocolo, 23, 187
- Punto de entrada, 23, 188
- Punto de inicio de ejecución, 24, 126, 189
- R (componente de inyección), 183
- Raíz de artefacto, 185
- Reformulación, 192
- Regla de conformidad, 9, 24, 193
- Regla de transformación estructural, 192
- Repositorio (componente lógico), 180
- Repository, 195
- Secuencia canónica de inicialización, 190
- Secuencia canónica de terminación, 190
- Sentencia, 26, 182
- Service, 195
- Servicio (componente lógico), 180
- Singleton Gathering, 23, 182
- Sinonimia heterotécnica, 177
- Slot (de inyección), 183
- Taxonomía de componentes, 9, 60
- Terminación, 23, 187
- Tipificación cerrada, 36, 56
- Tipificación propietaria, 38
- Tipo funcional, 22, 179
- Tipo primitivo, 23, 184
- Totalidad del código, 24, 195
- Transferencia heredada, 152, 184
- Transformación estructural, 140
- Tripartición procesamiento-interacción-acceso, 10, 32
- Unificación de estructuras de datos, 139, 191
- Utils, 196
- Verificabilidad de las reglas, 160
- Verificabilidad estructural, 25, 194
- Verificabilidad semántica, 25, 194
- View, 196
- Violación, 24, 194
- Violación de conformidad, 159
- Vista (componente lógico), 180
- Vía de comunicación, 141, 191